

SPCA 108N

**MASTER OF
COMPUTER APPLICATIONS**

FIRST YEAR

SECOND SEMESTER

CORE PAPER - VI

**DESIGN AND ANALYSIS
OF ALGORITHMS**



**INSTITUTE OF DISTANCE EDUCATION
UNIVERSITY OF MADRAS**

WELCOME

Warm Greetings.

It is with a great pleasure to welcome you as a student of Institute of Distance Education, University of Madras. It is a proud moment for the Institute of Distance education as you are entering into a cafeteria system of learning process as envisaged by the University Grants Commission. Yes, we have framed and introduced as per AICTE Choice Based Credit System(CBCS) in Semester pattern from the academic year 2020-21. You are free to choose courses, as per the Regulations, to attain the target of total number of credits set for each course and also each degree programme. What is a credit? To earn one credit in a semester you have to spend 30 hours of learning process. Each course has a weightage in terms of credits. Credits are assigned by taking into account of its level of subject content. For instance, if one particular course or paper has 4 credits then you have to spend 120 hours of self-learning in a semester. You are advised to plan the strategy to devote hours of self-study in the learning process. You will be assessed periodically by means of tests, assignments and quizzes either in class room or laboratory or field work. In the case of PG (UG), Continuous Internal Assessment for 20(25) percentage and End Semester University Examination for 80 (75) percentage of the maximum score for a course / paper. The theory paper in the end semester examination will bring out your various skills: namely basic knowledge about subject, memory recall, application, analysis, comprehension and descriptive writing. We will always have in mind while training you in conducting experiments, analyzing the performance during laboratory work, and observing the outcomes to bring out the truth from the experiment, and we measure these skills in the end semester examination. You will be guided by well experienced faculty.

I invite you to join the CBCS in Semester System to gain rich knowledge leisurely at your will and wish. Choose the right courses at right times so as to erect your flag of success. We always encourage and enlighten to excel and empower. We are the cross bearers to make you a torch bearer to have a bright future.

With best wishes from mind and heart,

DIRECTOR

MASTER OF COMPUTER APPLICATIONS

FIRST YEAR

SECOND SEMESTER

Core Paper - VI

DESIGN AND ANALYSIS OF ALGORITHMS

SYLLABUS

Objective of the course: This course introduces the basic concepts of Algorithm and discuss the implementation of different methods.

Course Outcomes: After successful completion of this course, the students should be able to Understand the basics of algorithm. Determine the appropriate data structure designs for the different concepts.

Unit-I: Introduction - Definition of Algorithm – pseudocode conventions – recursive algorithms – time and space complexity –big- “oh” notation – practical complexities – randomized algorithms – repeated element – primality testing - Divide and Conquer: General Method - Finding maximum and minimum – merge sort.**Unit-II:** Divide and conquer contd. – Quicksort, Selection, Strassen’s matrix multiplication – Greedy Method: General Method –knapsack problem - Tree vertex splitting - Job sequencing with deadlines – optimal storage on tapes.**Unit-III:** Dynamic Programming: General Method - multistage graphs – all pairs shortest paths – single source shortest paths -

String Editing – 0/1 knapsack. Search techniques for graphs – DFS- BFS-connected components – biconnected components.**Unit-IV:** Back Tracking: General Method – 8-queens - Sum of subsets - Graph Coloring – Hamiltonian cycles. Branch and Bound: General Method - Traveling Salesperson problem.**Unit-V:** Lower Bound Theory: Comparison trees - Oracles and advisory arguments - Lower bounds through reduction - Basic Concepts of NP-Hard and NP-Complete problems.

Recommended Texts

- 1) E. Horowitz, S. Sahni and S. Rajasekaran, 2008, Computer Algorithms, 2nd Edition, Universities Press, India.

Reference Books

- 1) G. Brassard and P. Bratley, 1997, Fundamentals of Algorithms, PHI, New Delhi.
- 2) A.V. Aho, J.E. Hopcroft, J.D. Ullmann, 1974, The Design and Analysis of Computer Algorithms, Addison Wesley, Boston.
- 3) S.E. Goodman and S.T. Hedetniemi, 1977, Introduction to the Design and Analysis of algorithms, Tata McGraw Hill Int. Edn, New Delhi.

E-learning resources

- 1) <http://www.cise.ufl.edu/~raj/BOOK.html>

MODEL QUESTION PAPER
MASTER OF COMPUTER APPLICATIONS
FIRST YEAR - SECOND SEMESTER
Core Paper - VI
DESIGN AND ANALYSIS OF ALGORITHMS

Time: 3 Hours

Maximum :80 marks

Part-A (10 X 2 =20)

Answer Any 10 Questions

1. Define space complexity.
2. Define P-type Problem.
3. Define Directed Acyclic Graph.
4. Define Minimum Spanning Tree.
5. What is the Least Cost search?
6. What are Binary search trees
7. What is Upper Bound?
8. What is sorting?
9. What is big O notation
10. Give the formula for Fast Fourier Transform.
11. What is an AVL tree?
12. What is pruning in backtracking?

(v)

Part-B (5 X 6 =30)

Answer Any 5 Questions

13. Explain divide and conquer principle for solving problems
14. What is dynamic programming approach? How is string editing done?
15. What is job sequencing with deadlines? Explain the greedy approach for solving the problem.
16. Explain the algorithm for Depth First Search
17. What is branch and bound algorithm? What are the problems solved by these method? Explain.
18. Explain about oracles and advisory arguments.
19. What are NP hard problems? Explain any one of them with example.

Part-C (3 X 10 =30)

Answer Any 3 Questions

20. Explain quick sort algorithm and compare with selection sort for complexities
21. Explain shortest path and tree vertex. Splitting problems and discuss Greedy approach for solving them
22. What is back tracking? Is it suitable for solving recursive procedure? Discuss how Graph coloring is done using this approach.
23. Discuss the procedure for solving 0/1 Knapsack problem using branch and bound technique.
24. Give algorithms for solving "Sum of Subset" problem using at least three different approaches.

MASTER OF COMPUTER APPLICATIONS

FIRST YEAR - SECOND SEMESTER

Core Paper - VI

DESIGN AND ANALYSIS OF ALGORITHMS

SCHEME OF LESSONS

Sl.No.	Title	Page
1	Introduction	008
2	Divide and Conquer	035
3	The Greedy Method	073
4	Dynamic Programming	115
5	Basic Traversal and Search Techniques	138
6	Backtracking	154
7	Branch-and-Bound	188
8	Algebraic Problems	233
9	Lower Bound Theory	289
10	NP-Hard and NP-Complete Problems	329

OVERVIEW

This package contains all the lessons as per the syllabus prescribed for.

V- Study Unit
Unit –1
Lesson - 1
INTRODUCTION

OBJECTIVE

After studying this lesson you can able to understand the following

- What is an algorithm
- How to analyze the performance of Algorithm.
 - Space & time complexity
 - Asympatic Notation
- Concepts of Randomized algorithm

1.1 WHAT IS AN ALGORITHM:

The word algorithm comes from the name of a Persian author, Abu Ja'far Mohammed ibn Musa al Khowarizmi Who wrote a textbook on mathematics. This word has taken on a special significance in computer science, where "algorithm " has come to refer to a method that can be used by a computer for the solution of a problem.

DEFINITION: ALGORITHM

An algorithm is a finite set of instructions that if followed accomplishes a particular task. In addition, all algorithms must satisfy the following criteria.

1. **Input:** Zero or more quantities are externally supplied.
2. **Output:** At least one quantity is produced.
3. **Definiteness:** Each instruction is clear and unambiguous.
4. **Finiteness:** If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a finite number of steps.
5. **Effectiveness:** Every instruction must be very basic so that it can be carried out, in principle, by a person using only pencil and paper. It is not enough that each operation is definite as in criterion 2; it also must be feasible.

The study of algorithms includes many important and active areas of research. There are four distinct areas of study one can identify:

1. How to devise algorithms

Creating an algorithm is an art, which may never be fully automated. Some of the techniques are especially useful in fields other than computer science such as operations research and electrical engineering. All of the approaches we consider have application in a variety of areas including computer science. But

some important design techniques such as linear, nonlinear, and integer programming are not covered here as they are traditionally covered.

2. How to validate algorithms

Once an algorithm is devised, it is necessary to show that it computes the correct answer for all possible legal inputs. We refer to this process as algorithm validation. The algorithm need not yet be expressed as a program. It is sufficient to state it in any precise way. The purpose of the validation is to assure us that this algorithm will work correctly independently of the issues concerning the programming language it will eventually be written in. Once the validity of the method has been shown, a program can be written and a second phase begins. This phase is referred to as **program proving** or **proof of program verifications**. A proof of correctness requires that the solution be stated in two forms. One form is usually as a program which is annotated by a set of assertions about the input and output variables of the program. These assertions are often expressed in the predicate calculus. The Second form is called a specification. A proof consists of showing that these two forms are equivalent in that for every given legal input, they describe the same output. A complete proof of program correctness requires that each statement of the programming language be precisely defined and all basic operations be proved correct. All these details may cause a proof to be very much longer than the program.

3. How to analyze algorithms

Analysis of algorithms or performance analysis refers to the task of determining how much computing time and storage an

algorithm requires. This is a challenging area, which sometimes requires great mathematical skill. An important result of this study is that it allows you to make quantitative judgements about the value of one algorithm over another. Another result is that it allows you to predict whether the software will meet any efficiency constraints that exist.

4. How to test a program

Testing a program consists of two phases: debugging and profiling.

Debugging is the process of executing programs on sample data sets to determine whether faulty results occur and, if so, to correct them. “Debugging can only point to the presence of errors, but not to their absence”. In cases in which we cannot verify the correctness of output on sample data, the following strategy can be employed: let more than one programmer develop programs for the same problem, and compare the outputs produced by these programs. If the outputs match, then there is a good chance that they are correct. A proof of correctness is much more valuable than a thousand tests, since it guarantees that the program will work correctly for all possible inputs.

Profiling or performance measurement is the process of executing a correct program on data sets and measuring the time and spaces it takes to compute the results. These timing figures are useful in that they may confirm a previously done analysis and point out logical places to perform useful optimization.

1.2 PERFORMANCE ANALYSIS:

DEFINITION: SPACE/TIME COMPLEXITY

The space complexity of an algorithm is the amount of memory it needs to run to completion. The time complexity of an algorithm is the amount of computer time it needs to run to completion.

Performance analysis can be loosely divided into two major phases: (1) a priori estimates (performance analysis) and (2) a posteriori testing (performance measurement).

1.2.1 SPACE COMPLEXITY :

Algorithm abc computes $a+b+b*c+(a+b-c)/(a+b)+4.0$;

Algorithm Sum computes $\sum_{i=1}^n a[i]$ iteratively, where the $a[i]$'s are real numbers;

Algorithm Rsum is a recursive algorithm that computes $\sum_{i=1}^n a[i]$.

Algorithm abc(a,b,c)

```
{
    return a + b + b * c + (a + b - c) / (a + b) + 4.0;
}
```

Algorithm Sum(a,n)

```
{
```



```

s:=0.0;
for i:=1 to n do
    s:=s+a[i];
return s;

```

```

Algorithm Rsum(a,n)
{
    if (n <=0) then return 0.0;
    else return Rsum(a,n-1) + a[n];
}

```

The space needed by each of these algorithms is seen to be the sum of the following components:

1. A fixed part that is independent of the characteristics (e.g., number, size) of the inputs and outputs. This part typically includes the instruction space (i.e., space for the code), space for simple variables and fixed-size component variables (also called aggregate), space for constants, and so on.
2. A variable part that consists of the space needed by component variables whose size is dependent on the particular problem instance being solved, the space needed by referenced variables (to the extent that this depends on instance characteristic), and the recursion stack space (in so far as this space depends on the instance characteristics).

The space requirement $S(P)$ of any algorithm P may therefore be written as

$S(P) = c + Sp(\text{instance characteristics})$, where c is a constant.

When analyzing the space complexity of an algorithm, we concentrate solely on estimating $Sp(\text{instance characteristics})$. For any given problem, we need first to determine which instance characteristics to use to measure the space requirements.

Example 1.1:

For algorithm `abc` the problem instance is characterized by the specific values of a , b , and c . Making the assumption that one word is adequate to store the values of each a , b , c , and the result, we see that the space needed by `abc` is independent of the instance characteristics. Consequently,

$Sp(\text{instance characteristics}) = 0$.

Example 1.2:

The problem instances for algorithm `Sum` are characterized by n , the number of elements to be summed. The space needed by n is one word, since it is of type integer. The space needed by a is the space needed by variable of type array of floating point numbers. This is at least n words, since a must be large enough to hold the n elements to be summed, so we obtain $\text{Sum}(n) \geq (n+3)$ (n for $a[]$, one each for n , i , and s).

Example 1.3:

Let us consider the algorithm `RSum`. As in the case of `Sum`, the instances are characterized by n . The recursion stack space

includes space for the formal parameters, the local variables, and the return address. Assume that the return address requires only one word of memory. Each call to RSum requires at least three words (including space for the values of n , the return address, and a pointer to $a[]$). Since the depth of recursion is $n+1$, the recursion stack space needed is $\geq 3(n+1)$.

1.2.2 TIME COMPLEXITY :

The time $T(P)$ taken by a program P is the sum of the compile time and the run (or execution) time. The compile time doesnot depend on the instance characteristics. Also, we may assume that a compiled program will be run several times without recompilation. Consequently, we concern ourselves with just the run time of the program. This run time is denoted by tp (instance characteristics).

Because many of the factors tp depends on are not known at the time the program is conceived, it is reasonable to attempt only to estimate tp . If we knew the characteristics of the compiler to be used, we could proceed to determine the number of addition, subtraction, multiplication, division, compares, loads, stores and so on, that would be made by the code for P . So, we could obtain an expression for $tp(n)$ of the form

$$tp(n) = caADD(n) + csSUB(n) + cmMUL(n) + cdDIV(n) + \dots$$

Where n denotes the instance characteristics, and ca , cs , cm , cd and so on, respectively, denote the time needed for an addition, subtraction, multiplication, division and so on, and ADD ,

SUB, MUL, DIV and so on are functions whose values are the numbers of addition, subtraction, multiplications, divisions and so on, that are performed when the code for P is used for an instance with characteristic n .

Obtaining such an exact formula is in itself an impossible task, since the time needed for an addition, subtraction, multiplication, division and so on. The value of $tp(n)$ for any given n can be obtained only experimentally. The program is typed, compiled and run on a particular machine. The execution time is physically clocked, and $tp(n)$ is obtained. Even with this experimental approach, one could face difficulties. In a multi-user system, the execution time depends on such factors as system load, the number of other programs running on the computer at the time the program P is run, the characteristics of these other programs and so on.

Given the minimal utility of determining the exact number of additions, subtractions and so on, that are needed to solve a problem instance with characteristics given by n , we might as well lump all the operations together (provided that the time required by each is relatively independent of the instant characteristics) and obtain a count for the total number of operations. We can go one step further and count only the number of program steps.

A program step is loosely defined as a syntactically or semantically meaningful segment of a program that has an execution time that is independent of the instance characteristics. For example, the entire statement

$\text{return } a + b + b * c + (a + b - c) / (a + b) + 4.0;$

could be regarded as a step since its execution time is independent of the instance characteristics.

The number of steps any program statement is assigned depends on the kind of statement. For example, comments count as zero steps; an assignment statement which does not involve any calls to other algorithms is counted as one step; in an iterative statement such as the **for**, **while** and **repeat-until** statements, we consider the step counts only for the control part of the statement. The control parts for **for** and **while** statements have the following forms:

for $i := \langle \text{expr} \rangle$ **to** $\langle \text{expr1} \rangle$ **do**

while $(\langle \text{expr} \rangle)$ **do**

Each execution of the control part of a **while** statement is given a step count equal to the number of step counts assignable to $\langle \text{expr} \rangle$. The step count for each execution of the control part of a **for** statement is one, unless the counts attributable to $\langle \text{expr} \rangle$ and $\langle \text{expr1} \rangle$ are functions of the instance characteristics.. In this latter case, the first execution of the control part of the **for** has a step count equal to the sum of the counts for $\langle \text{expr} \rangle$ and $\langle \text{expr1} \rangle$. Remaining executions of the **for** statement have a step count of one; and so on.

We can determine the number of steps needed by a program to solve a particular problem instance in one of two ways. In the first method, we introduce a new variable, count, into the program.

This is a global variable with initial value 0. Statements to increment count by the appropriate amount are introduced into the program. This is done so that each time a statement in the original program is executed, count is incremented by the step count of that statement.

Example 1.4 :

When the statement to increment count are introduced into Algorithm Sum the result algorithm is as follows. The change in the value of count by the time this program terminates is the number of steps executed by algorithm Sum.

```

ALGORITHM Sum(a,n)
{
    s:= 0.0;

count :=count+1;           // count is global it is initially
                           zero

for i:= 1 to n do
{
count := count+1;          //For for
    s := s+a[i]; count := count +1; // For assignment
}
count := count +1; // For last time of for
count := count +1; // For the return
return s;
}

```

Since we are interested in determining only the change in the value of count. Above algorithm may be simplified to following algorithm. It is easy to see in the for loop, the value of count will increase by a total of $2n$. If count is zero to start with, then it will be $2n+3$ on termination. So each invocation of Sum executes a total of $2n+3$ steps.

Algorithm sum (a,n)

```
{
    for i = 1 to n do count := count +2;

    count := count +3
}
```

Algorithm 1.5 :

When the statements to increment count are introduced into algorithm Rsum following algorithm is obtained.

ALGORITHM Rsum(a,n)

```
{
    count := Count +1;      // For the if statement
    if (n<=0) then
    {
        count := count +1;
        return 0.0;
```

```

    }
else
{
    count := count + 1;           // For the addition, function
                                // invocation and return

    return Rsum(a,n-1) + a[n];

}
}

```

Let $t_{\text{RSum}}(n)$ be the increase in the value of count when the algorithm terminates. We see that $t_{\text{RSum}}(0) = 2$. When $n > 0$ count increases by 2 plus whatever increase results from the invocation of Rsum from within the else clause. From the definition of t_{RSum} , it follows that this additional increase is $t_{\text{RSum}}(n-1)$. so, if the value of count is zero initially, its value at the time of termination is $2 + t_{\text{RSum}}(n-1)$, $n > 0$.

When analyzing a recursive program for its step count, we often obtain a recursive formula for the step count. For example,

$$t_{\text{Rsum}}(n) = \begin{cases} 2 & \text{if } n=0 \\ 2+t_{\text{Rsum}}(n-1) & \text{if } n>0 \end{cases}$$

These recursive formula are referred to as recurrence relations. One way of solving any such recurrence relation is to make repeated substitutions for each occurrence of the function t_{Rsum} on the right-hand side until all such occurrence disappear:

$$\begin{aligned}
t_{\text{Rsum}}(n) &= 2 + t_{\text{Rsum}}(n-1) \\
&= 2 + 2 t_{\text{Rsum}}(n-2) \\
&= 2(2) + t_{\text{Rsum}}(n-2) \\
&\quad \cdot \\
&\quad \cdot \\
&\quad \cdot \\
&\quad \cdot \\
&= n(2) + t_{\text{Rsum}}(0) \\
&= 2n + 2 \\
&= n \geq 0
\end{aligned}$$

so the step count for Rsum is $2n + 2$.

The step count is useful in that it tells us how the run time for a program changes with changes in the instance characteristics. From the step count for Sum, we see that if n is doubled, the run time also doubles; if n increases by a factor of 10, the run time increases by a factor of 10; and so on. So, the run time grows linearly in n . We say that Sum is a linear time algorithm.

DEFINITION: INPUT SIZE:

One of the instance characteristics that is frequently used in the literature is the input size. The input size of any instance of a problem is defined to be the number of words needed to describe that instance. The input size for the problem of summing an array

with n elements is $n+1$, n for listing the n elements and 1 for the value of n . If the input to any problem instance is a single element, the input size is normally taken to be the number of bits needed to specify that element.

1.2.3 ASYMPTOTIC NOTATION (O , Ω , Θ)

DEFINITION [Big “oh”]:

The function $f(n) = O(g(n))$ (read as “ f of n is big oh of g of n ”) iff there exist positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all n , $n \geq n_0$.

DEFINITION [Omega] :

The function $f(n) = \Omega(g(n))$ (reads as “ f of n is omega of g of n ”) iff there exist positive constants c and n_0 such that $f(n) \geq c \cdot g(n)$ for all n , $n \geq n_0$.

DEFINITION [Theta] :

The function $f(n) = \Theta(g(n))$ (read as “ f of n is theta of g of n ”) iff there exist positive constants c_1, c_2 , and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all n , $n \geq n_0$.

DEFINITION [Little “oh”] :

The function $f(n) = o(g(n))$ (read as “ f of n is little oh of g of n ”) iff

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

DEFINITION [Little omega] :

The function $f(n) = \omega(g(n))$ (read as “ f of n is little omega of g of n “) if

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$$

1.3 RANDOMIZED ALGORITHMS :

1.3.1 BASICS OF PROBABILITY THEORY :

Probability theory has the goal of characterizing the outcomes of natural or conceptual “experiments”. Each possible outcome of an experiment is called a sample point and the set of all possible outcomes is known as the sample space S . An event E is a subset of the sample space S .

Example 1 : [Tossing three coins] : When a coin is tossed, there are two possible outcomes : heads (H) and tails (T). Consider the experiment of throwing three coins. There are eight possible outcomes : HHH, HHT, HTH, HTT, THH, THT, TTH, and TTT. Each such outcome is a sample point. The sets {HHT, HTT, TTT}, {HHH, TTT}, and { } are three possible events. The third event has no sample points and is the empty set. For this experiment there are 2^3 possible events.

DEFINITION [PROBABILITY] :

The probability of an event E is defined to be $\frac{|E|}{|S|}$, Where

S is the sample space.

DEFINITION [MUTUAL EXCLUSION] :

Two events E1 and E2 are said to be mutually exclusive if they do not have any common sample points , that is, if $E1 \cap E2 = \phi$.

DEFINITION [CONDITIONAL PROBABILITY] :

Let E1 and E2 be any two events of an experiment. The conditional probability of E1 given E2, denoted by Prob [E1 | E2], is defined as

$$\frac{\text{Prop.}[E1 \cap E2]}{\text{Prob.}[E2]}$$

DEFINITION [INDEPENDENCE] :

Two Events E1 and E2 are said to be independent if $\text{Prob.}[E1 \cap E2] = \text{Prob.}[E1] * \text{Prob.}[E2]$

DEFINITION [RANDOM VARIABLE] :

Let S be the sample space of an experiment. A random variable on S is a function that maps the elements of S to the set of real numbers. For any sample point $s \in S$, X(s) denotes the

image of s under this mapping. If the range of X , that is, the set of values X can take, is finite, we say X is discrete.

Let the range of a discrete random variable X be $\{r_1, r_2, \dots, r_m\}$. Then, $\text{Prob.}[X=r_i]$, for any i , is defined to be the number of sample points whose image is r_i divided by the number of sample points in S . In this text we are concerned mostly with discrete random variables.

DEFINITION [EXPECTED VALUE] :

If the sample space of an experiment is $S = \{s_1, s_2, \dots, s_n\}$, the expected value or the mean of any random variable X is defined to be

$$\sum_{i=1}^n \text{Prob.}[s_i] * X(s_i) = 1/n \sum_{i=1}^n x(s_i).$$

DEFINITION [PROBABILITY DISTRIBUTION]:

Let X be a discrete random variable defined over the sample space S . Let $\{r_1, r_2, \dots, r_m\}$ be its range. Then, the probability distribution of X is the sequence

$$\text{Prob.}[X=r_1], \text{Prob.}[X=r_2], \dots, \text{Prob}[X=r_m]. \text{ Notice that } \sum_{i=1}^m \text{Prob}[X=r_i]=1.$$

DEFINITION [BINOMIAL DISTRIBUTION] :

A Bernoulli trial is an experiment that has two possible outcomes, namely, success and failure. The probability of success is p . Consider the experiment of conducting the Bernoulli trial n times. This experiment has a sample space S with 2^n sample points. Let X be a random variable on S defined to be the numbers of successes in the n trials. The variable X is said to have a binomial distribution with parameters (n, p) .

The expected value of X is np . Also,

$$\text{Pr ob}, [X = i] = \binom{n}{i} p^i (1-p)^{n-i}$$

Lemma 1 : [Markov's inequality] : If X is any nonnegative random variable whose mean is μ , then

$$\text{Pr ob}[X \geq x] \leq \frac{\mu}{x}$$

Lemma 2 : [Chernoff bounds] : If X is a binomial with parameters (n, p) , and $m \geq np$ is an integer, then

$$\text{Pr ob}(X > m) \leq \left(\frac{np}{m} \right)^m e^{(m-np)}$$

$$\text{also, Prob.}(X \leq \lfloor (1-\epsilon)pn \rfloor) \leq e^{(-\epsilon^2 np/2)}$$

$$\text{and Prob.}(X \geq \lceil (1+\epsilon)np \rceil) \leq e^{(-\epsilon^2 np/3)} \text{ for all } 0 < \epsilon < 1$$

1.3.2 RANDOMIZED ALGORITHMS : AN INFORMAL DESCRIPTION:

A Randomized algorithm is one that makes use of a randomizer (such as a random number generator). Some of the decisions made in the algorithm depend on the output of the randomizer. Since the output of an randomizer might differ in an unpredictable way from run to run, the output of a randomized algorithm could also differ from run to run for the same input. The execution time of a randomized algorithm could also vary from run to run for the same input.

Randomized algorithms can be categorized into two classes : The first is algorithms that always produce the same(correct) output for the same input. These are called Las Vegas algorithms. The execution time of a Las Vegas algorithm depends on the output of the randomizer. If we are lucky, the algorithm might terminate fast, and if not, it might run for a longer period of time. In general the execution time of a las Vegas algorithm is characterized as a random variable. The second is algorithm whose outputs might differ from run to run(for the same input). These are called Monte Carlo algorithms. Consider any problem for which there are only two possible answers, say, yes and no. If a Monte Carlo algorithm is employed to solve such a problem, then the algorithm might give incorrect answers depending on the output of the randomizer. We require that the probability of an incorrect answer from a Monte Carlo algorithm be low. Typically, for a fixed input, a Monte Carlo algorithm does not display much variation in execution time between runs, whereas in the case of a Las Vegas algorithm this variation is significant.

We can think of a randomized algorithm with one possible randomizer output to be different from the same algorithm with a different possible randomizer output. Therefore, a randomized algorithm can be viewed as a family of algorithms. The objective in the design of a randomized algorithm is to ensure that the number of bad algorithms in the family is only a small fraction of the total number of algorithms. If for any input we can show that at least $1 - \epsilon$ (ϵ being very close to zero) Fraction of algorithms in the family will run quickly on that input, then clearly, a random algorithm in the family will run quickly on any input with probability $\geq 1 - \epsilon$, where ϵ is called the error probability.

DEFINITION [The $\tilde{O}()$]:

Just like the $O()$ notation is used to characterize the run times of non randomized algorithms. $\tilde{O}()$ is used for characterizing the run times of Las Vegas algorithms. We say a Las Vegas algorithm has a resource (time, space, and so on) bound of $\tilde{O}(g(n))$. If there exists a constant c such the amount of resource used by the algorithm is no more than $c \alpha g(n)$ with probability $\geq 1 - n^{-\alpha}$. We shall refer to these bounds as high probability bounds.

DEFINITION [HIGH PROBABILITY]:

By high probability we mean a probability of $\geq 1 - n^{-\alpha}$ for any fixed α . We call α the probability parameter.

1.3.3 IDENTIFYING THE REPEATED ELEMENT:

Consider an array $a[]$ of n numbers that has $n/2$ distinct element and $n/2$ copies of another element. The problem is to identify the repeated element.


```

1   RepeatedElement(a,n)
2   // Finds the repeated element from a[1:n].
3   {
4   while (true) do
5   {
6   i := Random() mod n+1; j := Random() mod n+1;
7   // i and j are random numbers in the range [1,n]
8   if ((i < j) and (a[i] = a[j]) ) then return i;
9   }
10  }

```

ALGORITHM : Identifying the repeated array number.

1.3.4 PRIMALITY TESTING:

Any integer greater than one is said to be a prime if its only divisors are 1 and the integer itself. By convention, we take 1 to be a nonprime. Then 2,3,5,7,11, and 13 are the first six primes. Given an integer n , the problem of deciding whether n is a prime is known as primality testing.

THEOREM 1: [FERMAT] : If n is prime then $a^{n-1} \equiv 1 \pmod{n}$ for any integer $a < n$.

THEOREM 2 : The equation $x^2 \equiv 1 \pmod{n}$ has exactly two solutions, namely 1 and $n-1$, if n is prime.

Corollary : If the equation $x^2 \equiv 1 \pmod{n}$ has roots other than 1 and $n-1$, then n is composite.

Fermat's theorem suggests the following algorithm for primality testing: Randomly choose an $a < n$ and check whether $a^{n-1} \equiv 1 \pmod{n}$. If Fermat's equation is not satisfied, n is composite. If the equation is satisfied, we try some more random a 's. If on each a tried, Fermat's equation is satisfied, we output " n is prime"; otherwise we output " n is composite". In order to compute $a^{n-1} \pmod{n}$, we could employ Exponentiate with some minor modifications. The resultant primality testing algorithm is given as follows. Here large is a number sufficiently large that ensures a probability of correctness of $\geq 1 - n^{-\alpha}$.

1. Prime $O(n, \alpha)$
2. // Returns true if n is a prime and false otherwise
3. // is the probability parameter
4. {
5. $q := n-1$;
6. for $i := 1$ to large do // Specify large.
7. {
8. $m := q$; $y := 1$;
9. $a := \text{Random}() \pmod{q+1}$;

```

10. // Choose a random number in the range [1,n-1].
11. z:=a;
12. //Compute  $a^{n-1} \bmod n$ 
13. while (m > 0) do
14. {
15.   while (m mod 2 = 0) do
16.   {
17.     z:= $z^2 \bmod n$ ; m:= $\lfloor m/2 \rfloor$ ;
18.   }
19.   m:=m-1; y:=(y*z) mod n;
20. }
21. if (y < 1) then return false;
22. // if  $a^{n-1} \bmod n$  is not 1, n is not a prime.
23. }
24. return true;
25. }
```

ALGORITHM :Primality testing: First Attempt.

If the input is prime above algorithm will never output an incorrect answer.. If n is composite, Fermat's equation may be satisfied on the a chosen.

A slight modification of the above algorithm takes care of these problems. The modified primality testing algorithm is the same as `prime0` except that within the body of `Prime0` we also look for nontrivial square roots of n . The modified version is as follows.

`Prime ( $n, \alpha$ )`

`// Returns true if  $n$  is a prime and false otherwise`

`//  $\alpha$  is the probability parameter`

`{`

`$q := n-1$ ;`

`for  $i := 1$  to  $\alpha * \log(n) \infty$  // Specify large.`

`{`

`$m := q$ ;  $y := 1$ ;`

`$a := \text{Random}() \bmod q+1$ ;`

`// Choose a random number in the range  $[1, n-1]$ .`

`$z := a$ ;`

```

//Compute  $a^{n-1} \bmod n$ 

while (m > 0) do
{
    while (m mod 2 = 0) do
    {
        x := z; z :=  $z^2 \bmod n$  ;

        if( (z=1) and (x <> 1) and 'x' <> q) then

            return false;

        m :=  $\lfloor m/2 \rfloor$ ;
    }

    m := m-1; y := (y*z) mod n;
}

if(y <> 1) then return false;

// if  $a^{n-1} \bmod n$  is not 1, n is not a prime.

}

return true;

}

```

1.3.5 ADVANTAGES AND DISADVANTAGES :

Two of the most important advantages of using randomized algorithms are their simplicity and efficiency. You may have already gotten feel for this from the examples of repeated element identification and primality testing. A majority of the randomized algorithms found in the literature are simpler than the best deterministic algorithms for the same problems. Randomized algorithms have also been shown to yield better complexity bounds. For the repeated element identification problem any deterministic algorithm will have to spend $\Omega(n)$ time, whereas a simple randomized algorithm can solve the same problem in $\tilde{O}(\log n)$ time.

Not only do randomized algorithm yield superior asymptotic run-time bounds, but they also have been demonstrated to be competitive in practice. If we design a fast algorithm for a problem with an error probability much less than that of the hardware failure probability, then it is more likely for the machine to break down than for the algorithm to fail.

There are critical applications such as in nuclear reactor where even a small probability of error cannot be tolerated, Also, the use of a randomizer within an algorithm doesn't always result in better performance.

Lesson - 2

DIVIDE AND CONQUER

Objective :

After Studying this lesson you can able to understand the followig

- General method of Divide-and Conquer Strategy
- Finding the maximum and minimum
- Quicksort
- Selection
- Strassens matrix multiplication

2.1 GENERAL METHOD:

Given a function to compute on n inputs the divide and conquer strategy suggests splitting the inputs into k distinct subsets, $1 < k \leq n$, yielding k sub problems. These sub problems must be solved, and then a method must be found to combine sub solutions into a solution of the whole. If the sub problems are still relatively large, then the divide-and-conquer can possibly be reapplied. Often the sub problems resulting from a divide-and-conquer design are of the same type as the original problem. For those cases a recursive algorithm naturally expresses the reapplication of the divide-and-conquer principle. Now smaller and smaller

sub problems that are small enough to be solved without splitting are produced.

Suppose we consider the divide-and-conquer strategy when it splits the input into two sub problems of the same kind as the original problem. This splitting is typical of many of the problems we examine here. We can write a control abstraction that mirrors the way an algorithm based on divide-and-conquer will look.

Control Abstraction: It means a procedure whose flow of control is clear but whose primary operations are specified by other procedures whose precise meanings are left undefined.

Algorithm 2.1: Control Abstraction for Divide-and-conquer.

Algorithm DandC (P)

```
{
    If Small (P) then return S(P);
    Else
    {
        Divide P into smaller instances P1, P2, ....., PK, K > 1;
        Apply DandC to each of these sub problems;
        Return Combine (DandC (P1), DandC (P2), .....,
            DandC (PK));
    }
```


Above algorithm is initially invoked as DandC (P), Where

$P \rightarrow$ The problem to be solved.

Small (P) $\rightarrow$ Boolean-valued function that determines whether the input size is small enough that the answer can be computed without splitting or not.

Combine $\rightarrow$ function that determines the solution to P using the solutions to the k sub problems.

If Small (P) is true the function S is invoked. Otherwise the problem P is divided into smaller sub problems. These sub problems $P_1, P_2, \dots, P_k$ are solved by recursive applications of DandC. If the size of P is n and the sizes of the k sub problems are $n_1, n_2, \dots, n_k$, respectively, then the computing time of DandC is described by the recurrence relation

$$T(n) = \begin{cases} g(n) & n \text{ small} \\ T(n_1) + T(n_2) + \dots + f(n) & \text{otherwise} \end{cases} \quad \dots(2.1)$$

Where

$T(n) \rightarrow$ The time for DandC on any input size n

$g(n) \rightarrow$ The time to compute the answer directly for small inputs.

$f(n) \rightarrow$ The time for dividing P and combining the solution to sub problems.

The Complexity of many divide-and-conquer is given by recurrences of the form

$$T(n) = \begin{cases} T(1) & n = 1 \\ AT(n/b) + f(n) & n > 1 \end{cases} \quad \dots(2.2)$$

Where a and b are known constants.

$T(1)$ is known

n is a power of b (i.e., $n = b^k$)

One of the methods for solving any such recurrence relation is called the substitution method; this method repeatedly makes substitutions for each occurrence of the function T in the right-hand side until all such occurrences disappear.

Example 2.1

Consider the case in which $a=2$ and $b=2$. Let $T(1)=2$ and $f(n)=n$, we have

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &= 2\left[2T\left(\frac{n/2}{2}\right) + n/2\right] + n \text{ since sub } T(n) = 2T(n/2) + n \\ &= 2\left[2T(n/2 * 1/2) + n/2\right] + n \\ &= 2\left[2T(n/4) + n/2\right] + n \end{aligned}$$

$$= 4T(n/4) + 2n/2 + n$$

$$= 4T(n/4) + 2n$$

$$= 4[2T\left(\frac{n/4}{2}\right) + n/4] + 2n$$

$$= 4[2T(n/4 * 1/2) + n/4] + 2n$$

$$= 4[2T(n/8) + n/4] + 2n$$

$$= 8T(n/8) + 4n/4 + 2n$$

$$= 8T(n/8) + 3n$$

In general

$$T(n) = 2^i T(n/2^i) + in \quad \text{for any } \log_2 n > i > 1$$

If $i = \log_2 n$ then

$$T(n) = 2^{\log_2 n} T(n/2^{\log_2 n}) + n \log_2 n$$

$$T(n) = 2^{\log_2 2} T(2/2^{\log_2 2}) + 2 \log_2 2 \quad \text{since } \log_2 2 = 1$$

$$= 2T(2/2) + 2 \log_2 2$$

$$= 2T[1] + 2 \log_2 2$$

In general form, since $n = 2^{\log_2 n}$ for 2

$$= n * T[1] + n \log_2 n$$

$$= n * 2 + n \log_2 n \text{ since } T[1] = 2 \text{ from equation 2.1}$$

$$= 2n + n \log_2 n$$

Beginning with the recurrence 2.2 and using the substitution method it can be shown that

$$T(n) = n^{\log_b a} [T(1) + u(n)]$$

$$\text{Where } u(n) = \sum_{j=1}^k h(b^j)$$

$$\text{and } h(n) = f(n) / n^{\log_b a}$$

Following table tabulates the asymptotic value of $u(n)$ for various values of $h(n)$. This table allows one to easily obtain the asymptotic value of $T(n)$ for many of the recurrences one encounters when analyzing divide-and-conquer algorithms.

Table 2.1 : $u(n)$ values for various $h(n)$ values.

$h(n)$	$u(n)$
$O(n^r), r < 0$	$O(1)$
$\Theta((\log n)^i), i \geq 0$	$\Theta((\log n)^{i+1}/(i+1))$
$\Omega(n^r), r > 0$	$\Theta(h(n))$

Example 2.2

Look at the following recurrences when n is a power of 2.

$$T(n) = \begin{cases} T(1) & n = 1 \\ T(n/2) + c & n > 1 \end{cases}$$

Comparing with the equation 2.2, we see that $a = 1$, $b = 2$, and $f(n) = c$. So, $\log_b(a) = 0$ and $h(n) = f(n) / n^{\log_b a} = c = c(\log n)^0 = \Theta((\log n)^0)$. From the table we obtain $u(n) = \Theta(\log n)$. so, $T(n) = n^{\log_b a} [c + \Theta(\log n)] = \Theta(\log n)$.

Example 2.3 :

Next consider the case in which $a = 2$, $b=2$, and $f(n) = cn$. For this recurrence, $\log_b a = 1$ and $h(n) = f(n)/n = c = \Theta((\log n)^0)$. Hence $u(n) = \Theta(\log n)$ and $T(n) = n[T(1) + \Theta(\log n)] = \Theta(n \log n)$.

2.2 FINDING MAXIMUM AND MINIMUM:

Let us consider another simple problem that can be solved by the divide-and conquer technique. The problem is to find the maximum and minimum items in a set of n elements. Following algorithm is an Straightforward approach for this.

Algorithm 2.2 : Straight Forward Maximum and Minimum

Algorithm StraightMaxMin($a, n, \max, \min$)

// Set $\max$ to the maximum and $\min$ to the minimum of $a[1;n]$.

```

{
max:=min:=a[1];
for I:=2 to n do
{
If (a[i] > max) then max:=a[i];
If (a[i] < min) then min:=a[i];
}
}

```

In analyzing the time complexity of this algorithm, we once again concentrate on the number of element comparison. The justification for this is that the frequency count for other operations in this algorithm is of the same order as that for element comparison. More importantly, when the elements in $a[1:n]$ are polynomials, vectors, very large numbers, or strings of characters, the cost of an element comparison is much higher than the cost of the other operations. Hence the time is determined mainly by the total cost of the element comparisons.

StraightMaxMin requires $2(n-1)$ element comparison in the best, average, and worst cases. An immediate improvement is possible by realizing that the comparison $a[i] < \min$ is necessary only when $a[i] > \max$ is false. Hence we can replace the contents of the for loop by

```

If (a[i] > max) then max:=a[i];
Else if (a[i] < min) then min:=a[i];

```

Here the number of element comparisons is $n-1$. The worst case occurs when the elements are in decreasing order. In this case the number of element comparisons is $2(n-1)$. The average number of element comparisons is less than $2(n-1)$. On the average, $a[i]$ is greater than $\max$ half the time, and so the average number of comparisons is $3n/2-1$.

A divide-and-conquer algorithm for this problem would proceed as follows:

$$\begin{aligned} \text{i.e) } &= \frac{n-1+2n-2}{2} \\ &= \frac{3n-3}{2} = \frac{3n}{2} - \frac{3}{2} = \frac{3n}{2} - 1 \end{aligned}$$

Let $P=(n, a[i], \dots, a[j])$ denotes an arbitrary instance of the problem. Here n is the number of elements in the list $a[i], \dots, a[j]$ and we are interested in finding the maximum and minimum of the list. Let $\text{small}(p)$ be true when $n \leq 2$. In this case, the maximum and minimum are $a[i]$ if $n=1$. If $n=2$, the problem can be solved by making one comparison.

If the list has more than two elements, P has to be divided into smaller instances. For example, we might divide P into the two instances $P_1 = (\lfloor n/2 \rfloor, a[1], \dots, a[\lfloor n/2 \rfloor])$ and $P_2 = (n - \lfloor n/2 \rfloor, a[\lfloor n/2 \rfloor + 1], \dots, a[n])$. After having divided P into two smaller sub problems, we can solve them by recursively invoking the same divide-and-conquer algorithm. If $\text{MAX}(P)$ and $\text{MIN}(P)$ are the maximum and minimum of the elements in P , then $\text{MAX}(P)$

is the larger of $\text{MAX}(P1)$ and $\text{MAX}(P2)$. Also, $\text{MIN}(P)$ is the smaller of $\text{MIN}(P1)$ and $\text{MIN}(P2)$.

Following algorithm results from applying the strategy just described. MaxMin is a recursive algorithm that finds the maximum and minimum of the set of elements $\{a(i), a(i+1), \dots, a(j)\}$. The situation of set sizes ($i=j$) and two ($i=j-1$) are handled separately. For sets containing more than two elements, the midpoint is determined and two new sub problems are generated. When the maxima and minima of these sub problems are determined, the two maxima are compared and the two minima are compared to achieve the solution for the entire set.

Algorithm 2.3 : Recursively Finding the maximum and minimum.

Algorithm MaxMin($i, j, \text{max}, \text{min}$)

// $a[1:n]$ is a global array. Parameters i and j are integers.

// $1 \leq i \leq j \leq n$. The effect is to set max and min to the

// largest and smallest values in $a[i:j]$, respectively.

{

 if($i=j$) then $\text{max} := \text{min} := a[i]$; // Small(P)

 else if($i=j-1$) then // Another case of Small(P)

 {


```

if(a[i] < a[j]) then
{
    max := a[j]; min := a[i];
}
else
{
    max := a[i]; min := a[j];
}
}

else
{ // If P is not small, divide P into sub problems.

    // Find where to split the set
    mid :=  $\lfloor (i+j) / 2 \rfloor$ ;

    // solve the subproblems

    MaxMin(i, mid, max, min);

    MaxMin(mid+1, j, max1, min1);

```

```

// Combine the solutions.

If (max < max1) then max := max1;

If (min > min1) then min := min1;

    }

}

```

The problem is initially invoked by the statement

```
MaxMin (1,n, x, y);
```

Suppose we simulate MaxMin on the following nine elements.

a:	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	22	13	-5	-8	15	60	17	31	47

A good way of keeping track of recursive calls is to build a tree by adding a node each time a new call is made. For this algorithm each node has four items of information : i, j, max and min. On the array a[] above, the tree of Following figure is produced.

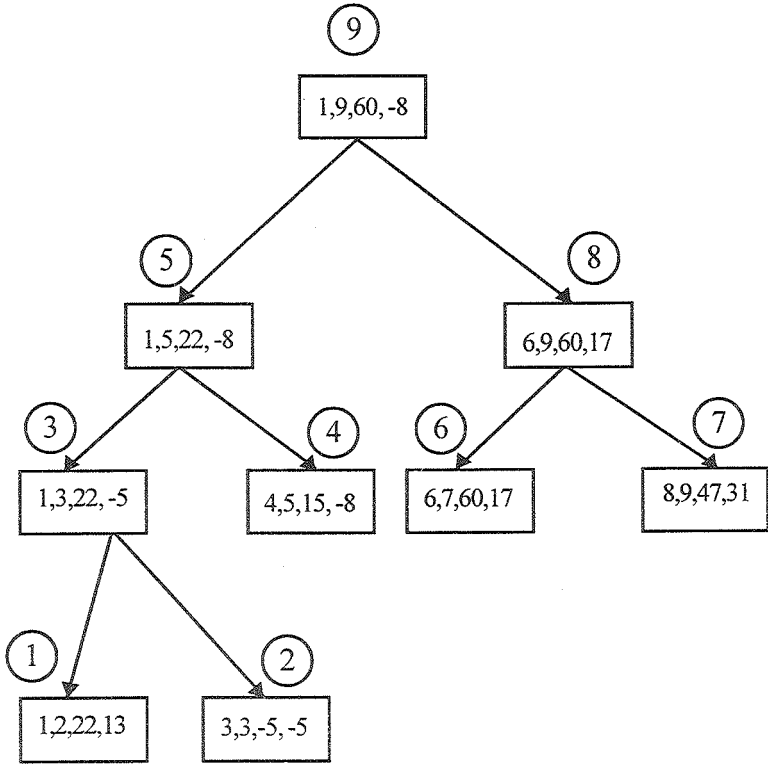


FIGURE : 2.1 Trees of recursive calls of MaxMin

Examining the fig, we see that the root node contains 1 and 9 as the values of i and j corresponding to the initial call to MaxMin. This execution produces two new calls to MaxMin, where i and j have the values 1, 5 and 6, 9, respectively and thus split the set into subsets of approximately the same size. From the tree we can immediately see that the maximum depth of recursion is four

(including the first call). The circled numbers in the upper left corner of each node represent the orders in which max and min are assigned values.

Now what is the number of element comparisons needed for MaxMin? If $T(n)$ represents this number, then the resulting recurrence relation is

$$T(n) = \begin{cases} T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + 2 & n > 2 \\ 1 & n = 2 \\ 0 & n = 1 \end{cases}$$

when n is a power of two, $n = 2^k$ for some positive k , then

$$\begin{aligned} T(n) &= 2T(n/2) + 2 \\ &= 2(2T(n/4) + 2) + 2 \\ &= 4T(n/4) + 4 + 2 \\ &\quad \cdot \\ &\quad \cdot \\ &\quad \cdot \\ &= 2^{k-1}T(2) + \sum_{1 \leq i < k-1} 2^i \\ &= 2^{k-1} + 2^k - 2 = 3n/2 - 2 \end{aligned}$$

Note that $3n/2 - 2$ is the best, average, and worst case number of comparisons when n is a power of two.

Compared with the $2n - 2$ comparisons for the straightforward method, this is a saving of 25% in comparisons. It can be shown that no algorithm based on comparisons uses less than $3n/2 - 2$ comparisons. So in this sense algorithm MaxMin is optional. In terms of storage, MaxMin is worse than the straightforward algorithm because it requires stack space for $i, j, \max, \min, \max1, \min1$. Given n elements, there will be $\lfloor \log_2 n \rfloor + 1$ levels of recursion and we need to save seven values for each recursive call.

Let us see what the count is when element comparisons have the same cost as comparisons between i and j . Let $C(n)$ be this number. First, we observe the lines 6 and 7 in the algorithm can be replaced with

If($i \geq j-1$) { //Small(P)

To achieve the same effect. Hence a single comparison between i and $j-1$ is adequate to implement the modified if statement. Assuming $n=2^k$ for some positive integer k , we get

$$C(n) = \begin{cases} 2C(n/2) + 3 & n > 1 \\ 2 & n = 1 \end{cases}$$

Solving this equation, we obtain

$$\begin{aligned} C(n) &= 2C(n/2) + 3 \\ &= 4C(n/4) + 6 + 3 \end{aligned}$$

$$\begin{aligned}
&= 2^{k-1} C(2) + \sum_{i=0}^{k-2} 2^i \\
&= 2^k + 3 * 2^{k-1} - 3 \\
&= 5n / 2 - 3
\end{aligned}$$

The comparative figure for StraightMaxMin is $3(n-1)$. This is larger than $5n/2-3$. Despite this, MaxMin will be slower than StraightMaxMin because of the overhead of stacking $i, j, \max$, and $\min$ for the recursion.

If comparisons among the elements of $a[]$ are much more costly than comparisons of integer variables, then the divide-and-conquer technique has yielded a more efficient algorithm. On the other hand, if this assumption is not true, the technique yields a less-efficient algorithm. Thus the divide-and-conquer strategy is seen to be only a guide to better algorithm design which may not always succeed.

Both MaxMin and Straight MaxMin are $\Theta(n)$, so the use of asymptotic notation is not enough of a discriminator in this situation.

2.3 QUICKSORT:

The divide-and-conquer approach can be used to arrive at an efficient sorting method different from merge sort. In merge sort, the file $a[1:n]$ was divided at its midpoint into sub arrays which were independently sorted and later merged. In quick sort,

the division into two sub arrays is made so that the sorted sub arrays do not need to be merged later. This is accomplished by rearranging the elements in $a[1:n]$ such that $a[i] \leq a[j]$ for all i between 1 and m and all j between $m+1$ and n for some m , $1 \leq m \leq n$. Thus the elements in $a[1:m]$ and $a[m+1:n]$ can be independently sorted. No merge is needed. The arrangement of the elements is accomplished by picking some element of a , say $t = a[s]$, and then reordering the other elements so that all elements appearing before t in a $[i:n]$ are less than or equal to t and all elements appearing after t are greater than or equal to t . This rearranging is referred as partitioning.

Function Partition algorithm accomplishes as in-place partitioning of the elements of $a[m:p-1]$. It is assumed that $a[p] \geq a[m]$ and that $a[m]$ is the partitioning element. If $m=1$ and $p-1 = n$, then $a[n+1]$ must be defined and must be greater than or equal to all elements in $a[1:n]$. The assumption that $a[m]$ is the partition element is merely for convenience ; other choice for partitioning element than the first item in the set are better in practice. The function $\text{Interchange}(a, i, j)$ exchanges $a[i]$ and $a[j]$.

Example 2.4 :

As an example of how partition works. Consider the following array of nine elements. The function is initially invoked as $\text{Partition}(a, 1, 10)$. The ends of the horizontal line indicate those elements which were interchanged to produce the next row. The element $a[1] = 65$ is the partitioning element and it is eventually

determined to be fifth smallest element of the set. Notice that the remaining elements are unsorted but partitioned about $a[5] = 65$.

(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	i	p
65	<u>70</u>	<u>75</u>	<u>80</u>	<u>85</u>	60	55	50	45	$+\infty$	2	9
65	45	<u>75</u>	<u>80</u>	<u>85</u>	60	55	<u>50</u>	70	$+\infty$	3	8
65	45	50	<u>80</u>	<u>85</u>	60	<u>55</u>	75	70	$+\infty$	4	7
65	45	50	55	<u>85</u>	<u>60</u>	80	75	70	$+\infty$	5	6
<u>65</u>	<u>45</u>	<u>50</u>	<u>55</u>	<u>60</u>	85	80	75	70	$+\infty$	6	5
60	45	50	55	65	85	80	75	70	$+\infty$		

Using Hoare's clever method of partitioning a set of elements about a chosen element, we can directly devise a divide-and-conquer method for completely sorting n elements. Following a call to the function Partition. Two sets S_1 and S_2 are produced. All elements in S_1 are less than or equal to the element in S_2 . Hence S_1 and S_2 can be sorted independently. Each set is sorted by reusing the function Partition. Following algorithm describes the complete process.

Algorithm : Partition the array $a[m:p-1]$ about $a[m]$

Algorithm Partition(a, m, p)

// Within $a[m]$, $a[m+1]$, ..., $a[p-1]$ the elements are rearranged in such a manner

// that if initially $t = a[m]$, then after completion $a[q] = t$ for some q between m

// and $p-1$, $a[k] \leq t$ for $m \leq k < q$, and $a[k] \geq t$ for $q < k < p$, q is returned. Set $a[p] = \infty$. {

$v := a[m]; i := m; j := p;$

repeat

{

repeat

$i := i + 1;$

until $(a[i] \geq v);$

repeat

$j := j - 1;$

until $(a[j] \leq v);$

if $(i < j)$ then Interchange(a, i, j);

} until $(i \geq j);$

$a[m] := a[j]; a[j] := v; \text{return } j;$

}

Algorithm Interchange(a, i, j)

```
// Exchange a[i] with a[j].
```

```
{
    p := a[i];
    a[i] := a[j]; a[j] := p;
}
```

Algorithm : Sorting by Partitioning

Algorithm QuickSort(p,q)

// Sorts the elements $a[p], \dots, a[q]$ which reside in the global array $a[i:n]$ into

// ascending order, $a[n+1]$ is considered to be defined and must be $\geq$ all the

// elements in $a[1:n]$.

```
{
if (p < q) then    // If there are more than one element
{
    // divide P into two sub problems.
    J := Partition(a,p,q+1);
    // j is a position of the partitioning element.
```

```
// Solve the Subproblems.
```

```
QuickSort(p,j-1);
```

```
QuickSort(j+1,q);
```

```
// There is no need for combining solutions.
```

```
}
```

```
}
```

First let us obtain the worst-case value $C_w(n)$ of $C(n)$. The number of element comparisons in each call of Partition is at most $p-m+1$. Let r be the total number of elements in all the calls to Partition at any level of recursion. At level one, only one call, Partition $(a, 1, n+1)$, is made and $r=n$; at level two at most two calls are made and $r = n-1$; and so on. At each level of recursion, $O(r)$ element comparisons are made by Partition. At each level, r is at least one less than r at the previous level as the partitioning elements of the previous levels are eliminated. Hence $C_w(n)$ is the sum on r as r varies from 2 to n , or $O(n^2)$.

The average value $C_A(n)$ of $C(n)$ is much less than $C_w(n)$. Under the assumptions made earlier, the partitioning element v has an equal probability of being the i -th smallest element, $1 \leq i \leq p-m$, in $a[m:p-1]$. Hence the two sub arrays remaining to be sorted are $a[m:j]$ and $a[j+1 : p-1]$ with probability $1/(p-m)$, $m \leq j < p$. From this we obtain the recurrence

$$C_A(n) = n+1 + \frac{1}{n} \sum [C_A(k-1) + C_A(n-k)] \quad \dots 1$$

$$1 \leq k \leq n$$

The number of element comparisons required by Partition on its first call is $n+1$. Note that $C_A(0) = C_A(1) = 0$. Multiplying both sides of above equation by n , we obtain.

$$nC_A(n) = n(n+1) + 2[C_A(0) + C_A(1) + \dots + C_A(n-1)] \quad \dots 2$$

Replacing n by $n-1$ in eqn 2 gives

$$(n-1) C_A(n-1) = n(n-1) + 2[C_A(0) + C_A(1) + \dots + C_A(n-2)]$$

Subtracting this from eqn 2 we get

$$nC_A(n) - (n-1) C_A(n-1) = 2n + 2C_A(n-1) \quad \dots 2$$

or

$$C_A(n)/(n+1) = C_A(n-1)/n + 2/(n+1).$$

Repeatedly using this qn to substitute for $C_A(n-1)$, $C_A(n-2)$,, we get

$$\begin{aligned} C_A(n)/n+1 &= C_A(n-2)/n-1 + 2/n + 2/n+1 \\ &= C_A(n-3)/n-2 + 2/n-1 + 2/n + 2/n+1 \\ &\quad \cdot \\ &\quad \cdot \\ &\quad \cdot \\ &= C_A(1)/2 + 2 \sum_{3 \leq k \leq n+1} 1/k \end{aligned}$$

$$= 2 \sum_{3 \leq k \leq n+1} 1/k$$

Since

$$\sum_{3 \leq k \leq n+1} 1/k \leq \int_2^{n+1} 1/x dx = \log_e(n+1) = \log_e 2$$

$$C_A(n) \leq 2(n+1)[\log_e^{(n+2)} - \log_e 2] = O(n \log n)$$

Even though the worst-case time is $O(n^2)$, the average time is only $O(n \log n)$. Let us now look at the stack space needed by the recursion. In the worst case the maximum depth of recursion may be $n-1$. This happens, for example, when the partition element on each call to Partition is the smallest value in $a[m:p-1]$. The amount of stack space needed can be reduced to $O(\log n)$ by using an iterative version of quicksort in which the smaller of the two sub arrays $a[p:j-1]$ and $a[j+1:q]$ is always sorted first. Also, the second recursive call can be replaced by some assignment statements and a jump to the beginning of the algorithm. With these changes, QuickSort takes the form of Algorithm as follows.

ALGORITHM : Iterative version of QuickSort.

Algorithm QuickSort2(p,q)

// Sorts the elements in $a[p:q]$

{

// stack is a stack of size $2 \log(n)$

repeat

{

while ($p < q$) do

{

$j := \text{Partition}(a, p, q+1);$

if ($((j-p) < (q-j))$) then

{

Add($j+1$); //Add $j+1$ to stack

Add(q); $q := j-1$; //Add q to stack

}

else

{

Add(p); //Add p to stack

Add($j-1$); $p := j+1$; // Add $j-1$ to stack

}

} //Sort the smaller subfile

if stack is empty then return;

```

Delete(q); Delete(p);    //Delete q and p from stack.

} until (false);

}

```

We can now verify that the maximum stack space needed is $O(\log n)$. Let $S(n)$ be the maximum stack space needed. Then it follows that

$$S(n) = \begin{cases} 2+S(\lfloor (n-1)/2 \rfloor) & n > 1 \\ 0 & n \leq 1 \end{cases}$$

which is less than $2 \log n$.

InsertionSort is exceedingly fast for n less than about 16. Hence Insertion Sort can be used to speed up QuickSort whenever $q - p < 16$. The exercises explore various possibilities for selection of the partition element.

PERFORMANCE MEASUREMENT

QuickSort and MergeSort were evaluated on a SUN workstation 10/30. In both cases the recursive versions were used. For QuickSort the Partition function was altered to carry out the median of three rule (i.e the partitioning element was the median of $a[m]$, $a[\lfloor (m+p-1)/2 \rfloor]$ and $a[p-1]$). Each data set consisted of random integers in the range (0,1000). Tables 2.1 & 2.2 record the actual computing times in milliseconds. Table 2.1 displays the average computing times. For each n , 50 random

data sets were used. Table 2.2 shows the worst-case computing times for the 50 data sets.

Scanning the tables, we immediately see that QuickSort is faster than Merge Sort for all values. Even though both algorithms require $O(n \log n)$ time on the average, QuickSort usually performs well in practice.

Randomized Sorting Algorithms

Though algorithm QuickSort has an average time of $O(n \log n)$ on n elements, its worst-case time is $O(n^2)$. On the other hand it does not make use of any additional memory as does MergeSort. A Possible input on which QuickSort displays worst-case behavior is one in which the elements are already in sorted order. In this case the partition will be such that there will be only one element in one part and the rest of the elements will fall in the other part. The performance of any divide-and-conquer algorithm will be good if the resultant subproblems are as evenly sized as possible. Can QuickSort be modified so that it performs well on every input. The answer is yes. Is the technique of using the median of the three elements $a[p]$, $a[\lfloor (q+p)/2 \rfloor]$, and $a[q]$ the solution? It is possible to construct inputs for which even this method will take $\Omega(n^2)$ time.

The solution is the use of a randomizer. While sorting array $a[p:q]$; instead of picking $a[m]$, pick a random element as the partition element. The resultant randomized algorithm (RquickSort) works on any input and runs in an expected $O(n \log n)$ time, The code for RquickSort is as follows.

ALGORITHM : Randomized quick sort algorithm

Algorithm RquickSort(p,q)

// Sorts the elements $a[p], \dots, a[q]$ which reside in the global array $a[1:n]$ into ascending

// order, $a[n+1]$ is considered to be defined and must be $\geq$ all the elements in $a[1:n]$

```
{
    if (p < q) then
    {
        if ((q - p) > 5) then
            Interchange(a, Random() mod (q - p + 1) + p, p);

        J := Partition(a, p, q + 1);

        // j is the position of the partitioning element.

        RquickSort(p, j - 1);

        RquickSort(j + 1, q);
    }
}
```

Note that this is a Las Vegas algorithm since it will always output the correct answer. Every call to the randomizer Random

takes a certain amount of time. If there are only a very few elements to sort, the time taken by the randomizer may be comparable to the rest of the computation. For this reason, we involve the randomizer only if $(q-p) > 5$. But 5 is not a magic number ; in the machine employed, this seems to give the best result. In general this number should be determined empirically.

The proof of the fact that RquickSort has an expected $O(n \log n)$ time is the same as the proof of the average time of QuickSort. Let $A(n)$ be the average time of RquickSort on any input of n elements. Then the number of elements in the second part will be $0, 1, 2, \dots, n-2$, or $n-1$, all with an equal probability of $1/n$. Thus the recurrence relation for $A(n)$ will be

$$A(n) = \frac{1}{n} \sum_{1 \leq k \leq n} (A(k-1) + A(n-k)) + n + 1$$

Its solution is $O(n \log n)$.

RquickSort and QuickSort were evaluated on a SUN 10/30 workstation . Following table 2.3 displays the times for the two algorithms in milliseconds aveaged over 100 runs. For each n , the input considered was the sequence of numbers $1, 2, \dots, n$. As we can see from the table. RquickSort performs much better than QuickSort. Note that the times shown in this table for QuickSort are much more than the corresponding entries in table 2.1 and 2.2. The reason is that QuickSort makes (n^2) comparisons on inputs that are already in sorted order. However, on random inputs its average performance is very good.

Table 2.1 Average computing times for two sorting algorithms on random inputs.

n	1000	2000	3000	4000	5000
MergeSort	72.8	167.2	275.1	378.5	500.6
QuickSort	36.6	85.1	138.9	205.7	269.0
N	6000	7000	8000	9000	10000
MergeSort	607.6	723.4	811.5	949.2	1073.6
QuickSort	339.4	411.0	487.7	556.3	645.2

Table 2.2 Worst Case computing times for two sorting algorithms on random inputs.

N	1000	2000	3000	4000	5000
MergeSort	105.7	206.4	335.2	422.1	589.9
QuickSort	41.6	97.1	158.6	244.9	397.8
N	6000	7000	8000	9000	10000
MergeSort	691.3	794.8	889.5	1067.2	1167.6
QuickSort	383.8	497.3	569.9	616.2	738.1

Table 2.3: Comparison of Quicksort and RQuickSort on the input $a[i] = 1; 1 \leq i \leq n$, times are in milliseconds.

N	1000	2000	3000	4000	5000
QuickSort	195.5	759.2	1728	3165	4829
RQuickSort	9.4	21.0	30.5	41.6	52.8

The performance of R quick Sort can be improved in various ways. For example, we could pick a small number of the elements in the array $a[]$ randomly and use the median of these elements as the partition element. These randomly chosen elements form a random sample of the array elements. We would expect that the medias of the sample would also be an approximately median of the array and hence result in an approximately even partitioning of the array.

An even more generalized version of the above random sampling technique is shown below.

Algorithm Rsort(a, n)

// Sort the elements $a[1:n]$

{

 Randomly sample s elements from $a[]$;

 Sort this sample;

 Partition the input using the sorted sample as partition keys;

Sort each part separately

}

Here we choose a random sample S of s elements from the input sequence x and sort them using Heap Sort, Merge Sort, or any other sorting algorithm. Let $l_1, l_2, \dots, l_s$ be the sorted sample. We partition X into $s+1$ parts using the sorted sample as partition keys. In particular $X_1 = \{x \in X / x \leq l_1\}$; $X_i = \{x \in X / l_{i-1} < x \leq l_i\}$; for $i=2, 3, \dots, s$; and $X_{s+1} = \{x \in X / x \geq l_s\}$. After having partitioned X into $s+1$ parts, we sort each part recursively. For a proper choice of s , the number of comparisons made in this algorithm is only $n \log n + \tilde{O}(n \log n)$.

Choose $S = \frac{n}{\log^2 n}$. The sample can be sorted in $O(s \log$

$s) = O(n/\log n)$ time and comparisons if we use Heap sort or merge sort. If we store the stored sample elements in an array, say $b[]$, for each $x \in X$, we can determine which part X_i it belongs to in $\leq \log n$ comparisons using binary search on $b[]$. Thus the partitioning process takes $n \log n + O(n)$ comparisons. Using Heap Sort or MergeSort to sort each of the X_i 's, the total cost of sorting the X_i 's is

$$\sum_{i=1}^{s+1} O(|X_i| \log |X_i|) = \max_{1 \leq i \leq s+1} \{ \log |X_i| \} \sum_{i=1}^{s+1} O(|X_i|)$$

E 5 Since each $|X_i|$ is $\tilde{O}(\log^3 n)$, the cost of sorting the $s+1$ parts is $\tilde{O}(n \log \log n) = \tilde{O}(n \log n)$.

2.4 SELECTION

In this problem, we are given n elements $a[1:n]$ and are required to determine the k th smallest element. If the partitioning element v is positioned at $a[j]$, then $j-1$ elements are less than or equal to $a[j]$ and $n-j$ elements are greater than or equal to $a[j]$. Hence if $k < j$, then the k th smallest element is the $(k-j)$ th smallest element in $a[1:j-1]$; if $k = j$ then $a[j]$ is the k th smallest element in $a[j+1:n]$. The resulting algorithm is function Select1. This function places the k th smallest element into position $a[k]$ and partitions the remaining elements so that $a[i] \leq a[k]$, $1 \leq i < k$, and $a[i] \geq a[k]$, $k < i \leq n$.

Example : Let us simulate Select1 as it operates on the same array used to test Partition. The array has the nine elements 65, 70, 75, 80, 85, 60, 55, 50 and 45, with $a[10] = \infty$. If $k = 5$ then the first call of Partition will be sufficient since 65 is placed into $a[i]$. Instead assume that we are looking for the seventh-smallest element of a that is $k = 7$. The invocation of Partition is Partition(6,10).

a:	(5)	(6)	(7)	(8)	(9)	(10)
	65	85	80	75	70	$+\infty$
	65	70	80	75	85	$+\infty$

This last call of Partition has uncovered the 9th smallest element of a . The next invocation is Partition(6,9)

a:	(5)	(6)	(7)	(8)	(9)	(10)
	65	<u>70</u>	80	75	85	$+\infty$
	65	70	80	75	85	$+\infty$

This time the 6th element has been found . Since $k > j$, another call to Partition is made, Partition(7,9).

a :	(5)	(6)	(7)	(8)	(9)	(10)
	65	70	<u>80</u>	<u>75</u>	85	$+\infty$
	65	70	75	80	85	$+\infty$

now, 80 is the Partition value and is correctly placed at $a[8]$, however Select1 has still not found the 7th smallest element. It needs one more call to Partion Which is Partition(7,8). This performs only an interchange between $a[7]$ and $a[8]$ and returns, having found the correct value.

In analyzing Select1 we make the same assumptions that were made for Quick Sort:

1. The n elements are distinct
2. The input distribution is such that the Partition element can be the i th smallest element of $a[m:p-1]$ with an equal probability for each i , $1 \leq i \leq p-m$

Partition requires $O(p-m)$ time. On each successive call to Partition, either m increases by at least one or j decreases by at least one. Initially $m=1$ and $j=n+1$. Hence, at most n calls to Partition can be made. Thus, the worst case complexity of Select2 is $O(n^2)$. The time is $\Omega(n^2)$, for example, when the input $a[1:n]$ is such that the partitioning element on the i th call to Partition is the i th smallest element and $k=n$. In this case, m increases by one

following each call to partition and j remains unchanged, Hence,

n calls are made for a total cost of $O\sum_1^n i = O(n^2)$. The average computing time of Select1 is, however, only $O(n)$. Before proving this fact, we specify more precisely what we mean by the average time.

Let $T_A^k(n)$ be the average time to find the k th smallest in $a[1:n]$. This average is taken over all $n!$ different permutations of n distinct elements. Now we define $T_A(n)$ and $R(n)$ as follows.

$$T_A(n) = \frac{1}{n} \sum_{1 \leq k \leq n} T_A^k(n)$$

$$\text{and } R(n) = \max_k \{T_A^k(n)\}$$

$T_A(n)$ is the average computing time of Select1. It is easy to see that $T_A(n) \leq R(n)$. We are now ready to show that $T_A(n) = O(n)$.

2.5 STRASSEN'S MATRIX MULTIPLICATION :

Let A and B be two $n \times n$ matrices. The product matrix $C = AB$ is also an $n \times n$ matrix i, j th element is formed by taking the elements in the i th row of A and the j th column of B and multiplying them to get

$$C(i, j) = \sum_{1 \leq k \leq n} A(i, k)B(k, j)$$

for all i and j between 1 and n . To compute $C(i,j)$ using this formula, we need n multiplications, As the matrix C has n^2 elements, the time for the resulting matrix multiplication algorithm, which we refer to as the conventional method is $\Theta(n^3)$.

The divide-and-conquer strategy suggests another way to compute the product of two $n \times n$ matrices. For simplicity we assume that n is a power of 2, that is, that there exists a nonnegative integer k such that $n = 2^k$. In case n is not a power of two, then enough rows and columns of zeros can be added to both A and B so that the resulting dimensions are a power of two. Imagine that A and B are each partitioned into four square submatrices, each submatrix having dimension $n/2 \times n/2$. Then the product AB can be computed by using the above formula for the product of 2×2 matrices if AB is

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} \quad \dots(2.1)$$

then

$$C_{11} = A_{11} B_{11} + A_{12} B_{21}$$

$$C_{12} = A_{11} B_{12} + A_{12} B_{22}$$

$$C_{21} = A_{21} B_{11} + A_{22} B_{21} \quad \dots(2.2)$$

$$C_{22} = A_{21} B_{12} + A_{22} B_{22}$$

If $n=2$, then formula (2.1) and (2.2) are computed using a multiplication operation for the elements of A and b . These

elements are typically floating point numbers. For $n > 2$, the elements of C can be computed using matrix multiplication and addition operations applied to matrices of size $n/2 \times n/2$. Since n is a power of 2, these matrix products can be recursively computed by the same algorithm we are using for the $n \times n$ case. This algorithm will continue applying itself to smaller-sized submatrices until n becomes suitably small ($n=2$) so that the product is computed directly.

To compute AB using (2.2), we need to perform eight multiplications of $n/2 \times n/2$ matrices and four additions of $n/2 \times n/2$ matrices. Since two $n/2 \times n/2$ matrices can be added in time cn^2 for some constant c , the overall computing time $T(n)$ of the resulting divide-and-conquer algorithm is given by the recurrence.

$$T(n) = \begin{cases} b & n \leq 2 \\ T(n/2) + cn^2 & n > 2 \end{cases}$$

Where b and c are constants.

This recurrence can be solved in the same way as earlier recurrence to obtain $T(n) = O(n^3)$. Hence no improvement over the conventional method has been made. Since matrix multiplications are more expensive than matrix additions ($O(n^3)$ versus $O(n^2)$), we can attempt to reformulate the equations for C_{ij} so as to have fewer multiplications and possibly more additions. Volker Strassen has discovered a way to compute the C_{ij} 's using only 7 multiplications and 18 additions or subtractions. His method

involves first computing the seven $n/2 \times n/2$ matrices P, Q, R, S, T, U, and V as following.. Then the C_{ij} 's are computed using the other formulae. As can be seen, P, Q, R, S, T, U, and V can be computed using 7 matrix multiplications and 10 matrix additions or subtractions. The C_{ij} 's require an additional 8 additions or subtractions.

$$P = (A_{11} + A_{22}) (B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22}) B_{11}$$

$$R = A_{11} (B_{12} - B_{22})$$

$$S = A_{22} (B_{21} - B_{11})$$

$$T = (A_{11} + A_{12}) B_{22}$$

$$U = (A_{21} - A_{11}) (B_{11} + B_{12})$$

$$V = (A_{12} - A_{22}) (B_{21} + B_{22})$$

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

The resulting recurrence relation for $T(n)$ is

$$T(n) = \begin{cases} b & n \leq 2 \\ 7T(n/2) + an^2 & n > 2 \end{cases}$$

where a and b are constants. Working with this formula, we get

$$\begin{aligned}
 T(n) &= an^2[1 + 7/4 + (7/4)^2 + \dots + (7/4)^{k-1}] + 7^k T(1) \\
 &\leq cn^2(7/4)^{\log_2 n} + 7^{\log_2 n}, \text{ } c \text{ a constant} \\
 &= cn^{\log_2 4 + \log_2 7 - \log_2 4} + n^{\log_2 7} \\
 &= O(n^{\log_7 2}) \\
 &\approx O(n^{2.81})
 \end{aligned}$$

Unit-II
Lesson - 3

THE GREEDY METHOD

OBJECTIVE :

After studying this lesson you can able to understand the following

- General Method of Greedy method
- Knapsack problem
- Tree Vertex Splitting
- Job sequencing with Deadline
- Single Source shortest path

3.1 THE GENERAL METHOD

The greedy method is perhaps the most straightforward design technique we consider in this text, and what's more it can be applied to a wide variety of problems. Most, though not all, of these problems have n inputs and require us to obtain a subset that satisfies some constraints. Any subset that satisfies these constraints is called a feasible solution. We need to find a feasible solution that either maximizes or minimizes a given objective function. A feasible solution that does this is called optimal solution. There is usually an obvious way to determine a feasible solution but not necessarily an optimal solution.

The greedy method suggests that one can devise an algorithm that works in stages, considering one input at a time. At each stage, a decision is made regarding whether a particular input is in an optimal solution. This is done by considering the inputs in an order determined by some selection procedure. If the inclusion of the next input into the partially constructed optimal solution will result in an infeasible solution, this input is not added to the partial solution. Otherwise, it is added. The selection procedure itself is based on some optimization measure. This measure may be the objective function. In fact, several different optimization measures may be plausible for a given problem. Most of these, however, will result in algorithms that generate suboptimal solutions. This version of the greedy technique is called the subset paradigm.

We can describe the subset paradigm abstractly, but more precisely than above, by considering the control abstraction in Algorithm 3.1

The function `Select` selects an input from $a[]$ and removes it. The selected input's value is assigned to x . `Feasible` is a Boolean-valued function that determines whether x can be included into the solution vector. The function `Union` combines x with the solution and updates the objective function. The function `Greedy` describes the essential way that a greedy algorithm will look once a particular problem is chosen and the functions `Select`, `Feasible`, and `Union` are properly implemented.

```

1   Algorithm Greedy (a,n)
2   // a[1:n] contains the n inputs
3   {
4       solution:= $\emptyset$ ; // Initialize the solution.
5       for i:=1 to n do
6           {
7               x:=Select(a);
8               if Feasible(solution, x) then
9                   solution := Union(solution, x);
10          }
11       return solution;
12  }
```

Algorithm 3.1 Greedy method control abstraction for the subset paradigm

For problems that do not call for the selection of an optimal subset, in the greedy method we make decisions by considering the inputs in some order. Each decision is made using an optimization criterion that can be computed using decision already made. Call this version of the greedy method the ordering paradigm.

3.2 KNAPSACK PROBLEM

Let us try to apply the greedy method to solve the knapsack problem. We are given n objects and a knapsack or bag. Object i has a weight w_i and the knapsack has a capacity m . If a fraction x_i , $0 \leq x_i \leq 1$, of object i is placed into the knapsack, then a profit of $p_i x_i$ is earned. The objective is to obtain a filling of the knapsack that maximizes the total profit earned. Since the knapsack capacity is m , we require the total weight of all chosen objects to be at most m . Formally, the problem can be stated as

$$\text{maximize } \sum_{1 \leq i \leq n} p_i x_i \quad (3.1)$$

$$\text{Subject } \sum_{1 \leq i \leq n} w_i x_i \leq m \quad (3.2)$$

$$\text{and } 0 \leq x_i \leq 1, 1 \leq i \leq n \quad (3.3)$$

The profits and weights are positive numbers.

A feasible solution (or filling) is any set $(x_1, \dots, x_n)$ satisfying (3.2) and (3.3) above. An optimal solution is a feasible solution for which (3.1) is maximized.

Example 3.1. Consider the following instance of the knapsack problem: $n=3, m=20$, $(p_1, p_2, p_3) = (25, 24, 15)$ and $(w_1, w_2, w_3) = (18, 15, 10)$. Four feasible solutions are:

	(x_1, x_2, x_3)	$\sum w_i x_i$	$\sum p_i x_i$
1.	$(1/2, 1/3, 1/4)$	16.5	24.25
2.	$(1, 2/15, 0)$	20	28.2
3.	$(0, 2/3, 1)$	20	31
4.	$(0, 1, 1/2)$	20	31.5

Of these four feasible solutions, solution 4 yields the maximum profit. As we shall soon see, this solution is optimal for the given problem instance.

Lemma 3.1 In case the sum of all the weights is $\leq m$, then $x_i = 1$, $1 \leq i \leq n$ is an optimal solution.

So let us assume the sum of weights exceeds m . Now all the x_i 's cannot be 1. Another observation to make is:

Lemma 3.2 All optimal solutions will fill the knapsack exactly.

Lemma 3.2 is true because we can always increase the contribution of some object i by a fractional amount until the total weight is exactly m .

Note that the knapsack problem calls for selecting a subset of the objects and hence fits the subset paradigm. In addition to selecting a subset, the knapsack problem also involves the selection of an x_i for each object. Several simple greedy strategies to obtain feasible solutions whose sums are identically m suggest themselves.

First, we can try to fill the knapsack by including next the object with largest profit. If an object under consideration doesn't fit, then a fraction of it is included to fill the knapsack. Thus each time an object is included (except possibly when the last object is included) into the knapsack, we obtain the largest possible increase in profit value. Note that if only a fraction of the last object is included, then it may be possible to get a bigger increase by using a different object. For example, if we have two units of space left and two objects with $(p_i=4, w_i=4)$ and $(p_j=3, w_j=2)$ remaining, then using j is better than using half of i .

Object one has the largest profit value ($p_1=25$). So it placed into the knapsack first. Then $x_1=1$ and a profit of 25 is earned. Only 2 units of knapsack capacity are left. Object two has the next largest profit ($p_2=24$). However, $w_2=15$ and it doesn't fit into the knapsack. Using $x_2=2/15$ fills the knapsack exactly with part of object 2 and the value of the resulting solution is 28.2. This is solution 2 and it is readily seen to be suboptimal. The method used to obtain this solution is termed a greedy method because at each step(except possibly the last one) we chose to introduce that object which would increase the objective function value the most. However, this greedy method did not yield an optimal solution. Note that even if we change the above strategy so that in the last step the objective function increases by as much as possible, an optimal solution is not obtained for Example 3.1

We can formulate at least two other greedy approaches attempting to obtain optimal solutions. From the preceding example, we note that considering objects in order of nonincreasing

profit values does not yield an optimal solution because even though the objective function value takes on large increases at each step, the number of steps is few as the knapsack capacity is used up at a rapid rate. So, let us try to be greedy with capacity and use it up as slowly as possible. This requires us to consider the objects in order of nondecreasing weights w_i . Using Example 4.1, solution 3 results. This too is suboptimal. This time, even though capacity is used slowly, profits aren't coming in rapidly enough.

Thus, our next attempt is an algorithm that strives to achieve a balance between the rate at which profit increases and the rate at which capacity is used. At each step we include that object which has the maximum profit per unit of capacity used. This means that objects are considered in order of the ratio p_i/w_i . Solution 4 of Example 3.1 is produced by this strategy. If the objects have already been sorted into nonincreasing order of p_i/w_i , then function GreedyKnapsack (Algorithm 3.2) obtains, solutions corresponding to this strategy. Note that solutions corresponding to the first two strategies can be obtained using this algorithm if the objects are initially in the appropriate order. Disregarding the time to initially sort the objects, each of the three strategies outlined above requires only $O(n)$ time.

We have seen that when one applies the greedy method to the solution of the knapsack problem, there are at least three different measures one can attempt to optimize when determining which objects to include next. These measures are total profit, capacity used, and the ratio of accumulated profit to capacity used. Once an optimization measure has been chosen, the greedy

```

1  Algorithm GreedyKnapsack(m,n)
2  //p[1:n] and w[1:n] contain the profits and weights
   respectively
3  // of the n objects ordered such that  $p[i]/w[i] \geq p[i+1]/w[i+1]$ .
4  // m is the knapsack size and x[1:n] is the solution vector.
5  {
6      for i:=1 to n do x[i]:=0.0; // Initialize x.
7      U:=m;
8      for i:=1 to n do
9          {
10             if (w[i]>U) then break;
11             x[i]:=1.0; U:=U-w[i];
12          }
13      if (i≤n) then x[i]:=U/w[i];
14  }
```

Algorithm 3.2 Algorithm for greedy strategies for the knapsack problem

method suggests choosing objects for inclusion into the solution in such a way that each choice optimizes the measure at that time. Thus a greedy method using profit as its measure will at each step choose an object that increases the profit the most. If the capacity measure is used, the next object included will increase

this the least. Although greedy-based algorithms using the first two measures do not guarantee optimal solutions for the knapsack problem, Theorem 3.1 shows that a greedy algorithm using strategy 3 always obtains an optimal solution. This theorem is proved using the following technique:

Compare the greedy solution with any optimal solution. If the two solutions differ, then find the first x_i at which they differ. Next, it is shown how to make the x_i in the optimal solution equal to that in the greedy solution without any loss in the total value. Repeated use of this transformation shows that the greedy solution is optimal.

This technique of proving solutions optimal is used often in this text. Hence, you should master it at this time.

Theorem 3.1 If $p_1/w_1 \geq p_2/w_2 \geq \dots \geq p_n/w_n$, then GreedyKnapsack generates an optimal solution to the given instance of the knapsack problem.

Proof: Let $x = (x_1, \dots, x_n)$ be the solution generated by GreedyKnapsack. If all the x_i equal one, then clearly the solution is optimal. So, let j be the least index such that $x_j \neq 1$. From the algorithm it follows that $x_i = 1$ for $1 \leq i \leq j$, $x_i = 0$ for $j < i \leq n$, and $0 \leq x_j < 1$. Let $y = (y_1, \dots, y_n)$ be an optimal solution. From Lemma 3.2, we can assume that $\sum w_i y_i = m$. Let k be the least index such that $y_k \neq x_k$. Clearly, such a k must exist. It also follows that $y_k < x_k$. To see this, consider the three possibilities $k < j$, $k = j$, or $k > j$.

1. If $k < j$, then $x_k = 1$. But, $y_k \neq x_k$, and so $y_k < x_k$.
2. If $k = j$, then since $\sum w_i x_i = m$ and $y_i = x_i$ for $1 \leq i \leq j$, it follows that either $y_k < x_k$ or $\sum w_i y_i > m$.
3. If $k > j$, then $\sum w_i y_i > m$, and this is not possible.

Now suppose we increase y_k to x_k and decrease as many of $(y_{k+1}, \dots, y_n)$ as necessary so that the total capacity used is still m . This results in a new solution $z = (z_1, \dots, z_n)$ with $z_i = x_i$, $1 \leq i \leq k$, and $\sum_{k < i \leq n} w_i (y_i - z_i) = w_k (z_k - y_k)$. Then, for z we have

$$\begin{aligned}
 \sum_{1 \leq i \leq n} p_i z_i &= \sum_{1 \leq i \leq n} p_i y_i + (z_k - y_k) w_k \frac{p_k}{w_k} - \sum_{k < i \leq n} (y_i - z_i) w_i \frac{p_i}{w_i} \\
 &\geq \sum_{1 \leq i \leq n} p_i y_i + [(z_k - y_k) w_k - \sum_{k < i \leq n} (y_i - z_i) w_i] \frac{p_k}{w_k} \\
 &= \sum_{1 \leq i \leq n} p_i y_i
 \end{aligned}$$

If $\sum p_i z_i > \sum p_i y_i$ then y could not have been an optimal solution. If these sums are equal, then either $z = x$ and x is optimal, or $z \neq x$. In the latter case, repeated use of the above argument will either show that y is not optimal, or transform y into x thus show that x too is optimal.

3.3 TREE VERTEX SPLITTING

Consider a directed binary tree each edge of which is labeled with a real number (called its weight). Trees with edge

weights are called weighted trees. A weighted tree can be used, for example, to model a distribution network in which electric signals or commodities such as oil are transmitted. Nodes in the tree correspond to receiving stations and edges correspond to transmission lines. It is conceivable that in the process of transmission some loss occurs (drop in voltage in the case of electric signals or drop in pressure in the case of oil). Each edge in the tree is labeled with the loss that occurs in traversing that edge. The network may not be able to tolerate losses beyond a certain level. In places where the loss exceeds the tolerance level, boosters have to be placed. The Tree Vertex Splitting Problem (TVSP) is to determine an optimal placement of boosters. It is assumed that the boosters can only be placed in the nodes of the tree.

The TVSP can be specified more precisely as follows: Let $T = (V, E, w)$ be a weighted directed tree, where V is the vertex set, E is the edge set, and w is the weight function for the edges. In particular, $w(i, j)$ is the weight of the edge $\langle i, j \rangle \in E$. The weight $w(i, j)$ is undefined for any $\langle i, j \rangle \notin E$. A source vertex is a vertex with in-degree zero, and a sink vertex is a vertex with out-degree zero. For any path P in the tree, its delay, $d(P)$, is defined to be the sum of the weights on that path. The delay of the tree T , $d(T)$, is the maximum of all the path delays.

Let T/X be the forest that results when each vertex u in X is split into two nodes u^i and u^o such that all the edges $\langle u, j \rangle \in E$ ($\langle j, u \rangle \in E$) are replaced by edges of the form $\langle u^o, j \rangle$ ($\langle j, u^i \rangle$). In other words, outbound edges from u now leave from u^o and inbound edges to u now enter at u^i . Figure 3.1 shows a tree before

and after splitting the node 3. A node that gets a split corresponds to a booster station. The TVSP is to identify a set $X \subseteq V$ of minimum cardinality for which $d(T/X) \leq \delta$, for some specified tolerance limit δ . Note that the TVSP has a solution only if the maximum edge weight is $= \delta$. Note that the TVSP naturally fits the subset paradigm.

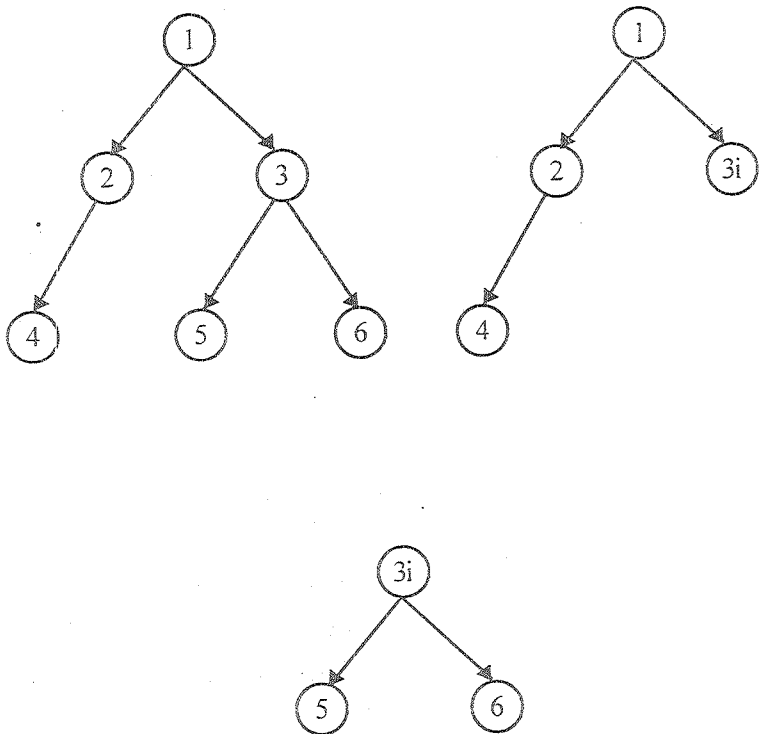


Figure 3.1 A Tree before and after splitting the node 3

Given a weighted tree $T(V, E, w)$ and a tolerance limit δ , any subset X of V is a feasible solution if $d(T/X) = \delta$. Given an X , we can compute $d(T/X)$ in $O(|V|)$ time. A trivial way of solving the TVSP is to compute $d(T/X)$ for each possible subset X of V . But there are $2^{|V|}$ such subsets! A better algorithm can be obtained using the greedy method.

For the TVSP, the quantity that is optimized (minimized) is the number of nodes in X . A greedy approach to solving this problem is to compute for each node $u \in V$, the maximum delay $d(u)$ from u to any other node in its subtree. If u has a parent v such that $d(u) + w(u, v) > \delta$, then the node u gets split and $d(u)$ is set to zero. Computation proceeds from the leaves toward the root.

In the tree of Figure 3.2 let $\delta = 5$. For each of the leaf nodes 7, 8, 5, 9, and 10 the delay is zero. The delay for any node is computed only after the delays for its children have been determined. Let u be any node and $C(u)$ be the set of all children of u . Then $d(u)$ is given by

$$d(u) = \max_{v \in C(u)} \left\{ d(u) + w(u, v) \right\}$$

Using the above formula, for the tree of Figure 3.2 $d(4) = 4$. Since $d(4) + w(2, 4) = 6 > \delta$, node 4 gets split. We set $d(4) = 0$. Now $d(2)$ can be computed and is equal to 2. Since $d(2) + w(1, 2)$ exceeds δ , node 2 gets split and $d(2)$ is set to zero. Then $d(6)$ is equal to 3. Also, since $d(6) + w(3, 6) > \delta$, node 6 has to

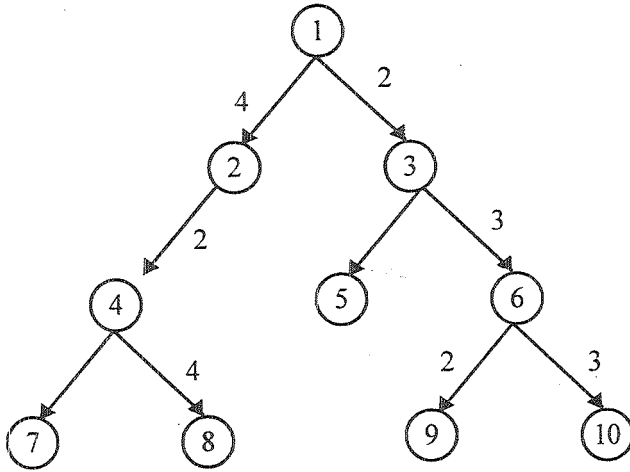


Fig. 3.2 An Example Tree

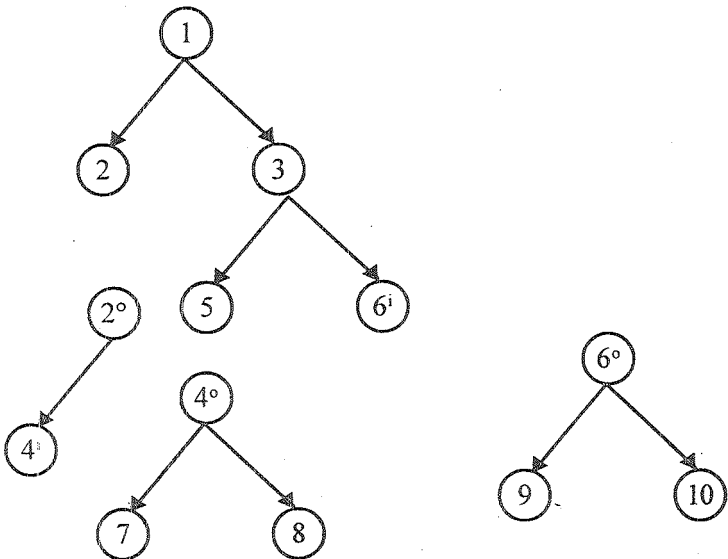


Fig. 3.3 The Final tree after splitting the nodes 2,4,6

be split. Set $d(6)$ to zero. Now $d(3)$ is computed as 3. Finally, $d(1)$ is computed as 4.

Figure 3.3 shows the final tree that results after splitting the nodes 2, 4, and 6. This algorithm is described in Algorithm 3.3, which is invoked as $\text{TVS}(\text{root}, \delta)$, root being the root of the tree. The order in which TVS visits (i.e., computes the delay values of) the nodes of the tree is called the post order.

```

1   Algorithm TVS(T,  $\delta$ )
2   // Determine and output the nodes to be split
3   //  $w()$  is the weighting function for the edges
4   {
5       if ( $T \neq 0$ ) then
6       {
7            $d[T] := 0$ ;
8           for each child  $v$  of  $T$  do
9           {
10              TVS( $v, \delta$ );
11               $d[T] := \max \{d[T], d[v] + w(T, v)\}$ ;
12          }
13          if (( $T$  is not the root) and
14              ( $d[T] + w(\text{parent}(T), T) > \delta$ )) then

```

```

15      {
16          write (T); d[T]:=0;
17      }
18      }
19  }
```

Algorithm 3.3 The tree vertex splitting algorithm

Algorithm TVS takes $\theta(n)$ time, where n is the number of nodes in the tree. This can be seen as follows: When TVS is called on any node T , only a constant number of operations are performed (excluding the time taken for the recursive calls). Also, TVS is called only once on each node T in the tree.

Algorithm 3.4 is a revised version of Algorithm 3.3 for the special case of directed binary trees. A sequential representation of the tree has been employed. The tree is stored in the array `tree[]` with the root at `tree[1]`. Edge weights are stored in the array `weight[]`. If `tree[i]` has a tree node, the weight of the incoming edge from its parent is stored in `weight[i]`. The delay of node i is stored in `d[i]`. The array `d[]` is initialized to zero at the beginning. Entries in the arrays `tree[]` and `weight[]` corresponding to nonexistent nodes will be zero. As an example, for the tree of Figure 4.2, `tree[]` will be set to $\{1, 2, 3, 0, 4, 5, 0, 0, 7, 8, 9, 10\}$ starting at cell 1. Also, `weight[]` will be set to $\{0, 4, 2, 0, 2, 1, 3, 0, 0, 1, 4, 0, 0, 2, 3\}$ at the beginning, starting from cell 1. The algorithm is invoked as `TVS(1,  $\delta$ )`. Now we show that `TVS(Algorithm 4.3)` will always split a minimal number of nodes.

```

1  Algorithm TVS(i,δ)
2  //Determine and output a minimum cardinality split set.
3  // The tree is realized using the sequential representation.
4  // Root is at tree[1]. N is the largest number such that
5  // tree[n] has a tree node.
6  {
7      if (tree[i]≠0) then // If the tree is not empty
8          if (2i>N) then d[i]:=0; // i is a leaf.
9          else
10             {
11                 TVS(2i,δ);
12                 d[i]:=max(d[i],d[2i] + weight[2i]);
13                 if (2i + 1 ≤ N) then
14                     {
15                         TVS(2i + 1,δ );
16                         d[i]:=max(d[i],d[2i + 1] +
17                             weight[2i + 1]);
18                     }
19             }
20 }

```

```

19          if(((tree[i] ≠ 1) and (d[i] + weight[i] > δ)) then
20              {
21                  write(tree[i]; d[i]:=0;
22              }
23  }
```

Algorithm 3.4 TVS for the special case of binary trees

Theorem 3.2 : Algorithm TVS outputs a minimum cardinality set U such that $d(T/U) \leq \delta$ on any tree T , provided no edge of T has weight $> \delta$.

Proof: The proof is by induction on the number of nodes in the tree. If the tree has a single node, the theorem is true. Assume the theorem for all trees of size $\leq n$. We prove it for trees of size $n+1$ also.

Let T be any tree of size $n+1$ and let U be the set of nodes split by TVS. Also let W be a minimum cardinality set such that $d(T/W) \leq \delta$. We have to show that $|U| \leq |W|$. If $|U| = 0$, this is true. Otherwise, let x be the first vertex split by TVS. Let T_x be the subtree rooted at x . Let T' be the tree obtained from T by deleting T_x except for x . Note that W has to have at least one node, say y , from T_x . Let $W' = W - \{y\}$. If there is W^* such that $|W^*| < |W'|$ and $d(T'/W^*) \leq \delta$, then since $d(T/(W^* + \{x\})) \leq \delta$, W is not a minimum cardinality split set for T . Thus, W' has to be a minimum cardinality split set such that $d(T'/W') \leq \delta$.

If algorithm TVS is run on tree T' , the set of split nodes output is $U \setminus \{x\}$. Since T' has $\leq n$ nodes, $U - \{x\}$ is a minimum cardinality split set for T' . This in turn means that $|W'| \geq |U| - 1$. In other words, $|W| \geq |U|$.

3.4 JOB SEQUENCING WITH DEADLINES

We are given a set of n jobs. Associated with job i is an integer deadline $d_i \geq 0$ and a profit $p_i > 0$. For any job i the profit p_i is earned iff the job is completed by its deadline. To complete a job, one has to process the job on a machine for one unit of time. Only one machine is available for processing jobs. A feasible solution for this problem is a subset J of jobs such that each job in this subset can be completed by its deadline. The value of a feasible solution J is the sum of the profits of the jobs in J , or $\sum_{i \in J} p_i$. An optimal solution is a feasible solution with maximum value. Here again, since the problem involves the identification of a subset, it fits the subset paradigm.

Example 3.2 Let $n=4$ $(p_1, p_2, p_3, p_4) = (100, 10, 15, 27)$ and $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$. The feasible solutions and their values are:

	feasible solution	processing sequence	value
1.	(1,2)	2,1	110
2.	(1,3)	1,3 or 3,1	115
3.	(1,4)	4,1	127

4.	(2,3)	2,3	25
5.	(3,4)	4,3	42
6.	(1)	1	100
7.	(2)	2	10
8.	(3)	3	15
9.	(4)	4	27

Solution 3 is optimal. In this solution only jobs 1 and 4 are processed and the value is 127. These jobs must be processed in the order job 4 followed by job 1. Thus the processing of job 4 begins at time zero and that of job 1 is completed at time 2.

To formulate a greedy algorithm to obtain an optimal solution, we must formulate an optimization measure to determine how the next job is chosen. As a first attempt we can choose the objective function $\sum_i \epsilon_i p_i$ as our optimization measure. Using this measure, the next job to include is the one that increases $\sum_i \epsilon_i p_i$ the most, subject to the constraint that the resulting J is a feasible solution. This requires us to consider jobs in nonincreasing order of the p_i 's. Let us apply this criterion to the data of Example 4.2. We begin with $J = \emptyset$ and $\sum_i \epsilon_i p_i = 0$. Job 1 is added to J as it has the largest profit and $J = \{1\}$ is a feasible solution. Next, job 4 is considered. The solution $J = \{1, 4\}$ is also feasible. Next, job 3 is considered for inclusion into J . It is discarded as $J = \{1, 2, 4\}$ is not feasible. Hence, we are left with the solution $J = \{1, 4\}$ with value 127. This is the optimal solution for the given problem instance. Theorem 4.4 proves that the greedy algorithm just described always obtains an optimal solution to this sequencing problem.

Before attempting the proof, let us see how we can determine whether a given J is a feasible solution. One obvious way is to try out all possible permutations of the jobs in J and check whether the jobs in J can be processed in any one of these permutations (sequences) without violating the deadlines. For a given permutation $\sigma = i_1, i_2, i_3, \dots, i_k$, this is easy to do, since the earliest time job $i_q, 1 \leq q \leq k$, will be completed is q . If $q > d_{i_q}$, then using σ , at least job i_q will not be completed by its deadline. However, if $|J| = i$, this requires checking $i!$ permutations. Actually, the feasibility of a set J can be determined by checking only one permutations in which jobs are ordered in nondecreasing order of deadlines.

Theorem 3.3. Let J be a set of k jobs and $\sigma = i_1, i_2, \dots, i_k$ a permutation of jobs in J such that $d_{i_1} \leq d_{i_2} \leq \dots \leq d_{i_k}$. Then J is a feasible solution iff the jobs in J can be processed in the order σ without violating any deadline.

Proof: Clearly, if the jobs in J can be processed in the order σ without violating any deadline, then J is a feasible solution. So, we have only to show that if J is feasible, then σ represents a possible order in which the jobs can be processed. If J is feasible, then there exists $\sigma' = r_1, r_2, \dots, r_k$, such that $d_{r_q} \geq q, 1 \leq q \leq k$. Assume $\sigma' \neq \sigma$. Then let a be the least index such that $r_a \neq i_a$. Let $r_b = i_a$. Clearly, $b > a$. In σ' we can interchange r_a and r_b . Since $d_{r_a} \geq d_{r_b}$, the resulting permutation $\sigma'' = s_1, s_2, \dots, s_k$ represents an order in which the jobs can be processed without violating a deadline. Continuing in this way, σ' can be transformed into σ without violating any deadline. Hence, the theorem is proved.

Theorem 3.3 is true even if the jobs have different processing times $t_i \geq 0$.

Theorem 3.4. The greedy method described above always obtains an optimal solution to the job sequencing problem.

Proof: Let (p_i, d_i) , $1 \leq i \leq n$, define any instance of the job sequencing problem. Let I be the set of jobs selected by the greedy method. Let J be the set of jobs in an optimal solution. We now show that both I and J have the same profit values and so I is also optimal. We can assume $I \neq J$ as otherwise we have nothing to prove. Note that if $J \subset I$, then J cannot be optimal. Also, the case $I \subset J$ is ruled out by the greedy method. So, there exist jobs a and b such that $a \in I$ and $a \notin J$, $a \in J$, $b \in I$ and $b \notin J$. Let a be the highest profit job such that $a \in I$ and $a \notin J$. It follows from the greedy method that $p_a \geq p_b$ for all jobs b that are in J but not in I . To see this, note that if $p_b > p_a$, then the greedy method would consider job b before job a and include it into I .

Now, consider feasible schedules S_I and S_J for I and J respectively. Let i be a job such that $i \in I$ and $i \notin J$. Let i be scheduled from t to $t+1$ in S_I and t' to $t'+1$ in S_J . If $t < t'$, then we can interchange the job (if any) scheduled in $[t', t'+1]$ in S_I with i . If no job is scheduled in $[t', t'+1]$ in I , then i is moved to $[t', t'+1]$. The resulting schedule is also feasible. If $t' < t$, then a similar transformation can be made in S_J . In this way, we can obtain schedules S'_I and S'_J with the property that all jobs common to I and J are scheduled at the same time. Consider the interval $[t_a, t_a+1]$ in S'_I in which the job a (defined above) is scheduled. Let b be the job (if any) scheduled in S'_J in this interval. From the

choice of a , $p_a \geq p_b$. Scheduling a from t_a to t_a+1 in S'_j and discarding job b gives us a feasible schedule for job set $J' = J - \{b\} \cup \{a\}$. Clearly, J' has a profit value no less than that of J and differs from I in one less job than J does.

By repeatedly using the transformation just described, J can be transformed into I with no decrease in profit value. So I must be optimal.

A high-level description of the greedy algorithm just discussed appears as Algorithm 3.5. This algorithm constructs an optimal set J of jobs that can be processed by their due times. The selected jobs can be processed in the order given by Theorem 3.3.

Now, let us see how to represent the set J and how to carry out the test of lines 7 and 8 in algorithm 3.5. Theorem 3.3 tells us how to determine whether all jobs in $J \cup \{i\}$ can be computed by their deadlines. We can avoid sorting the jobs in J each time by keeping the jobs in J ordered by deadlines. The set J itself can be represented by a 1d array $J[1..k]$ such that $J[r]$, $1 \leq r \leq k$ are the jobs in J and $d[J[1]] \leq d[J[2]] \leq \dots \leq d[J[k]]$. To test whether $J \cup \{i\}$ is feasible, we have just to insert i into J preserving the deadline ordering and then verify that $d[J[r]] \leq r$, $1 \leq r \leq k+1$. The insertion of i into J is simplified by the use of a fictitious job 0 with $d[0]=0$ and $J[0]=0$. Note also that if job i is to be inserted at position q , then only the positions of jobs $J[q]$, $J[q+1]$, ..., $J[k]$ are changed after the insertion. Hence, it is necessary to verify only that these jobs (and also job i) do not violate their deadlines following the

insertion. The algorithm that results from this discussion is function JS(Algorithm 4.6). The algorithm assumes that the jobs are already sorted such that $p_1 \geq p_2 \geq \dots \geq p_n$. Further it assumes that $n \geq 1$ and the deadline $d[i]$ of job i is at least 1. Note that no job with $d[i] < 1$ can ever be finished by its deadline. Theorem 3.5 proves that JS is a correct implementation of the greedy strategy.

```

1   Algorithm Greedy Job(d,J,n)
2   // is a set of jobs that can be completed by their deadlines
3   {
4       J:= {1};
5       for i:=2 to n do
6           {
7               if (all jobs in J U {i} can be completed
8               'by their deadlines) then J:=J U {i};
9           }
10  }
```

Algorithm 3.5 High-level description of job sequencing algorithm

Theorem 3.5 Function JS is a correct implementation of the greedy-based method described above

Proof: Since $d[i] \geq 1$, the job with the largest p_i will always be in the greedy solution. As the jobs are in nonincreasing order of the p_i 's, line 8 in Algorithm 3.6 includes the job with largest p_i . The for loop of line 10 considers the remaining jobs in the order required by the greedy method described earlier. At all times, the set of jobs already included in the solution is maintained in J . If $J[i]$, $1 \leq i \leq k$, is the set already included, then J is such that $d[j[i]] \leq d[j[i+1]]$, $1 \leq i \leq k$. This allows for easy application of the feasibility test of Theorem 3.3. When job i is being considered, the while loop of line 15 determines where in J this job has to be inserted. The use of a fictitious job 0 (line 7) allows easy insertion into position 1. Let w be such that $d[j[w]] \leq d[i]$ and $d[j[q]] > d[i]$, $w < q \leq k$. If job i is included into J , then jobs $J[q]$, $w < q \leq k$, have to be moved one position up in J (line 19). From Theorem 3.3, it follows that such a move retains feasibility of J iff $d[j[q]] \neq (q, w < q \leq k)$. This condition is verified in line 15. In addition, i can be inserted at position $w + 1$ iff $d[i] > w$. This is verified in line 16 (note $r=w$ on exit from the while loop if $d[j[q]] \neq (q, w < q \leq k)$). The correctness of JS follows from these observations.

1 Algorithm JS(d, j, n)

2 // $d[i] \geq 1$, $1 \leq i \leq n$ are the deadlines $n \geq 1$. The jobs

3 // are ordered such that $p[1] \geq p[2] \geq \dots \geq p[n]$. $J[i]$

4 // is the i th job in the optimal solution, $1 \leq i \leq k$.

5 // Also, at termination $d[j[i]] \leq d[j[i+1]]$, $1 \leq i \leq k$.

6 {

E 7

```

7      d[0]:=j[a]; // Initialize
8      J[1]:=1; // Include job 1.
9      k:=1;
10     for i:=2 to n do
11     {
12         // Consider jobs in nonincreasing order of p[i]. Find
13         // position for i and check feasibility of insertion
14         r:=k;
15         while ((d[J[r]]>d[i]) and (d[J[r]]≠(r))) do r:=r-1;
16         if ((d[j[r]]≤d[i]) and (d[i] > r)) then
17         {
18             // Insert i into J[].
19             for q:=k to (r+1) step -1 do J[q+1]:=J[q];
20             J[r+1]:=i; k:=k+1;
21         }
22     }
23     return k;
24 }
```

Algorithm 3.6 Greedy algorithm for sequencing unit time jobs with deadlines and profits.

For JS there are two possible parameters in terms of which its complexity can be measured. We can use n , the number of jobs, and s , the number of jobs included in the solution J . The while loop of line 15 in Algorithm 3.6 is iterated at most k times. Each iteration takes $\Theta(1)$ time. If the conditional of line 16 is true, then lines 19 and 20 are executed. These lines require $\Theta(k-r)$ time to insert job i . Hence, the total time for each iteration of the for loop of line 10 is $\Theta(k)$. This loop is iterated $n-1$ times. If s is final value of k , that is, s is the number of jobs in the final solutions, then the total time needed by algorithm JS is $\Theta(sn)$. Since $s \leq n$, the worst-case time, as a function of n alone is $\Theta(n^2)$. If we consider the job set $p_i = d_i = n-i+1$, $1 \leq i \leq n$, then algorithm JS takes $\Theta(n^2)$ time to determine J . Hence the worst-case computing time for JS is $\Theta(n^2)$. In addition to the space needed for d , JS needs $\Theta(s)$ amount of space for J . Note that the profit values are not needed by JS. It is sufficient to know that $p_i \geq p_{i+1}$, $1 \leq i \leq n$.

The computing time of JS can be reduced from $O(n^2)$ to nearly $O(n)$ by using the disjoint set union and find algorithm and a different method to determine the feasibility of a partial solution. If J is a feasible subset of jobs, then we can determine the processing times for each of the jobs using the rule; if job i hasn't been assigned a processing time, then assign it to the slot $[\alpha-1, \alpha]$, where α is the largest integer r such that $1 \leq r \leq d_i$ and the slot $[\alpha-1, \alpha]$ is free. This rule simply delays the processing of job i as much as possible. Consequently, when J is being built up job by job, jobs already in J do not have to be moved from their assigned slots to accommodate the new job. If for the new job being considered there is no α as defined above, then it cannot be

included in J . The proof of the validity of this statement is left as an exercise.

Example 3.3 Let $n = 5, (p_1, \dots, p_5) = (20, 15, 10, 5, 1)$ and $(d_1, \dots, d_5) = (2, 2, 1, 3, 3)$. Using the above feasibility rule, we have

J	assigned slots	job considered	action	profit
$\emptyset$	none	1	assign to $[1, 2]$	0
$\{1\}$	$[1, 2]$	2	assign to $[0, 1]$	20
$\{1, 2\}$	$[0, 1], [1, 2]$	3	cannot fit; reject	35
$\{1, 2\}$	$[0, 1], [1, 2]$	4	assign to $[2, 3]$	35
$\{1, 2, 4\}$	$[0, 1], [1, 2], [2, 3]$	5	reject	40

The optimal solution is $J = \{1, 2, 4\}$ with a profit of 40.

Since there are only n jobs and each job takes one unit of time, it is necessary only to consider the time slots $[i-1, i]$, $1 \leq i \leq b$, such that $b = \min\{n, \max\{d_i\}\}$. One way to implement the above scheduling rule is to partition the time slots $[i-1, i]$, $1 \leq i \leq b$ into sets. We use i to represent the time slots $[i-1, i]$. For any slot i , let n_i be the largest integer such that $n_i = i$ and slot n_i is free. To avoid end conditions, we introduce a fictitious slot $[-1, 0]$ which is always free. Two slots i and j are in the same set iff $n_i = n_j$. Clearly, if i and j , $i < j$, are in the same set, then $i, i+1, i+2, \dots, j$ are in the same set. Associated with each set k of slots is a value $f(k)$. Then

$f(k)=ni$ for all slots i in set k . Using the set representation, each set is represented as a tree. The root node identifies the set. The function f is defined only for root nodes. Initially, all slots are free and we have $b+1$ sets corresponding to the $b+1$ slots $[i-1, i]$, $0 \leq i \leq b$. At this time $f(i)=i$, $0 \leq i \leq b$. We use $p(i)$ to link slot i into its set tree. With the conventions for the union and find algorithms of Section 2.5, $p(i)=-1$, $0 \leq i \leq b$, initially. If a job with deadline d is to be scheduled, then we need to find the root of the tree containing the slot $\min\{n, d\}$. If this root is j , then $f(j)$ is the nearest free slot, provided $f(j) \neq 0$. Having used this slot, the set with root j should be combined with the set containing slot $f(j)-1$.

Example 3.4 The trees defined by the $p(i)$'s for the first three iterations in Example 3.3 are shown in Figure 4.4

The fast algorithm appears as FJS(Algorithm 3.7). Its computing time is readily observed to be $O(n\alpha(2n, n))$ (recall that $\leq(2n, n)$ is the inverse of Ackermann's function). It needs an additional $2n$ words of space for f and p .

```

1   Algorithm FJS(d,n,b,j)
2   // Find an optimal solution J[1:k]. It is assumed that
3   //  $p[1] \geq p[2] \geq \dots \geq p[n]$  and that  $b = \min\{n, \max_i(d[i])\}$ .
4   {
5   // Initially there are  $b+1$  single node trees.
6   for  $i:=0$  to  $b$  do  $f[i]:=i$ ;
```

```

7          k:=0; //Initialize
8          for i:=1 to n do
9              { //Use greedy rule.
10                 q:=CollapsingFind(min(n,d[i]));
11                 if (f[q]≠(0)) then
12                     {
13                         k:=K+1; J[k]:=i; // Select job i.
14                         m:=CollapsingFind(f[q]-1);
15                         WeightedUnion(m,q);
16                         f[q]:=f[m]; // q may be new root
17                     }
18                 }
19             }

```

Algorithm 3.7 Faster algorithm for job sequencing

3.5 SINGLE-SOURCE SHORTEST PATHS

Graphs can be used to represent the highway structure of a state or country with vertices representing cities and edges representing sections of highway. The edges can then be assigned weights which may be either the distance between the two cities

connected by the edge or the average time to drive along that section of highway. A motorist wishing to drive from city A to B would be interested in answers to the following questions:

- Is there a path from A to B?
- If there is more than one path from A to B, which is the shortest path?

The problem defined by these questions are special cases of the path problem we study in this section. The length of a path is now defined to be the sum of the weights of the edges on that path. The starting vertex of the path is referred to as the source, and the last vertex the destination. The graphs are digraphs to allow for one-way streets. In the problem we consider, we are given a directed graph $G=(V,E)$, a weighting function cost for the edges of G , and a source vertex v_0 . The problem is to determine the shortest paths from v_0 to all the remaining vertices of G . It is assumed that all the weights are positive. The shortest path between v_0 and some other node v is an ordering among a subset of the edges. Hence this problem fits the ordering paradigm.

Example 3.11. Consider the directed graph of Figure 3.4(a). The numbers of the edges are the weights. If node 1 is the source vertex, then the shortest path from 1 to 2 is 1,4,5,2. The length of this path is $10 + 15 + 20 = 45$. Even though there are three edges on this path, it is shorter than the path 1,2 which is of length 50. There is no path from 1 to 6. Figure 3.4(b) lists the shortest paths from node 1 to nodes 4,5,2, and 3, respectively. The paths have been listed in nondecreasing order of path length.

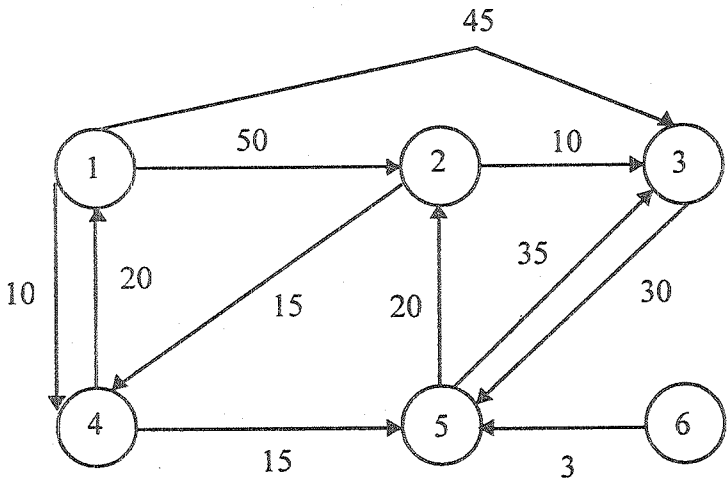


Figure 3.4 (a) Graph

	Path	Length
1)	1.4	10
2)	1.4.5	25
3)	1.4.5.2	45
4)	1.3	45

Figure 3.4 (b) Shortest Path from t

To formulate a greedy-based algorithm to generate the shortest paths. We must conceive of a multistage solution to the problem and also of an optimization measure. One possibility is to build the shortest paths one by one. As an optimization measure we can use the sum of the lengths of all paths so far generated. For this measure to be minimized, each individual path must be of minimum length. If we have already constructed i shortest paths, then using this optimization measure, the next path to be constructed should be the next shortest minimum length path. The greedy way (and also a systematic way) to generate the shortest paths from v_0 to the remaining vertices is to generate these paths in nondecreasing order of path length. First, a shortest path to the nearest vertex is generated. Then a shortest path to the second nearest vertex is generated, and so on. For the graph of Figure 3.4(a) the nearest vertex to $v_0 = 1$ is 4 (cost $[1,4]=10$). The path 1,4 is the first path generated. The second nearest vertex to node 1 is 5 and the distance between 1 and 5 is 25. The path 1,4,5 is the next path generated. In order to generate the shortest paths in this order, we need to be able to determine (1) the next vertex to which a shortest path must be generated and (2) a shortest path to this vertex. Let S denote the set of vertices (including v_0) to which the shortest paths have already been generated. For w not in S , let $\text{dist}[w]$ be the length of the shortest path starting from v_0 , going through only those vertices that are in S , and ending at w . We observe that:

1. If the next shortest path is to vertex u , then the path begins at v_0 , ends at u , and goes through only those vertices that are in S . To prove this, we must show

that all the intermediate vertices on the shortest path to u are in S . Assume there is a vertex w on this path that is not in S . Then, the v_0 to u path also contains a path from v_0 to w that is of length less than the v_0 to u path. By assumption the shortest paths are being generated in nondecreasing order of path length, and so the shorter path v_0 to w must already have been generated. Hence, there can be no intermediate vertex that is not in S .

2. The destination of the next path generated must be that of vertex u which has the minimum distance, $\text{dist}[u]$, among all vertices not in S . This follows from the definition of dist and observation 1. In case there are several vertices not in S with the same dist , then any of these may be selected.
3. Having selected a vertex u as in observation 2 and generated the shortest v_0 to u path, vertex u becomes a member of S . At this point the length of the shortest paths starting at v_0 , going through vertices only in S , and ending at a vertex w not in S may decrease; that is, the value of $\text{dist}[w]$ may change. If it does change, then it must be due to a shorter path starting at v_0 and going to u and then to w . The intermediate vertices on the v_0 to u path and the u to w path must all be in S . Further, the v_0 to u path must be the shortest such path; otherwise $\text{dist}[w]$ is not defined properly. Also, the u to w path can be chosen so as not to contain

any intermediate vertices. Therefore, we can conclude that if $\text{dist}[w]$ is to change (i.e., decrease), then it is because of a path from v_0 , to w , where the path from v_0 , to u in the shortest such path and the path from u to w is the edge (u, w) . The length of this path is $\text{dist}[u] + \text{cost}[u, w]$.

The above observations lead to a simple Algorithm 3.14 for the single source shortest path problem. This algorithm (known as Dijkstra's algorithm) only determines the lengths of the shortest paths from v_0 to all other vertices in G . The generation of the paths requires a minor extension to this algorithm and is left as an exercise. In the function `ShortestPaths` (Algorithm 3.14) it is assumed that the n vertices of G are numbered 1 through n . The set S is maintained as a bit array with $S[i]=0$ if vertex i is not in S and $s[i]=1$ if it is. It is assumed that the graph itself is represented by its cost adjacency matrix with $\text{cost}[i, j]$'s being the weight of the edge $\langle i, j \rangle$. The weight $\text{cost}[i, j]$ is set to some large number, ∞ , in case the edge $\langle i, j \rangle$ is not in $E(G)$. For $i=j$, $\text{cost}[i, j]$ can be set to any nonnegative number without affecting the outcome of the algorithm.

From our earlier discussion, it is easy to see that the algorithm is correct. The time taken by the algorithm on a graph with n vertices is $O(n^2)$. To see this, note that the for loop of line 7 in Algorithm 3.14 takes $\Theta(n)$ time. The for loop of line 12 is executed $n-2$ times. Each execution of this loop requires $O(n)$ time at lines 15 and 16 to select the next vertex and again at the for loop of line 18 to update dist . So the total time for this loop is

$O(n^2)$. In case a list t of vertices currently not in s is maintained, then the number of nodes on this list would at any time be $n - \text{num}$. This would speed up lines 15 and 16 and the for loop of line 18, but the asymptotic time would remain $O(n^2)$. This and other variations of the algorithm are explored in the exercises.

Any shortest path algorithm must examine each edge in the graph at least once since any of the edges could be in a shortest path. Hence, the minimum possible time for such an algorithm would be $\Omega(E)$. Since cost adjacency matrices were used to represent the graph, it takes $O(n^2)$ time just to determine which edges are in G , and so any shortest path algorithm using this representation must take $\Omega(n^2)$ time. For this representation then, algorithm ShortestPaths is optimal to within a constant factor. If a change to adjacency lists is made, the overall frequency of the for loop of line 18 can be brought down to $O(|E|)$ (Since dist can change only for vertices adjacent from u). If $V-S$ is maintained as a red-black tree each execution of lines 15 and 16 takes $O(\log n)$ time. Note that a red-black tree supports the following operations in $O(\log n)$ time : insert, delete (an arbitrary element), find-min, and search (for an arbitrary element). Each update in line 21 takes $O(\log n)$ time as well (since an update can be done using a delete and an insertion into the red-black tree). Thus the overall run time is $O((n+|E|) \log n)$.

- 1 Algorithm ShortestPaths($v, \text{cost}, \text{dist}, n$)
- 2 // $\text{dist}[j]$, $1 \leq j \leq n$, is set to the length of the shortest
- 3 // path from vertex v to vertex j in a digraph G with n


```

4    // vertices. dist[v] is set to zero. G is represented by its
5    // cost adjacency matrix cost[1:n,1:n].
6    {
7        for i:=1 to n do
8            { // Initialize S.
9                S[i]:=false; dist[i]:=cost[v,i];
10           }
11    S[v]:=true; dist[v]:=0.0; // Put v in S.
12    for num:=2 to n-1 do
13    {
14        //Determine n-1 paths from v.
15        choose u from among those vertices not
16        in S such that dist[u] is minimum;
17        S[u]:=true; // Put u in S.
18        for (each w adjacent to u with S[w]=false) do
19            // Update distances
20            if ((dist[w]>dist[u] + cost[u,w])) then
21                dist[w]:=dist[u] + cost[u,w];
22    }
23 }
```

Algorithm 3.14 Greedy algorithm to generate shortest paths

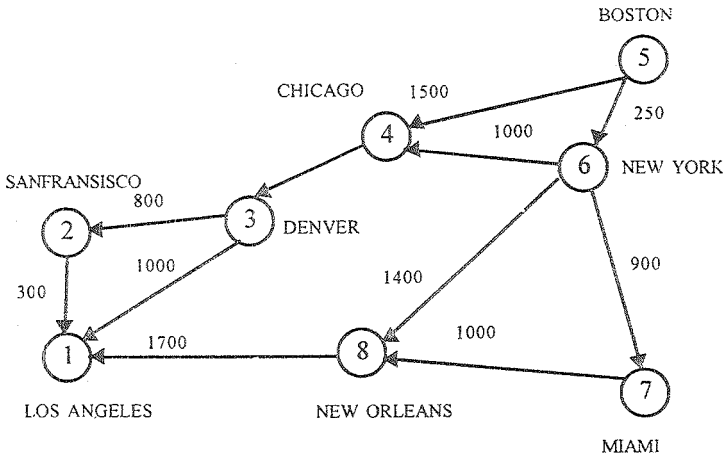


Figure 3.5 (a) Digraph

1	0						
2	300	0					
3	100	800	0				
4			1200	0			
5				1500	0	250	
6				1000		0	900
7							1000
8	1700						0

Figure 3.5 (b) Length Adjacency Matrix

Iteration	S	Vertex Selected	Distance							
			LA [1]	SF [2]	DEN [3]	CHI [4]	BOST [5]	NY [6]	MIA [7]	NO [8]
Initial	...	...	+∞	+∞	+∞	1500	0	250	+∞	+∞
1	[5]	6	+∞	+∞	+∞	1250	0	250	1150	1650
2	[5,6]	7	+∞	+∞	+∞	1250	0	250	1150	1650
3	[5,6,7]	4	+∞	+∞	2450	1250	0	250	1150	1650
4	[5,6,7,4]	8	3350	+∞	2450	1250	0	250	1150	1650
5	[5,6,7,4,8]	3	3350	3250	2450	1250	0	250	1150	1650
6	[5,6,7,4,8,3]	2	3350	3250	2450	1250	0	250	1150	1630

Figure 3.6

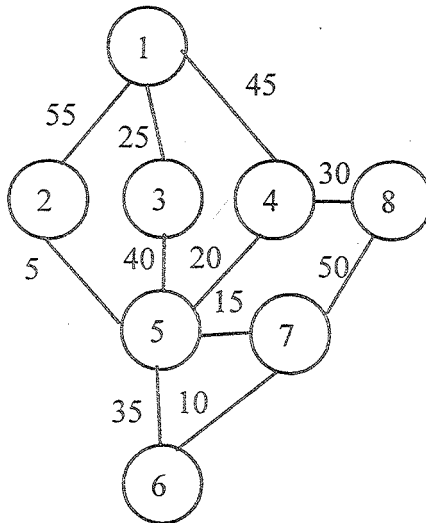


Figure 3.7(a) A Graph

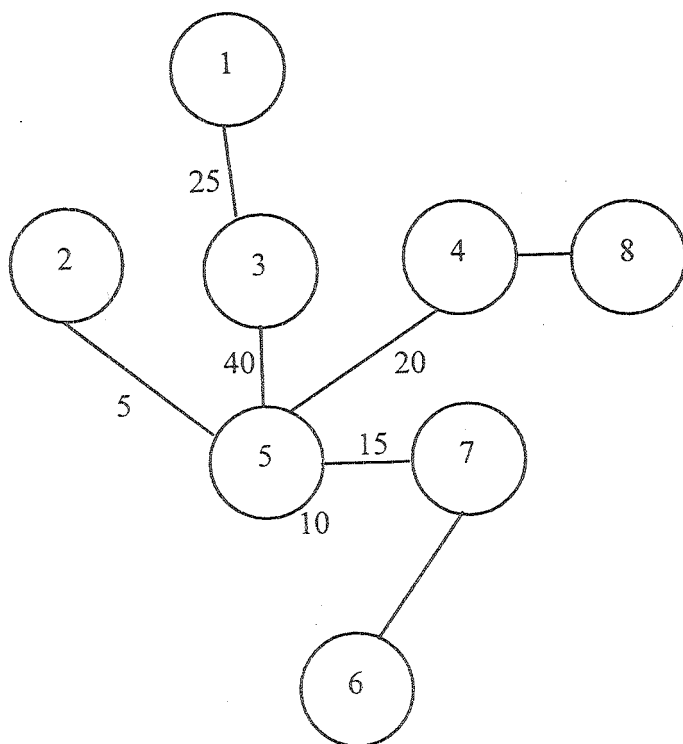


Figure 3.5 (b) Minimum Cost Spanning Tree

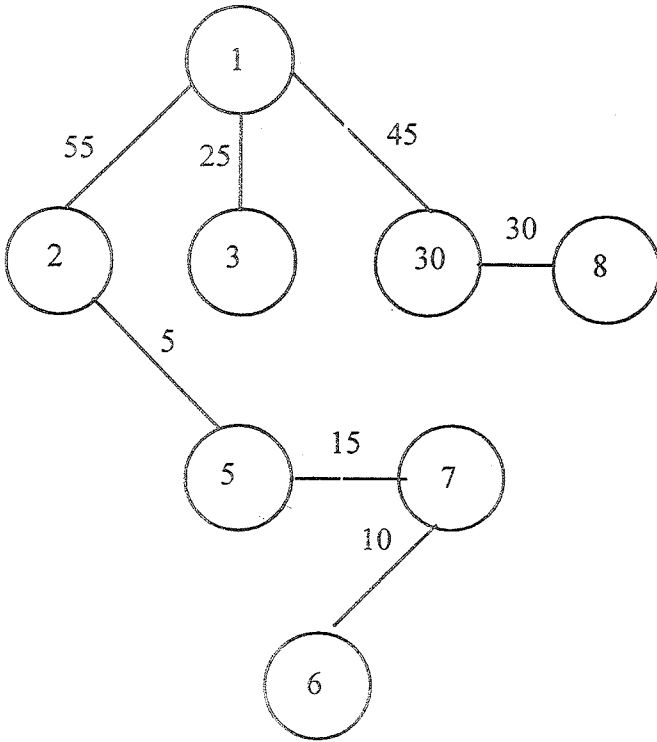


Figure 3.5 (c) Shortest Path Spanning Tree from Vertex 1

Example 3.12 Consider the eight vertex digraph of Figure 3.5(a) with cost adjacency matrix as in Figure 3.5(b). The values of dist and the vertices selected at each iteration of the for loop of line 12 in Algorithm 3.4 for finding all the shortest paths from Boston are shown in Figure 3.6. To begin with, S contains only Boston. In the first iteration of the for loop (that is, for $\text{num}=2$), the city u that is not in S and whose $\text{dist}[u]$ is minimum is identified

to be New York. New York enters the set S . Also the $\text{dist}[]$ values of Chicago, Miami, and New Orleans get altered since there are shorter paths to these cities via New York. In the next iteration of the for loop, the city that enters S is Miami since it has the smallest $\text{dist}[]$ value from among all the nodes not in S . None of the $\text{dist}[]$ values are altered. The algorithm continues in a similar fashion and terminates when only seven of the eight vertices are in S . By the definition of dist , the distance of the last vertex, in this case Los Angeles, is correct as the shortest path from Boston to Los Angeles can go through only the remaining six vertices.

One can easily verify that the edges on the shortest paths from a vertex v to all remaining vertices in a connected undirected graph G form a spanning tree of G . This spanning tree is called a shortest-path spanning tree. Clearly, this spanning tree may be different for different root vertices v . Figure 3.7 shows a graph G , its minimum-cost spanning tree, and a shortest-path spanning tree from vertex 1.

Lesson - 4

DYNAMIC PROGRAMMING**OBJECTIVE :**

After studying this lesson you can able to understand the following

- The General method of Dynamic Programming
- Multistage Graphs
- String Editing
- The Travelling Salesperson problem

4.1 THE GENERAL METHOD

Dynamic programming is an algorithm design method that can be used when the solution to a problem can be viewed as the result of a sequence of decisions. In earlier chapters we saw many problems that can be viewed this way. Here are some examples:

Example 4.1 [Knapsack] The solution to the knapsack problem can be viewed as the result of a sequence of decisions. We have to decide the values of x_i , $1 \leq i \leq n$. First we make a decision on x_1 , then on x_2 , then on x_3 , and so on. An optimal sequence of decisions maximizes the objective function $\sum p_i x_i$. (It also satisfies the constraints $\sum w_i x_i \leq m$ and $0 \leq x_i \leq 1$.)

Example 4.2 [Optimal merge patterns] An optimal merge pattern tells us which pair of files should be merged at each step. As a decision sequence, the problem calls for us to decide which pair of files should be merged first, which pair second, which pair third and so on. An optimal sequence of decisions is a least-cost sequence.

Example 4.3 [Shortest path] One way to find a shortest path from vertex i to vertex j in a directed graph G is to decide which vertex should be the second vertex, which the third, which the fourth, and so on, until vertex j is reached. An optimal sequence of decisions is one that results in a path of least length.

For some of the problems that may be viewed in this way, an optimal sequence of decisions can be found by making the decisions one at a time and never making an erroneous decision. This is true for all problems solvable by the greedy method. For many other problems, it is not possible to make stepwise decision (based only on local information) in such a manner that the sequence of decisions made is optimal.

Example 4.4 [Shortest path] Suppose we wish to find a shortest path from vertex i to vertex j . Let A_i be the vertices adjacent from vertex i . Which of the vertices in A_i should be the second vertex on the path? There is no way to make a decision at this time and guarantee that future decisions leading to an optimal sequence can be made. If on the other hand we wish to find a shortest path from vertex i to all other vertices in G , then at each step, a correct decision can be made.

One way to solve problems for which it is not possible to make a sequence of stepwise decisions leading to an optimal decision sequence is to try all possible decision sequences. We could enumerate all decision sequences and then pick out the best. But the time and space requirements may be prohibitive. Dynamic programming often drastically reduces the amount of enumeration by avoiding the enumeration of some decision sequences that cannot possibly be optimal. In dynamic programming an optimal sequence of decision is obtained by making explicit appeal to the principle of optimality.

Definition 4.1 [Principle of optimality] The principle of optimality state that an optimal sequence of decisions has the property that whatever the initial state and decision are, the remaining decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision.

Thus, the essential difference between the greedy method and dynamic programming is that in the greedy method only one decision sequence is ever generated. In dynamic programming, many decision sequence may be generated. However, sequences containing suboptimal subsequences cannot be optimal (if the principle of optimality holds) and so will (as far as possible) be generated.

Example 4.5 [Shortest path] Consider the shortest-path problem of Example 4.3. Assume that $i, i_1, i_2, \dots, i_k, j$ is a shortest path from i to j . Starting with the initial vertex i , a decision has been made to go to vertex i_1 . Following this decision the problem state is defined by vertex i_1 and we need to find a path from i_1 to

j. It is clear that the sequence $i_1, i_2, \dots, i_k, j$ must constitute a shortest i_1 to j path. If not, let $i_1, r_1, r_2, \dots, r_q, j$ be a shortest i_1 to j path. Then $i_1, i_1, r_1, \dots, r_q, j$ is an i to j path that is shorter than the path $i, i_1, \dots, i_k, j$. Therefore the principle of optimality applies for the problem.

Example 4.6 [0/1 knapsack] The 0/1 knapsack problem is similar to the knapsack problem except that the x_i 's are restricted to have a value of either 0 or 1. Using $\text{KNAP}(1, j, y)$ to represent the problem

$$\begin{aligned} & \text{maximize } \sum_{1 \leq i \leq j} p_i x_i \\ & \text{Subject to } \sum_{1 \leq i \leq j} w_i x_i = y \\ & x_i = 0 \text{ or } 1, 1 \leq i \leq j \end{aligned} \quad (5.1)$$

the knapsack problem is $\text{KNAP}(1, n, m)$. Let $y_1, y_2, \dots, y_n$ be an optimal sequence of 0/1 values for $x_1, x_2, \dots, x_n$, respectively. If $y_1 = 0$, then $y_2, y_3, \dots, y_n$ must constitute an optimal sequence for the problem $\text{KNAP}(2, n, m)$. If it does not, then $y_1, y_2, \dots, y_n$ is not an optimal sequence for the problem $\text{KNAP}(2, n, m - w_1)$. If it isn't, then there is another 0/1 sequence $z_2, z_3, \dots, z_n$ such that $\sum_{2 \leq i \leq n} w_i z_i \leq m - w_1$ and $\sum_{2 \leq i \leq n} p_i z_i > \sum_{2 \leq i \leq n} p_i y_i$. Hence, the sequence $y_1, z_2, z_3, \dots, z_n$ is a sequence for (4.1) with greater value. Again the principle of optimality applies.

Let S_0 be the initial problem state. Assume that n decisions d_i , $1 \leq i \leq n$, have to be made. Let $D_1 = \{r_1, r_2, \dots, r_j\}$ be the set of possible decision values for d_1 . Let S_1 be the problem state following the choice of decision r_1 , $1 \leq i \leq j$. Let Γ_1 be an optimal sequence of decisions with respect to the problem state S_1 . Then,

when the principle of optimality holds, an optimal sequence the decisions with respect to S_0 is the best of the decision sequences $r_i, \dots, 1 \leq i \leq j$.

4.2 MULTISTAGE GRAPHS

A multistage graph $G=(V,E)$ is a directed graph in which the vertices are partitioned into $k \geq 2$ disjoint sets $V_i, 1 \leq i \leq k$. In addition, if $\langle u,v \rangle$ is an edge in E , then $u \in V_i$ and $v \in V_{i+1}$ for some $i, 1 \leq i < k$. The sets V_1 and V_k are such that $|V_1| = |V_k| = 1$. Let s and t , respectively, be the vertices in V_1 and V_k . The vertex s is the source, and t the sink. Let $c(i,j)$ be the cost of edge $\langle i,j \rangle$. The cost of a path from s to t is the sum of the costs of the edges on the path. The multistage graph problem is to find a minimum-cost path from s to t . Each set V_i defines a stage in the graph. Because of the constraints on E , every path from s to t starts in stage 1, goes to stage 2, then to stage 3, then to stage 4, and so on, and eventually terminates in stage k . Figure 4.1 shows a five-stage graph. A minimum-cost s to t path is indicated by the broken edges.

Many problems can be formulated as multistage graph problems. We give only one example. Consider a resource allocation problem in which n units of resource are to be allocated to r projects. If $j, 0 \leq j \leq n$, units of the resource are allocated to project i , then the resulting net profit is $N(i,j)$. The problem is to allocate the resource to the r projects in such a way as to maximize total net profit. This problem can be formulated as an $r+1$ stage graph problem as follows. Stage $i, 1 \leq i \leq r$, represents project i . There are $n+1$ vertices $V(i,j), 0 \leq j \leq n$, associated with stage i ,

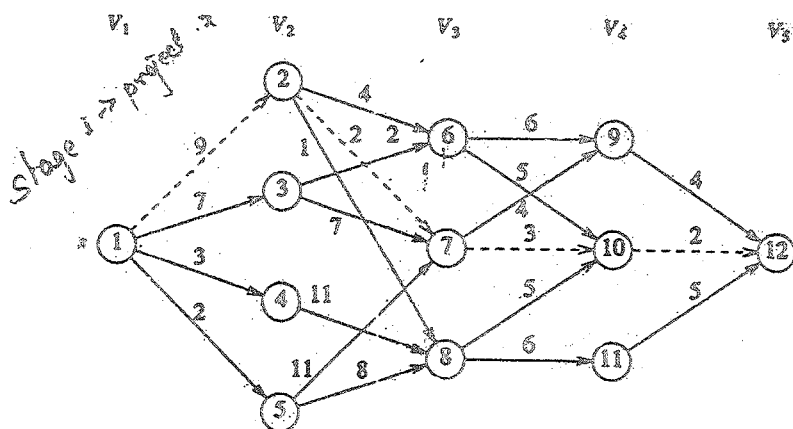


Figure 4.1 Five - Stage graph

$2 \leq i \leq r$. Stages 1 and $r+1$ each have one vertex, $V(1,0)=s$ and $V(r+1,n)=t$ respectively. Vertex $V(i,j)$, $2 \leq i \leq r$, represents the state in which a total of j units of resource have been allocated to projects $1, 2, \dots, i-1$. The edges in G are of the form $\langle V(i,j), V(i+1,l) \rangle$ (for all $j \leq l$ and $1 \leq i < r$). The edge $\langle V(i,j), V(i+1,l) \rangle$, $j \leq l$, is assigned a weight or cost of $N(i, l-j)$ and corresponds to allocating $l-j$ units of resources to project i , $1 \leq i \leq r$. In addition, G has edges of the type $\langle V(r,j), V(r+1,n) \rangle$. Each such edge is assigned a weight of $\max_{0 \leq p \leq n-j} \{ N(r, p) \}$. The resulting graph for a three project problem with $n=4$ is shown in Figure 4.2. It should be easy to see that an optimal allocation of resources is defined by a maximum cost s to t path. This is easily converted into a minimum-cost problem by changing the sign of all the edge costs.

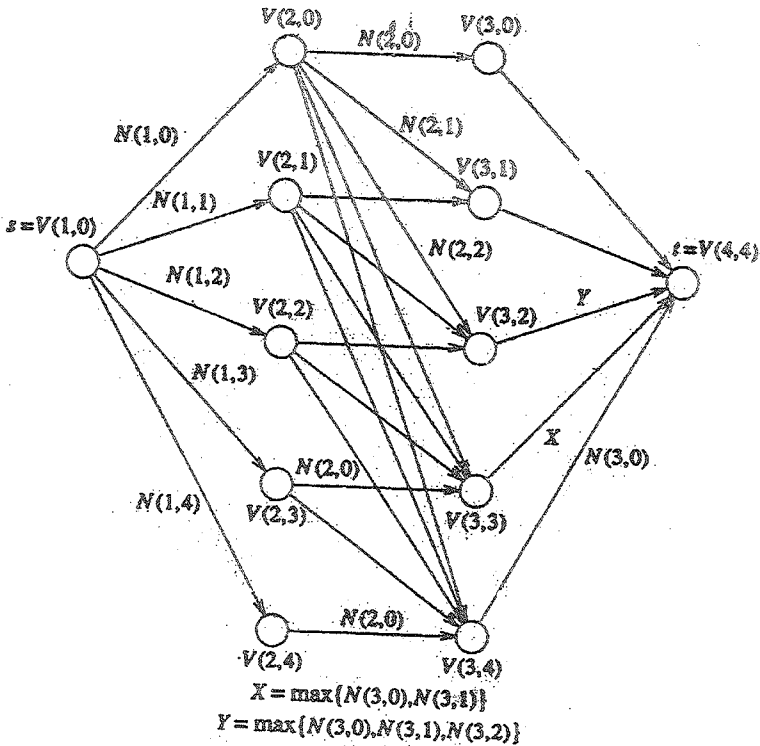


Figure 4.2 Four-stage graph corresponding to a three-project problem

A dynamic programming formulation for a k -stage graph problem is obtained by first noticing that every s to t path is the result of a sequence of $k-2$ decisions. The i^{th} decision involves determining which vertex in V_{i+1} , $1 \leq i \leq k-2$, is to be on the path. It is easy to see that the principle of optimality holds. Let $p(i,j)$ be a minimum-cost path from vertex j in V_i to vertex t . Let $\text{cost}(i,j)$

be the cost of this path. Then, using the forward approach, we obtain

$$\text{cost}(i,j) = \min \{ c(j,l) + \text{cost}(i+1,l) \} \quad (5.5)$$

$$l \in V_{i+1}$$

$$\langle j,l \rangle \in E$$

Since, $\text{cost}(k-1,j) = c(j,t)$ if $\langle j,t \rangle \in E$ and $\text{cost}(k-1,j) = \infty$ if $\langle j,t \rangle \notin E$, (5.5) may be solved for $\text{cost}(1,s)$ by first computing $\text{cost}(k-2,j)$ for all $j \in V_{k-2}$, then $\text{cost}(k-3,j)$ for all $j \in V_{k-3}$, and so on, and finally $\text{cost}(1,s)$. Trying this out on the graph of Figure 4.1, we obtain

$$\begin{aligned} \text{cost}(3,6) &= \min \{ 6 + \text{cost}(4,9), 5 + \text{cost}(4,10) \} \\ &= 7 \end{aligned}$$

$$\begin{aligned} \text{cost}(3,7) &= \min \{ 4 + \text{cost}(4,9), 3 + \text{cost}(4,10) \} \\ &= 5 \end{aligned}$$

$$\text{cost}(3,8) = 7$$

$$\begin{aligned} \text{cost}(2,2) &= \min \{ 4 + \text{cost}(3,6), 2 + \text{cost}(3,7), 1 + \text{cost}(3,8) \} \\ &= 7 \end{aligned}$$

$$\text{cost}(2,3) = 9$$

$$\text{cost}(2,4) = 18$$

$$\text{cost}(2,5) = 15$$

$$\begin{aligned}\text{cost}(1,1) &= \min \{ 9 + \text{cost}(2,2), 7 + \text{cost}(2,3), 3 + \\ &\quad \text{cost}(2,4), 2 + \text{cost}(2,5) \} \\ &= 16\end{aligned}$$

Note that in the calculation of $\text{cost}(2,2)$, we have reused the values of $\text{cost}(3,6)$, $\text{cost}(3,7)$, and $\text{cost}(3,8)$ and so avoided their recomputation. A minimum cost s to t path has a cost of 16. This path can be determined easily if we record the decision made at each state(vertex). Let $d(i,j)$ be the value of l (where l is a node) that minimizes $c(j,l) + \text{cost}(i+1,l)$ (see Equation 4.2). For Figure 4.1 we obtain

$$d(3,6) = 10; \quad d(3,7) = 10; \quad d(3,8) = 10;$$

$$d(2,2) = 7; \quad d(2,3) = 6; \quad d(2,4) = 8; \quad d(2,5) = 8;$$

$$d(1,1) = 2$$

Let the minimum-cost path be $s=1, v_2, v_3, \dots, v_{k-1}, t$. It is easy to see that $v_2 = d(1,1) = 2$, $v_3 = d(2, d(1,1)) = 7$, and $v_4 = d(3, d(2, d(1,1))) = d(3,7) = 10$.

Before writing an algorithm to solve (5.5) for a general k -stage graph, let us impose an ordering on the vertices in V . This ordering makes it easier to write the algorithm. We require that the n vertices in V are indexed 1 through n . Indices are assigned in order of stages. First, s is assigned index 1, then, vertices in v_2 are assigned indices, then vertices from V_3 , and so on. Vertex t

has index n . Hence, indices assigned to vertices in V_{i+1} are bigger than those assigned to vertices in V_i (see Figure 4.1). As a result of this indexing scheme, cost and d can be computed in the order $n-1, n-2, \dots, 1$. The first subscript in cost, p , and d only identifies the stage number and is omitted in the algorithm. The resulting algorithm, in pseudocode, is FGraph(Algorithm 4.1).

The Complexity analysis of the function FGraph is fairly straightforward. If G is represented by its adjacency lists, then r in line 9 of Algorithm 4.1 can be found in time proportional to the degree of vertex j . Hence, if G has $|E|$ edges, then the time for the for loop of line 7 is $\Theta(|V| + |E|)$. The time for the for loop of line 16 is $\Theta(k)$. Hence, the total time is $\Theta(|V| + |E|)$. In addition to the space needed for the input, space is needed for cost[], $d[]$, and $p[]$.

```

1   Algorithm FGraph( $G, k, n, p$ )
2   // The input is a  $k$ -stage graph  $G = (V, E)$  with  $n$  vertices
3   // indexed in order of stages.  $E$  is a set of edges and  $c[i, j]$ 
4   // is the cost of  $\langle i, j \rangle$ .  $p[i, k]$  is a minimum-cost path.
5   {
6       cost[n] := 0.0;
7       for  $j := n - 1$  to 1 step -1 do
8           { // Compute cost[j].

```



```

9      Let r be a vertex such the  $\langle j, r \rangle$  is an edge
10     of G and  $c[j, r] + \text{cost}[r]$  is minimum;
11      $\text{cost}[j] := c[j, r] + \text{cost}[r]$ ;
12      $d[j] := r$ ;
13 }
14 // Find a minimum-cost path.
15  $p[1] := 1; p[k] := n$ ;
16 for  $j := 2$  to  $k - 1$  do  $p[j] := d[p[j - 1]]$ ;
17 }
```

Algorithm 4.1 Multistage graph pseudocode corresponding to the forward approach

The multistage graph problem can also be solved using the backward approach. Let $bp(i, j)$ be a minimum-cost path from vertex s to a vertex j in V_i . Let $\text{bcost}(i, j)$ be the cost of $bp(i, j)$. From the backward approach we obtain

$$\text{bcost}(i, j) = \min \{ \text{bcost}(i-1, l) + c(l, j) \mid l \in V_{i-1}, \langle l, j \rangle \in E \} \quad (4.6)$$

Since $\text{bcost}(2, j) = c(1, j)$ if $\langle 1, j \rangle \in E$ and $\text{bcost}(2, j) = 8$ if $\langle 1, j \rangle \notin E$, $\text{bcost}(i, j)$ can be computed using (4.3) by first

computing bcost for $i=3$, then for $i=4$, and so on. For the graph of Figure 5.2, we obtain

$$\text{bcost}(3,6) = \min \{ \text{bcost}(2,2) + c(2,6), \text{bcost}(2,3) + c(3,6) \}$$

$$= \min \{ 9 + 4, 7 + 2 \}$$

$$= 9$$

$$\text{bcost}(3,7) = 11$$

$$\text{bcost}(3,8) = 10$$

$$\text{bcost}(4,9) = 15$$

$$\text{bcost}(4,10) = 14$$

$$\text{bcost}(4,11) = 16$$

$$\text{bcost}(5,12) = 16$$

The corresponding algorithm, in pseudocode, to obtain a minimum-cost s - t path is BGraph (Algorithm 4.2). The first subscript on bcost p , and d are omitted for the same reasons as before. This algorithm has the same complexity as FGraph provided G is now represented by its inverse adjacency lists (i.e., for each vertex v we have a list of vertices w such that $\langle w, v \rangle \in E$).

1 Algorithm BGraph(G, k, n, p)

2 // Same function as FGraph

```

3      {
4          bcost[1]:=0.0;
5          for j:=2 to n do
6              { // Compute bcost[j].
7                  Let r be such that  $\langle r, j \rangle$  is an edge of
8                  G and  $\text{bcost}[r] + c[r, j]$  is minimum;
9                   $\text{bcost}[j] := \text{bcost}[r] + c[r, j]$ ;
10                  $d[j] := r$ ;
11             }
12     // Find a minimum-cost path.
13     p[1]:=1; p[k]:=n;
14     for j:=k-1 to 2 do p[j]:=d[p[j+1]];
15 }

```

Algorithm 4.2 Multistage graph pseudocode corresponding to backward approach

It should be easy to see that both FGraph and BGraph work correctly even on a more generalized version of multistage graphs. In this generalization, the graph is permitted to have edges $\langle u, v \rangle$ such that $u \in V_i$, $v \in V_j$, and $i < j$.

Note: In the pseudocodes FGraph and BGraph, $\text{bcost}(i, j)$

is set to ∞ for any $\langle i, j \rangle \in E$. When programming these pseudocodes, one could use the maximum allowable floating point number for ∞ . If the weight of any such edge is added to some other costs, a floating point overflow might occur. Care should be taken to avoid such overflows.

4.3 STRING EDITING

We are given two strings $X = x_1, x_2, \dots, x_n$ and $Y = y_1, y_2, \dots, y_m$, where x_i , $1 \leq i \leq n$, and y_j , $1 \leq j \leq m$, are members of a finite set of symbols known as the alphabet. We want to transform X into Y using a sequence of edit operations on X . The permissible edit operations are insert, delete and change (a symbol of X into another), and there is cost associated with performing each. The cost of a sequence of operations is the sum of the costs of the individual operations in the sequence. The problem of string editing is to identify a minimum-cost sequence of edit operations that will transform X into Y .

Let $D(x_i)$ be the cost of deleting the symbol x_i from X , $I(y_j)$ be the cost of inserting the symbol y_j into X , and $C(x_i, y_j)$ be the cost of changing the symbol x_i of X into y_j .

Example 4.7 Consider the sequences $X = x_1, x_2, x_3, x_4, x_5 = a, a, b, a, b$ and $Y = y_1, y_2, y_3, y_4 = b, a, b, b$. Let the cost associated with each insertion and deletion be 1 (for any symbol). Also let the cost of changing any symbol to any other symbol be 2. One possible way of transforming X into Y is delete each x_i , $1 \leq i \leq 5$, and insert each y_j , $1 \leq j \leq 4$. The total cost of this edit sequence is 9. Another possible edit sequence is delete x_1 and x_2 and insert

y_4 at the end of string X . The total cost is only 3.

A solution to the string editing problem consists of a sequence of decisions, one for each edit operation. Let ξ be a minimum-cost edit sequence for transforming X into Y . The first operation, O in ξ is delete, insert, or change. If $\xi' = \xi - \{O\}$ and X' is the result of applying O on X , then ξ' should be a minimum-cost edit sequence that transforms X' into Y . Thus the principle of optimality holds for this problem. A dynamic programming solution for this problem can be obtained as follows. Define $\text{cost}(i,j)$ to be the minimum cost of any edit sequence for transforming $x_1, x_2, \dots, x_i$ into $y_1, y_2, \dots, y_j$ (for $0 \leq i \leq n$ and $0 \leq j \leq m$). Compute $\text{cost}(i,j)$ for each i and j . Then $\text{cost}(n,m)$ is the cost of an optimal edit sequence.

For $i = j = 0$, $\text{cost}(i,j) = 0$, since the two sequences are identical (and empty). Also, if $j=0$ and $i>0$, we can transform X into Y by a sequence of deletes. Thus, $\text{cost}(i,0) = \text{cost}(i-1,0) + D(x_i)$. Similarly, if $i=0$ and $j>0$, we get $\text{cost}(0,j) = \text{cost}(0,j-1) + I(y_j)$. If $i \neq 0$, and $j > 0$, $x_1, x_2, \dots, x_i$ can be transformed into $y_1, y_2, \dots, y_j$ in one of three ways:

1. Transform $x_1, x_2, \dots, x_{i-1}$ into $y_1, y_2, \dots, y_j$ using a minimum-cost edit sequence and then delete x_i . The corresponding cost is $\text{cost}(i-1, j) + D(x_i)$.
2. Transform $x_1, x_2, \dots, x_{i-1}$ into $y_1, y_2, \dots, y_{j-1}$ using a minimum-cost edit sequence and then change the symbol x_i to y_j . The associated cost is $\text{cost}(i-1, j-1) + C(x_i, y_j)$.

3. Transform $x_1, x_2, \dots, x_i$ into $y_1, y_2, \dots, y_{j-1}$ using a minimum-cost edit sequence and then insert y_j . This corresponds to a cost of $\text{cost}(i, j-1) + I(y_j)$.

The minimum cost of any edit sequence that transforms $x_1, x_2, \dots, x_i$ into $y_1, y_2, \dots, y_j$ (for $i > 0$ and $j > 0$) is the minimum of the above three costs, according to the principle of optimality. Therefore, we arrive at the following recurrence equation for $\text{cost}(i, j)$:

$$\text{cost}(i, j) = \begin{cases} 0 & i = j = 0 \\ \text{cost}(i-1, 0) + D(x_i) & j = 0, i > 0 \\ \text{cost}(0, j-1) + I(y_j) & i = 0, j > 0 \\ \text{cost}'(i, j) & i > 0, j > 0 \end{cases} \quad 4.14$$

$$\begin{aligned} \text{where } \text{cost}'(i, j) = \min \{ & \text{cost}(i-1, j) + D(x_i), \\ & \text{cost}(i-1, j-1) + c(x_i, y_j), \\ & \text{cost}(i, j-1) + I(y_j) \} \end{aligned}$$

we have to compute $\text{cost}(i, j)$ for all possible values of i and j ($0 \leq i \leq n$ and $0 \leq j \leq m$). There are $(n+1)(m+1)$ such values. These values can be computed in the form of a table, M , where each row of M corresponds to a particular value of i and each column of M corresponds to a specific value of j . $M(i, j)$ stores the value $\text{cost}(i, j)$. The zeroth row can be computed first since it corresponds to performing a series of insertions. Likewise the zeroth column can also be computed. After this, one could compute the entries of M in row-major order, starting from the

first row. Rows should be processed in the order $1, 2, \dots, n$. Entries in any row are computed in increasing order of column number.

The entries of M can also be computed in column-major order, starting from the first column. Looking at Equation 4.14, we see that each entry of M takes only $O(1)$ time to compute. Therefore the whole algorithm takes $O(mn)$ time. The value $\text{cost}(n, m)$ is the final answer we are interested in. Having computed all the entries of M , a minimum edit sequence can be obtained by a simple backward trace from $\text{cost}(n, m)$. This backward trace is enabled by recording which of the three options for $i > 0, j > 0$ yielded the minimum cost for each i and j .

	0	1	2	3	4
0	0	1	2	3	4
1	1	2	1	2	3
2	2	3	2	3	4
3	3	2	3	2	3
4	4	3	2	3	4
5	5	4	3	2	3

Figure 4.3 Cost table for Example 4.20

Example 4.20 Consider the string editing problem of Example 4.3 $X=a,a,b,a,b$ and $Y=b,a,b,b$. Each insertion and deletion has a unit cost and a change costs 2 units. For the cases $i=0, j > 1$, and $j=0, i > 1$, $\text{cost}(i,j)$ can be computed first (Figure 4.18). Let us compute the rest of the entries in row-major order. The next entry to be computed is $\text{cost}(1,1)$.

$$\begin{aligned}\text{cost}(1,1) &= \min \{ \text{cost}(0,1) + D(x_1), \text{cost}(0,0) + \\ &\quad C(x_1, y_1), \text{cost}(1,0) + I(y_1) \} \\ &= \min \{ 2, 2, 2 \} = 2\end{aligned}$$

Next is computed $\text{cost}(1,2)$.

$$\begin{aligned}\text{cost}(1,2) &= \min \{ \text{cost}(0,2) + D(x_1), \text{cost}(0,1) + \\ &\quad C(x_1, y_2), \text{cost}(1,1) + I(y_2) \} \\ &= \min \{ 3, 1, 3 \} = 1\end{aligned}$$

The rest of the entries are computed similarly. Figure 4.3 displays the whole table. The value $\text{cost}(5,4) = 3$. One possible minimum-cost edit sequence is delete x_1 , delete x_2 , and insert y_4 . Another possible minimum cost edit sequence is change x_1 to y_2 and delete x_4 .

4.4 THE TRAVELING SALESPERSON PROBLEM

We have seen how to apply dynamic programming to a subset selection problem (0/1 knapsack). Now we turn out attention to a permutation problems. Note that permutation problems usually are much harder to solve than subset problems as there are $n!$ different permutations of n objects whereas there

are only 2^n different subsets of n objects ($n! > 2^n$). Let $G=(V,E)$ be a directed graph with edge costs c_{ij} . The variable c_{ij} is defined such that $c_{ij} > 0$ for all i and j and $c_{ij} = \infty$ if $\langle i,j \rangle \notin E$. Let $|V|=n$ and assume $n>1$. A tour of G is a directed simple cycle that includes every vertex in V . The cost of a tour is the sum of the cost of the edges on the tour. The traveling salesperson problem is to find a tour of minimum cost.

As a second example, suppose we wish to use a robot arm to tighten the nuts of some piece of machinery on an assembly line. The arm will start from its initial position (which is over the first nut to be tightened), successively move to each of the remaining nuts, and return to the initial position. The path of the arm is clearly a tour on a graph in which vertices represent the nuts. A minimum-cost tour will minimize the time needed for the arm to complete its task (note that only the total arm movement time is variable; the nut tightening time is independent of the tour).

Our final example is from a production environment in which several commodities are manufactured on the same set of machines. The manufacture proceeds in cycles. In each production cycle, n different commodities are produced. When the machines are changed from production of commodity i to commodity j , a change over cost c_{ij} is incurred. It is desired to find a sequence in which to manufacture these commodities. This sequence should minimize the sum of change over costs (the remaining production costs are sequence independent). Since the manufacture proceeds cyclically, it is necessary to include the cost of starting the next cycle. This is just the change over cost from the last to the first

commodity. Hence, this problem can be regarded as a traveling salesperson problem on an n vertex graph with edge cost c_{ij} , 's being the changeover cost from commodity i to commodity j .

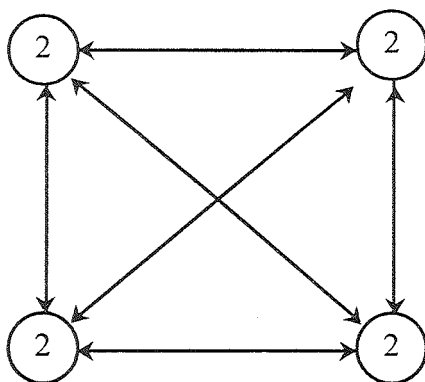
In the following discussion we shall, without loss of generality, regard a tour to be a simple path that starts and ends at vertex 1. Every tour consists of an edge $(1,k)$ for some $k \in V - \{1\}$ and a path from vertex k to vertex 1. The path from vertex k to vertex 1 goes through each vertex in $V - \{1,k\}$ exactly once. It is easy to see that if the tour is optimal, then the path from k to 1 must be a shortest k to 1 path going through all vertices in $V - \{1,k\}$. Hence, the principle of optimality holds. Let $g(i,S)$ be the length of a shortest path starting at vertex i , going through all vertices in S , and terminating at vertex 1. The function $g(1, V - \{1\})$ is the length of an optimal salesperson tour. From the principle of optimality it follows that

$$g(1, V - \{1\}) = \min_{2 \leq k \leq n} \{ c_{1k} + g(k, V - \{1,k\}) \} \quad (4.20)$$

Generalizing (4.20), we obtain (for $i \notin S$)

$$g(i,S) = \min \{ c_{ij} + g(j, S - \{j\}) \} \quad (4.21)$$

Equation 4.20 can be solved for $g(1, V - \{1\})$ if we know $g(k, V - \{1,k\})$ for all choices of k . The g values can be obtained by using (4.21). Clearly, $g(i, \emptyset) = c_{i1}$, $1 \leq i \leq n$. Hence, we can use (4.21) to obtain $g(i,S)$ for all S of size 1. Then we can obtain $g(i,S)$ for S with $|S| = 2$, and so on. When $|S| < n-1$, the values of i and S for which $g(i,S)$ is needed are such that $i \neq 1$, $1 \notin S$.



(a)

$$\begin{bmatrix} 0 & 10 & 15 & 20 \\ 5 & 0 & 9 & 10 \\ 6 & 13 & 0 & 12 \\ 8 & 8 & 9 & 0 \end{bmatrix}$$

(a)

Figure 4.4 Directed graph and edge length matrix c

Example 4.26 Consider the directed graph of Figure 5.21(a). The edge lengths are given by matrix c of Figure 5.21(b).

Thus $g(2, \emptyset) = c_{21} = 5$, $g(2, \emptyset) = c_{31} = 6$, and $g(4, \emptyset) = c_{41} = 8$. Using (5.21), we obtain

$$g(2, \{3\}) = c_{23} + g(3, \emptyset) = 15 \qquad g(2, \{4\}) = 18$$

$$g(3, \{2\}) = 18 \qquad g(3, \{4\}) = 20$$

$$g(4, \{2\}) = 13 \qquad g(4, \{3\}) = 15$$

Next, we compute $g(i, S)$ with $|S| = 2$, $i \neq 1$, $1 \notin S$ and $i \in S$.

$$g(2, \{3, 4\}) = \min \{c_{23} + g(3, \{4\}), c_{24} + g(4, \{3\})\} = 25$$

$$g(3, \{2, 4\}) = \min \{c_{32} + g(2, \{4\}), c_{34} + g(4, \{2\})\} = 25$$

$$g(4, \{2, 3\}) = \min \{c_{42} + g(2, \{3\}), c_{43} + g(3, \{2\})\} = 23$$

Finally, from (5.20) we obtain

$$\begin{aligned} g(1, \{2, 3, 4\}) &= \min \{c_{12} + g(2, \{3, 4\}), c_{13} + \\ &\quad g(3, \{2, 4\}), c_{14} + g(4, \{2, 3\})\} \\ &= \min \{35, 40, 43\} \\ &= 35 \end{aligned}$$

An optimal tour of the graph of Figure 5.21(a) has length 35. A tour of this length can be constructed if we retain with each $g(i, S)$ the value of j that minimizes the right-hand side of (5.21). Let $J(i, S)$ be this value. Then, $J(1, \{2, 3, 4\}) = 2$. Thus the tour starts from 1 and goes to 2. The remaining tour can be obtained from $g(2, \{3, 4\})$. So $J(2, \{3, 4\}) = 4$. Thus the next edge is $(2, 4)$. The remaining tour is for $g(4, \{3\})$. So $J(4, \{3\}) = 3$. The optimal tour is 1, 2, 4, 3, 1.

Let N be the number of $g(i, S)$'s that have to be computed before (4.20) can be used to compute $g(1, V - \{1\})$. For each value of $|S|$ there are $n-1$ choices for i . The number of distinct sets S of size k not including 1 and i is $\binom{n-2}{k}$. Hence

$$N = \sum_{k=1}^{n-2} (n-1) \binom{n-2}{k} = (n-1) 2^{n-2}$$

An algorithm that proceeds to find an optimal tour by using (4.20) and (4.21) will require $(n^2 2^n)$ time as the computation of $g(i, S)$ with $|S| = k$ requires $k-1$ comparison when solving (5.21). This is better than enumerating all $n!$ different tours to find the best one. The most serious drawback of this dynamic programming solution is the space needed, $O(n 2^n)$. This is too large even for modest values of n .

Unit - III

Lesson - 5

BASIC TRAVERSAL AND SEARCH TECHNIQUES

OBJECTIVES

After studying this lesson you can able to understand the following

- Basic Traversal and search Techniques
- Techniques for Binary Trees
- Techniques for Graphs
- Breadth first search and Traversal
- Depth first search and Traversal

5.1 INTRODUCTION

The techniques to be discussed in this chapter are divided into two categories. The first category includes techniques applicable only to binary trees. As described, these techniques involve examining every node in the given data object instance. Hence, these techniques are referred to as traversal methods. The second category includes techniques applicable to graphs (and hence also to trees and binary trees). These may not examine all vertices and so are referred to only as search methods. During

a traversal or search the fields of a node may be used several times. It may be necessary to distinguish uses of the fields of a node. During these uses, the node is said to be visited. Visiting a node may involve printing out its data field, evaluating the operation specified by the node in the case of a binary tree representing an expression, setting a mark bit to one or zero, and so on. Since we are describing traversals and searches of trees and graphs independently of their application, we use the term “visited” rather than the term for the specific function performed on the node at this time.

5.2 TECHNIQUES FOR BINARY TREES

The solution to many problems involves the manipulation of binary trees, trees, or graphs. Often this manipulation requires us to determine a vertex (node) or a subset of vertices in the given data object that satisfies a given property. For example, we may wish to find all vertices in a binary tree with a data value less than x or we may wish to find all vertices in a given graph G that can be reached from another given vertex v . The determination of this subset of vertices satisfying a given property can be carried out by systematically examining the vertices of the given data object. This often takes the form of a search in the data object. When the search necessarily

```
treenode = record
```

```
{
```

```
    Type data; // Type is the data type of data.
```

```
    treenode *lchild; treenode *rchild;
```

```

    }

1   Algorithm InOrder(t)
2   // t is a binary tree. Each node of t has
3   // three fields: lchild, data, and rchild.
4   {
5       if t≠0 then
6       {
7           InOrder(t.lchild);
8           Visit(t);
9           InOrder(t → rchild);
10      }
11  }

```

Algorithm 5.1 Recursive formulation of inorder traversal

involves the examination of every vertex in the object being searched it is called a traversal.

We have already seen an example of a problem whose solution required a search of a binary tree. An algorithm to search a binary search tree for an identifier x . This algorithm is not a traversal algorithm as it does not examine every vertex in the search tree. Sometimes, we may wish to traverse a binary search tree

(e.g., when we wish to list out all the identifiers in the tree). Algorithms for this are studied in this chapter.

There are many operations that we want to perform on binary trees. One that arises frequently is traversing a tree, or visiting each node in the tree exactly once. A traversal produces a linear order for the information in a tree. This linear order may be familiar and useful. When traversing a binary tree, we want to treat each node and its subtrees in the same fashion. If we let L,D, and R stand for moving left, printing the data, and moving right when at a node, then there are six possible combination of traversal: LDR, LRD, DLR, DRL, RDL, and RLD. If we adopt the convention that we traverse left before right, then only three traversals remain: LDR, LRD and DLR. To these we assign the names inorder, postorder, and preorder. Recursive functions for these three traversals are given in Algorithm 5.1 and 5.2.

```

1   Algorithm PreOrder(t)
2   // t is a binary tree. Each node of t has
3   // three fields: lchild, data, and rchild.
4   {
5       if t ( 0 then
6       {
7           Visit(t);
8           PreOrder(t→lchild);

```

```

9             PreOrder(t→rchild);
10         }
11     }

1  Algorithm PostOrder(t)
2      // t is a binary tree. Each node of t has
3      // three fields: lchild, data, and rchild.
4      {
5          if t ≠ 0 then
6              {
7                  PostOrder(t→lchild);
8                  PostOrder(t→rchild);
9                  Visit(t);
10             }
11     }

□

```

Algorithm 5.2 Preorder and postorder traversals

Figure 5.1 shows a binary tree and Figure 5.2 traces how InOrder works on it. This trace assumes that visiting a node requires only the printing of its data field. The output resulting

from this traversal is FDHGIBEAC. With Visit(t) replaced by a printing statement, the application of Algorithm 5.2 to the binary tree of Figure 5.1 results in the outputs ABDFGHI EC and FHIGDEB CA, respectively.

Theorem 5.1 Let $T(n)$ and $S(n)$ respectively represent the time and space needed by any one of the traversal algorithms when the input tree t has $n \leq 0$ nodes. If the time and space needed to visit a node are $\Theta(1)$, then $T(n) = \Theta(n)$ and $S(n) = O(n)$.

Proof: Each traversal can be regarded as a walk through the binary tree. During this walk, each node is reached three times: once from its parent (or as the start node in case the node is the root), once on returning from its left subtree and once on returning from its right subtree. In each of these three times a constant amount of work is done. So, the total time taken by the traversal is $\Theta(n)$. The only additional space needed is that for the recursion stack. If t has depth d , then this space is $\Theta(d)$. For an n -node binary tree, $d \leq n$ and so $S(n) = O(n)$.

Exercise 1 :

A triple order traversal of a binary tree t is defined recursively by algorithm 5.3. A very simple nonrecursive algorithm for such a traversal is given in Algorithm 5.4. In this algorithm p, q , and r point respectively to the present node, previously visited node, and next node to visit. The algorithm assumes that $t \neq 0$ and that an empty subtree of node p is represented by a link to p rather than a zero. Prove that algorithm 5.4 is a correct (Hint : Three links, l child, r child, and one from its parent, are associated with each node s . Each time s is visited the links are rotated

counter clockwise, and so after three visits they are restored to the original configuration and the algorithm backs up the tree).

5.3 TECHNIQUES FOR GRAPHS

A fundamental problem concerning graphs is the reachability problems. In its simplest form it requires us to determine whether there exists a path in the given graph $G = (V, E)$ such that this path starts at vertex v and ends at vertex u . A more general form is to determine for a given starting vertex v ($\forall$ all vertices u such that there is a path from v to u). This latter problem can be solved by starting at vertex v and systematically searching the graph G for vertices that can be reached from v . We describe two search methods for this.

```

1      Algorithm Triple(t)
2      {
3          if  $t \neq 0$  then
4              {
5                  Visit(t);
6                  Triple( $t \rightarrow lchild$ );
7                  Visit(t);
8                  Triple( $t \rightarrow rchild$ );
9                  Visit(t);
10         }
11     }
```

Algorithm 5.3 Triple-order traversal for Exercise 1

```

1   Algorithm Trip(t);
2   // It is assumed that lchild and rchild fields are >0.
3   {
4       p:=t;q:=-1;
5       while (p ≠ -1 ) do
6       {
7           Visit(p);
8           r:=(p→lchild); (p→lchild):=(p→rchild);
9           (p→rchild):=q; q:=p; p:=r;
10      }
11  }
```

Algorithm 5.4 A nonrecursive algorithm for the triple-order traversal for Exercise 1.

5.3.1 Breadth First Search and Traversal

In breadth first search we start at a vertex v and mark it as having been reached (visited). The vertex v is at this time said to be unexplored. A vertex is said to have been explored by an algorithm when the algorithm has visited all vertices adjacent from it. All unvisited vertices adjacent from v are visited next. These are new unexplored vertices. Vertex v has now been explored.

The newly visited vertices haven't been explored and are put onto the end of a list of unexplored vertices. The first vertex on this list is the next to be explored. Exploration continues until no unexplored vertex is left. The list of unexplored vertices operates as a queue and can be represented using any of the standard queue representations. BFS (Algorithm 5.5) describes, in pseudocode, the details of the search. It makes use of the queue representation given in Figure 5.1.

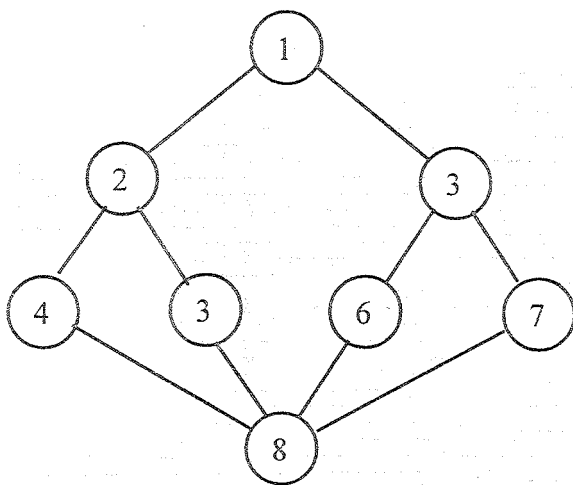
Example 5.1 Let us try out the algorithm on the undirected graph of Figure 5.1(a). If the graph is represented by its adjacency lists as in Figure 5.1(c), then the vertices get visited in the order 1,2,3,4,5,6,7,8. A breadth first search of the directed graph of Figure 5.1(b) starting at vertex 1 results in only the vertices 1, 2, and 3 being visited. Vertex 4 cannot be reached from 1.

Theorem 5.2 Algorithm BFS visits all vertices reachable from v .

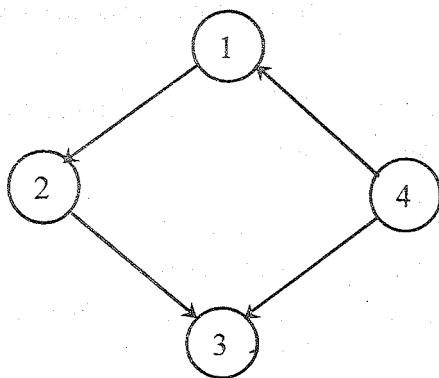
Proof: Let $G = (V, E)$ be a graph (directed or undirected) and let $v \in V$. We prove the theorem by induction on the length of the shortest path from v to every reachable vertex $w \in V$. The length (i.e., number of edges) of the shortest path from v to a reachable vertex w is denoted by $d(v, w)$.

Basic Step. Clearly all vertices w with $d(v, w) \leq 1$ get visited.

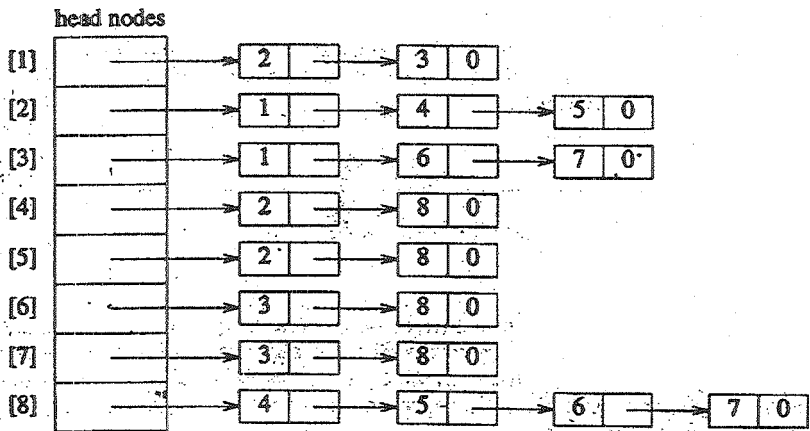
Induction Hypothesis. Assume that all vertices w with $d(v, w) \leq r$ get visited.



(a) Undirected Graph G



(b) Directed Graph



(c) Adjacency list for G

Figure 5.1 Example graphs and adjacency lists

Induction Step. We now show that all vertices w with $d(v, w) = r + 1$ also get visited.

Let w be a vertex in V such that $d(v, w) = r + 1$. Let u be a vertex that immediately precedes w on a shortest v to w path. Then $d(v, u) = r$ and so u gets visited by BFS. We can assume $u \neq v$ and $r \geq 1$. Hence, immediately before u gets visited, it is placed on the queue q of unexplored vertices. The algorithm doesn't terminate until q becomes empty. Hence, u is removed from q at some time and all unvisited vertices adjacent from it get visited in the for loop for line 11 of Algorithm 5.5. Hence, w gets visited.


```

1  Algorithm BFS(v)
2  // A breadth first search of G is carried out beginning
3  // at vertex v. For nay node i, visited[i] = 1 if i has
4  // already been visited. The graph G and array visited[]
5  // are global; visited[] is initialized to zero.
6  {
7      u:=v; // q is a queue of unexplored vertices.
8      visited[v]:=1;
9      repeat
10     {
11         for all vertices w adjacent from u do
12         {
13             if (visited[w]=0) then
14             {
15                 Add w to q; // w is unexplored.
16                 visited[w]:=1;
17             }
18         }
19         if q is empty then return; // No unexplored vertex.
20         Delete u from q; // Get first unexplored vertex.
21     } until (false);
22 }

```

Algorithm 5.5 Psedocode for breadth first search

Theorem 5.3 Let $T(n,e)$ and $S(n,e)$ be the maximum time and maximum additional space taken by algorithm BFS on any graph G with n vertices and e edges. $T(n,e) = \Theta(n+e)$ and $S(n,e) = \Theta(n)$ if G is represented by its adjacency lists. If G is represented by its adjacency matrix, then $T(n,e) = \Theta(n^2)$ and $S(n,e) = \Theta(n)$.

Proof: Vertices get added to the queue only in line 15 of Algorithm 5.5. A vertex w can get onto the queue only if $\text{visited}[w]=0$. Immediately following w 's addition to the queue, $\text{visited}[w]$ is set to 1 (line 16). Hence, each vertex can get onto the queue at most once. Vertex v never gets onto the queue and so at most $n-1$ additions are made. The queue space needed is at most $n-1$. The remaining variables take $O(1)$ space. Hence, $S(n,e) = O(n)$. If G is an n -vertex graph with v connected to the remaining $n-1$ vertices, then all $n-1$ vertices adjacent from v are on the queue at the same time. Furthermore, $\Theta(n)$ space is needed for the array visited . Hence $S(n,e) = \Theta(n)$. This result is independent of whether adjacency matrices or lists are used.

```

1   Algorithm BFT(G,n)
2   //Breadth first traversal of G
3   {
4       for i:=1 to n do // Mark all vertices unvisited.
5           visited[i]:=0;
6       for i:=1 to n do
7           if (visited[i]=0) then BFS(i);

```

8 }

Algorithm 5.6 Breadth first graph traversal

If adjacency lists are used, then all vertices adjacent from u can be determined in time $d(u)$, where $d(u)$ is the degree of u if G is undirected and $d(u)$ is the out-degree of u if G is directed. Hence, when vertex u is being explored, the time for the for loop of line 11 of Algorithm 5.5 is $\Theta(d(u))$. Since each vertex in G can be explored at most once, the total time for the repeat loop of line 9 is $O(\sum d(u)) = O(e)$. Then `visited[i]` has to be initialized to 0, $1 \leq i \leq n$. This takes $O(n)$ time. The total time is therefore: $O(n + e)$. If adjacency matrices are used, then it takes $\Theta(n)$ time to determine all vertices adjacent from u and the time becomes $O(n^2)$. If G is a graph such that all vertices are reachable from v , then all vertices get explored and the time is at least $O(n + e)$ and $O(n^2)$ respectively. Hence, $T(n, e) = \Theta(n + e)$ when adjacency lists are used, and $T(n, e) = \Theta(n^2)$ when adjacency matrices are used.

If BFS is used on a connected undirected graph G , then all vertices in G get visited and the graph is traversed. However, if G is not connected, then at least one vertex of G is not visited. A complete traversal of the graph can be made by repeatedly calling BFS each time with a new unvisited starting vertex. The resulting traversal algorithm is known as breadth first traversal (BFT) (see Algorithm 5.6). The proof of Theorem 5.3 can be used for BFT too to show that the time and additional space required by BFT on an n -vertex e -edge graph are $\Theta(n + e)$ and $\Theta(n)$ respectively if adjacency lists are used. If adjacency matrices are used, then the bounds are $\Theta(n^2)$ and $\Theta(n)$ respectively.

5.3.2 Depth First Search and Traversal

A depth first search of a graph differs from a breadth first search in that the exploration of a vertex v is suspended as soon as a new vertex is reached.

```

1   Algorithm DFS(v)
2   // Given an undirected (directed) graph  $G=(V,E)$  with
3   //  $n$  vertices and an array visited[] initially set
4   // to zero, this algorithm visits all vertices
5   // reachable from  $v$ .  $G$  and visited[] are global.
6   {
7       visited[v]:=1;
8       for each vertex  $w$  adjacent from  $v$  do
9           {
10              if(visited[w]=0) then DFS( $w$ );
11           }
12  }
```

Algorithm 5.7 Depth first search of a graph

At this time the exploration of the new vertex u begins. When this new vertex has been explored, the exploration of v continues. The search terminates when all reached vertices have

been fully explored. This search process is best described recursively as in Algorithm 5.7.

Example 5.2 A depth first search of the graph of Figure 5.4(a) starting at vertex 1 and using the adjacency lists of Figure 5.4(c) results in the vertices being visited in the order 1,2,4,8,5,6,3,7.

One can easily prove that DFS visits all vertices reachable from vertex v . If $T(n,e)$ and $S(n,e)$ represent the maximum time and maximum additional space taken by DFS for an n -vertex e -edge graph, then $S(n,e) = \Theta(n)$ and $T(n,e) = \Theta(n + e)$ if adjacency lists are used and $T(n,e) = \Theta(n^2)$ if adjacency matrices are used (see the exercises).

A depth first traversal of a graph is carried out by repeatedly calling DFS, with a new unvisited starting vertex each time. The algorithm for this (DFT) differs from BFT only in that the call to BFS(i) is replaced by a call to DFS(i). The exercises contain some problems that are solved best by BFS and others that are solved best by DFS. Later sections of this chapter also discuss graph problems solved best by DFS.

BFS and DFS are two fundamentally different search methods. In BFS a node is fully explored before the exploration of any other node begins. The next node to explore is the first unexplored node remaining. The exercise examine a search technique (D-search) that differs from BFS only in that the next node to explore is the most recently reached unexplored node. In DFS the exploration of a node is suspended as soon as a new unexplored node is reached. The exploration of this new node is immediately begun.

Lesson - 6

BACKTRACKING**OBJECTIVE**

After studying this lesson you can able to understand the following

- The General Method of Backtracking
- Sum of Subsets
- Graph coloring

6.1 THE GENERAL METHOD

In the search for fundamental principles of algorithm design, backtracking represents one of the most general techniques. Many problems which deal with searching for a set of solutions or which ask for an optimal solution satisfying some constraints can be solved using the backtracking formulation. The name backtrack was first coined by D.H. Lehmer in the 1950s. Early workers who studied the process were R. J. Walker, who gave an algorithmic account of it in 1960, and S. Golomb and L. Baumert who presented a very general description of it as well as a variety of applications.

In many applications of the backtrack method, the desired solution is expressible as an n -tuple $(x_1, \dots, x_n)$, where the x_i are chosen from some finite set S_i . Often the problem to be solved

calls for finding one vector that maximizes (or minimizes or satisfies) a criterion function $P(x_1, \dots, x_n)$. Sometimes it seeks all vectors that satisfy P . For example, sorting the array of integers in $a[1:n]$ is a problem whose solution is expressible by an n -tuple, where x_i is the index in a of the i th smallest element. The criterion function P is the inequality $a[x_i] \leq a[x_{i+1}]$ for $1 \leq i \leq n$. The set S_i is finite and includes the integers 1 through n . Though sorting is not usually one of the problems solved by backtracking, it is one example of a familiar problem whose solution can be formulated as n -tuple. In this chapter we study a collection of problems whose solutions are best done using backtracking.

Suppose m_i is the size of set S_i . Then there are $m = m_1 m_2 \dots m_n$ n -tuples that are possible candidates for satisfying the function P . The brute force approach would be to form all these n -tuples, evaluate each one with P , and save those which yield the optimum. The backtrack algorithm has as its virtue the ability to yield the same answer with far fewer than m trials. Its basic idea is to build up the solution vector one component at a time and to use modified criterion functions $P_i(x_1, \dots, x_i)$ (sometimes called bounding functions) to test whether the vector being formed has any chance of success. The major advantage of this method is this: if it is realized that the partial vector $(x_1, x_2, \dots, x_i)$ can in no way lead to an optimal solution, then $m_{i+1} \dots m_n$ possible test vectors can be ignored entirely.

Many of the problems we solve using backtracking require that all the solutions satisfy a complex set of constraints. For any problem these constraints can be divided into two categories: explicit and implicit.

Definition 6.1 Explicit constraints are rules that restrict each x_i to take on values only from a given set.

Common examples of explicit constraints are

$$x_i \geq 0 \text{ or } S_i = \{ \text{all nonnegative real numbers} \}$$

$$x_i = 0 \text{ or } 1 \quad \text{or} \quad S_i = \{0, 1\}$$

$$l_i \leq x_i \leq u_i \quad \text{or} \quad S_i = \{a: l_i \leq a \leq u_i\}$$

The explicit constraints depend on the particular instance I of the problems being solved. All tuples that satisfy the explicit constraints define a possible solution space for I .

Definition 6.2 The implicit constraints are rules that determine which of the tuples in the solution space of I satisfy the criterion function. Thus implicit constraints describe the way in which the x_i must relate to each other.

Example 6.1 [8-queen] A classic combinatorial problem is to place eight queens on an 8 (8 chessboard so that no two "attack," that is, so that no two of them are on the same row, column, or diagonal. Let us number the rows and columns of the chessboard 1 through 8 (Figure 6.1). The queens can also be numbered 1 through 8. Since each queen must be on a different row, we can without loss of generality assume queen i is to be placed on row i . All solutions to the 8-queens problem can therefore be represented as 8-tuples $(x_1, \dots, x_8)$, where x_i is the column on which queen i is placed. The explicit constraints using this formulation are $S_i = \{1, 2, 3, 4, 5, 6, 7, 8\}$, $1 \leq i \leq 8$. Therefore the solution space consists of 8^8 8-tuples. The implicit constraints

for this problem are that no two x_i 's can be the same (i.e., all queens must be on different columns) and no two queens can be on the same diagonal. The first of these two constraints implies that all solutions are permutation of the 8-tuple (1,2,3,4,5,6,7,8). This realization reduce the size of the solution space from 8^8 tuples to $8!$ tuples. We see later how to formulate the second constraint in terms of the x_i . Expressed as an 8-tuples, the solution in Figure 6.1 is (4,6,8,2,7,1,3,5).

		Column							
		1	2	3	4	5	6	7	8
row	1				Q				
	2						Q		
	3								Q
	4		Q						
	5							Q	
	6	Q							
	7			Q					
	8					Q			

Figure 6.1 One solution to the 8-queens problem

Example 6.2 [Sum of subsets] Given positive numbers w_i , $1 \leq i \leq n$, and m , this problem calls for finding all subsets of the w_i whose sums are m . For example, if $n=4$, $(w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$, and $m=31$, then the desired subsets are $(11, 13, 7)$ and $(24, 7)$. Rather than represent the solution vector by the w_i which sum to m , we could represent the solution vector by giving the indices of these w_i . Now the two solutions are described by the vectors $(1, 2, 4)$ and $(3, 4)$. In general, all solutions are k -tuples $(x_1, x_2, \dots, x_k)$, $1 \leq k \leq n$, and different solutions may have different-sized tuples. The explicit constraints require $x_i \in \{j \mid j \text{ is an integer and } 1 \leq j \leq n\}$. The implicit constraints require that no two be the same and that the sum of the corresponding w_i 's be m . Since we wish to avoid generating multiple instances of the same subset (e.g., $(1, 2, 4)$ and $(1, 4, 2)$ represent the same subset), another implicit constraint that is imposed is that $x_i < x_{i+1}$, $1 \leq i \leq k$.

In another formulation of the sum of subsets problem, each solution subset is represented by an n -tuple $(x_1, x_2, \dots, x_n)$ such that $x_i \in \{0, 1\}$, $1 \leq i \leq n$. Then $x_i = 0$ if w_i is not chosen and $x_i = 1$ if w_i is chosen. The solutions to the above instance are $(1, 1, 0, 1)$ and $(0, 0, 1, 1)$. This formulation expresses all solutions using a fixed-sized tuple. Thus we conclude that there may be several ways to formulate a problem so that all solutions are tuples that satisfy some constraints. One can verify that for both of the above formulations, the solution space consists of 2^n distinct tuples.

Backtracking algorithms determine problem solutions by systematically searching the solution space for the given problem instance. This search is facilitated by using a tree organization for the solution space. For a given solution space many tree organizations may be possible. The next two examples show some of the ways to organize a solution into a tree.

Example 6.3 [n-queens] The n-queens problem is a generalization of the 8-queens problem of Example 6.1. Now n queens are to be placed on an $n \times n$ chessboard so that no two attack; that is, no two queens are on the same row, column, or diagonal. Generalizing out earlier discussions, the solution space consists of all $n!$ permutations of the n -tuple $(1, 2, \dots, n)$. Figure 6.2 shows a possible tree organization for the case $n=4$. A tree such as this is called a permutation tree. The edges are labeled by possible values of x_i . Edges from level 1 to level 2 nodes specify the values for x_1 . Thus, the leftmost subtree contains all solutions with $x_1=1$; its leftmost subtree contains all solutions with $x_1=1$ and $x_2=1$, and so on. Edges from level i to level $i+1$ are labeled with the values of x_i . The solution space is defined by all paths from the root node to a leaf node. There are $4!=24$ leaf nodes in the tree of Figure 6.2.

Example 6.4 [Sum of subsets] In Example 6.2 we gave two possible formulations of the solution space for the sum of subsets problem. Figures 6.3 and 6.4 show a possible tree organization for each of these formulations for the case $n=4$. The tree of Figure 6.3 corresponds to the variable tuple size formulation. The edges are labeled such that an edge from a level i node to a level $i+1$ node represents a value for x_i . At each node, the solution space is partitioned into subsolution spaces. The solution space is defined by all paths from the root node to any node in the tree, since any such path corresponds to a subset satisfying the explicit constraints. The possible paths are $()$ (this corresponds to the empty path from the root to itself), (1) , $(1,2)$, $(1,2,3)$, $(1,2,3,4)$, $(1,2,4)$, $(1,3,4)$, (2) , $(2,3)$, and so on. Thus, the left-most subtree defines all subsets containing w_1 , the next subtree defines all subsets containing w_2 but not w_1 , and so on.

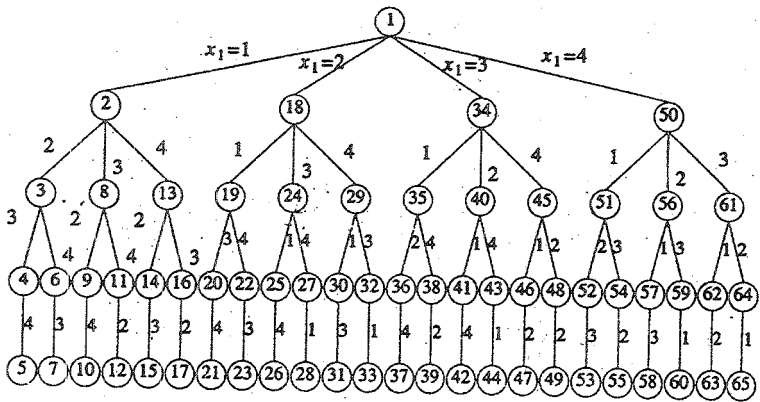


Figure 6.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

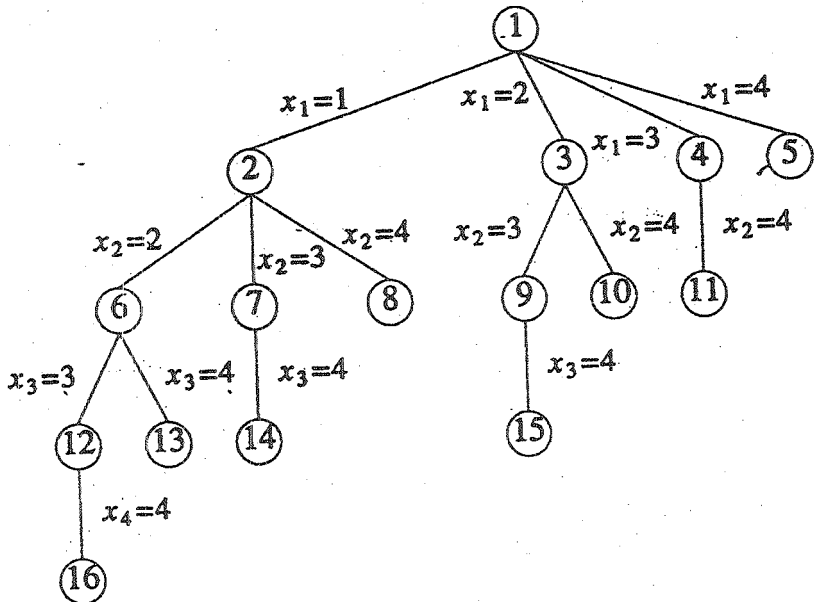


Figure 6.3 A possible solution space organization for the sum of subsets problem. Nodes are numbered as in breadth - first search

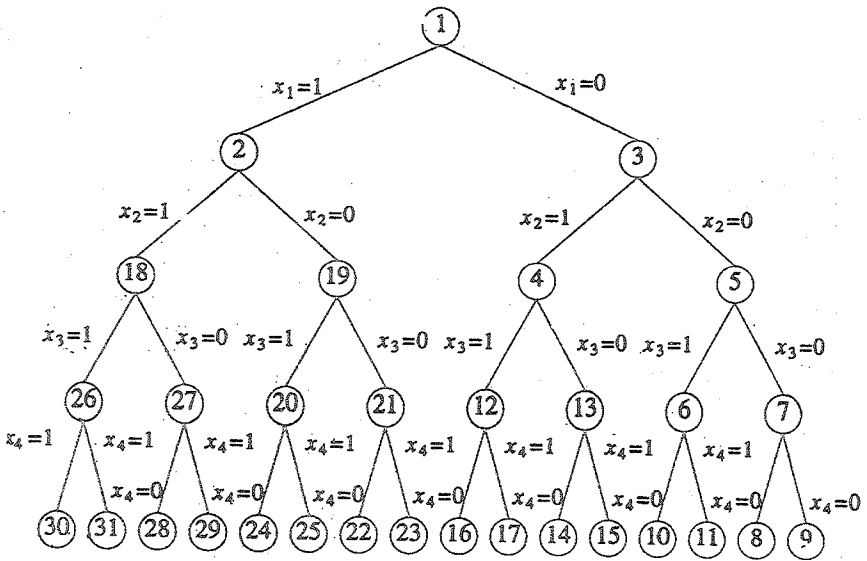


Figure 6.4 Another possible organization for the sum of subsets problems. Nodes are numbered as in D-search

The tree of Figure 6.4 corresponds to the fixed tuple size formulation. Edges from level i nodes to level $i + 1$ nodes are labeled with the value of x_i , which is either zero or one. All paths from the root to a leaf node define the solution space. The left subtree of the root defines all subsets containing w_1 , the right subtree defines all subsets not containing w_1 , and so on. Now there are 2^4 leaf nodes which represent 16 possible tuples.

At this point it is useful to develop some terminology regarding tree organizations of solution spaces. Each node in this tree defines a problem state. All paths from the root to other nodes define the state space of the problem. Solution states are

those problem states s for which the path from the root to s defines a tuple in the solution space. In the tree of Figure 6.3 all nodes are solution states whereas in the tree of Figure 6.4 only leaf nodes are solution states. Answer states are those solution states s for which the path from the root to s defines a tuple that is a member of the set of solutions (i.e., it satisfies the implicit constraints) of the problem. The tree organization of the solution space is referred to as the state space tree.

At each internal node in the space tree of Examples 6.3 and 6.4 the solution space is partitioned into disjoint sub-solution spaces. For example, at node 1 of Figure 6.2 the solution space is partitioned into four disjoint sets. Subtrees 2, 18, 34 and 50 respectively represent all elements of the solution space with $x_1 = 1, 2, 3$ and 54. At node 2 the sub-solution space with $x_1 = 1$ is further partitioned into three disjoint sets. Subtree 3 represents all solution space elements with $x_1 = 1$ and $x_2 = 2$. For all the state space trees we study in this chapter, the solution space is partitioned into disjoint sub-solution spaces at each internal node. It should be noted that this is not a requirement on a state space tree. The only requirement is that every element of the solution space be represented by at least one node in the state space tree.

The state space tree organizations described in Example 6.4 are called static trees. This terminology follows from the observation that the tree organizations are independent of the problem instance being solved. For some problems instances. In this case the tree organizations for different problem instance. In this case the tree organizations is determined dynamically as the solution space is being searched. Tree organizations that are

problem instance dependent are called dynamic trees. As an example, consider the fixed tuple size formulations for the sum of subsets problem (Example 6.4). Using a dynamic tree organizations, one problem instance with $n=4$ can be solved by means of the organization gives in Figure 6.4. An other problem instance with $n=4$ can be solved by means of a tree in which at level 1 the partitioning corresponds to $x_2=1$ and $x_2=0$. At level 2 the partitioning could correspond to $x_1=1$ and $x_1=0$, at level 3 it could correspond to $x_3=1$ and $x_3=0$, and so on.

Once a state space tree has been conceived of for any problem, this problem can be solved by systematically generating the problem states, determining which of these are solution states, and finally determining which solution states are answer states. There are two fundamentally different ways to generate the problem states. Both of these begin with the root node and generate other nodes. A node which has been generated and all of whose children have not yet been generated is called a live node. The live node whose children are currently being generated is called the E-node (node being expanded). A dead node is a generated node which is not to be expanded further or all of whose children have been generated. In both methods of generating problem states, we have a list of live nodes. In the first of these two methods as soon as a new child C of the current E-node R is generated, this child will become the new E-node. Then R will become the E-node again when the subtree C has been fully explored. This corresponds to a depth first generation of the problem states. In the second state generation methods, bounding functions are used to kill live nodes without generating all their children. This is done carefully enough that at the conclusion of the process at least one answer node is always generated or all

answer nodes are generated if the problem requires us to find all solutions. Depth first node generation with bounding functions is called backtracking. State generation methods in which the E-node remains the E-node until it is dead lead to branch-and-bound methods. The branch-and-bound technique is discussed in lesson 7.

The nodes of Figure 6.2 have been numbered in the order they would be generated in a depth first generation process. The nodes in Figures 6.3 and 6.4 have been numbered according to two generation methods in which the E-node remains the E-node until it is dead. In Figure 6.3 each new node is placed into a queue. When all the children of the current E-node have been generated, the next node at the front of the queue becomes the new E-node. In Figure 6.4 new nodes are placed in a stack instead of a queue. Current terminology is not uniform in referring to these two alternatives. Typically the queue method is called breadth first generation and the stack method is called D-search (depth search).

Example 6.4 [4-queens] Let us see how backtracking works on the 4-queens problem of Example 6.3 As a bounding function, we use the obvious criteria that if $(x_1, x_2, \dots, x_i)$ is the path to the current E-node, then all children nodes with parent-child labeling x_{i+1} are such that $(x_1, \dots, x_{i+1})$ represents a chessboard configuration in which no two queens are attacking. We start with the root node as the only live node. This becomes the E-node and the path is $()$. We generate one child. Let us assume that the children are generated in ascending order. Thus, node number 2 of Figure 6.2 is generated and the path is now (1) . This corresponds to placing queen 1 on column 1. Node 2

becomes the E-node. Node 3 is generated and immediately killed. The next node generated is node 8 and the path becomes (1,3). Node 8 becomes the E-node. However, it gets killed as all its children represent board configurations that cannot lead to an answer node. We backtrack to node 2 and generate another child, node 13. The path is now (1,4). Figure 6.5 shows the board configurations as backtracking proceeds. Figure 6.5 shows graphically the steps that the backtracking algorithm goes through as it tries to find a solution. The dots indicate placements of a queen which were tried and rejected because another queen was attacking. In Figure 6.5(b) the second queen is placed on columns 1 and 2 and finally settles on column 3. In Figure 6.5(c) the algorithm tries all four columns and is unable to place the next queen on a square. Backtracking now takes place. In Figure 6.5(d) the second queen is moved to the next possible column, column

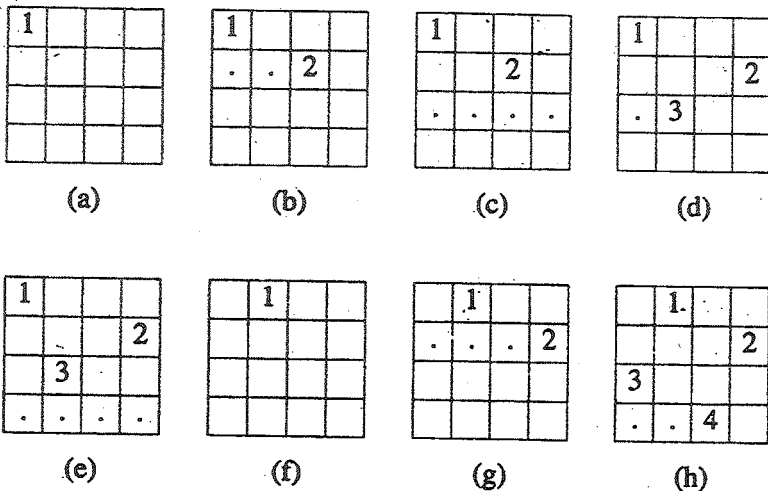


Figure 6.5 Example of a backtract solution to the 4-queens problem

4 and the third queen is placed on column 2. The boards in Figure 6.5(e), (f), (g), and (h) show the remaining steps that the algorithm goes through until a solution is found.

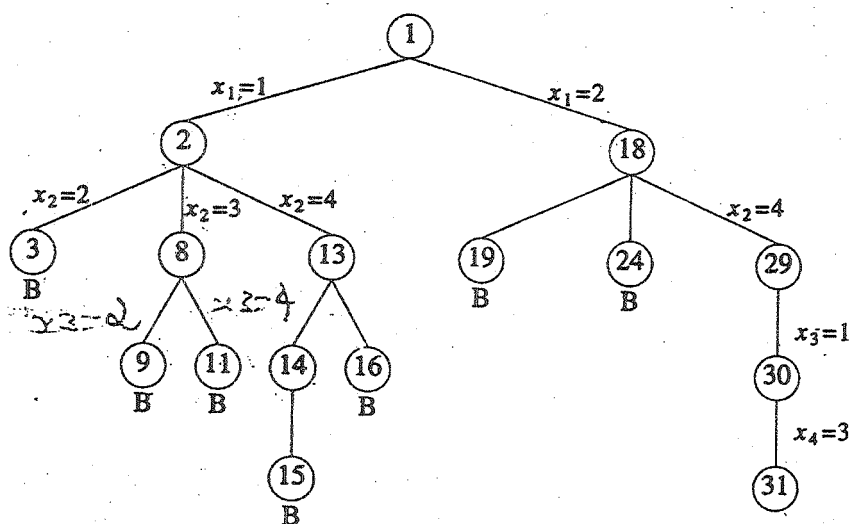


Figure 6.6 Portion of the tree of Figure 6.6 that is generated during backtracking

Figure 6.6 shows the part of the tree of Figure 6.2 that is generated. Nodes are numbered in the order in which they are generated. A node that gets killed as a result of the bounding function has a B under it. Contrast this tree with Figure 6.2 which contains 31 nodes.

With this example completed, we are now ready to present a precise formulation of the backtracking process. We continue to treat backtracking in a general way. We assume that all answer nodes are to be found and not just one. Let $(x_1, x_2, \dots, x_i)$ be a path from the root to a node in a state space tree. Let $T(x_1, x_2, \dots, x_i)$ be the set of all possible values for x_{i+1} such that $(x_1, x_2, \dots, x_{i+1})$ is also a path to a problem state. $T(x_1, x_2, \dots, x_n) = \emptyset$.

We assume the existence of bounding function B_{i+1} (expressed as predicates) such that if $B_{i+1}(x_1, x_2, \dots, x_{i+1})$ is false for a path $(x_1, x_2, \dots, x_{i+1})$ from the root node to a problem state, then the path cannot be extended to reach an answer node. Thus the candidates for position $i+1$ of the solution vector $(x_1, \dots, x_n)$ are those values which are generated by T and satisfy B_{i+1} . Algorithm 6.1 presents a recursive formulation of the backtracking technique. It is natural to describe backtracking in this way since it is essentially a postorder traversal of a tree. This recursive version is initially invoked by

Backtrack(1);

- 1 Algorithm Backtrack(k)
- 2 // This schema describes the backtracking process using
- 3 // recursion. On entering, the first $k-1$ values
- 4 // $x[1], x[2], \dots, x[k-1]$ of the solution vector
- 5 // $x[1:n]$ have been assigned. $x[]$ and n are global.
- 6 {

```

7      for (each  $x[k]$  (  $T(x[1], \dots, x[k-1])$ ) do
8      {
9          if ( $B_k(x[1], x[2], \dots, x[k]) \neq 0$ ) then
10         {
11             if ( $x[1], x[2], \dots, x[k]$  is a path to an answer
                node)
12                 then write  $x[1:k]$ );
13             if ( $k < n$ ) then Backtrack( $k+1$ );
14         }
15     }
16 }

```

Algorithm 6.1 Recursive backtracking algorithm

The solution vector $(x_1, \dots, x_n)$ is treated as a global array $x[1:n]$. All the possible elements for the k th position of the tuple that satisfy B_k are generated, one by one, and adjoined to the current vector $(x_1, \dots, x_{k-1})$. Each time x_k is attached, a check is made to determine whether a solution has been found. Then the algorithm is recursively invoked. When the for loop of line 7 is exited, no more values for x_k exist and the current copy of Backtrack ends. The last unresolved call now resumes, namely, the one that continues to examine the remaining elements assuming only $k-2$ values have been set.

Note that this algorithm causes all solutions to be printed and assumes that tuples of various sizes may make up a solution. If only a single solution is desired, then a flag can be added as a parameter to indicate the first occurrence of success.

```

1  Algorithm | Backtrack(n)
2  // This schema describes the backtracking process.
3  // All solutions are generated in  $x[1:n]$  and printed
4  // as soon as they are determined.
5  {
6       $k:=1$ ;
7      while ( $k \neq 0$ ) do
8          {
9              if (there remains an untried  $x[k] \in T(x[1], x[2], \dots,$ 
10              $x[k-1])$  and  $B_k(x[1], \dots, x[k])$  is true) then
11                  {
12                      if ( $x[1], \dots, x[k]$  is a path to an answer node)
13                          then write ( $x[1:k]$ );
14                       $k:=k+1$ ; // Consider the next set.
15                  }

```

```

16         else k:=k-1; // Backtrack to the previous set.
17     }
18 }

```

Algorithm 6.2 General iterative backtracking method.

An iterative version of Algorithm 6.1 appears in Algorithm 6.2. Note that $T()$ will yield the set of all possible values that can be placed as the first component x_1 of the solution vector. The component x_1 will take on those values for which the bounding function $B_1(x_1)$ is true. Also note how the elements are generated in a depth first manner. The variable k is continually incremented and a solution vector is grown until either a solution is found or no untried value of x_k remains. When k is decrement, the algorithm must resume the generation of possible elements for the k th position that have not yet been tried. Therefore one must develop a procedure that generates these values in some order. If only one solution is desired, replacing `write (x[1:k]);` with `{ write (x[1:k]); return; }` suffices.

The efficiency of both the backtracking algorithms we've just seen depends very much on four factors: (1) the time to generate the next x_k , (2) the number of x_k satisfying the explicit constraints, (3) the time for the bounding functions B_k , and (4) the number of x_k satisfying the B_k . Bounding functions are regarded as good if they substantially reduce the number of nodes that are generated. However, there is usually a trade-off in that bounding functions that are good also take more time to evaluate. What is desired is a reduction in the overall computing time and not just a reduction in the number of nodes generated. For many problems,

the size of the state space tree is too large to permit the generation of all nodes. Bounding functions must be used and hopefully at least one solution is found in a reasonable time span. Yet for many problems (e.g., n-queens) no sophisticated bounding methods are known.

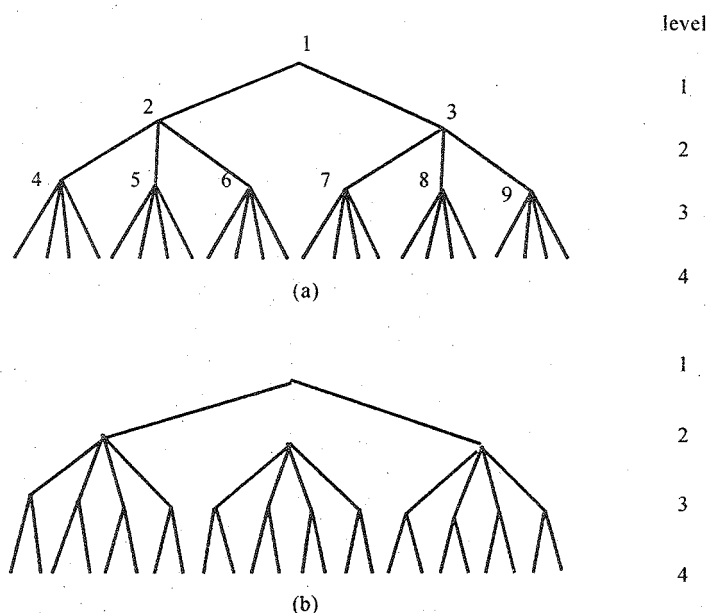


Figure 6.7 Rearrangement

One general principle of efficient searching is called rearrangement. For many problems the sets S_i can be taken in any order. This suggests that all other being equal, it is more efficient to make the next choice from the set with the fewest elements. This strategy doesn't pay off for the n-queens problem, and examples can be constructed that prove this principle won't always work. The potential value of this heuristic is exhibited in Figure 6.7 by the two backtracking search trees for the same problem.

If we are able to remove a node on level two of Figure 6.7(a), then we are effectively removing 12 possible 4-tuples from consideration. If we remove a node from level two of the tree in Figure 6.7(b), then only eight tuples are eliminated. More sophisticated rearrangement strategies are studied in conjunction with dynamic state space trees.

As stated previously, there are four factors that determine the time required by a backtracking algorithm. Once a state space tree organization, is selected, the first three of these are relatively independent of the problem instance being solved. Only the fourth, the number of nodes generated, varies from one problem instance to another. A backtracking algorithm on one problem instance might generate only $O(n)$ nodes whereas on a different (and even closely related) instance it might generate almost all the nodes in the state space tree. If the number of nodes in the solution space is 2^n or $n!$ the worst-case time for a backtracking algorithm will generally be $O(p(n)2^n)$ or $O(q(n)n!)$ respectively. The $p(n)$ and $q(n)$ are polynomials in n . The importance of backtracking lies in its ability to solve some instances with large n in a very small amount of time. The only difficulty is in predicting the behavior of a backtracking algorithm for the problem instance we wish to solve.

We can estimate the number of nodes that will be generated by a backtracking algorithm working on a certain instance by using Monte Carlo methods. The general idea in the estimation method is to generate a random path in the state space tree. Let X be a node on this random path. Assume that X is at level i of the state space tree. The bounding functions are used at node X to determine the number m_i of its children that do not get bounded.

The next node on the path is obtained by randomly selecting one of these m_1 children that do not get bounded. The path generation terminates at a node which is a leaf or all of whose children get bounded. Using these m_1 's we can estimate the total number m of nodes in the state space tree that will not get bounded. This number is particularly useful when all answer nodes are to be searched for. In this case all unbounded nodes need to be generated. When only a single solution is desired, m may not be such a good estimate for the number of nodes generated as the backtracking algorithm may arrive at a solution by generating only a small fraction of the m nodes. To estimate m from the m_1 's, we need to make an assumption on the bounding functions. We shall assume that these functions are static; that is, the backtracking algorithm does not change its bounding functions as it gathers information during its execution. Moreover, exactly the same function is used for all nodes on the same level of the state space tree. This assumption is not true of most backtracking algorithms. In most cases the bounding functions get stronger as the search proceeds. In these cases our estimate for m is higher than one that could be obtained if the change in the bounding functions were taken into consideration.

Continuing with the assumption of static bounding functions, we see that the number of unbounded nodes on level 2 is m_1 . If the search tree is such that nodes on the same level have the same degree, then we would expect each level 2 node to have on the average m_2 unbounded children. This yields a total of $m_1 m_2$ nodes on level 3. The expected number of unbounded nodes on level 4 is $m_1 m_2 m_3$. In general, the expected number of nodes on level $i + 1$ is $m_1 m_2 \dots m_i$. Hence, the estimated number m of unbounded

nodes that will be generated in solving the given problem instance I is $m = 1 + m_1 + m_1 m_2 + m_1 m_2 m_3 + \dots$

Function Estimate (Algorithm 6.3) determines the value m . It selects a random path from the root of the state space tree. The function Size returns the size of the set T_k . The function Choose makes a random choice of an element in T_k . The desired sum is built using the variables m and r .

```

1  Algorithm Estimate()
2  // This algorithm follows a random path
3  // in a state space tree and produces an
4  // estimate of the number of nodes in the tree.
5  {
6       $k:=1; m:=1; r:=1;$ 
7      repeat
8      {
9           $T_k := \{x[k] \mid x[k] \in T(x[1], x[2], \dots, x[k-1])$ 
10             and  $B_k(x[1], \dots, x[k])$  is true $\}$ ;
11             if ( $\text{Size}(T_k) = 0$ ) then return  $m$ ;
12              $r := r * \text{Size}(T_k); m := m + r;$ 
13              $x[k] := \text{Choose}(T_k); k := k + 1;$ 

```

```

14         } until (false);
15     }

```

Algorithm 6.3 Estimating the efficiency of backtracking

We use this estimator in later sections (such as Section 6.2) as we examine backtracking solutions to various problems. A better estimate of the number of unbounded nodes that will be generated by a backtracking algorithm can be obtained by selecting several different random paths (typically no more than 20) and determining the average of these values.

6.2 SUM OF SUBSETS

Suppose we are given n distinct positive numbers (usually called weights) and we desire to find all combinations of these numbers whose sums are m . This is called the sum of subsets problem. Examples 6.2 and 6.4 showed how we could formulate this problem using either fixed or variable-sized tuples. We consider a backtracking solution using the fixed tuple size strategy. In this case the element x_i of the solution vector is either one or zero depending on whether the weight w_i is included or not.

The children of any node in Figure 6.4 are easily generated. For a node at level i the left child corresponds to $x_i=1$ and the right to $x_i=0$.

A simple choice for the bounding functions is $B_k(x_1, \dots, x_k)$
 $= \text{true}$ iff

$$\sum_{i=1}^k w_i x_i + \sum_{i=k+1}^k w_i \geq m$$

Clearly $x_1, \dots, x_k$ cannot lead to an answer node if this condition is not satisfied. The bounding functions can be strengthened if we assume the w_i 's are initially in nondecreasing order. In this case $x_1, \dots, x_k$ cannot lead to an answer node if

$$\sum_{i=1}^k w_i x_i + w_{k+1} > m$$

The bounding functions we use are therefore

$$B_k(x_1, \dots, x_k) = \text{true if } \sum_{i=1}^k w_i x_i + \sum_{i=k+1}^k w_i \geq x_i$$

$$\text{and } \sum_{i=1}^k w_i x_i + w_{k+1} \leq m \quad (6.1)$$

Since our algorithm will not make use of B_n , we need not be concerned by the appearance of $w_n + 1$ in this function. Although we have now specified all that is needed to directly use either of the backtracking schemas, a simpler algorithm results if we tailor either of these schemas to the problem at hand. This simplification results from the realization that if $x_k = 1$, then

$$\sum_{i=1}^k w_i x_i + \sum_{i=k+1}^k w_i > m$$

For simplicity we refine the recursive schema. The resulting algorithm is Sum of Sub (Algorithm 6.6).

Algorithm SumOfSub avoids computing $\sum_{i=1}^k w_i x_i$ and

$\sum_{i=k+1}^k w_i$ each time by keeping these values in variables s and r respectively. The algorithm assumes $w_1 \leq m$ and

$\sum_{i=1}^k w_i \geq m$. The initial call is SumOfSub(0,1, $\sum_{i=1}^n w_i$). It is

interesting to note that the $i=1$ algorithm does not explicitly use the test $k > n$ to terminate the recursion. This test is not needed as on entry to the algorithm, $s \neq m$ and $s + r \geq m$. Hence, $r \neq 0$ and so k can be no greater than n . Also note that in the else if statement (line 11), since $s + w_k < m$ and $s + r \geq m$, it follows that $r \neq w_k$ and hence $k+1 \leq n$. Observe also that if $s + w_k = m$ (line 9), then $x_{k+1}, \dots, x_n$ must be zero. These zeros are omitted from the output

of line 9. In line 11 we do not test for $\sum_{i=1}^k w_i x_i + \sum_{i=k+1}^n w_i \geq m$, as

we already know $s+r \geq m$ and $x_k=1$

Example 6.6 Figure 6.8 shows the portion of the state space tree generated by function SumOfSub while working on the instance $n=6$, $m=30$, and $w[1:6] = \{5,10,12,13,15,18\}$. The rectangular nodes list the values of s, k , and r on each of the calls to SumOfSub. Circular nodes represent points at which subsets with sums m are printed out. At nodes A, B, and C the output is respectively $(1,1,0,0,1)$, $(1,0,1,1)$, and $(0,0,1,0,0,1)$. Note that the tree of Figure 6.10 contains only 23 rectangular

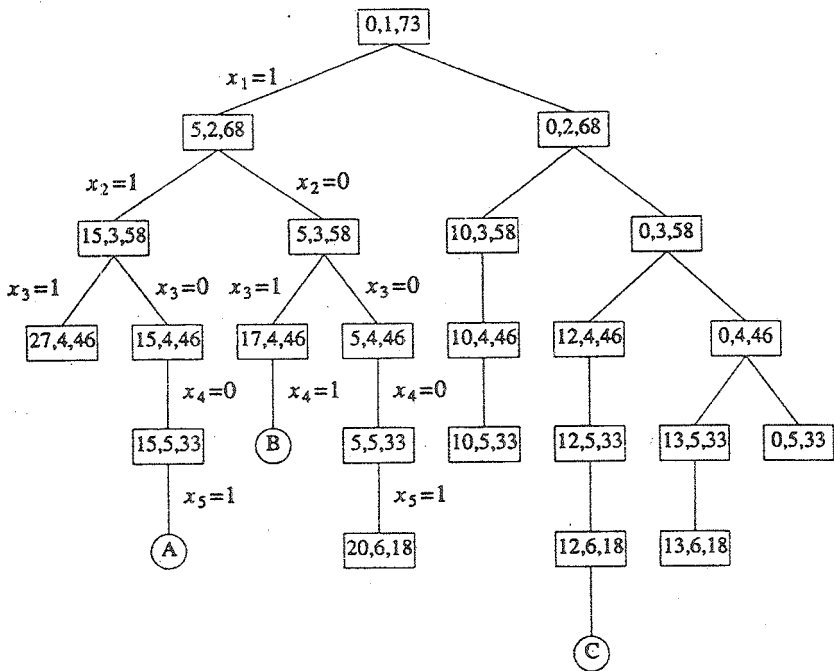


Figure 6.8 Portion of state space tree generated by SumOfSub

nodes. The full state space tree for $n=6$ contains $2^6 - 1 = 63$ nodes from which calls could be made (this count excludes the 64 leaf nodes as no call need be made from a leaf).

- ```

1 Algorithm SumOfSub(s,k,r)
2 // Find all subsets of w[1:n] that sum to m. The values of
 x[j],
3 // $1 \leq j < k$, have already been determined. $s = \sum_{j=1}^{k-1} w[j]$
 * x[j]

```

```

4 // and $r = \sum_{j=k}^n w[j]$. The $w[j]$'s are in nondecreasing order.
5 // It is assumed that $w[1] \leq m$ and $\sum_{i=1}^n w[i] \geq m$.
6 {
7 // Generate left child. Note: $s + w[k] \leq m$ since B_{k-1} is true

8 $x[k] := 1$;
9 if ($s + w[k] = m$) then write ($x[1:k]$); // Subset found
10 // There is no recursive call here as $w[j] > 0, 1 \leq j \leq n$.
11 else if ($s + w[k] + w[k + 1] \leq m$)
12 then SumOfSub ($s + w[k], k+1, r - w[k]$);
13 //Generate right child and evaluate B_k .
14 if ($((s+r-w[k] \geq m)$ and $(s + w[k + 1] \leq m)$) then
15 {
16 $x[k] := 0$;
17 SumOfSub($s, k+1, r-w[k]$);
18 }
19 }
```

Algorithm 6.6 Recursive backtracking algorithm for sum of subsets problem

### 6.3 GRAPH COLORING

Let  $G$  be a graph and  $m$  be a given positive integer. We want to discover whether the nodes of  $G$  can be coloured in such a way that no two adjacent nodes have the same color yet only  $m$  colors are used. This is termed the  $m$ -colorability decision problem. Note that if  $d$  is the degree of the given graph, then it can be colored with  $d+1$  colours. The  $m$ -colorability optimization problem asks for the smallest integer  $m$  for which the graph  $G$  can be colored. This integer is referred to as the chromatic number of the graph. For example, the graph of Figure 6.9 can be colored with three colors 1, 2, and 3. The color of each node is indicated next to it. It can also be seen that three colors are needed to color this graph and hence this graph's chromatic number is 3.

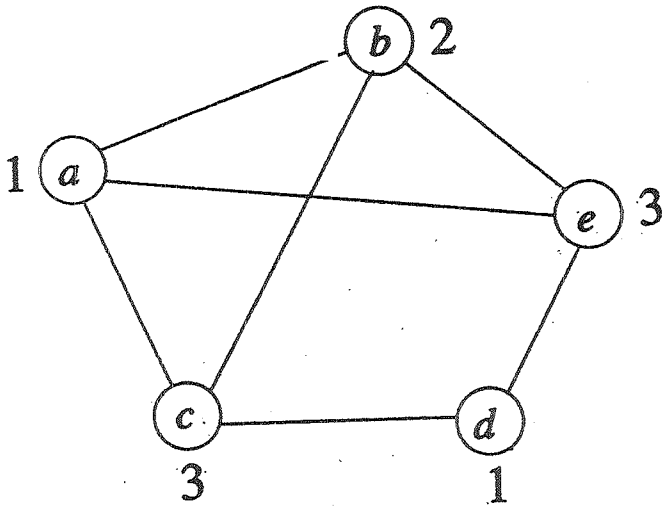


Figure 6.9 An example graph and its coloring



A graph is said to be planar iff it can be drawn in a plane in such a way that no two edges cross each other. A famous special case of the  $m$ -colorability decision problem is the 4-color problem for planar graphs. This problem asks the following question: given any map, can the regions be colored in such a way that no two adjacent regions have the same color yet only four colors are needed? This turns out to be a problem for which graphs are very useful, because a map can easily be transformed into a graph. Each region of the map becomes a node, and if two regions are adjacent, then the corresponding nodes are joined by an edge. Figure 6.10 shows a map with five regions and its corresponding graph. This map requires four colors. For many years it was known that five colors were sufficient to color any map, but no map that required more than four colors had ever been found. After several hundred years, this problem was solved by a group of mathematicians with the help of a computer. They showed that in fact four colors are sufficient. In this section we consider not only graphs that are produced from maps but all graphs. We are interested in determining all the different ways in which a given graph can be colored using at most  $m$  colors.

Suppose we represent a graph by its adjacency matrix  $G[1:n, 1:n]$ , where  $G[i, j] = 1$  if  $(i, j)$  is an edge of  $G$ ,  $G[i, j] = 0$  otherwise. The colors are represented by the integers  $1, 2, \dots, m$  and the solutions are given by the  $n$ -tuple  $(x_1, \dots, x_n)$ , where  $x_i$  is the color of node  $i$ . Using the recursive backtracking formulation as given in Algorithm 6.1, the resulting algorithm is  $m$ Coloring (Algorithm 6.7). The underlying state space tree used is a tree of degree  $m$  and height  $n+1$ . Each node at level  $i$  has  $m$  children corresponding to the  $m$  possible assignments to  $x_i$ ,  $1 \leq i \leq n$ .

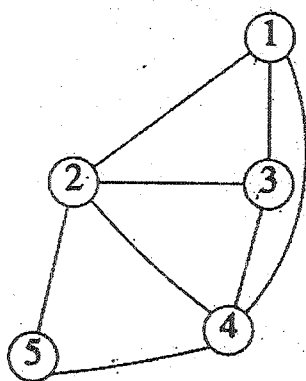
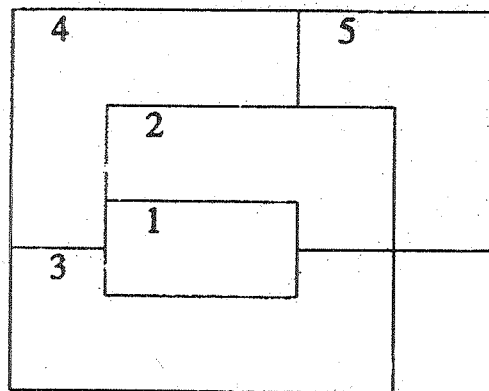


Figure 6.10 A map and its planar graph representation

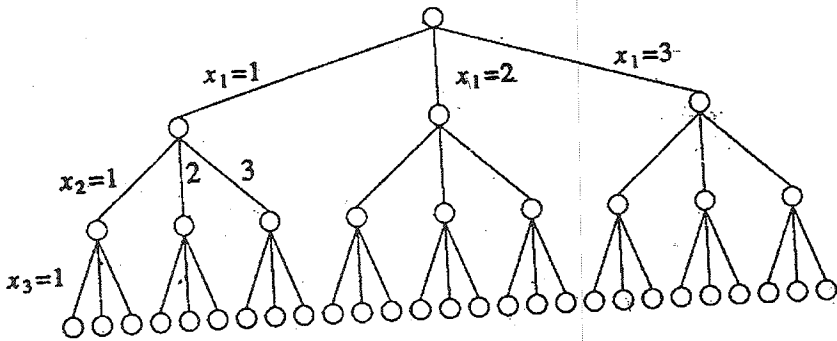


Figure 6.11 State space tree for mColoring  
when  $n=3$  and  $m=3$

Nodes at level  $n+1$  are leaf nodes. Figure 6.11 shows the state space tree when  $n=3$  and  $m=3$ .

Function mColoring is begun by first assigning the graph to its adjacency matrix, setting the array  $x[]$  to zero, and then invoking the statement mColoring(1);

Notice the similarity between this algorithm and the general form of the recursive backtracking schema of Algorithm 6.1. Function NextValue (Algorithm 6.8) produces the possible colors for  $x_k$  after  $x_1$  through  $x_{k-1}$  have been defined. The main loop of mColoring repeatedly picks an element from the set of possibilities, assigns it to  $x_k$ , and then calls mColoring recursively. For instance, Figure 6.12 shows a simple graph containing four nodes. Below that is the tree that is generated by mColoring. Each path to a leaf represents a coloring using at most three colors. Note that only 12 solutions exist with exactly three colors. In this tree, after

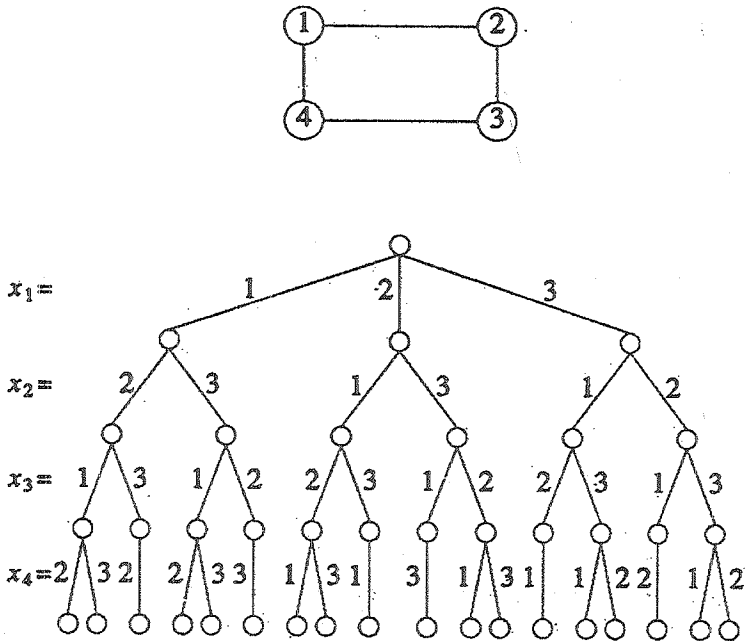


Figure 6.12 A 4-node graph and all possible 3-colorings

choosing  $x_1=2$  and  $x_2=1$ , the possible choices for  $x_3$  are 2 and 3. After choosing  $x_1=2$ ,  $x_2=1$ , and  $x_3=2$ , possible values for  $x_4$  are 1 and 3. And so on.

An upper bound on the computing time of mColoring can be arrived at by noticing that the number of internal nodes in the state space tree is  $\sum_{i=0}^{n-1} m^i$ . At each internal node,  $O(mn)$  time

is spent by NextValue to determine the children corresponding to legal colorings. Hence the total time is bounded by  $\sum_{i=0}^{n-1} m^{i+1}$   
 $n = \sum_{i=1}^n m^i n = n(m^{n+1} - 2)/(m - 1) = O(nm^n)$ .

```

1 Algorithm mColoring(k)

2 //This algorithm was formed using the recursive
 backtracking

3 //schema. The graph is represented by its boolean
 adjacency

4 //matrix G[1:n,1:n]. All assignments of 1,2,...,m to the

5 // vertices of the graph such that adjacent vertices are

6 // assigned distinct integers are printed. k is the index

7 // of the next vertex to color.

8 {

9 repeat

10 { // Generate all legal assignments for x[k].

11 NextValue(k); //Assign to x[k] a legal color.

12 if (x[k] = 0) then return; // No new color possible

```

```

13 if (k=n) then // At most m colors have been
14 // used to color the n vertices.
15 write (x[1:n]);
16 else mColoring (k+1);
17 } until (false);
18 }

```

Algorithm 6.7 Finding all m-colorings of a graph

```

1 Algorithm NextValue(k)
2 // x[1],...,x[k-1] have been assigned integer values in
3 // the range [1,m] such that adjacent vertices have distinct
4 // integers. A value for x[k] is determined in the range
5 [0,m]. x[k] is assigned the next highest numbered color
6 //while maintaining distinctness from the adjacent vertices
7 // of vertex k. If no such color exists, then x[k] is 0.
8 {
9 repeat

```

```

10 {
11 $x[k] := (x[k] + 1) \bmod (m+1)$; // Next highest color.
12 if ($x[k] = 0$) then return; // All colors have been
 used.
13 for j:=1 to n do
14 { // Check if this color is
15 // distinct from adjacent colors.
16 if ($(G[k,j] \neq 0) \text{ and } (x[k] = x[j])$)
17 // If (k,j) is an edge and if adj.
18 // vertices have the same color.
19 then break;
20 }
21 if ($j = n+1$) then return; // New color found
22 } until (false); // Otherwise try to find another color.
23 }
```

Algorithm 6.8 Generating a next color

**Unit - IV****Lesson - 7****BRANCH-AND-BOUND****OBJECTIVES**

After studying this lesson you can able to understand the following

- General Method of Branch and Bound
  - LC Search
  - The 15 puzzle : An example
  - Control Abstraction for LC - Search
  - Bounding
  - FIFO Branch - and - Bound
  - LC Branch - and - Bound
- 0/1 Knapsack Problem
  - LC Branch - and - Bound Solution
  - FIFO Branch - and - Bound Solution



## 7.1 GENERAL METHOD :

The term branch-and-bound refers to all state space search methods in which all children of the E-node are generated before any other live node can become the E-node. We have already seen two graph search strategies, BFS and D-search, in which the exploration of a new node cannot begin until the node currently being explored is fully explored. Both of these generalize to branch-and-bound strategies. In branch-and-bound terminology, a BFS-like state space search will be called FIFO (First In First Out) search as the list of live nodes is a first-in-first-out list (or queue). A D-search-like state space search will be called LIFO (Last In First Out) search as the list of live nodes is a last-in-first-out list (or stack). As in the case of backtracking, bounding functions are used to help avoid the generation of subtrees that do not contain an answer node.

**Example 7.1 [4-queens]** Let us see how a FIFO branch-and-bound algorithm would search the state space tree (Figure 7.2) for the 4-queens problem. Initially, there is only one live node, node 1. This represents the case in which no queen has been placed on the chessboard. This node becomes the E-node. It is expanded and its children, nodes 2, 18, 34 and 50, are generated. These nodes represent a chessboard with queen 1 in row 1 and columns 1, 2, 3 and 4 respectively. The only live nodes now are nodes 2, 18, 34 and 50. If the nodes are generated in this order, then the next E-node is node 2. It is expanded and nodes 3, 8, and 13 are generated. Node 3 is immediately killed using the bounding function. Nodes 8 and 13 are added to the queue of live nodes. Node 18 becomes the next E-node. Nodes 19, 24 and 29 are generated. Nodes 19 and 24 are killed as a result of

the bounding functions. Node 29 is added to the queue of live nodes. The E-node is node 34. Figure 7.1 shows the portion of the tree that is generated by a FIFO branch-and-bound search. Nodes that are killed as a result of the bounding functions have a "B" under them. Numbers inside the nodes correspond to the numbers in Figure 7.2. Numbers outside the nodes give the order in which the nodes are generated by FIFO branch-and-bound. At the time the answer node, node 31, is reached, the only live nodes remaining are nodes 38 and 54. A comparison of Figure 7.6 and 7.1 indicates that backtracking is a superior search method for this problem.

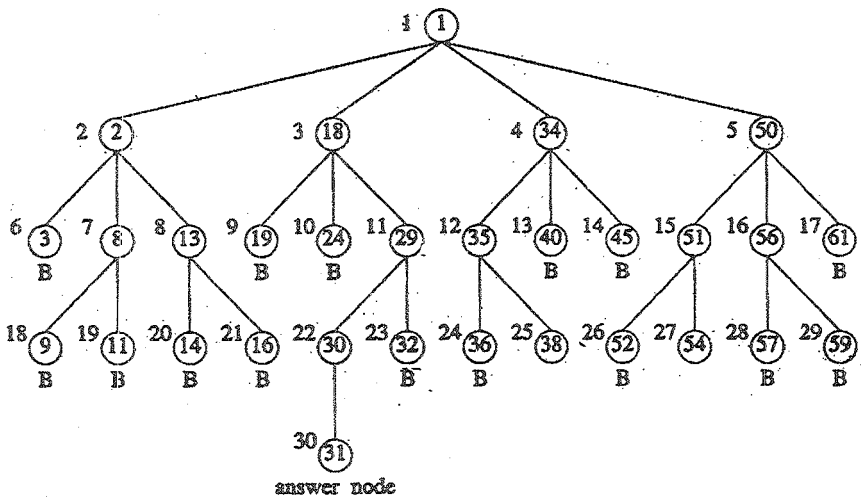


Figure 7.1 Portion of 4-queens state space tree generated by FIFO branch-and-bound

### 7.1.1 Least Cost (LC) Search

In both LIFO and FIFO branch-and-bound the selection rule for the next E-node is rather rigid and in a sense blind. The selection rule for the next E-node does not give any preference to a node that has a very good chance of getting the search to an answer node quickly. Thus, in Example 7.1, when node 30 is generated, it should have become obvious to the search algorithm that this node will lead to an answer node in one move. However, the rigid FIFO rule first requires the expansion of all live nodes generated before node 30 was expanded.

The search for an answer node can often be speeded by using an “intelligent” ranking function  $\hat{C}(\cdot)$  for live nodes. The next E-node is selected on the basis of this ranking function. If in the 4-queens example we use a ranking function that assigns node 30 a better rank than all other live nodes, then node 30 will become the E-node following node 29. The remaining live nodes will never become E-nodes as the expansion of node 30 results in the generation of an answer node (node 31).

The ideal way to assign ranks would be on the basis of the additional computational effort (or cost) needed to reach an answer node from the live node. For any node  $x$ , this cost could be (1) the number of nodes in the subtree  $x$  that need to be generated before an answer node is generated or, more simply, (2) the number of levels the nearest answer node (in the subtree  $x$ ) is from  $x$ . Using cost measure 2, the cost of the root of the tree of Figure 7.1 is 4 (node 31 is four levels from node 1). The costs of nodes 18 and 34, 29 and 35, 38 are respectively 3, 2 and 1. The costs of all remaining nodes on levels 2,3,4 are respectively

greater than 3, 2 and 1. Using these costs as the basis to select the next  $\epsilon$ - node, the  $\epsilon$  nodes are nodes 1, 18, 29 and 30 (in that order). The only other nodes to get generated are nodes 2, 34, 50, 19, 24, 32, and 31. It should be easy to see that if cost measure 1 is used, then the search would always generate the minimum number of nodes every branch-and-bound type algorithm must generate. If cost measure 2 is used, then the only nodes to become E-nodes are the nodes on the path from the root to the nearest answer node. The difficulty with using either of these ideal cost functions is that computing the cost of a node usually involves a search of the subtree  $x$  for an answer node. Hence, by the time the cost of a node is determined, that subtree has been searched and there is no need to explore  $x$  again. For this reason, search algorithms usually rank nodes only on the basis of an estimate  $\hat{g}(\cdot)$  of their cost.

Let  $\hat{g}(x)$  be an estimate of the additional effort needed to reach an answer node from  $x$ . Node  $x$  is assigned a rank using a function  $\hat{c}(\cdot)$  such that  $\hat{c}(x) = f(h(x)) + \hat{g}(x)$ , where  $h(x)$  is the cost of reaching  $x$  from the root and  $f(\cdot)$  is any nondecreasing function. At first, we may doubt the usefulness of using an  $f(\cdot)$  other than  $f(h(x)) = 0$  for all  $h(x)$ . We can justify such an  $f(\cdot)$  on the grounds that the effort already expended in reaching the live nodes cannot be reduced and all we are concerned with now is minimizing the additional effort we spend to find an answer node. Hence, the effort already expended need not be considered.

Using  $f(\cdot) \equiv 0$  usually biases the search algorithm to make deep probes into the search tree. To see this, note that we would normally expect  $\hat{g}(y) \leq \hat{g}(x)$  for  $y$ , a child of  $x$ . Hence, following

$x$ ,  $y$  will become the E-node, then one of  $y$ 's children will become the E-node, next one of  $y$ 's grandchildren will become the E-node, and so on. Nodes in subtree other than the subtree  $x$  will not get generated until the subtree  $x$  is fully searched. This would not be a cause of concern if  $\hat{g}(x)$  were the true cost of  $x$ . Then, we would not wish to explore the remaining subtrees in any case (as  $x$  is guaranteed to get us to an answer node quicker than any other existing live node). However,  $\hat{g}(x)$  is only an estimate of the true cost. So, it is quite possible that for two nodes  $w$  and  $z$ ,  $\hat{g}(w) < \hat{g}(z)$  and  $z$  is much closer to an answer node than  $w$ . It is therefore desirable not to overbias the search algorithm in favour of deep probes. By using  $f(.) \neq 0$ , we can force the search algorithm to favour a node  $z$  close to the root over a node  $w$  which is many levels below  $z$ . This would reduce the possibility of deep and fruitless searches into the tree.

A search strategy that uses a cost function  $\hat{c}(x) = f(h(x)) + \hat{g}(x)$  to select the next E-node would always choose for its next E-node a live node with least  $\hat{c}(.)$ . Hence, such a search strategy is called an LC-search (Least Cost search). It is interesting to note that BFS and D-search are special cases of LC-search. If we use  $\hat{g}(x) \equiv 0$  and  $f(h(x)) = \text{level of node } x$ , then a LC-search generates nodes by levels. This is essentially the same as a BFS. If  $f(h(x)) \equiv 0$  and  $\hat{g}(x) \geq \hat{g}(y)$  whenever  $y$  is a child of  $x$ , then the search is essentially a D-search. An LC-search coupled with bounding functions is called an LC branch-and-bound search.

E 13

In discussing LC-searches, we sometimes make reference to a cost function  $c(.)$  defined as follows: if  $x$  is an answer node,

then  $c(x)$  is the cost (level computational difficulty, etc) of reaching  $x$  from the root of the state space tree. If  $x$  is not an answer node, then  $c(x)=\infty$  providing the subtree  $x$  contains no answer node; otherwise  $c(x)$  equals the cost of a minimum-cost answer node in the subtree  $x$ . It should be easy to see that  $\hat{c}(\cdot)$  with  $f(h(x)) = h(x)$  is an approximation to  $c(\cdot)$ . From now on  $\hat{c}(x)$  is referred to as the cost of  $x$ .

### 7.1.2 The 15-puzzle: An Example

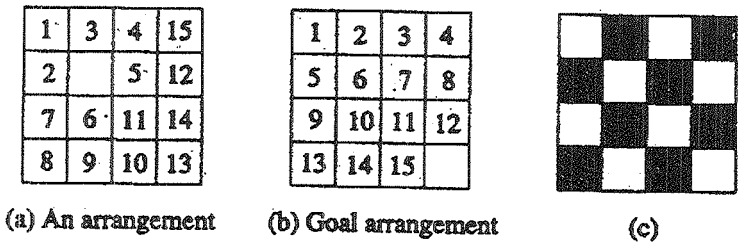


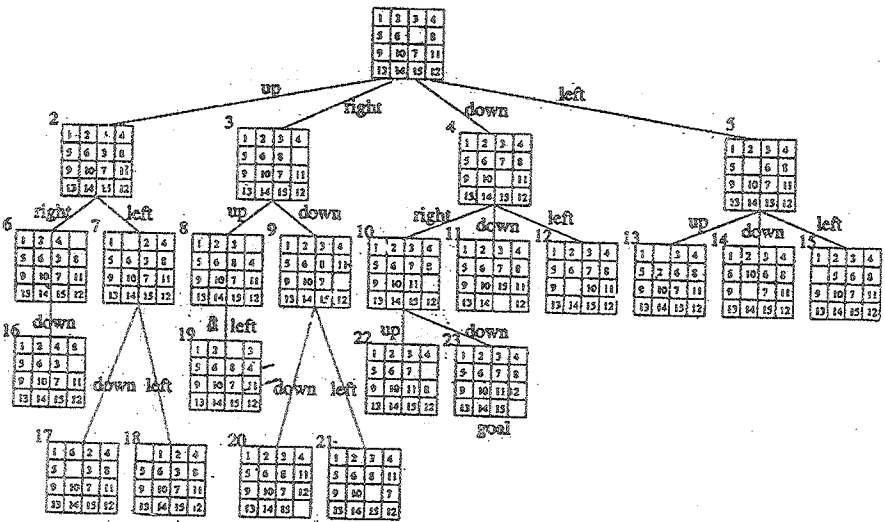
Figure 7.2 15-puzzle arrangements

The 15-puzzle (invented by Sam Loyd in 1878) consists of 15 numbered tiles on a square frame with a capacity of 16 tiles (Figure 7.2). We are given an initial arrangement of the tiles, and the objective is to transform this arrangement into the goal arrangement of Figure 7.2(b) through a series of legal moves. The only legal moves are ones in which a tile adjacent to the empty spot (ES) is moved to ES. Thus from the initial arrangement of Figure 7.2(a), four moves are possible. We can move any one of the tiles numbered 2, 3, 5, or 6 to the empty spot. Following this move, other moves can be made. Each move creates a new

arrangement of the tiles. These arrangements are called the states of the puzzle. The initial and goal arrangements are called the initial and goal states. A state is reachable from the initial state iff there is a sequence of legal moves from the initial state to this state. The state space of an initial state consists of all states that can be reached from the initial state. The most straightforward way to solve the puzzle would be to search the state space for the goal state and use the path from the initial state to the goal state as the answer. It is easy to see that there are  $16!$  ( $16! \approx 20.9 \times 10^{12}$ ) different arrangements of the tiles on the frame. Of these only one-half are reachable from any given initial state. Indeed, the state space for the problem is very large. Before attempting to search this state space for the goal state, it would be worthwhile to determine whether the goal state is reachable from the initial state. There is a very simple way to do this. Let us number the frame positions 1 to 16. Position  $i$  is the frame position containing tile numbered  $i$  in the goal arrangement of Figure 7.2(b). Position 16 is the empty spot. Let  $\text{position}(i)$  be the position number in the initial state of the tile numbered  $i$ . Then  $\text{position}(16)$  will denote the position of the empty spot.

For any state let  $\text{less}(i)$  be the number of tiles  $j$  such that  $j < i$  and  $\text{position}(j) > \text{position}(i)$ . For the state of Figure 7.2(a) we have, for example,  $\text{less}(1) = 0$ ,  $\text{less}(4) = 1$ , and  $\text{less}(12) = 6$ . Let  $x = 1$  if in the initial state the empty spot is at one of the shaded positions of Figure 7.2(c) and  $x = 0$  if it is at one of the remaining positions. Then, we have the following theorem:

**Theorem 7.1** The goal state of Figure 7.2(b) is reachable from the initial state iff  $\sum_{i=1}^{16} \text{less}(i) + x$  is even.



Edges are labeled according to the direction  
in which the empty space moves

Figure 7.3 Part of the state space tree for the 15-puzzle

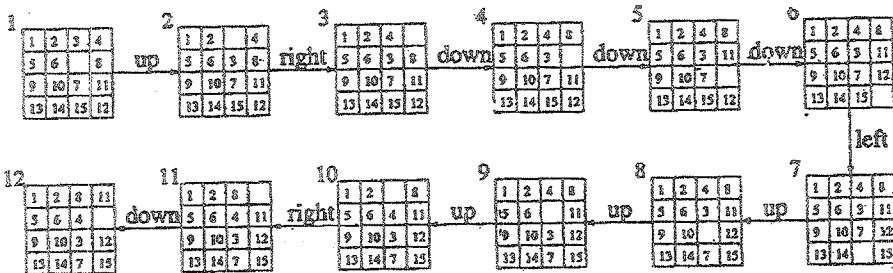


Figure 7.4 First ten steps in a depth first search



Proof : Left as an exercise.

Theorem 7.1 can be used to determine whether the goal state is in the state space of the initial state. If it is, then we can proceed to determine a sequence of moves leading to the goal state. To carry out this search, the state space can be organized into tree. The children of each node  $x$  in this tree represent the states reachable from state  $x$  by one legal move. It is convenient to think of a move as involving a move of the empty space rather than a move of a tile. The empty space, on each move, moves either up, right, down, or left. Figure 7.3 shows the first three levels of the state space tree of the 15-puzzle beginning with the initial state shown in the root. Parts of levels 4 and 5 of the tree are also shown. The tree has been pruned a little. No node  $p$  has a child state that is the same as  $p$ 's parent. The subtree eliminated in this way is already present in the tree and has root  $\text{parent}(p)$ . As can be seen, there is an answer node at level 4.

A depth first state space tree generation will result in the subtree of Figure 7.4 when the next moves are attempted in the order: move the empty space up, right, down and left. Successive board configurations reveal that each move gets us farther from the goal rather than closer. The search of the state space tree is blind. It will take the leftmost path from the root regardless of the starting configuration. As a result, an answer node may never be found (unless the leftmost path ends in such a node). In a FIFO search of the tree of Figure 7.3, the nodes will be generated in the order numbered. A breadth first search will always find a goal node nearest to the root. However, such a search is also blind in the sense that no matter what the initial configuration, the algorithm attempts to make the same sequence of moves. A FIFO search always generates the state space tree by levels.

What we would like, is a more “intelligent” search method, one that seeks out an answer node and adapts the path it takes through the state space tree to the specific problem instance being solved. We can associate a cost  $c(x)$  with each node  $x$  in the state space tree. The cost  $c(x)$  is the length of a path from the root to a nearest goal node (if any) in the subtree with root  $x$ . Thus, in Figure 7.3,  $c(1) = c(4) = c(10) = c(23) = 3$ . When such a cost function is available, a very efficient search can be carried out. We begin with the root as the E-node and generate a child node with  $c()$ -value the same as the root. Thus children nodes 2, 3 and 5 are eliminated and only node 4 becomes a live node. This becomes the next E-node. Its first child, node 10 has  $c(10) = c(4) = 3$ . The remaining children are not generated. Node 4 dies and node 10 becomes the E-node. In generating node 10’s children, node 22 is killed immediately as  $c(22) > 3$ . Node 23 is generated next. It is a goal node and the search terminates. In this search strategy, the only nodes to become E-nodes are nodes on the path from the root to a nearest goal node. Unfortunately, this is an impractical strategy as it is not possible to easily compute the function  $c(.)$  specified above.

We can arrive at an easy to compute estimate  $\hat{c}(x)$  of  $c(x)$ . We can write  $\hat{c}(x) = f(x) + \hat{g}(x)$ , where  $f(x)$  is the length of the path from the root to node  $x$  and  $\hat{g}(x)$  is an estimate of the length of a shortest path from  $x$  to a goal in the subtree with root  $x$ . One possible choice for  $\hat{g}(x)$  is

$$\hat{g}(x) = \text{number of nonblank tiles not in their goal position}$$

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  |    | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 7  |

Figure 7.5 Problem state

Clearly, at least  $\hat{g}(x)$  moves have to be made to transform state  $x$  to a goal state. More than  $\hat{g}(x)$  moves may be needed to achieve this. To see this, examine the problem state of Figure 7.5. There  $\hat{g}(x) = 1$  as only tile 7 is not in its final spot (the count for  $\hat{g}(x)$  excludes the blank tile). However, the number of moves needed to reach the goal state is many more than  $\hat{g}(x)$ . So  $\hat{c}(x)$  is a lower bound on the value of  $c(x)$ .

An LC-search of Figure 7.3 using  $\hat{c}(x)$  will begin by using node 1 as the E-node. All its children are generated. Node 1 dies and leaves behind the live nodes 2, 3, 4, and 5. The next node to become the E-node is a live node with least  $\hat{c}(x)$ . Then  $\hat{c}(2) = 1+4$ ,  $\hat{c}(3) = 1+4$ ,  $\hat{c}(4) = 1+2$ , and  $\hat{c}(5) = 1+4$ . Node 4 becomes the E-node. Its children are generated. The live nodes at this time are 2, 3, 5, 10, 11 and 12. So  $\hat{c}(10) = 2+1$ ,  $\hat{c}(11) = 2+3$ , and  $\hat{c}(12) = 2+3$ . The live node with least  $\hat{c}$  is node 10. This becomes the next E-node. Nodes 22 and 23 are generated next. Node 23 is determined to be a goal node and the search terminates. In this case LC-search was almost as efficient as using the exact function  $c()$ . It should be noted that with a suitable choice for  $\hat{c}()$ , and

LC-search will be far more selective than any of the other search methods we have discussed.

### 7.1.3 Control Abstractions for LC-Search

Let  $t$  be a state space tree and  $c()$  a cost function for the nodes in  $t$ . If  $x$  is a node in  $t$ , then  $c(x)$  is the minimum cost of any answer node in the subtree with root  $x$ . Thus,  $c(t)$  is the cost of a minimum-cost answer node in  $t$ . As remarked earlier, it is usually not possible to find an easily computable function  $c()$  as defined above. Instead, a heuristic  $\hat{c}$  that estimate  $c()$  is used. This heuristic should be easy to compute and generally has the property that if  $x$  is either an answer node or a leaf node, then  $c(x) = \hat{c}(x)$ . The algorithm uses two functions  $\text{Least}()$  and  $\text{Add}(x)$  to delete and add a live node from or to the list of live nodes, respectively.  $\text{Least}()$  finds a live node with least  $\hat{c}()$ . This node is deleted from the list of live nodes and returned.  $\text{Add}(x)$  adds the new live node  $x$  to the list of live nodes. The list of live nodes will usually be implemented as a min-heap. Algorithm  $\text{LCSearch}$  outputs the path from the answer node it finds to the root node  $t$ . This is easy to do if with each node  $x$  that becomes live, we associate a field  $\text{parent}$  which gives the parent of node  $x$ . When an answer node  $g$  is found, the path from  $g$  to  $t$  can be determined by following a sequence of parent values starting from the current E-node (which is the parent of  $g$ ) and ending at node  $t$ .

```
listnode = record {
```

```
 listnode *next, *parent; float cost;
```

```
}
```

```

1 Algorithm LCSearch(t)
2 // Search t for an answer node.
3 {
4 if *t is an answer node then output *t and return;
5 E:=t; //E-node
6 Initialize the list of live nodes to be empty;
7 repeat
8 {
9 for each child x of E do
10 {
11 if x is an answer node then output the path
12 from x to t and return;
13 Add(x); // x is a new live node.
14 (x→parent) :=E; // Pointer for path to root.
15 }
16 if there are no more live nodes then
17 {
18 write ("No answer node"); return;

```

```

19 }
20 E:=Least();
21 } until (false);
22 }
```

### Algorithm 7.1 LC-search

The correctness of algorithm LCSearch is easy to establish. Variable E always points to the current E-node. By the definition of LC-search, the root node is the first E-node (line 5). Line 6 initializes the list of live nodes. At any time during the execution of LCSearch, this list contains all live nodes except the E-node. Thus, initially this list should be empty (line 6). The for loop of line 9 examines all the children of the E-node. If one of the children is an answer node, then the algorithm outputs the path from x to t and terminates. If a child of E is not an answer node, then it becomes a live node. It is added to the list of live nodes (line 13) and its parent field set to E (line 14). When all the children of E have been generated, E-becomes a dead node and line 16 is reached. This happens only if none of E's children is an answer node. So, the search must continue further. If there are no live nodes left, then the entire state space tree has been searched and no answer nodes found. The algorithm terminates in line 18. Otherwise, Least(), by definition, correctly chooses the next E-node and the search continues from here.

From the preceding discussion, it is clear that LCSearch terminates only when either an answer node is found or the entire state space tree has been generated and searched. Thus,

termination is guaranteed only for finite state space trees. Termination can also be guaranteed for infinite state space trees that have at least one answer node provided a “proper” choice for the cost function  $\hat{c}()$  is made. This is the case, for example, when  $\hat{c}(x) > \hat{c}(y)$  for every pair of nodes  $x$  and  $y$  such that the level number of  $x$  is “sufficiently” higher than that of  $y$ . For infinite state space trees with no answer nodes, LCSearch will not terminate. Thus, it is advisable to restrict the search to find answer nodes with a cost no more than a given bound  $C$ .

One should note the similarity between algorithm LCSearch and algorithms for a breadth first search and D-search of a state space tree. If the list of live nodes is implemented as a queue with  $\text{Least}()$  and  $\text{Add}(x)$  being algorithms to delete an element from and add an element to the queue, then LCSearch will be transformed to a FIFO search schema. If the list of live nodes is implemented as a stack with  $\text{Least}()$  and  $\text{Add}(x)$  being algorithms to delete and add elements to the stack, then LCSearch will carry out a LIFO search are essentially the same. The only difference is in the implementation of the list of live nodes. This is to be expected as the three search methods differ only in the selection rule used to obtain the next E-node.

#### 7.1.4 Bounding

A branch-and-bound method searches a state space tree using any search mechanism in which all the children of the E-node are generated before another node becomes the E-node. We assume that each answer node  $x$  has a cost  $c(x)$  associated with it and that a minimum-cost answer node is to be found. Three common search strategies are FIFO, LIFO, and LC. (Another

method, heuristic search, is discussed in the exercises.) A cost function  $\hat{c}(\cdot)$  such that  $c(x) \leq \hat{c}(x)$  is used to provide lower bounds on solutions obtainable from any node  $x$ . If upper is an upper bound on the cost of a minimum-cost solution, then all live nodes  $x$  with  $\hat{c}(x) > \text{upper}$  may be killed as all answer nodes reachable from  $x$  have cost  $c(x) \geq \hat{c}(x) > \text{upper}$ . The starting value for upper can be obtained by some heuristic or can be set to  $\infty$ . Clearly, so long as the initial value for upper is no less than the cost of a minimum-cost answer node, the above rules to kill live nodes will not result in the killing of a live node that can reach a minimum-cost answer node. Each time a new answer node is found, the value of upper can be updated.

Let us see how these ideas can be used to arrive at branch-and-bound algorithms for optimization problems. In this section we deal directly only with minimization problems. A maximization problem is easily converted to a minimization problem by changing the sign of the objective function. We need to be able to formulate the search for an optimal solution as a search for a least-cost answer node in a state space tree. To do this, it is necessary to define the cost function  $c(\cdot)$  such that  $c(x)$  is minimum for all nodes representing an optimal solution. The easiest way to do this is to use the objective function itself for that feasible solution. For nodes representing infeasible solutions,  $c(x) = \infty$ . For nodes representing partial solutions,  $c(x)$  is the cost of the minimum-cost node in the subtree with root  $x$ . Since  $c(x)$  is in general as hard to compute as the original optimization problem is to solve, the branch-and-bound algorithm will use an estimate  $\hat{c}(x)$  such that  $\hat{c}(x) \leq c(x)$  for all  $x$ . In general then, the  $\hat{c}(\cdot)$  function used in a branch-and-bound solution to optimization functions will estimate the objective



function value and not the computational difficulty of reaching an answer node. In addition, to be consistent with the terminology used in connection with the 15-puzzle, any node representing a feasible solution (a solution node) will be an answer node. However, only minimum-cost answer node will correspond to an optimal solution. Thus, answer nodes and solution nodes are indistinguishable.

As an example optimization problem, consider the job sequencing with deadlines problem introduced in Section 4.4. We generalize this problem to allow jobs with different processing times. We are given  $n$  jobs and one processor. Each job  $i$  has associated with it a three tuple  $(p_i, d_i, t_i)$ . Job  $i$  requires  $t_i$  units of processing time. If its processing is not completed by the deadline  $d_i$ , then a penalty  $p_i$  is incurred. The objective is to select a subset  $J$  of the  $n$  jobs such that all jobs in  $J$  can be completed by their deadlines. Hence, a penalty can be incurred only on those jobs not in  $J$ . The subset  $J$  should be such that the penalty incurred is minimum among all possible subsets  $J$ . Such a  $J$  is optimal.

Consider the following instance:  $n=4, (p_1, d_1, t_1) = (5, 1, 1), (p_2, d_2, t_2) = (10, 3, 2), (p_3, d_3, t_3) = (6, 2, 1),$  and  $(p_4, d_4, t_4) = (3, 1, 1)$ . The solution space for this instance consists of all possible subsets of the job index set  $\{1, 2, 3, 4\}$ . The solution space can be organized into a tree by means of either of the two formulations used for the sum of subsets problem (Example 7.2). Figure 7.6 corresponds to the variable tuple size formulation while Figure 7.7 corresponds to the fixed tuple size formulation. In both figures square nodes represent infeasible subsets. In Figure 7.6 all nonsquare nodes are answer nodes. Node 9 represents an optimal solution and is the only minimum-cost answer node. For this node

$J=\{2,3\}$  and the penalty (cost) is 8. In Figure 7.7 only nonsquare leaf nodes are answer nodes node 25 represents the optimal solution and is also a minimum cost answer node. This corresponds to  $J=\{2,3\}$  and a penalty of 8. The cost of the answer nodes of Figure 7.7 are given below the nodes.

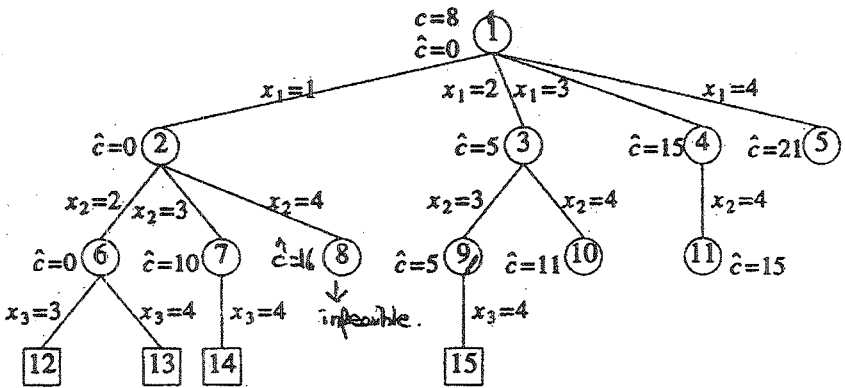


Figure 7.6 State space tree corresponding to variable tuple size formulation

We can define a cost function  $c()$  for the state space formulations of Figures 7.6 and 7.7. For any circular node  $x$ ,  $c(x)$  is the minimum penalty corresponding to any node in the subtree with root  $x$ . The value of  $c(x) = \infty$  for a square node. In the tree of Figure 7.6,  $c(3)=8$ ,  $c(2)=9$ , and  $c(1)=8$ . In the tree of Figure 7.7,  $c(1)=8$ ,  $c(2)=9$ ,  $c(5)=13$ , and  $c(6)=8$ . Clearly,  $c(1)$  is the penalty corresponding to an optimal selection  $J$ .

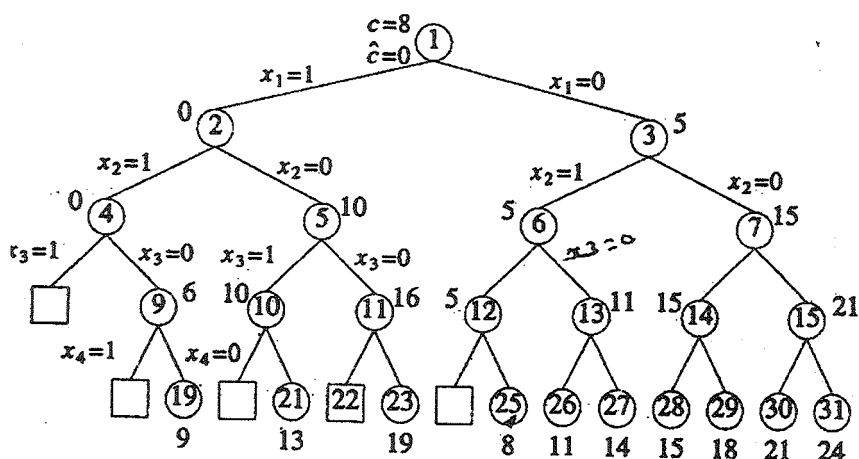


Figure 7.7 State space tree corresponding to fixed tuple size formulation

A bound  $\hat{c}(x)$  such that  $\hat{c}(x) \leq c(x)$  for all  $x$  is easy to obtain. Let  $S_x$  be the subset of jobs selected for  $J$  at node  $x$ . If  $m = \max \{i | i \in S_x\}$ , then  $\hat{c}(x) = \sum_{i \in S_x} i < mp_i$  is an estimate for  $c(x)$

with the property  $\hat{c}(x) \leq c(x)$ . For each circular node  $x$  in Figures 7.6 and 7.7, the value of  $\hat{c}(x)$  is the number outside node  $x$ . For a square node,  $\hat{c}(x) = \infty$ . For example, in Figure 7.6 for node 6,

$S_6 = \{1, 2\}$  and hence  $m = 2$ . Also,  $\sum_{i \in S_2} i < 2p_i = 0$ . For node 7,

$S_7 = \{1, 3\}$  and  $m = 3$ . Therefore  $\sum_{i \in S_2} i < 2p_i = p_2 = 10$ . And so on.

In Figure 7.7 node 12 corresponds to omission of jobs 1 and hence a penalty of 5, node 13 corresponds to the omission of jobs 1 and 3 and hence a penalty of 11 and so on.

A simple upper bound  $u(x)$  on the cost of a minimum-cost answer node in the subtree  $x$  is  $u(x) = \sum_{i \in S_x} p_i$ . Note that  $u(x)$  is the cost of the solution  $S_x$  corresponding to node  $x$ .

### 7.1.5 FIFO Branch-and-Bound

A FIFO branch-and-bound algorithm for the job sequencing problem can begin with  $\text{upper} = \infty$  (or  $\text{upper} = \sum_{1 \leq i \leq n} p_i$ ) as an upper bound on the cost of a minimum-cost answer node. Starting with node 1 as the E-node and using the variable tuple size formulation of Figure 7.6, nodes 2,3,4 and 5 are generated (in that order). Then  $u(2)=19$ ,  $u(3)=14$ ,  $u(4)=18$ , and  $u(5)=21$ . For example, node 2 corresponds to the inclusion of job 1. Thus  $u(2)$  is obtained by summing the penalties of all the other jobs. The variable upper is updated to 14 when node 3 is generated. Since  $\hat{c}(4)$  and  $\hat{c}(5)$  are greater than upper, nodes 4 and 5 get killed (or bounded). Only nodes 2 and 3 remain alive. Node 2 becomes the next E-node. Its children, nodes 6,7, and 8 are generated. Then  $u(6)=9$  and so upper is updated to 9. The cost  $\hat{c}(7)=10 > \text{upper}$  and node 7 gets killed. Node 8 is infeasible and so it is killed. Next, node 3 becomes the E - node, Nodes 9&10 are now generated. Then  $u(9)=8$  and so upper becomes.. The cost  $\hat{c}(10)=11 > \text{upper}$ , and this node is killed. The next E-node is node 6. Both its children are infeasible. Node 9's only child is also infeasible. The minimum-cost answer node is node. It has a cost of 8.

When implementing a FIFO branch-and-bound algorithm, it is not economical to kill live nodes with  $\hat{c}(x) > \text{upper}$  each time upper is updated. This is so because live nodes are in the queue in the order in which they were generated. Hence, nodes with

$\hat{c}(x) > \text{upper}$  are distributed in some random way in the queue. Instead, live node with  $\hat{c}(x) > \text{upper}$  can be killed when they are about to become E-nodes.

From here on we shall refer to the FIFO-based branch-and-bound algorithm with an appropriate  $\hat{c}(\cdot)$  and  $u(\cdot)$  as FIFOBB.

### 7.1.6 LC Branch-and-Bound

An LC branch-and-bound search of the tree of Figure 7.6 will begin with  $\text{upper} = \infty$  and node 1 as the first E-node. When node 1 is expanded, nodes 2, 3, 4 and 5 are generated in that order. As in the case of FIFOBB, upper is updated to 14 when node 3 is generated and nodes 4 and 5 are killed as  $\hat{c}(4) > \text{upper}$  and  $\hat{c}(5) > \text{upper}$ . Node 2 is the next E-node as  $\hat{c}(2) = 0$  and  $\hat{c}(3) = 5$ . Nodes 6, 7, and 8 are generated and upper is updated to 9 when node 6 is generated. So, node 7 is killed as  $\hat{c}(7) = 10 > \text{upper}$ . Node 8 is infeasible and so killed. The only live nodes now are nodes 3 and 6. Node 6 is the next E-node as  $\hat{c}(6) = 0 < \hat{c}(3)$ . Both its children are infeasible. Node 3 becomes the next E-node. When node 9 is generated, upper is updated to 8 as  $u(9) = 8$ . So, node 10 with  $\hat{c}(10) = 11$  is killed on generation. Node 9 becomes the next E-node. Its only child is infeasible. No live nodes remain. The search terminates with node 9 representing the minimum-cost answer node.

From here on we refer to the LC(LIFO)-based branch-and-bound algorithm with an appropriate  $\hat{c}(\cdot)$  and  $u(\cdot)$  as LCBB (LIFOBB).

## 7.2 0/1 KNAPSACK PROBLEM

To use the branch-and-bound technique to solve any problem, it is first necessary to conceive of a state space tree for the problem. We have already seen two possible state space tree organizations for the knapsack problem (Section 7.6). Still, we cannot directly apply the techniques of Section 7.1 since these were discussed with respect to minimization problems whereas the knapsack problem is a maximization problem. This difficulty is easily overcome by replacing the objective function  $\sum p_i x_i$  by the function  $-\sum p_i x_i$ . Clearly,  $\sum p_i x_i$  is maximized iff  $-\sum p_i x_i$  is minimized. This modified knapsack problem is stated as (7.1).

$$\begin{aligned} &\text{minimize} \quad -\sum_{i=1}^n p_i x_i \\ &\text{subject to} \quad \sum_{i=1}^n w_i x_i \leq m \\ &\quad x_i = 0 \text{ or } 1, 1 \leq i \leq n \end{aligned} \tag{7.1}$$

We continue the discussion assuming a fixed tuple size formulation for the solution space. The discussion is easily extended to the variable tuple size formulations. Every leaf node in the state space tree representing an assignment for which  $\sum_{1 \leq i \leq n} w_i x_i \leq m$  is an answer (or solution) node. All other leaf nodes are infeasible. For a minimum-cost answer node to correspond to any optimal solution, we need to define  $c(x) = -\sum_{1 \leq i \leq n} p_i x_i$  for every answer node  $x$ . The cost  $c(x) = \infty$  for infeasible leaf nodes. For nonleaf nodes,  $c(x)$  is recursively defined to be  $\min \{ c(\text{lchild}(x)), c(\text{rchild}(x)) \}$ .

We now need two functions  $\hat{c}(x)$  and  $u(x)$  such that  $\hat{c}(x) \leq c(x) \leq u(x)$  for every node  $x$ . The cost  $\hat{c}(\cdot)$  and  $u(\cdot)$  satisfying this requirement may be obtained as follows. Let  $x$  be a node at level  $j$ ,  $1 \leq j \leq n+1$ . At node  $x$  assignments have already been made to  $x_i$ ,  $1 \leq i < j$ . The cost of these assignments is  $-\sum_{1 \leq i < j} p_i x_i$ . So,  $c(x) = -\sum_{1 \leq i < j} p_i x_i$  and we may use  $u(x) = -\sum_{1 \leq i < j} p_i x_i$ . If  $q = -\sum_{1 \leq i < j} p_i x_i$ , then an improved upper bound function  $u(x)$  is  $u(x) = \text{Ubound}(q, \sum_{1 \leq i < j} w_i x_i, j-1, m)$ , where  $\text{Ubound}$  is defined in Algorithm 7.2. As for  $c(x)$ , it is clear that  $\text{Bound}(-q, \sum_{1 \leq i < j} w_i x_i, j-1) \leq c(x)$ , where  $\text{Bound}$  is as given in Algorithm 7.11.

```

1 Algorithm Ubound(cp,cw,k,m)

2 // cp→is the current profit total, cw is the current
 // weight total, k is the index of the last - remove. Then
 // and m is the knapscale size

3 // Algorithm 7.11 w[i] and p[i] are respectively

4 // the weight and profit of the ith object.

5 {

6 b:=cp; c:=cw;

7 for i:=k+1 to n do

8 {

9 if (c+ w[i] ≤ m) then

10 {

```

```

11 c:=c+w[i];b:=b-p[i];
12 }
13 }
14 return b;
15 }

```

Algorithm 7.2 Function  $u(\cdot)$  for knapsack problem

### 7.2.1 LC Branch-and-Bound Solution

**Example 7.2 [LCBB]** Consider the knapsack instance  $n=4, (p_1, p_2, p_3, p_4) = (10, 10, 12, 18), (w_1, w_2, w_3, w_4) = (2, 4, 6, 9),$  and  $m=15$ . Let us trace the working of an LC branch-and-bound search using  $\hat{c}(\cdot)$  and  $u(\cdot)$  as defined previously. We continue to use the fixed tuple size formulation. The search begins with the root as the E-node. For this node, node 1 of Figure 7.8, we have  $\hat{c}(1) = -38$  and  $u(1) = -32$ .

The computation of  $u(1)$  and  $\hat{c}(1)$  is done as follows. The bound  $u(1)$  has a value  $Ubound(0, 0, 0, 15)$ .  $Ubound$  scans through the objects from left to right starting from  $j$ ; it adds these objects into the knapsack until the first object that doesn't fit is encountered. At this time, the negation of the total profit of all the objects in the knapsack plus  $cw$  is returned. In Function  $Ubound$ ,  $c$  and  $b$  start with a value of zero. For  $i=1, 2,$  and  $3$ ,  $c$  gets incremented by  $2, 4$  and  $6$  respectively. The variable  $b$  also gets decremented by  $10, 10,$  and  $12$  respectively. When  $i=4$ , the test  $(c + w[i] \leq m)$  fails and hence the value returned is  $-32$ . Function



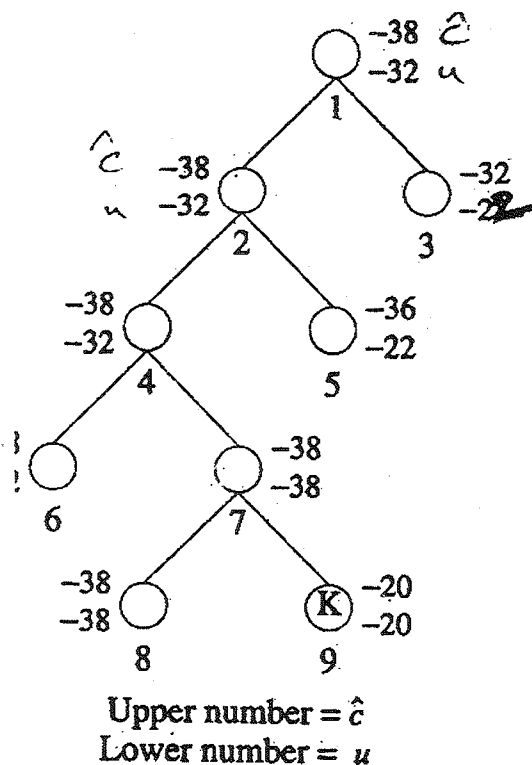


Figure 7.8 LC branch-and-bound tree for Example 7.2

Bound is similar to Ubound, except that it also considers a fraction of the first object that doesn't fit the knapsack. For example, in computing  $\hat{c}(1)$ , the first object that doesn't fit is 4 whose weight is 9. The total weight of the objects 1, 2, and 3 is 12. So, Bound considers a fraction  $3/9$  of the object 4 and hence returns  $-32 - 3/9 * 18 = -38$ .

Since node 1 is not a solution node, LCBB sets  $ans=0$  and  $upper = -32$  ( $ans$  being a variable to store intermediate answer

nodes). The E-node is expanded and its two children, nodes 2 and 3, generated. The cost  $\hat{c}(2) = -38$ ,  $\hat{c}(3) = -32$ ,  $u(2) = -32$ , and  $u(3) = -27$ . Both nodes are put onto the list of live nodes. Node 2 is the next E-node. It is expanded and nodes 4 and 5 generated. Both nodes get added to the list of live nodes. Node 4 is the live node with least  $\hat{c}$  value and becomes the next E-node. Node 6 and 7 are generated. Assuming node 6 is generated first, it is added to the list of live nodes. Next, node 7 joins this list and upper is updated to  $-38$ . The next E-node will be one of nodes 6 and 7. Let us assume it is node 7. Its two children are nodes 8 and 9. Node 8 is a solution node. Then upper is updated to  $-38$  and node 8 is put onto the live nodes list. Node 9 has  $\hat{c}(9) > \text{upper}$  and is killed immediately. Node 6 and 8 are two live nodes with least  $\hat{c}$ . Regardless of which becomes the next E-node,  $\hat{c}(E) \geq \text{upper}$  and the search terminates with node 8 the answer node. At this time, the value  $-38$  together with the path 8,7,4,2,1 is printed out and the algorithm terminates. From the path one cannot figure out the assignment of values to the  $x_i$ 's such that  $\sum p_i x_i = \text{upper}$ . Hence, a proper implementation of LCBB has to keep additional information from which the values of the  $x_i$ 's can be extracted. One way is to associate with each node a one bit field, tag. The sequence of tag bits from the answer node to the root give the  $x_i$  values. Thus, we have  $\text{tag}(2) = \text{tag}(4) = \text{tag}(6) = \text{tag}(8) = 1$  and  $\text{tag}(3) = \text{tag}(5) = \text{tag}(7) = \text{tag}(9) = 0$ . The tag sequence for the path 8,7,4,2,1 is 1 0 1 1 and so  $x_4=1$ ,  $x_3=0$ ,  $x_2=1$ , and  $x_1=1$ .

To use LCBB to solve the knapsack problem, we need to specify (1) the structure of nodes in the state space tree being searched, (2) how to generate the children of a given node, (3)

how to recognize a solution node, and (4) a representation of the list of live nodes and a mechanism for adding a node into the list as well as identifying the least-cost node. The node structure needed depends on which of the two formulations for the state space tree is being used. Let us continue with a fixed size tuple formulation. Each node  $x$  that is generated and put onto the list of live nodes must have a parent field. In addition, as noted in Example 7.2, each node should have a one bit tag field. This field is needed to output the  $x_i$  values corresponding to an optimal solution. To generate  $x$ 's children, we need to know the level of node  $x$  in the state space tree. For this we shall use a field level. The left child of  $x$  is chosen by setting  $x_{\text{level}(x)} = 1$  and the right child by setting  $x_{\text{level}(x)} = 0$ . To determine the feasibility of the left child, we need to know the amount of knapsack space available at node  $x$ . This can be determined either by following the path from node  $x$  to the root or by explicitly retaining this value in the node. Say we choose to retain this value in a field  $cu$  (capacity unused). The evaluation  $\hat{c}(x)$  and  $u(x)$  requires knowledge of the profit  $\sum_{1 \leq i \leq \text{level}(x)} p_i x_i$  earned by the filling corresponding to node  $x$ . This can be computed by following the path from  $x$  to the root. Alternatively, this value can be explicitly retained in a field  $pe$ . Finally, in order to determine the live node with least  $\hat{c}$  value or to insert nodes properly into the list of live nodes, we need to know  $\hat{c}(x)$ . Again, we have a choice. The value  $\hat{c}(x)$  may be stored explicitly in a field  $ub$  or may be computed when needed. Assuming all information is kept explicitly, we need nodes with size fields each: parent, level, tag,  $cu$ ,  $pe$ , and  $ub$ .

Using this six-field node structure, the children of any live node  $x$  can be easily determined. The left child  $y$  is feasible iff

$cu(x) \geq w_{level(x)}$ . In this case,  $parent(y) = x$ ,  $level(y) = level(x) + 1$ ,  $cu(y) = cu(x) \geq w_{level(x)}$ ,  $pe(y) = pe(x) + p_{level(x)}$ ,  $tag(y) = 1$ , and  $ub(y) = ub(x)$ . The right child can be generated similarly. Solution nodes are easily recognized too. Node  $x$  is a solution node iff  $level(x) = n + 1$ .

We are now left with the task of specifying the representation of the list of live nodes. The functions we wish to perform on this list are (1) test if the list is empty, (2) add nodes, and (3) delete a node with least  $ub$ . We have seen a data structure that allows us to perform these three functions have seen a data structure that allows us to perform these three functions efficiently: a min-heap. If there are  $m$  live nodes, then function (1) can be carried out in  $\theta(1)$  time, whereas functions (2) and (3) require only  $O(\log m)$  time.

### 7.2.2 FIFO Branch-and-Bound Solution

**Example 7.3** Now, let us trace through the FIFOBB algorithm using the same knapsack instance as in Example 7.2. Initially the root node, node 1 of Figure 7.9, is the E-node and the queue of live nodes is empty. Since this is not a solution node, upper is initialized to  $u(1) = -32$ .

We assume the children of a node are generated left to right. Nodes 2 and 3 are generated and added to the queue (in that order). The value of upper remains unchanged. Node 2 becomes the next E-node. Its children, nodes 4 and 5, are generated and added to the queue. Node 3, the next E-node is expanded. Its children nodes are generated. Node 6 gets added to the queue. Node 7 is immediately killed as  $\hat{c}(7) > upper$ . Node 4 is expanded next. Nodes 8 and 9 are generated and added to

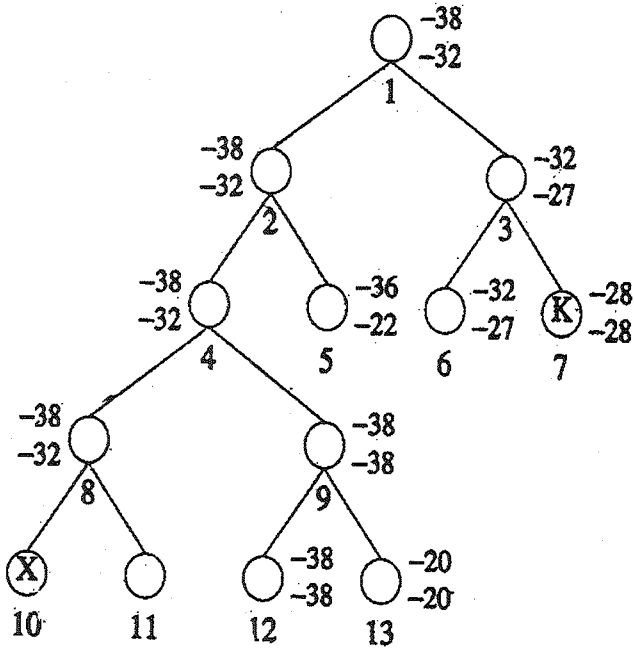


Figure 7.9 FIFO branch-and-bound tree for Example 7.3

the queue. Then upper is updated to  $u(9) = -38$ . Nodes 5 and 6 are the next two nodes to become E-nodes. Neither is expanded as for each  $c \gg$  upper node 8 is the next e node. Nodes 10 and 11 are generated. Node 10 is infeasible and so killed. Node 11 has  $\hat{c}(11) > \text{upper}$  and so is also killed. Node 9 is expanded next. When node 12 is generated, upper and ans are updated to  $-38$  and  $12$  respectively. Node 12 joins the queue of live nodes. Node 13 is killed before it can get onto the queue of live nodes as  $\hat{c}(13) > \text{upper}$ . The only remaining live node is node 12. It has

no children and the search terminates. The value of upper and the path from node 12 to the root is output. As in the case of Example 7.2, additional information is needed to determine the  $x_i$  values on this path.

As first we may be tempted to discard FIFOBB in favor of LCBB in solving knapsack. Our intuition leads us to believe that LCBB will examine fewer nodes either is expanded as for each  $c > u_{\text{upper}}$  node 8 is the next  $e$  node. Nodes in its quest for and optimal solution. However, we should keep in mind that insertions into and deletions from a heap are far more expensive (proportional to the logarithm of the heap size) than the corresponding operations on a queue ( $\theta(1)$ ). Consequently, the work done for each E-node is more in LCBB than in FIFOBB. Unless LCBB uses far fewer E-nodes than FIFOBB, FIFOBB will outperform (in terms of real computation time) LCBB.

We have now seen four different approaches to solving the knapsack problem: dynamic programming, backtracking, LCBB, and FIFOBB. If we compare the dynamic programming algorithm DKnap (Algorithm 5.7) and FIFOBB, we see that there is a correspondence between generating the  $s^{(i)}$ 's and generating nodes by levels.  $S^{(i)}$  contains all pairs  $(P, W)$  corresponding to nodes on level  $i + 1$ ,  $0 \leq i \leq$ . Hence, both algorithms generate the state space tree by levels. The dynamic programming algorithm, however, keeps the nodes on each level ordered by their profit earned ( $P$ ) and capacity used ( $W$ ) values. No two tuples have the same  $P$  or  $W$  value. In FIFOBB we may have many nodes on the same level with the same  $P$  or  $W$  values. However, the bounding rules can easily be incorporated into DKnap. some simple heuristics to determine whether a pair  $(P, W) \in S^{(i)}$  should be

killed. These heuristics are readily seen to be bounding functions of the type discussed here. Let the algorithm resulting from the inclusion of the bounding functions into Dknap be DKnap1. DKnap1 is expected to be superior to FIFOBB as it uses the dominance rule in addition to the bounding functions. In addition, the overhead incurred each time a node is generated is less.

To determine which of the knapsack algorithms is best, it is necessary to program them and obtain real computing times for different data sets. Since the effectiveness of the bounding functions and the dominance rule is highly data dependent, we expect a wide variation in the computing time for different problem instances having the same number of objects  $n$ . To get representative times, it is necessary to generate many problem instances for a fixed  $n$  and obtain computing times for these instances. The generation of these data sets and the problem of conducting the tests is discussed in a programming project at the end of this section. The results of some tests can be found in the references to this chapter.

Before closing our discussion of the knapsack problem, we briefly discuss a very effective heuristic to reduce a knapsack instance with large  $n$  to an equivalent one with smaller  $n$ . This heuristic, Reduce, uses some of the ideas developed for the branch-and-bound algorithm. It classifies the objects  $\{1, 2, \dots, n\}$  into one of three categories  $I_1, I_2$ , and  $I_3$ .  $I_1$  is a set of objects for which  $x_i$  must be 1 in every optimal solution  $I_2$  is a set for which  $x_i$  must be 0.  $I_3$  is  $\{1, 2, \dots, n\} - I_1 - I_2$ . Once  $I_1, I_2$ , and  $I_3$  have been determined, we need to solve only the reduced knapsack instance

$$\begin{aligned}
& \text{maximize} && \sum_{i \in I} p_i x_i \\
& \text{subject to} && \sum_{i \in I} w_i x_i \leq m - \sum_{i \in I_1} w_i x_i \quad (7.2) \\
& && x_i = 0 \text{ or } 1
\end{aligned}$$

From the solution to (7.2) an optimal solution to the original knapsack instance is obtained by setting  $x_i = 1$  if  $i \in I_1$  and  $x_i = 0$  if  $i \in I_2$ .

Function Reduce (Algorithm 7.3) makes use of two functions Ubb and Lbb. The bound Ubb( $I_1, I_2$ ) is an upper bound on the value of an optimal solution to the given knapsack instance with added constraints  $x_i = 1$  if  $i \in I_1$  and  $x_i = 0$  if  $i \in I_2$ . The bound Lbb( $I_1, I_2$ ) is a lower bound under the constraints of  $I_1$  and  $I_2$ . Algorithm Reduce needs no further explanation. It should be clear that  $I_1$  and  $I_2$  are such that from an optimal solution to (7.2), we can easily obtain an optimal solution to the original knapsack problem.

The time complexity of Reduce is  $O(n^2)$ . Because the reduction procedure is very much like the heuristics used in DKnap1 and the knapsack algorithms of this chapter, the use of Reduce does not decrease the overall computing time by as much as may be expected by the reduction in the number of objects. These algorithms do dynamically what Reduce does. The exercises explore the value of Reduce further.

- 1     Algorithm Reduce ( $p, w, n, m, I_1, I_2$ )
- 2     // Variables are as described in the discussion



```

3 // $p[i]/w[i] \geq p[i+1]/w[i+1]$, $1 \leq i < n$
4 {
5 $I1 := I2 := \emptyset$;
6 $q := \text{Lbb}(\emptyset, \emptyset)$;
7 $k :=$ largest j such that $w[1] + \dots + w[j] < m$;
8 for $i := 1$ to k do
9 {
10 if $(\text{Ubb}(\emptyset, \{i\}) < q)$ then $I1 := I1 \cup \{i\}$;
11 else if $(\text{Lbb}(\emptyset, \{i\}) > q)$ then $q := \text{Lbb}(\emptyset, \{i\})$;
12 }
13 for $i := k+1$ to n do
14 {
15 if $(\text{Ubb}(\{i\}, \emptyset) < q)$ then $I2 := I2 \cup \{i\}$;
16 else if $(\text{Lbb}(\{i\}, \emptyset) > q)$ then $q := \text{Lbb}(\{i\}, \emptyset)$;
17 }
18 }

```

Algorithm 7.3 Reduction pseudocode for knapsack problem

### 7.3 TRAVELLING SALESPERSON (\*)

An  $O(n^2 2^n)$  dynamic programming algorithm for the traveling salesperson problem was arrived at in Section 5.9. We now investigate branch-and-bound algorithms for this problem. Although the worst-case complexity of these algorithm will not be any better than  $O(n^2 2^n)$ , the use of good bounding functions will enable these branch-and-bound algorithms to solve some problem instances in much less time than required by the dynamic programming algorithm.

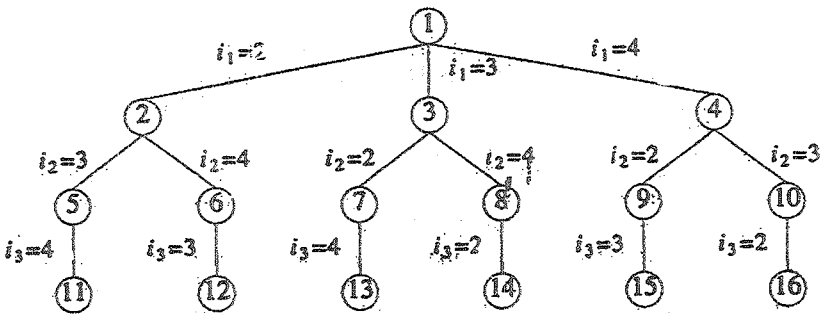


Figure 7.10 State space tree for the traveling salesperson problem with  $n=4$  and  $i_0=i_4=1$

Let  $G=(V,E)$  be a directed graph defining an instance of the traveling salesperson problem. Let  $c_{ij}$  equal the cost of edge  $\langle i,j \rangle$ ,  $c_{ij} = \infty$  if  $\langle i,j \rangle \notin E$ , and let  $|V|=n$ . Without loss of generality,

we can assume that every tour starts and ends at vertex 1. So, the solution space  $S$  is given by  $S = \{1, \pi, 1 | \pi \text{ is a permutation of } (2, 3, \dots, n)\}$ . Then  $|S| = (n-1)!$ . The size of  $S$  can be reduced by restricting  $S$  so that  $(1, i_1, i_2, \dots, i_{n-1}, 1) \in S$  iff  $\langle i_j, i_{j+1} \rangle \in E$ ,  $0 \leq j \leq n-1$ , and  $i_0 = i_n = 1$ .  $S$  can be organized as a state space tree similar to that for the  $n$ -queens problem (see Figure 7.2). Figure 7.10 shows the tree organization for the case of a complete graph with  $|V|=4$ . Each leaf node  $L$  is a solution node and represents the tour defined by the path from the root to  $L$ . Node 14 represents the tour  $i_0=1$ ,  $i_1=3$ ,  $i_2=4$ ,  $i_3=2$ , and  $i_4=1$ .

To use LCBB to search the traveling salesperson state space tree, we need to define a cost function  $c(\cdot)$  and two other functions  $\hat{c}(\cdot)$  and  $u(\cdot)$  such that  $\hat{c}(r) \leq c(r) \leq u(r)$  for all nodes  $r$ . The cost  $c(\cdot)$  is such that the solution node with least  $c(\cdot)$  corresponds to a shortest tour in  $G$ . One choice for  $c(\cdot)$  is

$$c(A) = \begin{cases} \text{length of tour defined by the path from the root to } A, & \text{if } A \text{ is a leaf} \\ \text{cost of a minimum-cost leaf in the subtree } A, & \text{if } A \text{ is not a leaf} \end{cases}$$

A simple  $\hat{c}(\cdot)$  such that  $\hat{c}(A) \leq c(A)$  for all  $A$  is obtained by defining  $\hat{c}(A)$  to be the length of the path defined at node  $A$ . For example, the path defined at node 6 of Figure 7.10 is  $i_0, i_1, i_2 = 1, 2, 4$ . It consists of the edges  $\langle 1, 2 \rangle$  and  $\langle 2, 4 \rangle$ . A better  $\hat{c}(\cdot)$  can be obtained by using the reduced cost matrix corresponding to  $G$ . A row (column) is said to be reduced iff it contains at least one zero and all remaining entries are non-negative. A matrix is reduced iff every row and column is reduced. As an example of how to reduce the cost matrix of a given graph  $G$ , consider the matrix of Figure 7.11(a). This corresponds to a graph with five

vertices. Since every tour on this graph includes exactly one edge  $\langle i, j \rangle$  with  $i=k, 1 \leq k \leq 5$ , and exactly one edge  $\langle i, j \rangle$  with  $j=k, 1 \leq k \leq 5$ , subtracting a constant  $t$  from every entry in one column or one row of the cost matrix reduces the length of every tour by exactly  $t$ . A minimum-cost tour remains a minimum-cost tour following this subtraction operation. If  $t$  is chosen to be the minimum entry in row  $i$  (column  $j$ ), then subtracting it from all entries in row  $i$  (column  $j$ ) introduces a zero into row  $i$  (column  $j$ ). Repeating this as often as needed, the cost matrix can be reduced. The total amount subtracted from the column and rows is a lower bound on the length of a minimum-cost tour and can be used as the  $\hat{c}$  value for the root of the state space tree. Subtracting 10, 2, 2, 3, 4, 1, and 3 from rows 1, 2, 3, 4 and 4 and columns 1 and 3 respectively of the matrix of Figure 7.11(a) yields the reduced matrix of Figure 7.11(b). The total amount subtracted is 25. Hence, all tours in the original graph have a length at least 25.

We can associate a reduced cost matrix with every node in the traveling salesperson state space tree. Let  $A$  be the reduced cost matrix for node  $R$ . Let  $S$  be a child of  $R$  such that the tree edge  $(R, S)$  corresponds to including edge  $\langle i, j \rangle$  in the tour. If  $S$  is not a leaf, then the reduced cost matrix for  $S$  may be obtained as follows: (1) Change all entries in row  $i$  and column  $j$  of  $A$  to  $\infty$ . This prevents the use of any more edges leaving vertex  $i$  or entering vertex  $j$ . (2) Set  $A(j, 1)$  to  $\infty$ . This prevents the use of edge  $\langle j, 1 \rangle$ . (3) Reduce all rows and columns in the resulting matrix except for rows and columns containing only  $\infty$ . Let the resulting matrix be  $B$ . Steps (1) and (2) are valid as no tour in the subtree  $s$  can contain edges of the type  $\langle i, k \rangle$  or  $\langle k, j \rangle$  or  $\langle j, 1 \rangle$  (except for edge  $\langle i, j \rangle$ ). If  $r$  is the total amount subtracted in step (3) then  $\hat{c}(S) = \hat{c}(R) + A(i, j) + r$ . For leaf nodes,  $\hat{c}(\cdot) = c(\cdot)$  is easily computed

as each leaf defines a unique tour. For the upper bound function  $u$ , we can use  $u(R)=\infty$  for all nodes  $R$ .

|                                                                                                                                                                          |                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix}$ | $\begin{bmatrix} \infty & 10 & 17 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix}$ |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

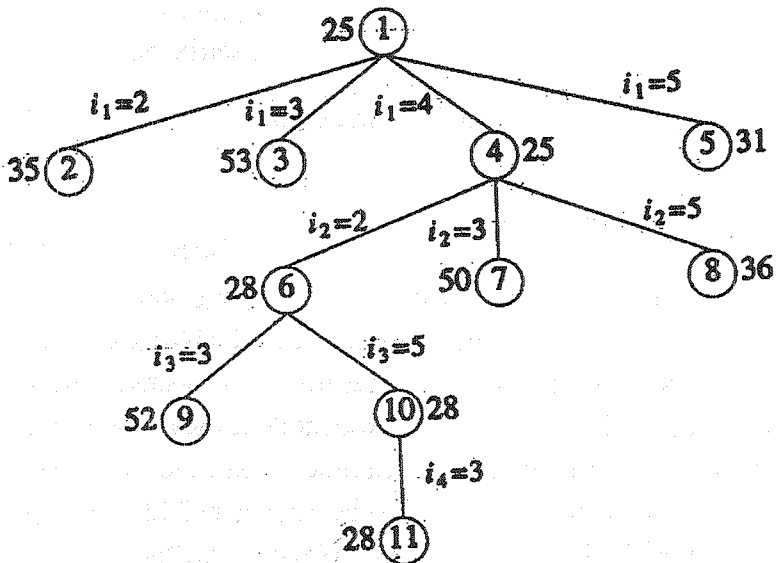
(a) Cost Matrix

(b) Reduced cost matrix = 25

Figure 7.11 An example

Let us now trace the progress of the LCBB algorithm on the problem instance of Figure 7.11(a). We use  $\hat{c}$  and  $u$  as above. The initial reduced matrix is that of Figure 7.11(b) and upper =  $\infty$ . The portion of the state space tree that gets generated is shown in Figure 7.12. Starting with the root node as the E-node, nodes 2,3,4 and 5 are generated (in that order). The reduced matrices corresponding to these nodes are shown in Figure 7.13. The Figure 7.13(b) is obtained from that of 7.11(b) by (1) setting all entries in row 1 and column 3 to  $\infty$ , (2) setting the element at position (3,1) to  $\infty$ , and (3) reducing column 1 by subtracting by 11. The  $\hat{c}$  for node 3 is therefore  $25 + 17$  (the cost of edge  $\langle 1,3 \rangle$  in the reduced matrix) + 11 = 53. The matrices and  $\hat{c}$  value for nodes

2, 4, and 5 are obtained similarly. The value of upper is unchanged and node 4 becomes the next E-node. Its children 6, 7 and 8 are generated. The live nodes at this time are nodes 2, 3, 5, 6, 7, and 8. Node 6 has least  $\hat{c}$  value and becomes the next E-node. Nodes 9 and 10 are generated. Node 10 is the next E-node. The solution node, node 11, is generated. The tour length for this node is  $\hat{c}(11)=28$  and upper is updated to 28. For the next E-node, node 5,  $\hat{c}(5)=31 > \text{upper}$ . Hence, LCBB terminates with 1, 4, 2, 5, 3, 1 as the shortest length tour.



Numbers outside the node are  $\hat{c}$  values

7.12 State space tree generated by procedure LCBB

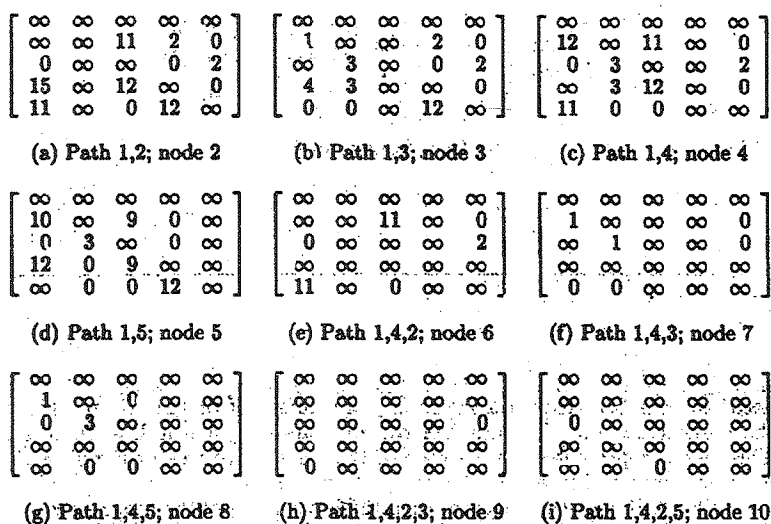


Figure 7.13 Reduced cost matrices  
corresponding to nodes in Figure 7.12

An exercise examines the implementation considerations for the LCBB algorithm. A different LCBB algorithm can be arrived at by considering a different tree organization for the solution space. This organization is reached by regarding a tour as a collection of  $n$  edges. If  $G = (V, E)$  has  $e$  edges, then every tour contains exactly  $n$  of the edges. However, for each  $i$ ,  $1 \leq i \leq n$ , there is exactly one edge of the form  $\langle i, j \rangle$  and one of the form  $\langle k, i \rangle$  in every tour. A possible organization for the state space is a binary tree in which a left branch represents the inclusion of a particular edge while the right branch represents the exclusion of that edge. Figure 7.14(b) and the right branch represents the exclusion of that edge. Figure 7.14(b) and (c) represents the first two levels of two possible state space trees for the three vertex graph of Figure 7.14(a). As is true of all problems, many state

space trees are possible for a given problem formulation. Different trees differ in the order in which decisions are made. Thus, in Figure 7.14(c) we first decide the fate of edge  $\langle 1,2 \rangle$ . Rather than use a static state space tree, we now consider a dynamic state space tree (see Section 7.1). This is also a binary tree. However, the order in which edges are considered depends on the particular problem instance being solved. We compute  $\hat{c}$  in the same way as we did using the earlier state space tree formulation.

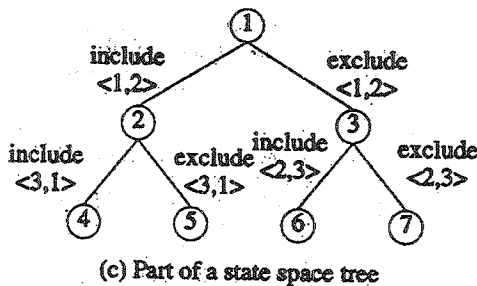
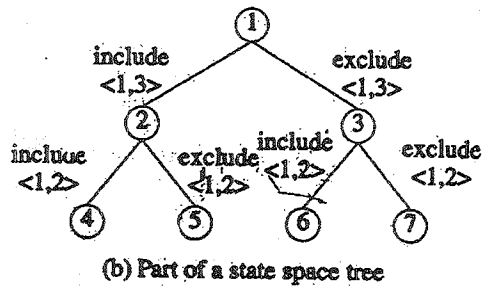
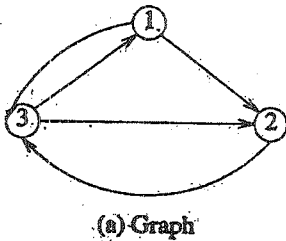


Figure 7.14 An example



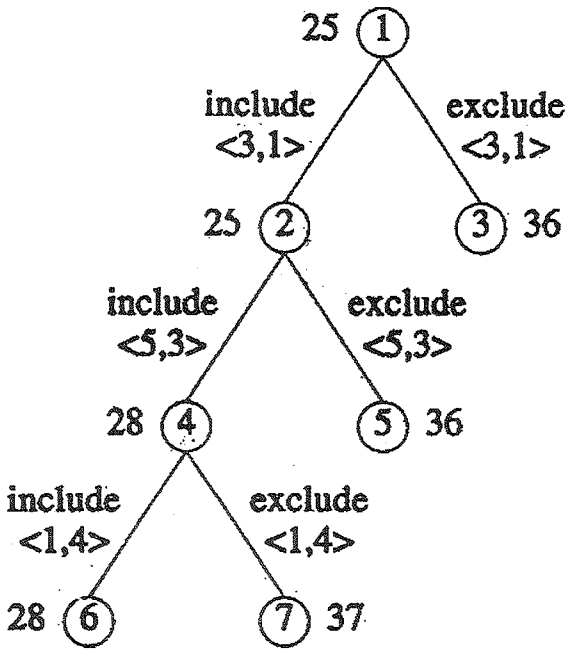


Figure 7.15 State space tree for Figure 7.11 (a)

As an example of how LCBB would work on the dynamic binary tree formulation, consider the cost matrix of Figure 7.11(a). Since a total of 25 needs to be subtracted from the rows and columns of this matrix to obtain the reduced matrix of Figure 7.11(b), all tours have a length at least 25. This fact is represented by the root of the state space tree of Figure 7.15. Now, we must decide which edge to use to partition the solution space into two subsets. If edge  $\langle i,j \rangle$  is used, then the left subtree of the root represents all tours including edge  $\langle i,j \rangle$  and the right subtree represents all tours that do not include edge  $\langle i,j \rangle$ . If an optimal tour is included in the left subtree, then only  $n-1$  edges remain to

be selected. If all optimal tours lie in the right subtree, then we have still to select  $n$  edges. Since the left subtree selects fewer edges, it should be easier to find an optimal solution in it than to find one in the right subtree. Consequently, we would like to choose as the partitioning edge an edge  $\langle i, j \rangle$  that has the highest probability of being in an optimal tour. Several heuristics for determining such an edge can be formulated. A selection rule that is commonly used is select that edge which results in a right subtree that has highest  $\hat{c}$  value. The logic behind this is that we soon have right subtrees (perhaps at lower levels) for which the  $\hat{c}$  value is higher than the length of an optimal tour. Another possibility is to choose an edge such that the difference in the  $\hat{c}$  values for the left and right subtrees is maximum. Other selection rules are also possible.

When LCBB is used with the first of the two selection rules stated above and the cost matrix of Figure 7.11(a), the tree of Figure 7.15 is generated. At the root node, we have to determine an edge  $\langle i, j \rangle$  that will maximize the  $\hat{c}$  value of the right subtree. If we select an edge  $\langle i, j \rangle$  whose cost in the reduced matrix (Figure 7.11(b)) is positive, then the  $\hat{c}$  value of the right subtree will remain 25. This is so as the reduced matrix for the right subtree will have  $B(i, j) = \infty$  and all other entries will be identical to those in Figure 7.11(b). Hence  $B$  will be reduced and  $\hat{c}$  cannot increase. So, we must choose an edge with reduced cost 0. If we choose  $\langle 1, 4 \rangle$ , the  $B(1, 4) = \infty$  and we need to subtract 1 from row 1 to obtain a reduced matrix. In this case  $\hat{c}$  will be 26. If  $\langle 3, 1 \rangle$  is selected, then 11 needs to be subtracted from column 1 to obtain the reduced matrix for the right subtree. So,  $\hat{c}$  will be 36. If  $A$  is the reduced cost matrix for node  $R$ , then the selection

of edge  $\langle i,j \rangle$  ( $A(i,j)=0$ ) as the next partitioning edge will increase the  $\hat{c}$  of the right subtree by  $\Delta = \min_{k \neq j} \{A(i,k) + \min_{k \neq i} \{A(k,j)\}$  as this much needs to be subtracted from row  $i$  and column  $j$  to introduce a zero into both. For edges  $\langle 1,4 \rangle, \langle 2,5 \rangle, \langle 3,1 \rangle, \langle 3,4 \rangle, \langle 4,5 \rangle, \langle 5,2 \rangle$ , and  $\langle 5,3 \rangle$ ,  $\Delta = 1, 2, 11, 0, 3, 3$ , and  $11$  respectively. So, either of the edges  $\langle 3,1 \rangle$  or  $\langle 5,3 \rangle$  can be used. Let us assume that LCBB selects edge  $\langle 3,1 \rangle$ . The  $\hat{c}$  (2) (Figure 7.15) can be computed in a manner similar to that for the state space tree of Figure 7.12. In the corresponding reduced cost matrix all entries in row 3 and column 1 will be  $\infty$ . Moreover the entry (1,3) will also be  $\infty$  as inclusion of this edge will result in a cycle. The reduced matrices corresponding to nodes 2 and 3 are given in Figure 7.16(a) and (b). The  $\hat{c}$  values for nodes 2 and 3 (as well as for all other nodes) appear outside the respective nodes.

Node 2 is the next E-node. For edges  $\langle 1,4 \rangle, \langle 2,5 \rangle, \langle 4,5 \rangle, \langle 5,2 \rangle$ , and  $\langle 5,3 \rangle$ ,  $\Delta = 3, 2, 3, 3$  and  $11$  respectively. Edge  $\langle 5,3 \rangle$  is selected and nodes 4 and 5 generated. The corresponding reduced matrices are given in Figure 7.16(c) and (d). Then  $\hat{c}$  (4) becomes 28 as we need to subtract 3 from column 2 to reduce this column. Note that entry (1,5) has been set to  $\infty$  in Figure 7.16(c). This is necessary as the inclusion of edge  $\langle 1,5 \rangle$  to the collection  $\{\langle 3,1 \rangle, \langle 5,3 \rangle\}$  will result in a cycle. In addition, entries in column 3 and row 5 are set to  $\infty$ . Node 4 is the next E-node. The  $\Delta$  values corresponding to edges  $\langle 1,4 \rangle, \langle 2,5 \rangle$ , and  $\langle 4,2 \rangle$  are 9, 2 and 0 respectively. Edge  $\langle 1,4 \rangle$  is selected and nodes 6 and 7 generated. The edge selection at node 6 is  $\{\langle 3,1 \rangle, \langle 5,3 \rangle, \langle 1,4 \rangle\}$ . This corresponds to the path 5,3,1,4. So, entry (4,5) is set to  $\infty$  in Figure 7.16(e). In general if edge  $\langle i,j \rangle$  is selected, then the entries in row  $i$  and column  $j$  are set to  $\infty$  in the left

subtree. In addition one more entry needs to be set to  $\infty$ . This is an entry whose inclusion in the set of edges would create a cycle (Exercise 4 examines how to determine this). The next E-node is node 6. At this time three of the five edges have already been selected. The remaining two may be selected directly. The only

In the preceding example, LCBB was modified slightly to handle nodes close to a solution node differently from other nodes. Node 6 is only two levels from a solution node. Rather than evaluate  $\hat{c}$  at the children of 6 and then obtain their grandchildren, we just obtained an optimal solution for that subtree by a complete search with no bounding. We could have done something similar when generating the tree of Figure 7.12. Since node 6 is only two levels from the leaf nodes, we can simply skip computing  $\hat{c}$  for the children and grandchildren of 6, generate all of them, and pick the best. This works out to be quite efficient as it is easier to generate a subtree with a small number of nodes and evaluate all the solution nodes in it than it is to compute  $\hat{c}$  for one of the children of 6. This latter statement is true of many applications of branch-and-bound. Branch-and-bound is used on large subtree. Once a small subtree is reached (say one with 4 or 6 nodes in it), then that subtree is fully evaluated without using the bounding functions.

We have now seen several branch-and-bound strategies for the traveling salesperson problem. It is not possible to determine analytically which of these is the best. The exercises describes computer experiments that determine empirically the relative performance of the strategies suggested.

## Lesson - 8

**ALGEBRAIC PROBLEMS****OBJECTIVE**

After studying this lesson you can able to understand the following

- The General Method of Algebraic problems.
- Evaluation and Interpolation
- The Fast- Fourier Transform
- Moduloar Arithmetic

**8.1 THE GENERAL METHOD**

In this lesson we shift our attention away from the problems we've dealt with previously to concentrate on methods for dealing with numbers and polynomials. Though computers have the ability already built-in to manipulate integers and reals, they are not directly equipped to manipulate symbolic mathematical expressions such as polynomials. One must determine a way to represent them and then write procedures that perform the desired operations. A system that allows for the manipulation of mathematical expressions (usually including arbitrary precision integers, polynomials, and rational functions) is called a mathematical symbol manipulation system. These systems have been fruitfully used to solve a variety of scientific problems for

many years. The technique we study here have often led to efficient ways to implement the operations offered by these systems.

The first design technique we present is called algebraic transformation. Assume we have an input  $I$  that is a member of set  $S_1$  and a function  $f(I)$  that describes what must be computed. Usually the output  $f(I)$  is also a member of  $S_1$ . Though a method may exist for computing  $f(I)$  using operations on elements in  $S_1$ , this method may be inefficient. The algebraic transformation technique suggests that we alter the input into another form to produce a member of set  $S_2$ . The set  $S_2$  contains exactly the same elements as  $S_1$  except it assumes a different representation for them. Why would we transform the input into another form? Because it may be easier to compute the function  $f$  for elements of  $S_2$  than for elements for  $S_1$ . Once the answer in  $S_2$  is computed, an inverse transformation is performed to yield the result in set  $S_1$ .

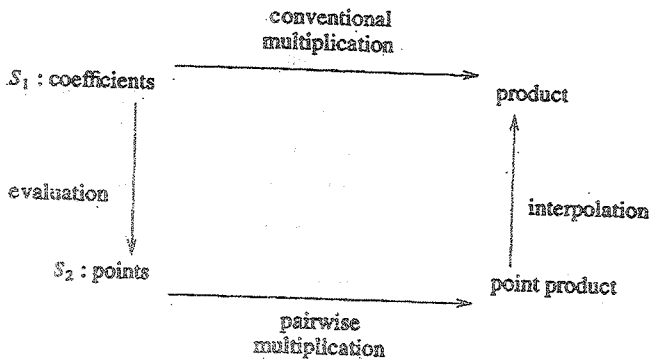


Figure 8.1 Transformation technique for polynomial products

Example 8.1. Let  $S_1$  be the set of integers represented using decimal notation, and  $S_2$  the set of integers using binary notation. Given two integers from set  $S_1$ , plus any arithmetic operations to carry out on these numbers, today's computers can transform the numbers into elements of set  $S_2$ , perform the operations, and transform the result back into decimal form. The algorithms for transforming the numbers are familiar to most students of computer science. To go from elements of set  $S_1$  to set  $S_2$ , repeated division by 2 is used, and from set  $S_2$  to set  $S_1$ , repeated multiplication is used. The value of binary representation is the simplification that results in the internal circuitry of a computer.

Example 8.2 Let  $S_1$  be the set of  $n$ -degree polynomials ( $n \geq 0$ ) with integer coefficients represented by a list of their coefficients; e.g.,

$$A(x) = a_n x^n + \dots + a_1 x + a_0$$

The set  $S_2$  consists of exactly the same set of polynomials but is represented by their values at  $2n+1$  points; that is, the  $2n+1$  pairs  $(x_i, A(x_i))$ ,  $1 \leq i \leq 2n+1$ , would represent the polynomial  $A$ . (At this stage we won't worry about what the values of  $x_i$  are, but for now you can consider them consecutive integers.) The function  $f$  to be computed is the one that determines the product of two polynomials  $A(x)$  and  $B(x)$ , assuming the set  $S_1$  representation to start with. Rather than forming the product directly using the conventional method (which requires  $O(n^2)$  operations, where  $n$  is the degree of  $A$  and  $B$  and any possible growth in the size of the coefficients is ignored), we could transform the two polynomials into elements of the set  $S_2$ . We do this by evaluating  $A(x)$  and  $B(x)$  at  $2n+1$  points. The product can now be computed simply, by multiplying the corresponding points together. The

representation of  $A(x)B(x)$  in set  $S_2$  is given by the tuples  $(x_i, A(x_i)B(x_i))$ ,  $1 \leq i \leq 2n+1$ , and requires only  $O(n)$  operations to compute. We can determine the product of  $A(x)B(x)$  in coefficient form by finding the polynomial that interpolates (or satisfies) these  $2n+1$  points. It is easy to show that there is a unique polynomial of degree  $\leq 2n$  that goes through  $2n+1$  points.

Figure 8.1 describes these transformation in a graphical form indicating the two paths one can take to reach the coefficient product domain, either directly by conventional multiplication or indirectly by algebraic transformation. The transformation in one direction is effected by evaluation whereas the inverse transformation is accomplished by interpolation. The value of the scheme rests entirely on whether these transformations can be carried out efficiently.

For instance, if  $A(x) = 3x^2 + 4x + 1$  and  $B(x) = x^2 + 2x + 5$ , these can be represented by the pairs  $(0,1), (1,8), (2,21), (3,40)$ , and  $(4,65)$  and  $(0,5), (1,8), (2,13), (3,20)$  and  $(4,29)$ , respectively. Then  $A(x)B(x)$  in  $S_2$  takes the form  $(0,5), (1,64), (2,273), (3,800)$ , and  $(4,1885)$ .

The world of algebraic algorithms is so broad that we only attempt to cover a few of the interesting topics. In section 8.2 we discuss the question of polynomial evaluation at one or more points and the inverse operation of polynomial interpolation at  $n$  points. Then in Section 8.3 we discuss the same problems as in Section 8.2 but this time assuming the  $n$  points are  $n$ th roots of unity. This is shown to be equivalent to computing the Fourier transform. We also show how the divide-and-conquer strategy leads to the fast Fourier transform algorithm. In Section 8.4 we shift our



attention to integer problem, in this case the processes of modular arithmetic. Modular arithmetic can be viewed as a transformation scheme that is useful for speeding up large precision integer arithmetic operations. Moreover we see that transformation into and out of modular form is a special case of evaluation and interpolation. Thus there is an algebraic unity to Sections 8.2, 8.3 and 8.4. Finally, in Section 8.5 we present asymptotically efficient algorithms for n-points evaluation and interpolation.

## 8.2 EVALUATION AND INTERPOLATION

In this section we examine the operations on polynomials of evaluation and interpolation. As we search for efficient algorithms, we see examples of an other design strategy called algebraic simplification. When applied to algebraic problems, algebraic simplification refers to the process of reexpressing computational formulas so that the required number of operations to compute these formulas is minimized. One issue we ignore here is the numerical stability of the resulting algorithms. Though this is often an important consideration, it is too far from our purposes.

A univariate polynomial is generally written as

$$A(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

where  $x$  is an indeterminate and the  $a_i$  may be integers, floating point numbers, or more generally elements of a commutative ring or a field. If  $a_n \neq 0$ , then  $n$  is called the degree of  $A$ .

When considering the representation of a polynomial by its coefficients, there are atleast two alternatives. The first calls for storing the degree followed by degree + 1 coefficients:

$$(n, a_n, a_{n-1}, \dots, a_1, a_0)$$

This is termed the dense representation because it explicitly stores all coefficients whether or not they are zero. We observe that for a polynomial such as  $x^{1000} + 1$  the dense representation is wasteful since it requires 1002 locations although there are only two nonzero terms.

The second representation calls for storing only each nonzero coefficient and its corresponding exponent; for example, if all the  $a_i$  are nonzero, then the polynomial is stored as

$$(n, a_n, n-1, a_{n-1}, \dots, 1, a_1, 0, a_0)$$

This is termed the sparse representation because the storage depends directly on the number of nonzero terms and not on the degree. For a polynomial

```

1 Algorithm StraightEval(A,n,v)
2 {
3 r:=1; s:=a0;
4 for i:=1 to n do
5 {
6 r:=r * v;
```

```

7 s:=s + ai * r;
8 }
9 return s;
10 }

```

### Algorithm 8.1 Straightforward evaluation

of degree  $n$ , all of whose coefficients are nonzero, this second representation requires roughly twice the storage of the first. However, that is the worst case. For high-degree polynomials with few nonzero terms, the second representation is many times better than the first.

Secondarily we note that the terms of a polynomial will often be linked together rather than sequentially stored. However, we will avoid this complication in the following algorithms and assume that we can access the  $i$ th coefficient by writing  $a_i$ .

Suppose we are given the polynomial  $A(x) = a_n x^n + \dots + a_0$  and we wish to evaluate it at a point  $v$ , that is, compute  $A(v)$ . The straightforward or right-or-left method adds  $a_1 v$  to  $a_0$  and  $a_2 v^2$  to this sum and continues as described in Algorithm 8.1. The analysis of this algorithm is quite simple:  $2n$  multiplications,  $n$  additions, and  $2n + 2$  assignments are made (excluding the for loop).

An improvement to this procedure was devised by Isaac Newton in 1711. The same improvement was used by W. G. Horner in 1819 to evaluate the coefficient of  $A(x + c)$ . The method came to be known as Horner's rule. They rewrote the polynomial as

$$A(x) = (\dots((a_n x + a_{n-1})x + a_{n-2})x + \dots + a_1)x + a_0$$

This is our first and perhaps most famous example of algebraic simplification. The function for evaluation that is based on this formula is given in Algorithm 8.2.

Horner's rule requires  $n$  multiplications,  $n$  additions, and  $n+1$  assignments (excluding the for loop). Thus we see that it is an improvement over the straightforward method by a factor of 2. In fact in lesson 9.

```

1 Algorithm Horner (A,n,v)
2 {
3 s:=an;
4 for i:=n-1 to 0 step -1 do
5 {
6 s:=s * v + a;
7 }
8 return s;
9 }
```

#### Algorithm 8.2 Horner's rule

we see that Horner's rule yields the optimal way to evaluate an  $n$ th-degree polynomial.

Now suppose we consider the sparse representation of a polynomial,  $A(x) = a_m x^{e_m} + \dots + a_1 x^{e_1}$ , where the  $a_i \neq 0$  and  $e_m >$

$e_{m-1} > \dots > e_1 \geq 0$ . The straightforward algorithm (Algorithm 8.1), when generalized to this sparse case, is given in Algorithm 8.3

```

1 Algorithm SstraightEval(A,m,v)
2 //Sparse straightforward evaluation
3 // m is number of nonzero terms
4 {
5 s:=0;
6 for i:=1 to m do
7 {
8 s:=s + ai * Power(v,ei);
9 }
10 return s;
11 }
```

#### Algorithm 8.3 Sparse evaluation

Power(v,e) returns  $v^e$ . Assuming that  $v^e$  is computed by repeated multiplication with v, this operation requires e-1 multiplications and Algorithm 8.3 requires  $e_m + e_{m-1} + \dots + e_1$  multiplications, m additions, and m+1 assignments. This is horribly inefficient and can easily be improved

```

1 Algorithm NstraightEval(A,m,v)
2 {
3 s:=e0:=0; t:=1;
4 for i:=1 to m do
5 {
6 r:=Power(v,ei-ei-1);
7 t:= t * r;
8 s:= s+ ai * t;
9 }
10 return s;
11 }

```

Algorithm 8.4 Evaluating a polynomial represented in coefficient exponent form

by an algorithm based on computing

$$v^{e_1}, v^{e_2-e_1} v^{e_1}, v^{e_3-e_2} v^{e_2}, \dots$$

Algorithm 8.4 requires  $e_m + m$  multiplications,  $3m + 3$  assignments,  $m$  additions, and  $m$  subtractions.

A more clever scheme is to generalize Horner's strategy in the revised formula.

$$A(x) = (\dots((a_m x^{e_m - e_{m-1}} + a_{m-1})x^{e_{m-1} - e_{m-2}} + \dots + a_2)x^{e_2 - e_1} + a_1)x^{e_1}$$

The function of Algorithm 8.5 is based on this formula. The number of multiplication required is

$$(e_m - e_{m-1} - 1) + \dots + (e_1 - e_0 - 1) + m = e_m$$

Which is the degree of  $A$ . In addition there are  $m$  additions,  $m$  subtractions, and  $m+2$  assignments. Thus we see that Horner's rule is easily adapted to either the sparse or the dense polynomial model and in both cases the number of operations is bounded and linear in the degree. With a little more work one can find an even better method, assuming a sparse representation, which requires only  $m + \log_2 e_m$  multiplications. (See the exercises for a hint).

Given  $n$  points  $(x_i, u_i)$  the interpolation problem is to find the coefficients of the unique polynomial  $A(x)$  of degree  $\leq n-1$  that goes through these  $n$  points. Mathematically the answer to this problem was given by Lagrange:

```

1 Algorithm SHorner(A,m,v)
2 {
3 s:=e0:=0;
4 for i:=m to 1, step -1 do
5 {
6 s:=(s + a_i) * Power(v, e_i - e_{i-1});
7 }

```

```

8 return s;
9 }

```

Algorithm 8.5 Horner's rule for a sparse representation

$$A(x) = \sum_{1 \leq i \leq n} \left( \prod_{\substack{i \neq j \\ 1 \leq i \leq n}} \frac{(x - x_j)}{(x_i - x_j)} \right) Y_i \quad \dots(8.1)$$

To verify that  $A(x)$  does satisfy the  $n$  points, we observe that

$$A(x) = \left( \prod_{\substack{i \neq j \\ 1 \leq i \leq n}} \frac{(x - x_j)}{(x_i - x_j)} \right) Y_i = Y_i \quad \dots(8.2)$$

since every other term becomes zero. The numerator of each term is a product of  $n-1$  factors and hence the degree of  $A$  is  $\leq n-1$ .

Example 8.3 Consider the input  $(0,1)$ ,  $(1,10)$ , and  $(2,21)$ . Using Equation 8.1, we get

$$A(x) = \frac{(x-1)(x-2)}{(0-1)(0-2)} 5 + \frac{(x-0)(x-2)}{(1-0)(1-2)} 10 + \frac{(x-0)(x-1)}{(2-0)(2-1)} 21$$



$$= \frac{5}{2} (x^2 - 3x + 2) - 10 (x^2 - 2x) + \frac{21}{2} (x^2 - x)$$

$$= 3x^2 + 2x + 5$$

we can verify that  $A(0)=5$ ,  $A(1)=10$ , and  $A(2)=21$ .

We now give an algorithm (Algorithm 8.6) that produces the coefficients of  $A(x)$  using Equation 8.1. We need to perform some addition and multiplication of polynomials. So we assume that the operators and  $+$ ,  $*$ ,  $/$ , and  $=$  have been overloaded to take polynomials as operands.

```

1 Algorithm Lagrange(X,Y,n,A)
2 // X and Y are one-dimensional arrays containing
3 // n points (xi,yi), 1 ≤ i ≤ n. A is a
4 // polynomial that interpolates these points.
5 {
6 // poly is a polynomial.
7 A:=0;
8 for i:=1 to n do
9 {
10 poly:=1; denom:=1;

```

```

11 for j:=1 to n do
12 if (i ≠ j) then
13 {
14 poly := poly * (x-X[j]);
15 // x- X[j] is a degree one polynomial in x.
16 denom :=denom * (X[i]-X[j]);
17 }
18 A :=A +(poly * Y[i]/denom);
19 }
20 }

```

### Algorithm 8.6 Lagrange interpolation

An analysis of the computing time of Lagrange is instructive. The if statement is executed  $n^2$  times. The time to compute each new value of denom is one subtraction and one multiplication, but the execution of  $*$  (as applied to polynomials) requires more than constant time per call. Since the degree of  $x-X[j]$  is one, the time for one execution of  $*$  is proportional to the degree of poly, which is at most  $j-1$  on the  $j$ th iteration.

Therefore the total cost of the polynomial multiplication step is

$$\begin{aligned} \sum_{1 \leq i \leq n} \sum_{1 \leq j \leq n} (j-1) &= \sum_{1 \leq i \leq n} \left( \frac{n(n+1)}{2} - n \right) \\ &= n^2(n+1)/2 - n^2 \end{aligned} \quad 8.3$$

$$\text{Thus } \sum_{1 \leq i \leq n} \sum_{1 \leq j \leq n} (j-1) = O(n^3)$$

This result is discouraging because it is so high. Perhaps we should search for a better method. Suppose we already have an interpolating polynomial  $A(x)$  such that  $A(x_i) = y_i$  for  $1 \leq i \leq n$  and we want add just one more point  $(x_{n+1}, y_{n+1})$ . How would we compute this new interpolating polynomial given the fact that  $A(x)$  was already available? If we could solve this problem efficiently, then we could apply our solution  $n$  times to get an  $n$ -point interpolating polynomial.

Let  $G_{j-1}(x)$  interpolate  $j-1$  points  $(x_k, y_k)$ ,  $1 \leq k < j$ , so that  $G_{j-1}(x_k) = y_k$ . Also let  $D_{j-1}(x) = (x-x_1) \dots (x-x_{j-1})$ . Then we can compute  $G_j(x)$  by the formula.

$$G_j(x) = [y_j - G_{j-1}(x_j)] \frac{D_{j-1}(x)}{D_{j-1}(x_j)} + G_{j-1}(x) \quad (9.3)$$

We observe that

$$G_j(x_k) = [y_j - G_{j-1}(x_j)] \frac{D_{j-1}(x)}{D_{j-1}(x_j)} + G_{j-1}(x_k)$$

but  $D_{j-1}(x_k) = 0$  for  $1 \leq k < j$ . So

$$G_j(x_k) = G_{j-1}(x_k) = y_k$$

Also we observe that

$$\begin{aligned} G_j(x_j) &= [y_j - G_{j-1}(x_j)] \frac{D_{j-1}(x)}{D_{j-1}(x_j)} + G_{j-1}(x_j) \\ &= [y_j - G_{j-1}(x_j)] + G_{j-1}(x_j) \\ &= y_j \end{aligned}$$

Example 8.4 Consider again the input (0,5), (1,10), and (2,21). Here  $G_1(x) = 5$  and  $D_1(x) = (x-x_1) = x$

$$G_2(x) = [y_2 - G_1(x_2)] \frac{D_1(x)}{D_1(x_2)} + G_1(x) = (10-5) \frac{x}{1} + 5 = 5x + 5$$

Also,  $D_2(x) = (x-x_1)(x-x_2) = (x-0)(x-1) = x^2 - x$ . Finally

$$\begin{aligned} G_3(x) &= [y_3 - G_2(x_3)] \frac{D_2(x)}{D_2(x_3)} + G_2(x) \\ &= [21-15] \frac{x^2 - x}{1} + (5x + 5) = 3x^2 + 2x + 5 \end{aligned}$$

Having verified that this formula is correct, we present an algorithm (Algorithm 8.7) for computing the interpolating polynomial that is based on Equation 8.4. Notice that from the equation, two applications of Horner's rule are required, one for evaluating  $G_{j-1}(x)$  at  $x_j$  and the other for evaluating  $D_{j-1}(x)$  at  $x_j$ .

1     Algorithm Interpolate( $X, Y, n, G$ )

```

2 // Assume $n \geq 2$. $X[1:n]$ and $Y[1:n]$ are the
3 // n pairs of points. The unique interpolating
4 // polynomial of degree $< n$ is returned in G .
5 {
6 // D is a polynomial.
7 $G := Y[1]$; // G begins as a constant.
8 $D := x - X[1]$; // D is a linear polynomial.
9 for $j := 2$ to n do
10 {
11 $\text{denom} := \text{Horner}(D, j-1, X[j])$; // Evaluate D at $X[j]$.
12 $\text{num} := \text{Horner}(G, j-2, X[j])$; // Evaluate G at $X[j]$.
13 $G := G + (D * (Y[j] - \text{num} / \text{denom}))$;
14 $D := D * (x - X[j])$;
15 }
16 }

```

### Algorithm 8.7 Newtonian interpolation

On the  $j$ th iteration  $D$  has degree  $j-1$  and  $G$  has degree  $j-2$ . Therefore the invocations of Horner require.

$$\sum_{1 \leq j \leq n-1} (j-1 + j-2) = n(n-1) - 3(n-1) = n^2 - 4n + 3 \quad (8.5)$$

multiplications in total. The term  $(Y[j] - \text{num}) / \text{denom}$  in Algorithm 9.7 is a constant. Multiplying this constant by  $D$  requires  $j$  multiplications and multiplying  $D$  by  $x - X[j]$  requires  $j$  multiplications. The addition with  $G$  requires zero multiplications.

Thus the remaining steps require

$$\sum_{1 \leq j \leq n-1} (2j) = n(n-1) \quad (8.6)$$

operations, so the entire algorithm Interpolate requires  $O(n^2)$  operations.

In conclusion we observe that for a dense polynomial of degree  $n$ , evaluation can be accomplished using  $O(n)$  operations or, for a sparse polynomial with  $m$  nonzero terms and degree  $n$ , evaluation can be done using at most  $O(m+n) = O(n)$  operations. Also, given  $n$  points, we can produce the interpolating polynomial in  $O(n^2)$  time.

### 8.3 THE FAST FOURIER TRANSFORM

If one is able to devise an algorithm that is an order of magnitude faster than any previous method, that is a worthy accomplishment. When the improvement is for a process that has many applications, then that accomplishment has a significant impact on researchers and practitioners. This is the case of the fast Fourier transform. No algorithm improvement has had a greater impact in the recent past than this one. The Fourier transform is used by electrical engineers in a variety of ways including speech transmission, coding theory, and image processing. But before this fast algorithm was developed, the use of this transform was considered impractical.

The Fourier transform of a continuous function  $a(t)$  is given by

$$A(f) = \int_{-\infty}^{\infty} a(t) e^{2\pi i f t} dt \quad (8.7)$$

whereas the inverse transform of  $A(f)$  is

$$a(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} A(f) e^{-2\pi i f t} df \quad (8.8)$$

The  $i$  in the above two equations stands for the square root of  $-1$ . The constant  $e$  is the base of the natural logarithm. The variable  $t$  is often regarded as time, and  $f$  is taken to mean frequency. Then the Fourier transform is interpreted as taking a function of time into a function of frequency.

Corresponding to this continuous Fourier transform is the discrete Fourier transform which handles sample points of  $a(t)$ , namely,  $a_0, a_1, \dots, a_{N-1}$ . the discrete Fourier transform is defined by

$$A_j = \sum_{0 \leq k \leq N-1} a_k e^{2\pi i j k / N}, \quad 0 \leq j \leq N-1 \quad (9.9)$$

and the inverse is

$$a_k = \frac{1}{N} \sum_{0 \leq j \leq N-1} A_j e^{-2\pi i j k / N}, \quad 0 \leq k \leq N-1 \quad (9.10)$$

In the discrete case a set of  $N$  sample points is given and a resulting set of  $N$  points is produced. An important fact to observe is the close connection between the discrete Fourier transform and polynomial evaluation. If we imagine the polynomial

$$a(x) = a_{N-1} x^{N-1} + a_{N-2} x^{N-2} + \dots + a_1 x + a_0$$

these the Fourier element  $A_j$  is the value of  $a(x)$  at  $x = w^j$ , where  $w = e^{2\pi i/N}$ . Similarly for the inverse Fourier transform if  $w$  imagine the polynomial with the Fourier coefficients

$$A(x) = A_{N-1} x^{N-1} + A_{N-2} x^{N-2} + \dots + A_1 x + A_0$$

then each  $a_k$  is the value of  $A(x)/N$  at  $x = (w^{-1})^k$ , where  $w = e^{2\pi i/N}$ . Thus, the discrete Fourier transform corresponds exactly to the evaluation of a polynomial at  $N$  points:  $w^0, w^1, \dots, w^{N-1}$ .

From the preceding section we know that we can evaluate an  $N$ th-degree polynomial at  $N$  points using  $O(N^2)$  operations. We apply Horner's rule once for each point. The fast Fourier transform (abbreviated FFT) is an algorithm for computing these  $N$  values using only  $O(N \log N)$  operations. This algorithm was popularized by J. M. Cooley and J. W. Tukey in 1965, and the long history of this method was traced by J.M Cooley, P.A. Lewis and P.D Welch.

A hint that the Fourier transform can be computed faster than by Horner's rule comes from observing that the evaluation points are not arbitrary but are in fact very special. They are the  $N$  powers  $w^j$  for  $0 \leq j \leq N-1$ , where  $w = e^{2\pi i/N}$ . The point  $w$  is a primitive  $N$ th root of unity in the complex plane.

**Definition 8.1** An element  $w$  in a commutative ring is called a primitive  $N$ th root of unity. If



1.  $w \neq 1$
2.  $w^N = 1$
3.  $\sum_{0 \leq p \leq N-1} w^{jp} = 0, 1 \leq j \leq N-1$

Example 8.5 Let  $N=4$ . Then,  $w = e^{i\pi/2} = \cos(\pi/2) + i \sin(\pi/2) = i$ . Thus,  $w \neq 1$ , and  $w^4 = i^4 = 1$ . Also,  $\sum_{0 \leq p \leq 3} w^{jp} = 1 + i^j + i^{2j} + i^{3j} = 0$ .

We now present two simple properties of  $N$ th roots from which we can see how the FFT algorithm can easily be understood.

Theorem 8.1. Let  $N=2n$  and suppose  $w$  is a primitive  $N$ th root of unity. Then  $-w^j = w^{j+n}$

Proof: Here  $(w^{j+n})^2 = (w^j)^2 (w^n)^2 = (w^j)^2 (w^{2n}) = (w^j)^2$  since  $w^n = 1$ . Since the  $w^j$  are distinct, we know that  $w^j \neq w^{j+n}$ , so we can conclude that  $w^{j+n} = -w^j$

Theorem 8.2 Let  $N=2n$  and  $w$  a primitive  $N$ th root of unity. Then  $w^2$  is a primitive  $n$ th root of unity.

Proof: Since  $w^N = w^{2n} = 1$ ,  $(w^2)^n = 1$ ; this implies  $w^2$  is an  $n$ th root of unity. In addition we observe that  $(w^2)^j \neq 1$  for  $1 \leq j \leq n-1$  since otherwise we would have  $w^k = 1$  for  $1 \leq k \leq 2n = N$  which would contradict the fact that  $w$  is a primitive  $N$ th root of unity. Therefore  $w^2$  is a primitive  $n$ th root of unity.

From this theorem we can conclude that if  $w^j$ ,  $0 < j \leq N-1$ , are the primitive  $N$ th roots of unity and  $N=2n$ , then  $w^{2j}$ ,  $0 < j \leq n-1$ , are primitive  $n$ th roots of unity. Using these two theorems, we are ready to show how to derive a divide-and-conquer

algorithm for the Fourier transform. The complexity of the algorithm is  $O(N \log N)$ , an order of magnitude faster than the  $O(N^2)$  of the conventional algorithm which uses polynomial evaluation.

Again let  $a_{N-1}, \dots, a_0$  be the coefficients to be transformed and let  $a(x) = a_{N-1}x^{N-1} + \dots + a_1x + a_0$ . We break  $a(x)$  into two parts, one of which contains even-numbered exponents and the other odd-number exponents.

$$\begin{aligned} a(x) &= (a_{N-1}x^{N-1} + a_{N-3}x^{N-3} + \dots + a_1x) \\ &\quad + (a_{N-2}x^{N-2} + \dots + a_2x^2 + a_0) \end{aligned}$$

Letting  $y=x^2$ , we can rewrite  $a(x)$  as a sum of two polynomials.

$$\begin{aligned} a(x) &= (a_{N-1}y^{n-1} + a_{N-3}y^{n-2} + \dots + a_1)x \\ &\quad + (a_{N-2}y^{n-1} + a_{N-4}y^{n-2} + \dots + a_0) \\ &= c(y)x + b(y) \end{aligned}$$

Recall that the values of the Fourier transform are  $a(w^j)$ ,  $0 \leq j \leq N-1$ . Therefore the values of  $a(x)$  at the points  $w^j$ ,  $0 \leq j \leq n-1$ , are now expressible as

$$\begin{aligned} a(w^j) &= c(w^{2j})w^j + b(w^{2j}) \\ a(w^{j+n}) &= -c(w^{2j})w^j + b(w^{2j}) \end{aligned}$$

These two formulas are computationally valuable in that they reveal how to take a problem of size  $N$  and transform it into two identical problems of size  $n=N/2$ . These subproblems are

the evaluation of  $b(y)$  and  $c(y)$ , each of degree  $n-1$ , at the points  $(w^2)^j$ ,  $0 \leq j \leq n-1$ , and these points are primitive  $n$ th roots. This is an example of divide-and-conquer, and we can apply the divide-and-conquer strategy again as long as the number of points remains even. This leads us to always choose  $N$  as a power of 2,  $N=2^m$ , for then we can continue to carry out the splitting procedure until a trivial problem is reached, namely, evaluating a constant polynomial.

FFT( Algorithm 8.8) combines all these ideas into a recursive version of the fast Fourier transform algorithm. Dense representation for polynomials is assumed. We overload the operators  $+$ ,  $-$ ,  $*$ , and  $=$  with regard to complex numbers.

```

1 Algorithm FFT(N,a(x),w,A)
2 // $N = 2^m$, $a(x) = a_{N-1}x^{N-1} + \dots + a_0$, and w is a
3 // primitive N th root of unity. $A[0:N-1]$ is set to
4 // the values $a(w^j)$, $0 \leq j \leq N-1$
5 {
6 // b and c are polynomial
7 // B, C and wp are complex arrays.
8 if $N=1$ then $A[0]:=a_0$;
9 else
10 {
```

```

11 n:=N/2;
12 b(x):=aN-2xn-1+....+a2x+a0;
13 c(x):=aN-1xn-1+....+a3x+a1;
14 FFT(n,b(x),w2,B);
15 FFT(n,c(x), w2,C);
16 wp[-1]:=1/w;
17 for j:=0 to n-1 do
18 {
19 wp[j]:=w * wp[j-1];
20 A[j]:=B[j] + wp[j] * C[j];
21 A[j+n]:=B[j] - wp[j] * C[j];
22 }
23 }
24 }
```

### Algorithm 8.8 Recursive fast Fourier transform

Now let us derive the computing time of FFT. Let  $T(N)$  be the time for the algorithm applied to  $N$  inputs. Then we have

$$T(N) = 2T(N/2) + DN$$

where  $D$  is a constant and  $DN$  is a bound on the time needed to form  $b(x)$ ,  $c(x)$ , and  $A$ . Since  $T(1) = d$ , where  $d$  is another constant, we can repeatedly simplify this recurrence relation to get

$$\begin{aligned}
 T(2^m) &= 2T(2^{m-1}) + D2^m \\
 &= \\
 &\quad \vdots \\
 &\quad \vdots \\
 &= Dm2^m + T(1)2^m \\
 &= DN \log_2 N + dN \\
 &= O(N \log_2 N)
 \end{aligned}$$

Suppose we return briefly to the problem considered at the beginning of this chapter, the multiplication of polynomials. The transformation technique calls for evaluating  $A(x)$  and  $B(x)$  at  $2N+1$  points (where  $N$  is the degree of  $A$  and  $B$ ), computing the  $2N+1$  products  $A(x_i)B(x_i)$ , and then finding the product  $A(x)B(x)$  in coefficient form by computing the interpolating polynomial that satisfies these points. In section 8.2 we saw that  $N$ -point evaluation and interpolation required  $O(N^2)$  operations, so that no asymptotic improvement is gained by using this transformation over the conventional multiplication algorithm. However, in this section we have seen that if the points are chosen to be the  $N=2^m$  distinct powers of a primitive  $N$ th root of unity, then evaluation and interpolation can be done using at most  $O(N \log N)$  operations. Therefore by using the fast Fourier transform

algorithm, we can multiply two  $N$ -degree polynomials in  $O(N \log N)$  operations.

The divide-and-conquer strategy plus some simple properties of primitive  $N$ th roots of unity leads to a very nice conceptual framework for understanding the FFT. The above analysis shows that asymptotically it is better than the direct method by an order of magnitude. However the version we have produced uses auxiliary space for  $b, c, B$ , and  $C$ . We need to study this algorithm more closely to eliminate this overhead.

**Example 8.6** Consider the case in which  $a(x) = a_3x^3 + a_2x^2 + a_1x + a_0$ . Let us walk through the execution of Algorithm 8.8 on this input. Here  $N=4$ ,  $n=2$ , and  $w=i$ . The polynomials  $b$  and  $c$  are constructed as  $b(x) = a_2x + a_0$  and  $c(x) = a_3x + a_1$ . Function FFT is invoked on  $b(x)$  and  $c(x)$  to get  $B[0]=a_0 + a_2$ ,  $B[1]=a_0 + a_2w^2$ ,  $C[0]=a_1 + a_3$ , and  $C[1]=a_1 + a_3w^2$ .

In the for loop, the array  $A[]$  is modified. When  $j=0$ ,  $wp[0]=1$ . Thus,  $A[0] = B[0] + C[0] = a_0 + a_1 + a_2 + a_3$  and  $A[2] = B[0] - C[0] = a_0 + a_2 - a_1 - a_3 = a_0 + a_1w^0 + a_2w^2 + a_3w^3$  (since  $w^2=-1$ ,  $w^4=1$ , and  $w^6=-1$ ). When  $j=1$ ,  $wp[1]=w$ . Then  $A[1] = B[1] + wC[1] = a_0 + a_2w^2 + w(a_1 + a_3w^2) = a_0 + a_1w + a_2w^2 + a_3w^3$  and  $A[3] = B[1] - wC[1] = a_0 + a_2w^2 - w(a_1 + a_3w^2) = a_0 - a_1w + a_2w^2 - a_3w^3 = a_0 + a_1w^3 + a_2w^6 + a_3w^9$  (since  $w^2=-1$ ,  $w^4=1$ , and  $w^6=-1$ ).

### 8.3.1 An In-place Version of the FFT

Recall that if we view the elements of the vector  $(a_0, \dots, a_{N-1})$  to be transformed as coefficients of a polynomial  $a(x)$ , then the Fourier transform is the same as computing  $a(w^j)$  for  $0 \leq j \leq N$ . This transformation is also equivalent to computing the

remainder when  $a(x)$  is divided by the linear polynomial  $x-w^j$ , for if  $q(x)$  and  $c$  are the quotient and remainder such that

$$a(x) = (x-w^j)q(x) + c$$

then  $a(w^j) = 0 * q(x) + c = c$ . We could divide  $a(x)$  by these  $N$  linear polynomials, but that would require  $O(N^2)$  operations. Instead we make use of the principle called balancing and compute these remainders with the help of a process that is structured like a binary tree.

Consider the product of the linear factors  $(x-w^0)(x-w^1)\dots(x-w^7)=x^8-w^0$ . All the intermediate terms cancel and leave only the exponents eight and zero with nonzero coefficients. If we select out from this product the even-and-odd-degree terms, a similar phenomenon occurs:  $(x-w^0)(x-w^2)(x-w^4)(x-w^6)=(x^4-w^0)$  and  $(x-w^1)(x-w^3)(x-w^5)(x-w^7)=(x^4-w^4)$ . Continuing in a similar fashion, we see in Figure 9.2 that the selected products have only two nonzero terms and we can continue this splitting until only linear factors are present.

Now suppose we want to compute the remainders of  $a(x)$  by eight linear factors  $(x-w^0), \dots, (x-w^7)$ . We begin by computing the remainder of  $a(x)$  divided by the product  $d(x)=(x-w^0)\dots(x-w^7)$ . If  $a(x)=q(x)d(x) + r(x)$ , then  $a(w^j)=r(w^j)$ ,  $0 \leq j \leq 7$ , since  $d(w^j)=0$  and the degree of  $r(x)$  is less than the degree of  $d(x)$  which equals 8. Now we divide  $r(x)$  by  $x^4-w^0$  and obtain  $s(x)$ , and by  $x^4-w^4$  and obtain  $t(x)$ . Then  $a(w^j) = r(w^j) = s(w^j)$  for  $j=0,2,4$ , and  $6$  and  $a(w^j) = r(w^j) = t(w^j)$  for  $j=1,3,5$  and  $7$  and degrees of  $s$  and  $t$  are less than 4. Next we divide  $s(x)$  by  $x^2-w^0$  and  $x^2-w^4$  and obtain remainders  $u(x)$  and  $v(x)$ , where  $a(w^j) = u(w^j)$  for  $j=0$  and  $4$  and  $a(w^j) = v(w^j)$  for  $j=2$  and  $6$ . Notice how

each divisor has only two nonzero terms and so the division process will be fast. Continuing in this way, we eventually conclude with the eight values  $a(x) \bmod (x - w^j)$  for  $j=0,1,\dots,7$ .

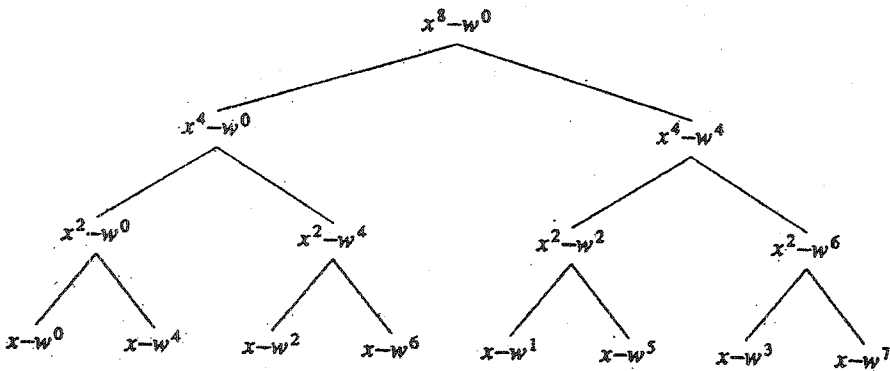


Figure 8.2 Divisors in the FFT algorithm of size eight

By carrying out successive divisions down the binary tree of Figure 9.2, we eventually arrive at the appropriate coefficients of the Fourier transform. The order of these coefficients is permuted in the same way the  $x - w^j$  appear at the bottom of the tree, but this can be corrected at the end of the algorithm. Since this permutation caused the polynomials at each node of the tree to have such a simple form, the division at each stage is simple and the resulting computation time for the entire transform reduces to  $O(N \log N)$ . One can see this in a simple way by observing that the tree has  $\log N$  levels and  $2^i$  nodes on each level, where a dividend polynomial on level  $i$  has at most  $2^{m-i}$  terms. Thus the work on the  $i$ th level is proportional to  $2^i 2^{m-i} = 2^m = N$  and hence



$O(N \log N)$  bounds the time for the entire algorithm. Algorithm 9.9 uses this point of view to produce an FFT algorithm that is iterative in nature.

The NFFT is an in-place, iterative version of the fast Fourier transform. Note that the elements of the vector to be transformed are initially stored in the array  $a[1:N]$ . The NNFFT begins by rearranging the input so that at the end of the algorithm the correct values are in their proper positions. Complex arithmetic is assumed, and  $w = e^{2\pi i/N}$  is expressed in terms of sines and cosines. To verify that the complexity of the NFFT is truly  $O(N \log N)$ , assume  $N=2^m$  and examine the triply nested for loops. The statements contained in the innermost for loop require no more than constant time for each iteration. The innermost for loop is executed no more than  $\lceil N/2^i \rceil < 2^{m-i+1}$  times. This implies that the total time of NFFT is bounded by

```

1 Algorithm NFFT (a,m)
2 // In-place FFT algorithm where a[1:N] has
3 // the input coefficients. $N=2^m$. The elements
4 // of the transform are computed inplace.
5 {
6 $N:=2^m; j:=1;$
7 for $i:=1$ to $N-1$ do
8 { // Permute the input
9 if $i < j$ then
10 {
```

```

11 t:=a[j];a[j]:=a[i];a[i]:=t;
12 }
13 q:=[N/2];
14 while (q<j) do
15 {
16 j:=j-q; q:=[q/2];
17 }
18 j:=j+q;
19 }
20 for i:=1 to m do
21 {
22 pow:=[2i/2];
23 r:=1;
24 s:=cos(π /pow) + i sin (π /pow);
25 // An Nth root of unity
26 for j:=1 to pow do
27 {
28 for q:=j to N step 2i do
29 {
30 t:=a[q + pow] * r;
31 a[q + pow]:=a[q]-t;
32 // Compute the next pair

```

```

33 a[q]:=a[q] + t;
34 }
35 r:=r * s;
36 }
37 }
38 }

```

### Algorithm 8.9 In-place nonrecursive FFT

$$\sum_{1 \leq i \leq m} \sum_{1 \leq j \leq 2} c 2^{m-i+j} = \sum_{1 \leq i \leq m} c 2^m = c 2^m m = O(N \log N)$$

Example 8.7 Now suppose we simulate the algorithm as it works on the particular case  $N=4$ . We assume as inputs the symbolic quantities  $a[1]=a_0$ ,  $a[2]=a_1$ ,  $a[3]=a_2$ , and  $a[4]=a_3$ . Initially  $m=2$  and  $N=4$ . After the first for loop is completed, the array contains the elements permuted as  $a[1]=a_0$ ,  $a[2]=a_2$ ,  $a[3]=a_1$ , and  $a[4]=a_3$ . The main for loop is executed for  $i=1$  and  $i=2$ . After the  $i=1$  pass is completed, the array contains  $a[1]=a_0 + a_2$ ,  $a[2]=a_0 - a_2$ ,  $a[3]=a_1 + a_3$ , and  $a[4]=a_1 - a_3$ . At this point we observe that in general that  $w^{N/2} = -1$  and in this case  $w^2 = -1$  and the complex number expressed as the 2-tuple  $\cos \pi + i \sin \pi$  is equal to  $w$ . At the end of the algorithm the final values in the array are  $a[1]=a_0 + a_1 + a_2 + a_3$ ,  $a[2]=a_0 + wa_1 + w^2a_2 + w^3a_3$ ,  $a[3]=a_0 + w^2a_1 + a_2 + w^2a_3$ , and  $a[4]=a_0 + w^3a_1 + w^2a_2 + wa_3$ .

### 8.3.2 Some Remaining Points

Up to now we have been treating the value  $w$  as  $e^{2\pi i/N}$ . This is a complex number (it has an imaginary part) and its value cannot be represented exactly in a digital computer. Thus the arithmetic operations performed in the Fourier transform algorithm were assumed to be operations on complex numbers, and this implies they are approximations to the actual values. When the inputs to be transformed are readings from a continuous signal, approximations of  $w$  do not cause any significant loss in accuracy. However, there are occasions when we would prefer an exact result, for instance, when we are using the FFT for polynomial multiplication in a mathematical symbol manipulation system. It is possible to circumvent the need for approximate, complex arithmetic by working in a finite field.

Let  $p$  be chosen such that it is a prime that is less than your computer's word size and such that the integers  $0, 1, \dots, p-1$  contain a primitive  $n$ th root of unity. By doing all the arithmetic of the fast Fourier transform modulo  $p$ , all the results are single precision. By choosing  $p$  to be a prime, the integers  $0, 1, \dots, p-1$  form a field and all arithmetic operations including division can be performed. If all values during the computation are bounded by  $p-1$ , then the exact answer is formed since  $x \bmod p = x$  if  $0 \leq x < p$ . However, if one or more values exceed  $p-1$ , the exact answer can still be produced by repeating the transform using several different primes followed by the Chinese Remainder Theorem as described in the next section. So the question that remains is, given an  $N$ , can one find a sufficient number of primes of a certain size that contain  $N$ th roots? From finite field theory  $\{0, 1, \dots, p-1\}$  contains a primitive  $N$ th root if and only if  $N$  divides  $p-1$ . Therefore,

in the following way. Given a set of congruences  $a_i \bmod p_i$ ,  $1 \leq i \leq r$ , we let OneStepCRA be called  $r-1$  times with the following set of values for the parameters.

|                  | a         | p                       | b     | q     | output    |
|------------------|-----------|-------------------------|-------|-------|-----------|
| First time       | $a_1$     | $p_1$                   | $a_2$ | $p_2$ | $c_1$     |
| Second time      | $c_1$     | $p_1 p_2$               | $a_3$ | $p_3$ | $c_2$     |
| Third time       | $c_2$     | $p_1 p_2 p_3$           | $a_4$ | $p_4$ | $c_3$     |
| ...              | ...       | ...                     | ...   | ...   | ...       |
| ( $r-1$ )st time | $c_{r-2}$ | $p_1 p_2 \dots p_{r-1}$ | $a_r$ | $p_r$ | $c_{r-1}$ |

```

1 Algorithm OneStepCRA(a,p,b,q)
2 // a and b are in GF(p), gcd(p,q)=1. This function
3 // returns a c such that c mod p =a and c mod q =b.
4 {
5 t:=a mod q; pb:=p mod q; s:=ExEuclid(pb,q);
6 u:=((b-t) * s) mod q; if (u < 0) then u:= u + q;
7 t:= u * p + a; return t;
8 }
```

Algorithm 8.12 One-step Chinese Remainder Algorithm

we can perform the inverse transformation by the Chinese Remainder Algorithm.

Suppose we consider how to compute the value in the Chinese Remainder Theorem for only two moduli: Given  $a \bmod p$  and  $b \bmod q$ , we wish to determine the unique  $c$  such that  $c \bmod p = a$  and  $c \bmod q = b$ . The value for  $c$  that satisfies these two constraints is easily seen to be

$$c = (b-1)sp + a$$

where  $s$  is the multiplicative reciprocal of  $p \bmod q$ ; that is,  $s$  satisfies  $ps \bmod q = 1$ . To show that this is correct, we note that

$$((b-1)sp + a) \bmod p = a$$

since the term  $(b-1)sp$  has  $p$  as a factor. Secondly

$$\begin{aligned} ((b-1)sp + a) \bmod p &= (b-1)sp \bmod q + a \bmod q \\ &= (b-1) \bmod q + a \bmod q \\ &= (b-1 + a) \bmod q \\ &= b \end{aligned}$$

OneStepCRA (Algorithm 8.12) uses ExEuclid and arithmetic modulo  $p$  to compute the formula we have just described. The computing time is dominated by the call to ExEuclid which requires  $O(\log q)$  operations.

E 18

The simplest way to use this procedure to implement the Chinese Remainder Theorem for  $r$  moduli is to apply it  $r-1$  times

to transform a sequence of size  $N=2^m$  primes of the form  $p=2^e k+1$ , where  $m \leq e$ , must be found. Call such a number a Fourier prime. J. Lipson has shown that there are more than  $x/(2^{e-1} \ln x)$  Fourier primes less than  $x$  with exponent  $e$ , and hence there are more than enough for any reasonable application. For example, if the word size is 32 bits, let  $x=2^{31}$  and  $e=20$ . Then there are approximately 182 primes of the form  $2^f k+1$ , where  $f \leq 20$ . Any of these Fourier primes would suffice to compute the FFT of a sequence of at most  $2^{20}$ . See the exercises for more details.

## 8.4 MODULAR ARITHMETIC

Another example of a useful set of transformations is modular arithmetic. Modular arithmetic is useful in one context because it allows the reformulation of the way addition, subtraction, and multiplication are performed. This reformulation is one that exploits parallelism whereas the normal methods for doing arithmetic are serial. The growth of special computers that make it desirable to perform parallel computation make modular arithmetic attractive. A second use of modular arithmetic is with systems that allow symbolic mathematical computation. These software packages usually provide operations that permit arbitrarily large integers and rational numbers as operands. Modular arithmetic has been found to yield efficient algorithms for the manipulation of large numbers. Finally there is an intrinsic interest in finite field arithmetic (the integers  $0, 1, \dots, p-1$ ), where  $p$  is a prime form a field) by number theorists and electrical engineers specializing in communications and coding theory. In this section we study this subject from a computer scientist's point of view, namely, the development of efficient algorithms for the required operations.

The mode operator is defined as

$$x \bmod y = x - y[x/y], \quad \text{if } y \neq 0$$

$$x \bmod 0 = x$$

Note that  $x/y$  corresponds to fixed point integer division which is commonly found on most current-day computers.

We denote the set of integers  $\{0, 1, \dots, p-1\}$ , where  $p$  is a prime, by  $GF(p)$  (the Galois field with  $p$  elements), named after the mathematician E. Galois who studied and characterized the properties of these fields. Also we assume that  $p$  is a single precision number for the computer you plan to execute on. It is, in fact, true that the set  $GF(p)$  forms a field under the following definitions of addition, subtraction, multiplication, and division, where  $a, b \in GF(p)$

$$(a + b) \bmod p = \begin{cases} a + b & \text{if } a + b < p \\ a + b & \text{if } a + b \geq p \end{cases}$$

$$(a - b) \bmod p = \begin{cases} a - b & \text{if } a - b \geq 0 \\ a - b + p & \text{if } a - b < 0 \end{cases}$$

$(ab) \bmod p = r$  such that  $r$  is the remainder when the product  $ab$  is divided by  $p$ ;  $ab = qp + r$ , where  $0 \leq r < p$ .

$(a/b) \bmod p = (ab^{-1}) \bmod p = r$ , the unique remainder when  $ab^{-1}$  is divided by  $p$ ;  $ab^{-1} = qp + r$ ,  $0 \leq r < p$ .



The factor  $b^{-1}$  is the multiplicative inverse of  $b$  in  $GF(p)$ . For every element  $b$  in  $GF(p)$  except zero, there exists a unique element called  $b^{-1}$  such that  $bb^{-1} \bmod p = 1$ .

Now what are the computing times of these operations? We have assumed that  $p$  is a single precision integer; this implies that all  $a, b \in GF(p)$  are also single precision integers. The time for addition, subtraction, and multiplication mode  $p$ , given the formulas above, is easily seen to be  $O(1)$ . But before we can determine the time for division, we must develop an algorithm to compute the multiplicative inverse of an element  $b \in GF(p)$ .

By definition we know that to find  $x = b^{-1}$ , there must exist an integer  $k$ ,  $0 \leq k < p$ , such that  $bx = kp + 1$ . For example, if  $p=7$ ,

|            |   |   |   |   |   |             |
|------------|---|---|---|---|---|-------------|
| b:         | 1 | 2 | 3 | 4 | 5 | 6 (element) |
| $b^{-1}$ : | 1 | 4 | 5 | 2 | 3 | 6 (inverse) |
| k:         | 0 | 1 | 2 | 1 | 2 | 5           |

An algorithm for computing the inverse of  $b$  in  $GF(p)$  is provided by generalizing Euclid's algorithm for the computation of greatest common divisors (gcds). Given two nonnegative integers  $a$  and  $b$ , Euclid's algorithm computes their gcd. The essential step that guarantees the validity of his method consists of showing that the greatest common divisor of  $a$  and  $b$  ( $a \geq b \geq 0$ ) is equal to  $a$  if  $b$  is zero and is equal to the greatest common divisor of  $b$  and the remainder of  $a$  divided by  $b$  if  $b$  is nonzero.

**Example 8.8**

$$\gcd(22,8) = \gcd(8,6) = \gcd(6,2) = \gcd(2,0) = 2$$

$$\text{and } \gcd(21,13) = \gcd(13,8) = \gcd(8,5) = \gcd(5,3)$$

$$= \gcd(3,2) = \gcd(2,1) = \gcd(1,0) = 1$$

Expressing this process as a recursive function gives Algorithm 8.10.

Using Euclid's algorithm, it is also possible to compute two more integers  $x$  and  $y$  such that  $ax + by = \gcd(a,b)$ . Letting  $a$  be a prime  $p$  and  $b \in \text{GF}(p)$ , the  $\gcd(p,b) = 1$  (since the only divisors of a prime are itself and one), and Euclid's generalization reduces to finding integers  $x$  and  $y$  such that  $px + by = 1$ . This implies that  $y$  is the multiplicative inverse of  $b \bmod p$ .

A close examination of ExEuclid (Algorithm 8.11) shows that Euclid's gcd algorithm is carried out by the steps  $q := \lfloor c/d \rfloor$ ;  $e := c - d * q$ ;  $c := d$ ; and  $d := e$ . The only other steps are the updating of  $x$  and  $y$  as the algorithm proceeds. To analyze the time for ExEuclid, we need to know the number of divisions Euclid's algorithm may require. This was answered in the worst case by G. Lamé in 1845.

```

1 Algorithm GCD(a,b)
2 // Assume $a > b \geq 0$.
3 {
4 if $b \neq 0$ then return GCD(b, a mod b);
```

```

5 else return a;
6 }

```

Algorithm 8.10 Algorithm to compute the gcd of two numbers

```

1 Algorithm ExEuclid(b,p)
2 //b is in GF(p), p being a prime. ExEuclid is a function
3 // whose result is the integer x such that $bx + kp = 1$.
4 {
5 c:=p; d:=b; x:=0; y:=1;
6 while (d $\neq$ 1) do
7 {
8 q:=[c/d];
9 e:=c- d*q;
10 w:=x-y * q;
11 c:=d; d:=e; x:=y; y:=w;
12 }
13 if y < 0 then y:=y +p;
14 return y;
15 }

```

### Algorithm 8.11 Extended Euclidean algorithm

Theorem 8.3[G. Lamé 1845] for  $n \geq 1$ . let  $a$  and  $b$  be integers  $a > b > 0$ , such that Euclid's algorithm applied to  $a$  and  $b$  requires  $n$  division steps. Then  $n \geq 5 \log_{10} b$ .

Thus the while loop is executed no more than  $O(\log_{10} p)$  times, and this is the computing time for the extended Euclidean algorithm and hence for modular division. By modular arithmetic we mean the operations of addition, subtraction, multiplication, and division modulo  $p$  as previously defined.

Now let's see how we can use modular arithmetic as a transformation technique to help us work with integers. We begin by looking at how we can represent integers using a set of moduli, then how we perform arithmetic on this representation, and finally how we can produce the proper integer result.

Let  $a$  and  $b$  be integers and suppose that  $a$  is represented by the  $r$ -tuple  $(a_1, \dots, a_r)$ , where  $a_i = a \bmod p_i$ , and  $b$  is represented by  $(b_1, \dots, b_r)$ , where  $b_i = b \bmod p_i$ . The  $p_i$  are typically single precision primes. This is called a mixed radix representation which contrasts with the conventional representation of integers using a single radix such as 10 (decimal) or 2 (binary). The rules for addition, subtraction, and multiplication using a mixed radix representation are as follows:

$$(a_1, \dots, a_r) + (b_1, \dots, b_r) = ((a_1 + b_1) \bmod p_1, \dots, (a_r + b_r) \bmod p_r)$$

$$(a_1, \dots, a_r) (b_1, \dots, b_r) = ((a_1 b_1) \bmod p_1, \dots, (a_r b_r) \bmod p_r)$$

Example 8.9 For example, let the moduli be  $p_1 = 3$ ,  $p_2 = 5$ , and  $p_3 = 7$  and suppose we start with the integers 10 and 15.

$$10 = (10 \bmod 3, 10 \bmod 5, 10 \bmod 7) = (1, 0, 3)$$

$$15 = (15 \bmod 3, 15 \bmod 5, 15 \bmod 7) = (0, 0, 1)$$

Then

$$10 + 15 = (25 \bmod 3, 25 \bmod 5, 25 \bmod 7) = (1, 0, 4)$$

$$= (1 + 0 \bmod 3, 0 + 0 \bmod 5, 3 + 1 \bmod 7) = (1, 0, 4)$$

$$\text{Also } 15 - 10 = (5 \bmod 3, 5 \bmod 5, 5 \bmod 7) = (2, 0, 5)$$

$$= (0 - 1 \bmod 3, 0 - 0 \bmod 5, 1 - 3 \bmod 7) = (2, 0, 5)$$

$$\text{Also } 10 * 15 = (150 \bmod 3, 150 \bmod 5, 150 \bmod 7) = (0, 0, 3)$$

$$= (1 * 0 \bmod 3, 0 * 0 \bmod 5, 3 * 1 \bmod 7) = (0, 0, 3)$$

After we have performed some desired sequence of arithmetic operations using these  $r$ -tuples, we are left with some  $r$ -tuple  $(c_1, \dots, c_r)$ . We now need some way of transforming back from modular form with the assurance that the resulting integer is the correct one. The ability to do this is guaranteed by the following theorem which was first proven in full generality by L. Euler in 1734.

**Theorem 8.4 [Chinese Remainder Theorem]** Let  $p_1, \dots, p_r$  be positive integers that are pairwise relatively prime (no two integers have a common factor). Let  $p = p_1 \dots p_r$  and let  $b, a_1, \dots, a_r$  be integers. Then, there is exactly one integer  $a$  that satisfies the conditions

$$b \leq a < b + p \text{ and } a \equiv a_i \pmod{p_i} \text{ for } 1 \leq i \leq r$$

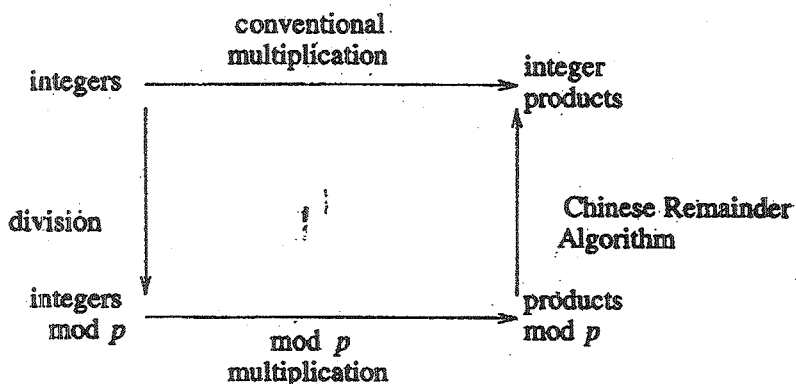


Figure 8.3 Integer multiplication by mod  $p$  transformations

**Proof:** Let  $x$  be another integer, different from  $a$ , such that  $a \equiv x \pmod{p_i}$  for  $1 \leq i \leq r$ . Then  $a - x$  is a multiple of  $p_i$  for all  $i$ . Since the  $p_i$  are pairwise relatively prime, it follows that  $a - x$  is a multiple of  $p$ . Thus, there can be only one solution that satisfies these relations. We show how to construct this value in a moment.

A pictorial view of these transformations when applied to integer multiplication is given in Figure 9.3. Instead of using conventional multiplication, which requires  $O((\log a)^2)$  operations ( $a = \max(a, b)$ ), we choose a set of primes  $p_1, \dots, p_r$  and compute  $a_i = a \bmod p_i$ ,  $b_i = b \bmod p_i$ , and then  $c_i = a_i b_i \bmod p_i$ . These are all single precision operations and so they require  $O(r)$  steps. The  $r$  must be sufficiently large so that  $ab < p_1 \dots p_r$ . The precision of  $a$  is proportional to  $\log a$  and hence the precision of  $ab$  is no more than  $2 \log a = O(\log a)$ . Thus  $r = O(\log a)$  and the time for transformation into modular form and computing the  $r$  products is  $O(\log a)$ . Therefore the value of this method resets on how fast

The final result  $c_{r-1}$  is an integer such that  $c_{r-1} \bmod p_i = a_i$  for  $1 \leq i \leq r$  and  $cr-1 < p_1 \dots p_r$ . The total computing time is  $O(r \log q) = O(r^2)$ .

Example 8.10 Suppose we wish to take 4, 6 and 8 and compute  $4 + 8 * 6 = 52$ . Let  $p_1 = 7$ , and  $p_2 = 11$ .

$$4 = (4 \bmod 7, 4 \bmod 11) = (4, 4)$$

$$6 = (6 \bmod 7, 6 \bmod 11) = (6, 6)$$

$$8 = (8 \bmod 7, 8 \bmod 11) = (1, 8)$$

$$8 * 6 = (6 * 1 \bmod 7, 8 * 6 \bmod 11) = (6, 4)$$

$$4 + 8 * 6 = (4 + 6 \bmod 7, 4 + 4 \bmod 11) = (3, 8)$$

So, we must convert the 2-tuple (3,8) back to integer notation. Using OneStepCRA with  $a=3$ ,  $b=8$ ,  $p=7$  and  $q=11$ , we get

$$t = a \bmod q = 3 \bmod 11 = 3$$

$$pb = p \bmod q = 7 \bmod 11 = 7$$

$$s = \text{ExEuclid}(pb, q) = 8; k = 5$$

$$u = ((b-t)s) \bmod q = (8-3)8 \bmod 11 = 40 \bmod 11 = 7$$

$$\text{return } (u * p + a) = 7 * 7 + 3 = 52.$$

In conclusion we review the computing times for modular arithmetic. If  $a, b \in \text{GF}(p)$ , where  $p$  is single precision then

| operation                | computing time |
|--------------------------|----------------|
| $a + b$                  | $O(1)$         |
| $ab$                     | $O(1)$         |
| $a/b$                    | $O(\log p)$    |
| $c := (c_1, \dots, c_r)$ | $O(r \log c)$  |
| $c_i = c \bmod p_i$      |                |
| $c := (c_1, \dots, c_r)$ | $O(r^2)$       |

## 8.5 EVEN FASTER EVALUATION AND INTERPOLATION

In this section we study four problems:

1. From an  $n$ -precision integer compute its residues modulo  $n$  single precision primes.
2. From an  $n$ -degree polynomial compute its values at  $n$  points
3. From  $n$  single precision residues compute the unique  $n$ -precision integer that is congruent to the residues.
4. From  $n$  points compute the unique interpolating polynomial through those points.

We saw in Sections 8.2 and 8.4 that the classical methods for problems 1 to 4 take  $O(n^2)$  operations. Here we show how to use the fast Fourier transform to speed up all four problems. In



particular we derive algorithms for problems 1 and 2 whose times are  $O(n(\log n)^2)$  and for problems 3 and 4 whose times are  $O(n(\log n)^3)$ . These algorithm rely on the fast Fourier transform as it is used to perform  $n$ -precision integer multiplication in time  $O(n \log n \log \log n)$ . This algorithm, developed by H. Schonhage and V. Strassen, is the fastest known way to multiply. Because this algorithm is complex to describe and already appears in several places (see, e.g., D. E. Knuth cited in References and Readings at the end of this chapter), we simply assume it existence here. Moreover to simplify things some what , we assume that for  $n$ -precision integers and for  $n$ -degree polynomials the time to add or subtract is  $O(n)$  and the time to multiply or divide is  $O(n \log n)$ . In addition we assume that an extended gcd algorithm is available (see Algorithm 8.11) for integers or polynomials whose computing times are  $O(n (\log n)^2)$ .

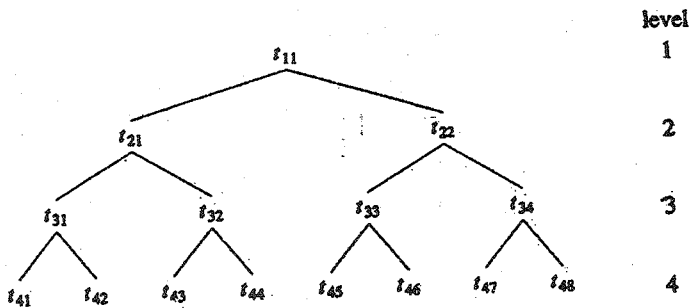


Figure 8.4 A binary tree

Now consider the binary tree as shown in Figure 9.4. As we go down the tree, the level numbers increase, while the root of the tree is at the top at level 1. The  $i$ th level has  $2^{i-1}$  nodes and a tree with  $m$  levels has a total of  $2^{m-1}$  nodes. We are interested in computing different functions at every node of such a binary tree. Algorithm 9.13 is an algorithm for moving up the tree.

Subsequently we are concerned about the cost of the operation  $*$ , which is denoted by  $C(*)$ . Given the value of  $C(*)$  on the  $i$ th level (call it  $C_i(*)$ ) and Algorithm 9.13, the total time needed to compute every node in a tree is

$$\sum_{1 \leq i \leq m-1} 2^{i-1} C_i(*) \quad (9.13)$$

```

1 Algorithm MoveUpATree(t,n)
2 // n = 2m-1 values are stored in t[1:m,1:n] in locations
3 // t[m,1:n]. The algorithm causes the nodes of a binary tree
4 // to be visited so that at each node an abstract binary
 operation
5 //denoted by * is performed. The resulting values are stored
6 // in the array as indicated in Figure 9.4.
7 {
8 for i:= m - 1 to 1 step -1 do
9 {
10 p:=1;
```

```

11 for j:=1 to 2^{i-1} do
12 {
13 $t[i,j]:=t[i+1,p] * t[i+1,p+1]$;
14 $p:=p+2$;
15 }
16 }
17 }

```

#### Algorithm 8.13 Moving up a tree

Similarly Algorithm 8.14 is an algorithm that computes elements as we go down the tree. We now proceed to the specific problems

```

1 Algorithm MoveDownATree (s,t,m)
2 // $n=2^{m-1}$ and $t[1,1]$ is given. Also, $s[1:m,1:n]$ is given
3 // containing a binary tree of values. The algorithm produces
4 // elements and stores them in the array $t[1:m,1:n]$ at the
5 // positions that correspond to the nodes of the binary tree
6 // in Figure 9.4
7 {
8 for i:= 2 to m do
9 {
10 $p:=1$;

```

```

11 for j:=1 to 2^{i-1} step 2 do
12 {
13 $t[i,j] := s[i,j] * t[i-1,p];$
14 $t[i,j+1] := s[i,j+1] * t[i-1,p];$
15 $p := p + 1;$
16 }
17 }
18 }

```

#### Algorithm 8.14 Moving down a tree

**Problem 1** Let  $u$  be an  $n$ -precision integer and  $p_1, \dots, p_n$  be single precision primes. We wish to compute the  $n$  residues  $u_i = u \bmod p_i$  that give the mixed radix representation for  $u$ . We consider the binary tree in Figure 8.5. Starting from the leaves of the tree, we move up the tree, computing the products indicated at each node of the tree.

If  $n = 2^{m-1}$ , then products on the  $i$ th level have precision  $2^{m-i}$ ,  $q \leq i \leq m$ . Using our fast integer multiplication algorithm, we can compute the elements going up the tree. Therefore  $C_i(*)$  is  $2^{m-i-1} (m-i-1)$  and the total time to complete the tree is

$$\begin{aligned}
 & \sum_{1 \leq i \leq m-1} 2^{i-1} 2^{m-i-1} (m-i-1) \\
 &= 2^{m-2} \left( \frac{(m-1)(m-2)}{2} \right) = O(n (\log n)^2) \quad \dots 8.14
 \end{aligned}$$

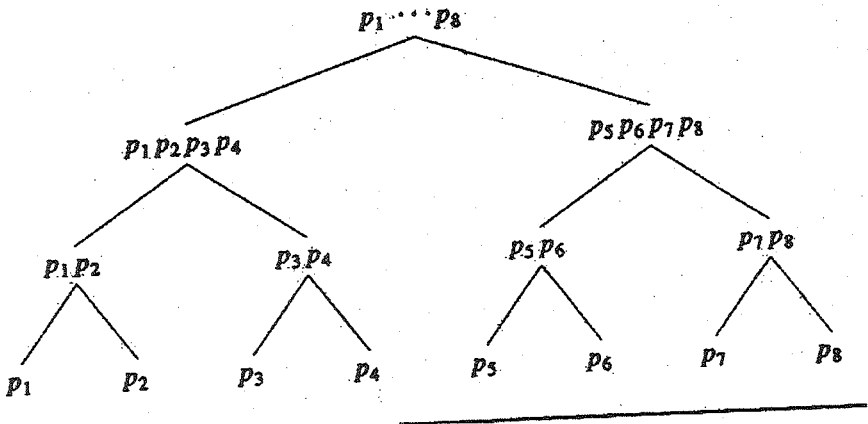


Figure 8.15

Now to compute the  $n$  residues  $u_i = u \bmod p_i$ , we reverse direction and proceed to compute functions down the tree. Since  $u$  is  $n$ -precision and the primes are all near the maximum size of a single precision number, we first compute  $u \bmod p_1 \dots p_n = u_b$ . Then the algorithm continues by computing

$$u_{2,1} = u_b \bmod p_1 \dots p_{n/2} \text{ and } u_{2,2} = u_b \bmod p_{n/2+1} \dots p_n$$

Then we compute

$$u_{3,1} = u_{2,1} \bmod p_1 \dots p_{n/4}, \quad u_{3,2} = u_{2,1} \bmod p_{n/4+1} \dots p_{n/2}$$

$$u_{3,3} = u_{2,2} \bmod p_{n/2+1} \dots p_{3n/4}, \quad u_{3,4} = u_{2,2} \bmod p_{3n/4+1} \dots p_n$$

and so on down the tree until we have

$$u_{m,1} = u_1, u_{m,2} = u_2, \dots, u_{m,2^{(m-1)}} = u_n$$

A node on level  $i$  is computed using the previously computed product of primes at that position plus the element  $u_{j,i-1}$  at the descendant node. The computation requires a division operation so  $C_i(.)$  is  $2^{m-i+1}(m-i+1)$  and the total time for problem 1 is

$$= 2^m \left( m^2 - \frac{m(m-1)}{2} \right) = O(n(\log n)^2) \quad \dots 8.15$$

$$\sum_{1 \leq i \leq m} 2^{i-1} 2^{m-i+1} (m-i+1)$$

Problem 2 Let  $P(x)$  be an  $n$ -degree polynomial and  $x_1, \dots, x_n$  be  $n$  single precision points. We wish to compute the  $n$  values  $P(x_i)$ ,  $1 \leq i \leq n$ . We can use the binary tree in Figure 8.6 to perform this computation.

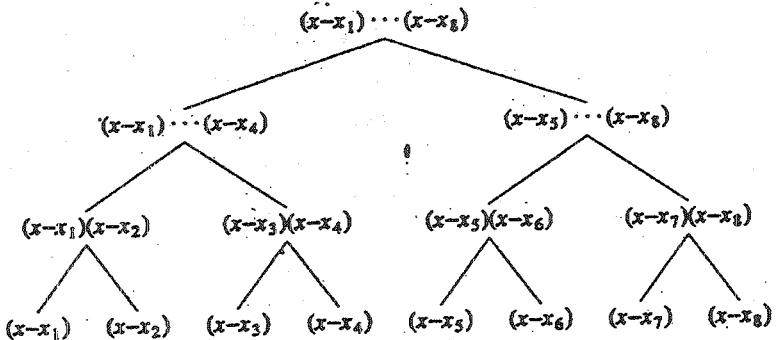


Figure 8.6 A binary tree with linear moduli

First, we move up the tree and compute the products indicated at each node of the tree. If  $n = 2^{m-1}$ , the products on the  $i$ th level have degree  $2^{m-i}$ . Using fast polynomial multiplication, we compute the elements going up the tree. Therefore  $C_i(*)$  is  $2^{m-i-1}(m-i-1)$  and the total time to complete the tree is

$$\begin{aligned} \sum_{i=m-1} 2^{i-1} 2^{m-i-1}(m-i-1) \\ = 2^{m-2} \left( \frac{(m-1)(m-2)}{2} \right) = O(n(\log n)^2) \dots 8.16 \end{aligned}$$

Now to compute the  $n$  values  $P(x_i)$ , we reverse direction and proceed to compute functions down the tree. If  $D(x) = (x-x_1)\dots(x-x_n)$ , then we can divide  $P(x)$  by  $D(x)$  and obtain the quotient and remainder

$$P(x) = D(x)Q(x) + R_{11}(x)$$

where the degree of  $R_{11}$  is less than the degree of  $D$ . By substitution it follows that

$$P(x_i) = R_{11}(x_i), \quad 1 \leq i \leq n$$

The algorithm would continue by dividing  $R_{11}(x)$  by the first  $n/2$  factors of  $D(x)$  and then by second  $n/2$  factors. Calling these polynomials  $D_1(x)$  and  $D_2(x)$ , we get the quotients and remainders

$$R_{11}(x) = D_1(x)Q_1(x) + R_{12}(x)$$

$$R_{11}(x) = D_2(x)Q_2(x) + R_{22}(x)$$

By the same argument we see that

$$P(x_i) = \begin{cases} R_{12}(x_i) & 1 \leq i \leq n/2 \\ R_{22}(x_i) & n/2 + 1 \leq i \leq n \end{cases} \quad \dots 9.17$$

Eventually we arrive at constants  $R_{m,1}, \dots, R_{m,2^{(m-1)}}$ , where  $P(x_i) = R_{m,i}$  for  $1 \leq i \leq n$ . Since the time for multiplication and division of polynomials is the same,  $C_i(*)$  is  $2^{m-i}(m-i)$  and the total for problem 2 is

$$\begin{aligned} & \sum_{i=1}^{m-1} 2^{i-1} 2^{m-i-1}(m-i) \\ &= 2^{m-1} \left( m^2 - \frac{m(m+1)}{2} \right) = O(n(\log n)^2) \quad \dots 9.18 \end{aligned}$$

**Problem 3** Given  $n$  residues  $u_i$  of  $n$  single precision primes  $p_i$ , we wish to find the unique  $n$ -precision integer  $u$  such that  $u \bmod p_i = u_i$ ,  $1 \leq i \leq n$ . It follows from the Chinese Remainder Theorem, Theorem 8.4, that this integer exists and is unique. For this problem, as for problem 1, we assume do is go up the tree and at each node compute a new integer that is congruent to the product of the integers at the children nodes for example at the first level let  $u_i = u_{m,i}$ ,  $1 \leq i \leq n \leq 2^{m-1}$ . Then for  $i$  odd, we compute from  $u_{m,i} \bmod p_i$  and  $u_{m,i+1} \bmod p_{i+1}$ . The unique integer  $u_{m-1,i} = um, i \bmod p_j$  and  $um-1, i = u + \bmod p$ . Thus  $u_{m-1,i}$  lies in the range  $[0, p_{i+1})$ . Repeating this process up the tree, we eventually produce the integer  $u$  in the interval  $[0, p_1 \dots p_n)$ . So we need to develop an algorithm that proceeds from level  $i$  to  $i-1$ . But we already have such an algorithm, the one-step Chinese Remainder Algorithm OneStepCRA. The time for this algorithm was shown



to be dominated by the time for ExEuclid. Using our assumption that ExEuclid can be done in  $O(n(\log n)^2)$  operations, where  $n$  is the maximum precision of the moduli, then this is also the time for OneStepCRA. Note the difference between its use in this section and in Section 8.4. In Section 8.4 only one of the moduli was growing.

We now apply this one-step algorithm to an algorithm that proceeds up the tree of Figure 9.5. The total time for problem 3 is seen to be

$$\begin{aligned} & \sum_{1 \leq i \leq m-1} 2^{i-1} 2^{m-i-1} (m-i-1)^2 \\ &= 2^{m-2} \sum_{1 \leq i \leq m-1} (m-i-1)^2 = O(n(\log n)^3) \end{aligned} \quad (8.19)$$

**Problem 4** Given  $n$  values  $y_1, \dots, y_n$  at  $n=2m-1$  points  $(x_1, \dots, x_n)$ , we wish to compute the unique interpolating polynomial  $P(x)$  of degree  $\leq n-1$  such that  $P(x_i) = y_i$ . For this problem, as for problem 2, we assume that the binary tree in Figure 9.6 has already been computed. Again we need an algorithm that goes up the tree and at each node computes a new interpolating polynomial from its two children. For example, at level  $m$  we compute polynomial  $R_{m1}(x), \dots, R_{mn}(x)$  such that  $R_{mi}(x_i) = y_i$ ,  $1 \leq i \leq n$ . Then at level  $m-1$  we compute  $R_{m-1,1}, \dots, R_{m-1,n/2}$  such that, for  $1 \leq i \leq n/2$ ,

$$R_{m-1,i}(x_{2i-1}) = y_{2i-1}$$

$$R_{m-1,i}(x_{2i}) = y_{2i}$$

and so on, until  $R_{11}(x) = P(x)$ . Therefore we need an algorithm that combines two interpolating polynomials to give a

third that interpolates at both sets of points. This requires a generalization, Algorithm 9.15, of algorithm Interpolate, Algorithm 9.7. In this algorithm, the operators  $+$ ,  $-$ ,  $*$  and  $\text{mod}$  have been overloaded to take polynomial operands. Also  $Q1(x) = (x-x_1)(x-x_2)\dots(x-x_{k/2})$  and  $Q2(x) = (x-x_{k/2+1})\dots(x-x_k)$  with  $\text{gcd}(Q1, Q2) = 1$ . `BalancedInterp` returns a polynomial  $A$  such that  $A(x_i) = U1(x_i)$  for  $1 \leq i \leq k/2$  and  $A(x_i) = U2(x_i)$  for  $k/2+1 \leq i \leq k$ . The degree of  $A$  is  $\leq k-1$ .

We note that lines 5.7 and 8 of Algorithm 8.15 imply that there exist quotients  $C1, C2$ , and  $C3$  such that

$$U1 = Q2 * C1 + P1, \quad \deg(P1) < \deg(Q2) \quad (a)$$

$$Q1 = Q2 * C2 + P2, \quad \deg(P2) < \deg(Q2) \quad (b)$$

$$P4 * P2 + C3 * Q2 = 1, \quad \deg(P4) < \deg(Q2) \quad (c)$$

$P4$  is the multiplicative inverse of  $P2 \text{ mod } Q2$ . Therefore

$$A = U1 + (U2 - P1) * P4 * Q1 \quad (i)$$

$$A = U1 + (U2 + Q2 * C1 - U1) ((1 - C3 * Q2)/P2) * Q1 \quad (ii)$$

1     Algorithm `BalancedInterp`( $U1, U2, Q1, Q2, k$ )

2     //  $U1, U2, Q1$  and  $Q2$  are all polynomials in  $x$ .  $U1$  interpolates

3     // the first  $k/2$  points and  $U2$  interpolates the next  $k/2$  points.

4     {

```

5 P1:=U1 mod Q2;

6 // a mod b computes the poly. remainder of a(x)/
b(x).

7 P2:=Q1 mod Q2;

8 P3:=ExEuclid(P2,Q2);

9 //The extended Euclidean alg. for
polynomials.

10 P4:=P3 mod Q2;

11 return U1 + (U2-P1) * P4 * Q1;

12 }

```

### Algorithm 8.15 Balanced interpolation

using (a) and (c). By (i).  $A(x_i) = U_1(x_i)$  for  $1 \leq i \leq k/2$  since  $Q_1(x)$  evaluated at those points is zero. By (ii), it is easy to see that  $A(x) = U_2(x)$  at points  $x_{k/2+1}, \dots, x_k$ .

Now lines 5 and 7 take  $O(k \log k)$  operations. To compute the multiplicative inverse of  $P_2$ , we use the extended gcd algorithm for polynomials which takes  $O(k (\log k)^2)$  operations. The time for line 10 is no more than  $O(k \log k)$  so the total time for one-step interpolation is  $O(k(\log k)^2)$ .

Applying this one-step algorithm as we proceed up the tree gives a total computing time for problem 4 of

$$\sum_{1 \leq i \leq m-1} 2^{i-1} 2^{m-i-1} (m-i-1)^2 = O(n(\log n)^3) \quad (9.20)$$

The exercises show how one can further reduce the time for problems 3 and 4 using preconditioning.

**Unit - V****Lesson - 9****LOWER BOUND THEORY****OBJECTIVES**

After studying this lesson you can able to understand the following:

- Comparison trees
  - Ordering searching
  - Sorting
- Oracle and Adversary arguments
  - Merging
  - Largest and Second Largest
  - State Space method
  - Selection
- Lower Bounds through Reductions
  - Finding the Convex hull
  - Disjoint sets problem

- On-line Medium finding
- Multiplying Triangular Matrices.
- Investing a Lower Triangular Matrix
- Computing the transitive closure.

## 9.1 INTRODUCTION :

In the previous chapters eight lessons we surveyed a broad range of problems and their algorithmic solution. Our main task for each problem was to obtain a correct and efficient solution. If two algorithms for solving the same problem were discovered and their times differed by an order of magnitude, then the one with the smaller order was generally regarded as superior. But still we are left with the question: is there a faster method? The purpose of this chapter is to expose you to some techniques that have been used to establish that a given algorithm is the most efficient possible. The way this is done is by discovering a function  $g(n)$  that is a lower bound on the time that any algorithm must take to solve the given problem. If we have an algorithm whose computing time is the same order as  $g(n)$ , then we know that asymptotically we can do no better.

Recall from Chapter one that there is a mathematical notation for expressing lower bounds. If  $f(n)$  is the time for some algorithm, then we write  $f(n) = \Omega(g(n))$  to mean that  $g(n)$  is a lower bound for  $f(n)$ . Formally this equation can be written if there exist positive constants  $c$  and  $n_0$  such that  $|f(n)| \geq c|g(n)|$  for all  $n > n_0$ . In addition to developing lower bounds to within a constant factor,

we are also concerned with determining more exact bounds whenever this is possible.

Deriving good lower bounds is often more difficult than devising efficient algorithms. Perhaps this is because a lower bound states a fact about all possible algorithms for solving a problem. Usually we cannot enumerate and analyze all these algorithms, so lower bound proofs are often hard to obtain.

However, for many problems it is possible to easily observe that a lower bound identical to  $n$  exists, where  $n$  is the number of inputs (or possibly outputs) to the problem. For example, consider all algorithms that find the maximum of an unordered set of  $n$  integers. Clearly every integer must be examined at least once, so  $\Omega(n)$  is a lower bound for any algorithm that solves this problem. or, suppose we wish to find an algorithm that efficiently multiplies two  $n \times n$  matrices. Then  $\Omega(n^2)$  is a lower bound on any such algorithm since there are  $2n^2$  inputs that must be examined and  $n^2$  outputs that must be computed. Bounds such as these are often referred to as trivial lower bounds because they are so easy to obtain. We know how to find the maximum of  $n$  elements by an algorithm that uses only  $n-1$  comparisons so there is no gap between the upper and lower bounds for this problem. But for matrix multiplication the best-known algorithm requires  $O(n^{2+\epsilon})$  operations ( $\epsilon > 0$ ), and so there is no reason to believe that a better method cannot be found.

## 9.2 COMPARISON TREES

In this section we study the use of comparison trees for deriving lower bounds on problems that are collectively called

sorting and searching. We see how these trees are especially useful for modeling the way in which a large number of sorting and searching algorithms work. By appealing to some elementary facts about trees, the lower bounds are obtained.

Suppose that we are given a set  $S$  of distinct values on which an ordering relation  $<$  holds. The sorting problem calls for determining a permutation of integers 1 to  $n$ , say  $p(1)$  to  $p(n)$ , such that the  $n$  distinct values from  $S$  stored in  $A[1:n]$  satisfy  $A[p(1)] < A[p(2)] < \dots < A[p(n)]$ . The ordered searching problem asks whether a given element  $x \in S$  occurs within the elements in  $A[1:n]$  that are order so that  $A[1] < \dots < A[n]$ . If  $x$  is in  $A[1:n]$ , then we are to determine an  $i$  between 1 and  $n$  such that  $A[i] = x$ . The merging problem assumes that two ordered sets of distinct input from  $S$  are given in  $A[1:m]$  and  $B[1:n]$  such that  $A[1] < \dots < A[m]$  and  $B[1] < \dots < B[n]$ ; these  $m+n$  values are to be rearranged into an array  $C[1:m+n]$  so that  $C[1] < \dots < C[m+n]$ . For all these problems we restrict the class of algorithms we are considering to those which work solely by making comparisons between elements. No arithmetic involving elements is permitted, though it is possible for the algorithm to move elements around. These algorithms are referred to as comparison-based algorithms. We rule out algorithms such as radix sort that decompose the values into subparts.

### 9.2.1 Ordered Searching

In obtaining the lower bound for the ordered searching problem, we consider only those comparison-based algorithms in which every comparison between two elements of  $S$  is of the type “compare  $x$  and  $A[i]$ ”. The progress of any searching



algorithm that satisfies this restriction can be described by a path in a binary tree. Each internal node in this tree represents a comparison between  $x$  and an  $A[i]$ . There are three possible outcomes of this comparison:  $x < A[i]$ ,  $x = A[i]$ , or  $x > A[i]$ . We can assume that if  $x = A[i]$ , the algorithm terminates. The left branch is taken if  $x < A[i]$ , and the right branch is taken if  $x > A[i]$ . If the algorithm terminates following a left or right branch (but before another comparison between  $x$  and an element of  $A[]$ ), then no  $i$  has been found such that  $x = A[i]$  and the algorithm must declare the search unsuccessful.

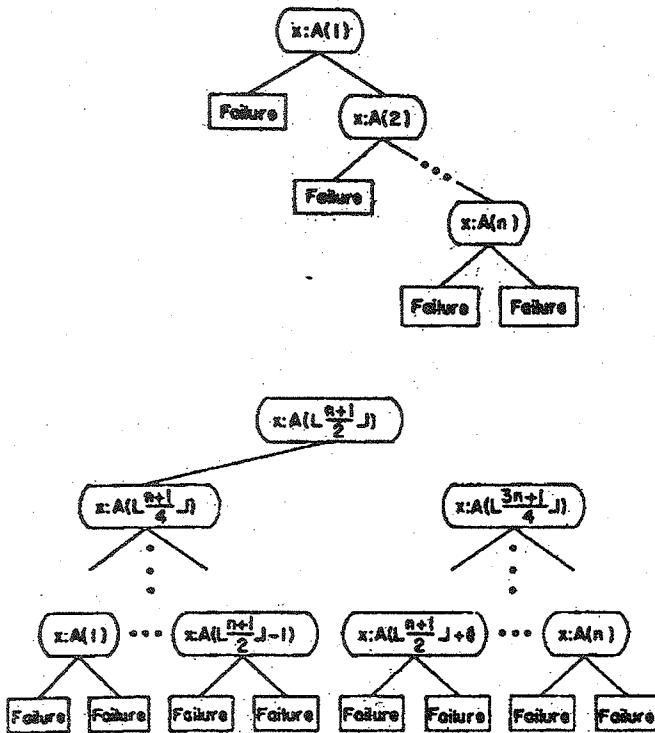


Figure 9.1 Comparison trees for two searching algorithms

Figure 9.1 shows two comparison trees, one modeling a linear search algorithm and the other a binary search (see Algorithm 3.2). It should be easy to see that the comparison tree for any search algorithm must contain at least  $n$  internal nodes corresponding to the  $n$  different values of  $i$  for which  $x = A[i]$  and at least one external node corresponding to an unsuccessful search.

**Theorem 9.1** Let  $A[1:n]$ ,  $n \geq 1$ , contain  $n$  distinct elements, ordered so that  $A[1] < \dots < A[n]$ . Let  $\text{FIND}(n)$  be the minimum number of comparisons needed, in the worst case, by any comparison-based algorithm to recognize whether  $x \in A[1:n]$ . Then  $\text{FIND}(n) \geq \lceil \log(n+1) \rceil$ .

**Proof:** Consider all possible comparison trees that model algorithms to solve the searching problem. The value of  $\text{FIND}(n)$  is bounded below by the distance of the longest path from the root to a leaf in such a tree. There must be  $n$  internal nodes in all these trees corresponding to the  $n$  possible successful occurrences of  $x$  in  $A$ . If all internal nodes of a binary tree are at levels less than or equal to  $k$ , then there are at most  $2^k - 1$  internal nodes. Then  $n \leq 2^k - 1$  and  $\text{FIND}(n) = k \geq \lceil \log(n+1) \rceil$ .

From Theorem 9.1 we can conclude that binary search is an optimal worst-case algorithm for solving the searching problem.

### 9.2.2 SORTING

Now let's consider the sorting problem. We can describe any sorting algorithm that satisfies the restrictions of the comparison tree model by a binary tree. Consider the case in which the  $n$  numbers  $A[1:n]$  to be sorted are distinct. Now, any comparison between  $A[i]$  and  $A[j]$  must result in one of two possibilities: either

$A[i] < A[j]$  or  $A[i] > A[j]$ . So, the comparison tree is binary tree in which each internal node is labeled by the pair  $i:j$  which represents the comparison of  $A[i]$  with  $A[j]$ . If  $A[i]$  is less than  $A[j]$ , then the algorithm proceeds down the left branch of the tree; otherwise it proceeds down the right branch. The external nodes represent termination of the algorithm. Associated with every path from the root to an external node is a unique permutation. To see that this permutation is unique, note that the algorithms we allow are only permitted to move data and make comparison. The data movement on any path from the root to an external node is the same no matter what the initial input values are. As there are  $n!$  different possible permutations of  $n$  items, and any one of these might legitimately be the only correct answer for the sorting problem on a given instance, the comparison tree must have at least  $n!$  external nodes.

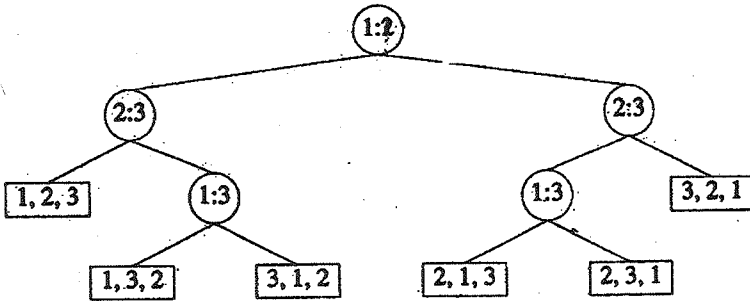


Figure 9.2 A comparison tree for sorting three items

Figure 9.2 shows a comparison tree for sorting three items. The first comparison is  $A[1] : A[2]$ . If  $A[1]$  is less than  $A[2]$ , then the next comparison is  $A[2]$  with  $A[3]$ . If  $A[2]$  is less than  $A[3]$ , then the left branch leads to an external node containing 1,2,3. This implies that the original set was already sorted for  $A[1] < A[2] < A[3]$ . The other five external nodes correspond to the other possible orderings that could yield a sorted set.

Example 9.1 Let  $A[1]=21$ ,  $A[2]=13$ , and  $A[3]=18$ . At the root of the comparison tree (in Figure 9.2) 21 and 13 are compared, and as a result, the computation proceeds to the right subtree. Now, 13 and 18 are compared and the computation proceeds to the left subtree. Then  $A[1]$  and  $A[3]$  are compared and the computation proceeds to the right subtree to yield the permutation  $A[2], A[3], A[1]$ .

We consider the worst case for all comparison-based sorting algorithms. Let  $T(n)$  be the minimum number of comparisons that are sufficient to sort  $n$  items in the worst case. Using our knowledge of binary trees once again, if all internal nodes are at levels less than  $k$ , then there are at most  $2^k$  external nodes (one more than the number of internal nodes). Therefore, if we let  $k=T(n)$ .

$$n! \leq 2^{T(n)}$$

Since  $T(n)$  is an integer, we get the lower bound

$$T(n) \geq \lceil \log n! \rceil$$

By Stirling's approximation (see Exercise 7) it follows that

$$\lceil \log n! \rceil = n \log n - n/\ln 2 + (1/2) \log n + O(1)$$

where  $\ln 2$  refers to the natural logarithm of 2 whereas  $\log n$  is the logarithm to the base 2 of  $n$ . This formula shows that  $T(n)$  is of the order  $n \log n$ . Hence we say that any comparison-based sorting algorithm need  $\Omega(n \log n)$  more complex than comparisons are allowed, for example, operations  $n$ ) time. (This bound can be shown to hold even when operations such as addition, subtraction, and in some cases arbitrary analytic functions.)

How close do the known sorting methods get to this lower bound of  $T(n)$ ? Consider the bottom-up version of merge sort which first orders consecutive pairs of elements and then merges adjacent groups of size 2, 4, 8, .... until the entire sorted set is produced. The worst-case number of comparison required by this algorithm is bounded by

$$\sum_{1 \leq i \leq k} (n/2^i)(2^i - 1) \leq n \log n - O(n) \quad (10.1)$$

Thus we know at least one algorithm that requires slightly less than  $n \log n$  comparison. Is there still a better method?

The sorting strategy called binary insertion sorting works in the following way. The next unsorted item is chosen and a binary search is performed on the sorted set to determine where to place this new item. Then the sorted items are moved to make room for the new value. This algorithm requires  $O(n^2)$  data movements to sort the entire set but far fewer comparisons. Let  $\text{BISORT}(n)$  be the number of comparisons it requires. Then by the results of Section 3.2

$$\text{BISORT}(n) = \sum_{1 \leq i \leq k} [\log_2 k] \quad (10.2)$$

which is equal to

$$n[\log n] - 2^{\lceil \log n \rceil} + 1$$

Now suppose we compare  $\text{BISORT}(n)$  with the theoretical lower bound. This is done in Table 9.1. Scanning Table 9.1, we observe that for  $n=1, 2, 3$  and  $4$  the values are the same so binary insertion is optimal. But for  $n=5$ , there is a difference of one and so we are left with the question of whether  $7$  or  $8$  is the minimum number of comparisons in the worst case needed to sort five items. This question has been answered by L. Ford, Jr., and S. Johnson, who presented a sorting algorithm that requires even fewer comparisons than the binary insertion method. In fact their method requires exactly  $T(n)$  comparisons for  $1 \leq n \leq 11$  and  $20 \leq n \leq 21$ .

---

|                    |   |   |   |   |   |    |    |    |    |    |    |    |    |
|--------------------|---|---|---|---|---|----|----|----|----|----|----|----|----|
| $n$                | 1 | 2 | 3 | 4 | 5 | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 |
| $\{\log n!\}$      | 0 | 1 | 3 | 5 | 7 | 10 | 13 | 16 | 19 | 22 | 26 | 29 | 33 |
| $\text{Bisort}(n)$ | 0 | 1 | 3 | 5 | 8 | 11 | 14 | 17 | 21 | 25 | 29 | 33 | 37 |

---

Table 9.1 Bounds for minimum comparison sorting

To see how the Ford-Johnson method works, we consider the sorting of 17 items that originally reside in  $\text{SORTED}[1:17]$ . We begin by comparing consecutive pairs  $\text{SORTED}[1]:\text{SORTED}[2]$ ,  $\text{SORTED}[3]:\text{SORTED}[4]$ , ...,  $\text{SORTED}[15]:\text{SORTED}[16]$  and placing the larger items into the array  $\text{HIGH}$  and the smaller items into the array  $\text{LOW}$ . The item  $\text{LOW}[9]$

gets SORTED[17]. Then we sort the array HIGH using this algorithm recursively in nonincreasing order, so that HIGH[1] has the largest element. Permute the array LOW also according to this permutation. When this is done, we have  $LOW[1] < HIGH[1] < HIGH[2] < \dots < HIGH[8]$ . Though LOW[2] through LOW[9] remain unsorted, we do know that  $LOW[i] = HIGH[i]$  for  $2 \leq i \leq 8$ . Now inserting LOW[2] into the sorted set possibly requires two comparisons and at the same time causes the insertion of LOW[3] to possibly require three more comparisons for a total of five. A better approach is to insert first LOW[3] among the items LOW[1], HIGH[1], and HIGH[2] using binary insertion and then insert LOW[2]. Each insertion requires only two comparisons and the merged elements are stored back into the array SORTED. This gives us the new relationships  $SORTED[1] < SORTED[2] < \dots < SORTED[6] < HIGH[4] < HIGH[5] < HIGH[6] < HIGH[7] < HIGH[8]$  and  $LOW[i] = HIGH[i]$ , for  $4 \leq i \leq 8$ . Eleven items are now sorted and six remain to be merged. If we insert LOW[4] followed by LOW[5], three and four comparisons may be needed respectively. Once again it is more economical to first insert LOW[5] and the LOW[4]; each insertion requires at most three comparisons. This gives us the new situation  $SORTED[1] < \dots < SORTED[10] < HIGH[6] < HIGH[7] < HIGH[8]$  and  $LOW[i] < HIGH[i]$  for  $6 \leq i \leq 8$ . Inserting LOW[7] requires only four comparisons, and then inserting LOW[8] requires five comparisons. However if we insert LOW[9] and then LOW[8], LOW[7], and LOW[6], then each item requires at most four comparisons. We do the insertions in the order LOW[9] to LOW[6] and get the completely sorted set of 17 items.

A count of the total number of comparisons needed to sort the 17 items is 5 to compare  $\text{SORTED}[i]:\text{SORTED}[i+1]$ , 16 to sort  $\text{HIGH}[1:8]$  using merge insertion recursively, 4 to insert  $\text{LOW}[3]$  and  $\text{LOW}[2]$ , 6 to insert  $\text{LOW}[5]$  and  $\text{LOW}[4]$ , and 16 to insert  $\text{LOW}[9]$  to  $\text{LOW}[6]$ —a total of 50. The value of  $[\log n!]=17$  is 49, and so merge insertion requires only one more comparison than the theoretical lower bound.

In general, merge insertion can be summarized as follows: Let the items to be sorted be  $\text{SORTED}[1:n]$ . Make pairwise comparisons of  $\text{SORTED}[i]$  and  $\text{SORTED}[i+1]$ ; place the larger items into an array  $\text{HIGH}$  and the smaller items into array  $\text{LOW}$ . If  $n$  is odd, then the last item of  $\text{SORTED}$  is appended to  $\text{LOW}$ . Now apply merge insertion to the elements of  $\text{HIGH}$ . Permute  $\text{LOW}$  using the same permutation. Then we know that  $\text{HIGH}[1] < \text{HIGH}[2] < \dots < \text{HIGH}[\lceil n/2 \rceil]$  and  $\text{LOW}[i] < \text{HIGH}[i]$  for  $1 \leq i \leq \lceil n/2 \rceil$ . Now we insert the items of  $\text{LOW}$  into the  $\text{HIGH}$  array using binary insertion. However, the order in which we insert the  $\text{LOW}$ s is important. We want to select the maximum number of items in  $\text{LOW}$  so that the number of comparisons required to insert each one into the already sorted list is a constant  $j$ . As we have seen from our example, the insertion proceeds in the order  $\text{LOW}(t_j), \text{LOW}(t_j-1), \dots, \text{LOW}(t_j-1+1)$ , where the  $t_j$  are a set of increasing integers. In fact  $t_j$  has the form  $t_j = 2^j - 1$ , and in the exercise it is shown that this recurrence relation can be solved to give the formula  $t_j = (2^{j+1} + (-1)^j)/3$ . Thus items are inserted in the order  $\text{LOW}[3], \text{LOW}[2]; \text{LOW}[5], \text{LOW}[4]; \text{LOW}[11], \text{LOW}[10], \text{LOW}[9], \text{LOW}[8], \text{LOW}[7], \text{LOW}[6]$ ; and so on.



It can be shown that the time for this algorithm is

$$\sum_{1 \leq k \leq n} \left\lceil \log_2 \frac{3k}{4} \right\rceil \quad (10.3)$$

For  $n=1$  to 21, the values of this sum are

0,1,3,5,7,10,13,16,19,22,26,30,34,38,42,46,50,54,58,62,66

Comparing these values with  $\lceil \log n! \rceil$ , we see that merge insertion is truly optimal for  $1 \leq n \leq 11$  and  $n=20,21$ .

### 9.2.3 SELECTION

From our previous discussion it should be clear that any comparison tree that models comparison-based algorithms for finding the maximum of  $n$  elements has at least  $2n-1$  external nodes since each path from the root to an external node must contain at least  $n-1$  internal nodes. This implies at least  $n-1$  comparisons for otherwise at least two of the input items never lose a comparison and the largest is not yet found.

Now suppose we  $L_k(n)$  denote a lower bound for the number of comparisons necessary for a comparison-based algorithm to determine the largest, second largest, ...,  $k$ th largest out of  $n$  elements, in the worst case.  $L_1(n)=n-1$  as previously. Since the comparison tree must contain enough external nodes to allow for any possible permutation of the input, it follows immediately that  $L_k(n) \geq \lceil \log n(n-1) \dots (n-k+1) \rceil$ .

Theorem 9.2  $L_k(n) \geq n-k + \lceil \log n(n-1) \dots (n-k+2) \rceil$  for all integers  $k$  and  $n$ , where  $1 \leq k \leq n$ .

Proof: As before internal nodes of the comparison tree contain integers of the form  $i:j$  that imply a comparison between the input items  $A[i]$  and  $A[j]$ . If  $A[i] < A[j]$  then the algorithm proceeds down the left branch; otherwise it proceeds down the right branch. Now consider the set of all possible inputs and place inputs into the same equivalence class if their  $k-1$  largest values appear in the same positions. There will be  $n(n-1)\dots(n-k+2)$  equivalence classes which we denote by  $E_i, i=1,2,\dots$ . Now consider the external nodes for the set of inputs in the equivalence class  $E_i$  (for some  $i$ ). The external nodes of the entire tree are also partitioned into classes called  $X_i$ . For all external nodes in  $X_i$  the positions of the largest,  $\dots, k-1$ st-largest elements are identical. If we examine the subtree of the original comparison tree that defines the class  $X_i$ , then we observe that all comparisons are made on the position of the  $n-k+1$  smallest elements; in essence we are trying to determine the  $k$ th-largest element. Therefore this subtree can be viewed as a comparison tree for finding the largest of  $n-k+1$  elements and it has at least  $2^{n-k}$  external nodes.

Hence the original tree contains at least  $n(n-1)\dots(n-k+2)2^{n-k}$  external nodes and the theorem follows.

### 9.3 ORACLES AND ADVERSARY ARGUMENTS

One of the proof techniques that is useful for obtaining lower bounds consists of making use of an oracle. The most famous oracle in history was called the Delphic oracle, located in Delphi, Greece. This oracle can still be found, situated in the side of a hill embedded in some rocks. In olden times people would approach the oracle and ask it a question. After some period of time elapsed, the oracle would reply and a caretaker would interpret the oracle's answer.

A similar phenomenon takes place when we use an oracle to establish a lower bound. Given some model of computation such as comparison trees, the oracle tells us the outcome of each comparison. To derive a good lower bound, the oracle tries its best to cause the algorithm to work as hard as it can. It does this by choosing as the outcome of the next test, the result that causes the work to be required to determine the final answer. And by keeping track of the work that is done, a worst-case lower bound for the problem can be derived.

### 9.3.1 Merging

Now we consider the merging problem. Given the sets  $A[1:m]$  and  $B[1:n]$ , where the items in  $A$  and the items in  $B$  are sorted, we investigate lower bounds for algorithms that merge these two sets to give a single sorted set. As was the case for sorting, we assume that all the  $m+n$  elements are distinct and that

$A[1] < A[2] < \dots < A[m]$  that there are  $\binom{m+n}{m}$  ways that the

$A$ 's and  $B$ 's can merge together while  $B[1] < B[2] < \dots < B[n]$ . It is possible that after these two sets are merged, the  $n$  elements of  $B$  can be interleaved within  $A$  in every possible way. Elementary combinatorics tells us preserving the ordering within  $A$  and  $B$ . For example, if  $m=3, n=2$ ,  $A[1]=x, A[2]=y, A[3]=z$ ,  $B[1]=u$ , and  $B[2]=v$ , there are  $\binom{3+2}{3} = 10$  ways in which  $A$  and  $B$  can merge:  $u,v,x,y,x$ ;  $u,x,v,y,z$ ;  $u,x,y,v,z$ ;  $u,x,y,z,v$ ;  $x,u,v,y,z$ ;  $x,u,y,v,z$ ;  $x,u,y,z,v$ ;  $x,y,v,z$ ;  $x,y,u,z,v$ ; and  $x,y,z,u,v$ .

Thus if we use comparison trees as our model for merging algorithms, then there will be  $\binom{m+n}{m}$  external nodes, and therefore at least

$$\left\lceil \log \binom{m+n}{m} \right\rceil$$

comparisons are required by any comparison-based merging algorithm. The conventional merging algorithm that was given in (Algorithm 3.8) takes  $m+n-1$  comparisons. If we let  $\text{MER } E(m,n)$  be the minimum number of comparisons needed to merge  $m$  items with  $n$  items, then we have the inequality

$$\left\lceil \log \binom{m+n}{m} \right\rceil \leq \text{MER } E(m,n) \leq m+n-1$$

The exercises show that these upper and lower bound can get arbitrarily far apart as  $m$  gets much smaller than  $n$ . This should not be a surprise because the conventional algorithm is designed to work best when  $m$  and  $n$  are approximately equal. In the extreme case when  $m=1$ , we observe that binary insertion would require the fewest number of comparisons needed to merge  $A[1]$  into  $B[1], \dots, B[n]$ .

When  $m$  and  $n$  are equal, the lower bound given by the comparison tree model is too and the number of comparisons for the conventional merging algorithm can be shown to be optimal.

**Theorem 9.3**  $\text{MER } E(m,m) = 2m-1$ , for  $m \leq 1$ .

**Proof:** Consider any algorithm that merges the two sets  $A[1] < \dots < A[m]$  and  $B[1] < \dots < B[m]$ . We already have an algorithm that requires  $2m-1$  comparisons. If we can show that  $\text{MER } E(m,m) = 2m-1$ , then the theorem follows. Consider any comparison-based algorithm for solving the merging problem and

an instance for which the final result is  $B[1] < A[1] < B[2] < A[2] < \dots < B[m] < A[m]$ , that is, for which the B's and A's alternate. Any merging algorithm must make each of the  $2m-1$  comparisons  $B[1]:A[1]$ ,  $A[1]:B[2]$ ,  $B[2]:A[2]$ , ...,  $B[m]:A[m]$  while merging the given inputs. To see this, suppose that a comparison of type  $B[i]:A[i]$  is not made for some  $i$ . Then the algorithm cannot distinguish between the previous ordering and the one in which

$$B[1] < A[1] < \dots < A[i-1] < A[i] < B[i] < B[i+1] < \dots < B[m] < A[m]$$

So the algorithm will not necessarily merge the A's and B's properly. If a comparison of type  $A[i]:B[i+1]$  is not made, then the algorithm will not be able to distinguish between the cases in which  $B[1] < A[1] < B[2] < \dots < B[m] < A[m]$  and in which  $B[1] < A[1] < B[2] < A[2] < \dots < A[i-1] < B[i] < B[i+1] < A[i] < A[i+1] < \dots < B[m] < A[m]$ . So any algorithm must make all  $2m-1$  comparisons to produce this final result.

## 9.2 Largest and Second Largest

For another example that we can solve using oracles, consider the problem of finding the largest and the second largest elements out of a set of  $n$ . What is a lower bound on the number of comparisons required by any algorithm that finds these two quantities? Theorem 9.2 has already provided us with an answer using comparison trees. An algorithm that makes  $n-1$  comparisons to find the largest and then  $n-2$  to find the second largest gives an immediate upper bound of  $2n-3$ . So a large gap still remains.

This problem was originally stated in terms of a tennis tournament in which the values are called players and the largest value is interpreted as the winner, and the second largest as the runner-up. Figure 9.2 shows a sample tournament among eight players. The winner of each match (which is the larger of the two values being compared) is promoted up the tree until the final round, which in this case, determines McMahon as the winner. Now, who are the candidates for second place? The runner-up must be someone who lost to McMahon but who did not lose to anyone else. In Figure 9.3 that means that either Guttag, Rosen, or Francez are the possible candidates for second place.

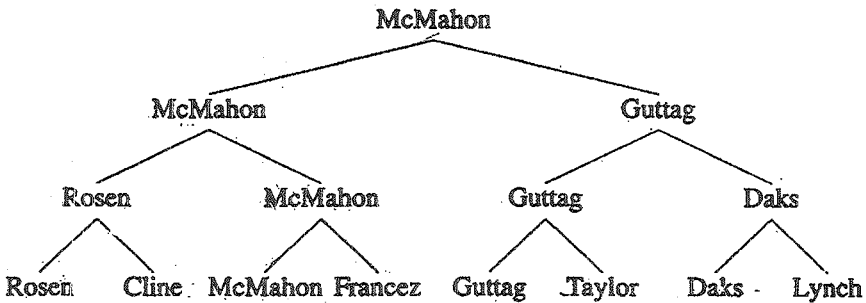


Figure 9.3 A tennis tournament

Figure 9.3 leads us to another algorithm for determining the runner-up once the winner of a tournament has been found. The players who have lost to the winner play a second tournament to determine the runner-up. This second tournament need only be replayed along the path that the winner, in this case McMahon, followed as he rose through the tree. For a tournament with  $n$  players, there are  $\lceil \log n \rceil$  levels, and hence only  $\lceil \log n \rceil - 1$

comparisons are required for this second tournament. This new algorithm, which was first suggested by J. Schreier in 1932, requires a total of  $n-2 + \lceil \log n \rceil$  comparisons. Therefore we have an identical agreement between the known upper and lower bounds for this problem.

Now we show how the same lower bound can be derived using an oracle.

**Theorem 9.4** Any comparison-based algorithm that computes the largest and second largest of a set of  $n$  unordered elements requires  $n-2 + \lceil \log n \rceil$  comparisons.

**Proof:** Assume that a tournament has been played and the largest element and the second-largest element obtained by some method. Since we cannot determine the second-largest element without having determined the largest element, we see that at least  $n-1$  comparisons are necessary. Therefore all we need to show is that there is always some sequence of comparisons that forces the second largest to be found in  $\lceil \log n \rceil - 1$  additional comparisons.

Suppose that the winner of the tournament has played  $x$  matches. Then there are  $x$  people who are candidates for the runner-up position. The runner-up has lost only once, to the winner, and the other  $x-1$  candidates must have lost to one other person. Therefore we produce an oracle that decides the results of matches in such a way that the winner plays  $\lceil \log n \rceil$  other people.

In a match between  $a$  and  $b$  the oracle declares  $a$  the winner if  $a$  is previously undefeated and  $b$  has lost at least once or if both  $a$  and  $b$  are undefeated but  $a$  has won more matches than  $b$ . In

any other case the oracle can decide arbitrarily as long as it remains consistent.

Now, consider a tournament in which the outcome of each match is determined by the above oracle. Imagine drawing a directed graph with  $n$  vertices corresponding to this tournament. Each vertex corresponds to one of the  $n$  players. Draw a directed edge from vertex  $b$  to  $a$ ,  $b \neq a$ , if and only if either player  $a$  has defeated  $b$  or  $a$  has defeated another player who has defeated  $b$ . It is easy to see by induction that any player who has played and won only  $x$  matches can have at most  $2^{x-1}$  edges pointing into her or his corresponding node. Since for the overall winner there must be an edge from each of the remaining  $n-1$  vertices, it follows that the winner must have played at least  $\lceil \log n \rceil$  matches.

### 9.3.3 State Space Method

Another technique for establishing lower bounds that is related to oracles is the state space description method. Often it is possible to describe any algorithm for solving a given problem by a set of  $n$ -tuples. A state space description is a set of rules that show the possible states ( $n$ -tuples) that an algorithm can assume from a given state and a single comparison. Once the state transitions are given, it is possible to derive lower bounds by arguing that the finish state cannot be reached using any fewer transitions. As an example of the state space description method, we consider a problem originally defined and solved in Section 3.3.; given  $n$  distinct items, find the maximum and the minimum. Recall that the divide-and-conquer-based solution required  $\lceil 3n/2 \rceil - 2$  comparisons. We would like to show that this algorithm is indeed optimal.



**Theorem 9.5** Any algorithm that computes the largest and smallest elements of a set of  $n$  unordered elements requires  $\lceil 3n/2 \rceil - 2$  comparisons.

**Proof:** The technique we use to establish a lower bound is to define an oracle by a state table. We consider the state of a comparison-based algorithm as being described by a 4-tuple  $(a,b,c,d)$ , where  $a$  is the number of items that have never been compared,  $b$  is the number of items that have won but never lost,  $c$  is the number of items that have lost but never won, and  $d$  is the number of items have both won and lost. Originally the algorithm is in state  $(n,0,0,0)$  and concludes with  $(0,1,1,n-2)$ . Then, after each comparison the tuple  $(a,b,c,d)$  can make progress only if it assumes one of the five possible states shown in Figure 10.4.

|                                          |               |                         |
|------------------------------------------|---------------|-------------------------|
| $(a-2, b+1, c+1, d)$                     | if $a \geq 2$ | //Two items from $a$    |
|                                          |               | //are compared.         |
| $(a-1, b, c+1, d)$ or $(a-1, b+1, c, d)$ | if $a \geq 1$ | //An item from $a$      |
| or $(a-1, b, c, d+1)$                    |               | //is compared with      |
|                                          |               | //one from $b$ or $c$ . |
| $(a, b-1, c, d+1)$                       | if $b \geq 2$ | //Two items from $b$    |
|                                          |               | //are compared.         |
| $(a, b, c-1, d+1)$                       | if $c \geq 2$ | //Two items from $c$    |
|                                          |               | //are compared.         |

Figure 9.4 States for max-min problem

To get the state  $(0,1,1,n-2)$  from the state  $(n,0,0,0)$ ,  $\lceil 3n/2 \rceil - 2$  comparisons are needed. To see this, observe that the quickest way to get the  $a$  component to zero requires  $n/2$  state changes yielding the tuple  $(0,n/2,n/2,0)$ . Next the  $b$  and  $c$  components are reduced; this requires an additional  $n-2$  state changes.

### 9.3.4 Selection

We end this section by deriving another lower bound on the selection problem. We originally studied this problem in chapter 3 where we presented several solutions. One of the algorithms presented there has a worst-case complexity of  $O(n)$  no matter what value is being selected. Therefore we know that asymptotically any selection algorithm requires  $\theta(n)$  time. Let  $SEL_k(n)$  be the minimum number of comparisons needed for finding the  $k$ th element of an unordered set of size  $n$ . We have already seen that for  $k=1$ ,  $SEL_1(n)=n-1$  and, for  $k=2$ ,  $SEL_2(n)=n-2 + \lceil \log n \rceil$ . In the following paragraphs we present a state table that

shows that  $n-k+(k-1)\left\lceil \log \frac{n}{2^{(k-1)}} \right\rceil = SEL_k(n)$ . We continue

to use the terminology that refers to an element of the set as a player and to comparison between two players as a match that must be won by one of the players. A procedure for selecting the  $k$ th-largest element is referred to as a tournament that finds the  $k$ th-best player.

To derive this lower bound on the selection problem, an oracle is constructed in the form of a state transition table that will cause any comparisonbased algorithm to make at least  $n-k+(k-$

$1)\left\lceil \log \frac{n}{2^{(k-1)}} \right\rceil$  comparisons. The tuple size for states in this case is two, (it was four for the max-min problems), and the components of a tuple, say (Map, Set), are Map, a mapping from the integers  $1,2,\dots,n$  onto itself, and Set, an ordered subset of the input. The initial state is the identity mapping (that is,

$\text{Map}(i)=1, 1 \leq i \leq n$ ) and the empty set. Intuitively, at any given time, the player in Set are the top players (from among all). In particular, the  $i$ th player that enters Set is the  $i$ th-best player. Candidates for entering Set are chosen according to their Map values. At any time period  $t$  the oracle is assumed to be given two unordered elements from the input, say  $a$  and  $b$ , and the oracle acts as follows:

1. If  $a$  and  $b$  are both in Set at time  $t$ , then  $a$  wins iff  $a > b$ . The tuple  $(\text{Map}, \text{Set})$  remains unchanged.
2. If  $a$  is in Set and  $b$  is not in Set, then  $a$  wins and the tuple  $(\text{Map}, \text{Set})$  remains unchanged.
3. If  $a$  and  $b$  are both not in Set and if  $\text{Map}(a) > \text{Map}(b)$  at time  $t$ , then  $a$  wins. If  $\text{Map}(a) = \text{Map}(b)$ , then it doesn't matter who wins as long as no inconsistency with any previous decision is made. In either case, if  $\text{Map}(a) + \text{Map}(b) \geq n/(k-1)$  at time  $t$ , then Map is unchanged and the winner is inserted into Set as a new member. If  $\text{Map}(a) + \text{Map}(b) < n/(k-1)$ . Set stays the same and we set  $\text{Map}(\text{the loser}) := 0$  at time  $t+1$  and  $\text{Map}(\text{the winner}) := \text{Map}(a) + \text{Map}(b)$  at time  $t+1$  and, for all items  $w, w \neq a, w \neq b$ ,  $\text{Map}(w)$  stays the same.

Lemma 9.1 Using the oracle just defined, the  $k-1$  best

players will have played at least  $(k-1) \left\lceil \log \frac{n}{2(k-1)} \right\rceil$  matches when the tournament is completed.

Proof: At time  $t$  the number of matches won by any player  $x$  is greater than or equal to  $\lceil \log \text{Map}(x) \rceil$ . The elements in  $\text{Set}$  are ordered so that  $x_1 < \dots < x_j$ . Now for all  $w$  in the input  $\sum_w \text{Map}(w) = n$ . Let  $W = \{y : y \text{ is not in Set but } \text{Map}(y) > 0\}$ . Since for all  $w$  in the input  $\text{Map}(w) < n/(k-1)$ , it follows that the size of  $\text{Set}$  plus the size of  $W$  is greater than  $k-1$ . However, since the elements  $y$  in  $W$  can only be less than some  $x_i$  in  $\text{Set}$ , if the size of  $\text{Set}$  is less than  $k-1$  at the end of the tournament, then any player in  $\text{Set}$  or  $W$  is a candidate to be one of the  $k-1$  best players. This is a contradiction, so it follows that at the end of the tournament  $|\text{Set}| \geq k-1$ .

Let  $x$  be any element of  $\text{Set}$ . If  $x$  has entered  $\text{Set}$  by defeating  $y$ , it can only be because  $\text{Map}(x) + \text{Map}(y) \geq n/(k-1)$  and  $\text{Map}(x) \geq \text{Map}(y)$ . This in turn means that  $\text{Map}(x) \geq \frac{n}{2(k-1)}$  implying

that  $x$  has played at least  $\left\lceil \log \frac{n}{2(k-1)} \right\rceil$  matches. This is true for every member of  $\text{Set}$  and  $|\text{Set}| \geq (k-1)$ .

We are now in a position to establish the main theorem.

Theorem 9.6 [Hayfi!] The function  $\text{SEL}_k(n)$  satisfies

$$\text{SEL}_k(n) = n - k + (k-1) \left\lceil \log \frac{n}{2(k-1)} \right\rceil$$

Proof: According to Lemma 10.1, the  $k-1$  best players have played at least  $(k-1) \left\lceil \log \frac{n}{2(k-1)} \right\rceil$  matches. Any player who is not among the  $k$  best players has lost at least one match against a player who is not among the  $k-1$  best. Thus there are  $n-k$  additional matches that were not included in the count of the matches played by the  $k-1$  top players. The theorem follows.

## 9.4 LOWER BOUNDS THROUGH REDUCTIONS

Here we discuss a very important technique that can be used to derive lower bounds. This technique calls for reducing the given problem to another problem for which a lower bound is already known.

**Definition 9.1** Let  $P_1$  and  $P_2$  be any two problems. We say  $P_1$  reduces to  $P_2$  (also written  $P_1 \propto P_2$ ) in time  $T(n)$  if an instance of  $P_1$  can be converted into an instance of  $P_2$  and a solution for  $P_1$  can be obtained from a solution for  $P_2$  in time  $\leq r(n)$ .

**Example 9.2** Let  $P_1$  be the problem of selection (discussed in Section 3.6) and  $P_2$  be the problem of sorting. Let the input have  $n$  numbers. If the numbers are sorted, say in an array  $A[]$ , the  $i$ th-smallest element of the input can be obtained  $A[i]$ . Thus  $P_1$  reduces to  $P_2$  in  $O(1)$  time.

**Example 9.3** Let  $S_1$  and  $S_2$  be the two sets with  $m$  elements each. The problem  $P_1$  is to check whether the two sets are disjoint, that is whether  $S_1 \cap S_2 = \emptyset$ .  $P_2$  is the sorting problem. We can

show that  $P_1 \propto P_2$  in  $O(m)$  time as follows.

Let  $S1 = \{k_1, k_2, \dots, k_m\}$  and  $S2 = \{\ell_1, \ell_2, \dots, \ell_m\}$ . The instance of  $P_2$  to be created has  $n = 2m$  and the sequence of keys to be sorted is  $X = (k_1, 1), (k_2, 1), \dots, (k_m, 1), (\ell_1, 2), (\ell_2, 2), \dots, (\ell_m, 2)$ . In other words, each key in  $X$  is a tuple and the sorting has to be done in lexicographic order. The conversion of  $P_1$  to  $P_2$  takes  $O(m)$  time, since it involves the creation of  $2m$  tuples.

Now we have to show that a solution to  $P_1$  is obtainable from the solution to  $P_2$  in  $O(m)$  time. Let  $X'$  be  $X$  in sorted order. Once  $X'$  has been computed, what remains to be done is to sequentially go through the elements of  $X'$  from left to right and check whether there are two successive elements  $(x, 1)$  and  $(y, 2)$  such that  $x=y$ . If there are no such elements, then  $S_1$  and  $S_2$  are disjoint; otherwise they are not.

Note: If  $P_1$  reduces to  $P_2$  in  $\tau(n)$  time and if  $T(n)$  is a lower bound on the solution time of  $P_1$ , then, clearly,  $T(n) - \sigma(n)$  is a lower bound on the solution time of  $P_2$ . In Example 10.3, we can infer that  $P_2$  has a lower bound of  $T(n) - O(n)$ , where  $T(n)$  is a lower bound for the solution of  $P_1$  on two sets with  $m=n/2$  elements each. In Lesson 10 we revisit the notion of reduction in the context of NP-hard problems.

Now we present several examples to illustrate the above technique of reduction in deriving lower bounds. The first problem is that of computing the convex hull of points in the plane (see Section 3.8) and the second problem is  $P_1$  of Example 9.3

### 9.4.1 Finding the Convex Hull

Given  $n$  points in the plane, recall that the convex hull problem is to identify the vertices of the hull in some order (clockwise or counterclockwise). We now show that any algorithm for this problem takes  $\Omega(n \log n)$  time. If we call the convex hull problem  $P_2$  and the problem of sorting  $P_1$ , then this slower bound is obtained by showing that  $P_1$  reduces to  $P_2$  in  $O(n)$  time. Since sorting of  $n$  points needs  $\Omega(n \log n)$  time (see Section 10.1) the result readily follows.

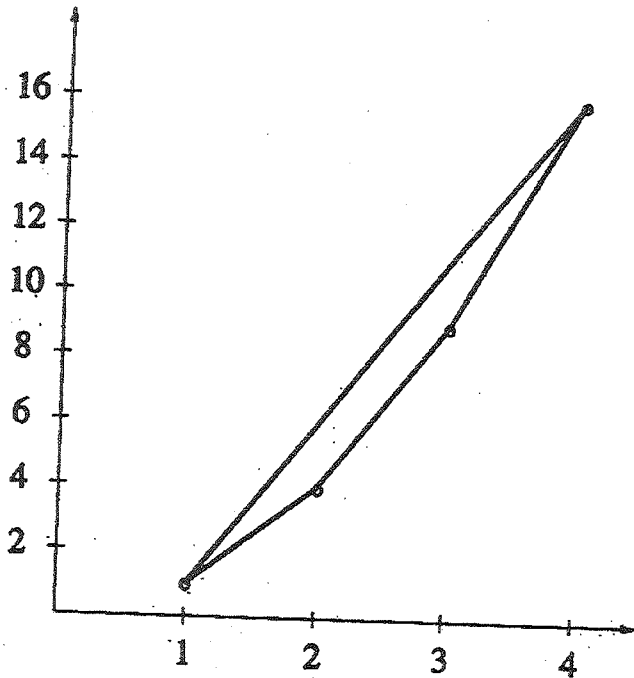


Figure 9.5 Sorting numbers using the convex hull reduction

Let  $P_1$  be the problem of sorting the sequence of numbers  $K = k_1, k_2, \dots, k_n$ .  $P_2$  takes as input points in the plane. We convert the  $n$  numbers into the  $n$  points  $(k_1, k_1^2), (k_2, k_2^2), \dots, (k_n, k_n^2)$ . Construction of these points takes  $O(n)$  time. These points lie on the parabola  $y = x^2$ .

The convex hull of these points has all the  $n$  points as vertices and moreover they are ordered according to their  $x$ -coordinate values (that is  $k_i$  values). If  $X = (\ell_1, \ell_1^2), (\ell_2, \ell_2^2), \dots, (\ell_n, \ell_n^2)$  is the output (in counterclockwise order) for  $P_2$ , we can identify the point of  $X$  with the least  $x$ -coordinate in  $O(n)$  time. If  $(\ell^1, \ell^{1^2})$  is this point, the sorted order of  $K$  is the  $x$ -coordinates of points in  $X$  starting from  $\ell^1$  and moving counterclockwise. Thus determining the output for  $P_1$  also takes  $O(n)$  time.

**Example 9.4** Consider the problem of sorting the numbers 2, 3, 1 and 4. The four points created are (2, 4), (3, 9), (1, 1), and (4, 16). The convex hull of these points is shown in Figure 9.5. All the four points are on the hull. A counterclockwise ordering of the hull is (3, 9), (4, 16), (1, 1), and (2, 4), from which the sorted order of the points can be retrieved.

Therefore we arrive at the following lemma.

**Lemma 9.2** Computing the convex hull of  $n$  given points in the plan needs  $\Omega(n \log n)$  time.

### 9.4.2 Disjoint Sets Problem

In Example 9.3 we showed a reduction from the problem  $P_1$  of deciding whether two given sets are disjoint to the problem  $P_2$  of sorting. This reduction can then be used to derive a lower



bound on the solution time of  $P_2$ , making use of any known lower bound for the solution of  $P_1$ . But the only lower bound we have proved so far is for  $P_2$  and not for  $P_1$ . Now we would like to derive a lower bound for  $P_1$ .

**Lemma 9.3** Any algorithm for solving  $P_1$ , when the sets are of size  $n$  each needs  $\Omega(n \log n)$  time.

**Proof:** We show that  $P_2 \propto P_1$  in  $O(n)$  time. Let  $K = k_1, k_2, \dots, k_n$  be any sequence of  $n$  numbers that we want to sort. Also let  $X = x_1, x_2, \dots, x_n$  be the sorted order of the sequence  $K$ . To construct an instance of  $P_1$ , we let  $S_1 = \{(k_1, 0), (k_2, 0), \dots, (k_n, 0)\}$  and  $S_2 = \{(k_1, 1), (k_2, 1), \dots, (k_n, 1)\}$ .

Any algorithm for  $P_1$  must compare  $(x_i, 1)$  with  $(x_{i+1}, 0)$  for each  $i$ ,  $1 \leq i \leq n-1$ . If not, we can replace  $(x_i, 1)$  in  $S_2$  with  $(x_{i+1}, 0)$  and force the algorithm to output an incorrect answer. Replacing  $(x_i, 1)$  with  $(x_{i+1}, 0)$  does not in any way alter the outcomes of other comparisons made by the algorithm.

Our claim is that the above comparisons are sufficient to sort  $K$ . The sorted order of  $K$  can be obtained by constructing a graph  $G(V, E)$  as follows:  $V = \{k_1, k_2, \dots, k_n\}$  and there is a directed edge from  $k_i$  to  $k_j$  if the algorithm for  $P_1$  has determined that  $k_i < k_j$  (for any  $1 \leq i, j \leq n$ ). This graph is constructible in  $O(n + T(n))$  time, where  $T(n)$  is the run time of the algorithm for  $P_1$ . To obtain the elements of  $K$  in sorted order, find the smallest element  $x_1$  in  $O(n)$  time. Find the smallest among all the neighbors of  $x_1$  in  $G$ ; this will be  $x_2$ . Then find the smallest among all the neighbors of  $x_2$  in  $G$ ; this will be  $x_3$ . And so on. The total time spent is clearly  $O(n + |V| + |E|) = O(n + T(n))$ .

### 9.4.3 On-line Median Finding

In this problem, at every time step we are given a new key. We are required to compute the median of all the given keys before the next key is given. So, if  $n$  keys are given in all, we have to compute  $n$  medians.

**Example 9.5.** Let  $n=7$ . Say the first key input is 7. We output the median as 7. After this we are given the second key, say 15. The median is either 7 or 15. Say the next 5 keys input are 3, 17, 8, 11, and 5, in this order. The corresponding medians output will be 7, 7 or 15, 8, 8, or 11, and 8 respectively.

**Lemma 9.4** The on-line median finding problem (call it  $P_2$ ) requires  $\Omega(n \log n)$  time for its solution, where  $n$  is the total number of keys input.

**Proof:** The lemma is proved by reducing  $P_1$  to  $P_2$ , where  $P_1$  is the problem of sorting. Let  $P_1$  be the Problem of sorting the sequence of keys  $K=k_1, k_2, \dots, k_n$ . The instance of  $P_2$  should be such that each key of  $K$  is an on-line median. Also, the smallest key of  $K$  should be output before the second-smallest key, which in turn should be output before the third smallest key, and so on. Such an instance of  $P_2$  is created by extending the sequence  $K$  with  $-\infty$ s and  $\infty$ s as follows:

$$\infty, -\infty, \dots, -\infty, k_1, k_2, \dots, k_n, \infty, \infty, \infty, \infty, \dots, \infty, \infty$$

That is, the input consists of  $n$   $-\infty$ s followed by  $K$  and then  $2n$   $\infty$ s. This instance of  $P_2$  can be generated in  $O(n)$  time. The solution to  $P_2$  consists of  $4n$  medians. Note that when the first  $\infty$  is input, the median of all given keys is the smallest key of  $K$ .

Also, when the third  $\infty$  is input, the median is the second-largest key of  $K$ . And so on. In other words the sorted order of  $K$  is nothing but the  $2n+1^{\text{st}}$  median, the  $2n+3^{\text{rd}}$  median, ..., the  $4n-1^{\text{st}}$  median output by any algorithm for  $P_2$ . Therefore,  $P_1$  can be solved given a solution to  $P_2$ , and hence  $P_1$  reduces to  $P_2$  in  $O(n)$  time.

If  $S(n)$  is the time needed to sort  $n$  keys and  $M(m)$  is the solution time of  $P_2$  on a total of  $m$  keys, then the above reduction shows that  $S(n) \leq M(4n) + O(n)$ . But we know that  $S(n) \geq cn \log n$  for some constant  $c$ . Therefore,  $M(4n) \geq cn \log n - O(n)$ . That is,  $M(n) = c \frac{n}{4} \log \frac{n}{4} - O(\frac{n}{4})$ ; this implies that  $M(n) = \Omega(n \log n)$ .

#### 9.4.4 Multiplying Triangular Matrices

An  $n \times n$  matrix  $A$  whose elements are  $\{a_{ij}\}$ ,  $1 \leq i, j \leq n$ , is said to be upper triangular if  $a_{ij} = 0$  whenever  $i > j$ . It is said to be lower triangular if  $a_{ij} = 0$  for  $i < j$ . A matrix that is either upper triangular or lower triangular is said to be triangular.

We are interested in deriving lower bounds for the problem of multiplying two triangular matrices. We restrict our discussion to lower triangular matrices. But the results to be derived hold for upper triangular matrices also. Let  $A$  and  $B$  be two  $n \times n$  lower triangular matrices. If  $M(n)$  is the time needed to multiply two  $n \times n$  full matrices (call this problem  $P_1$ ) and  $M_t(n)$  is the time needed to multiply two lower triangular matrices (call this problem  $P_2$ ), then clearly  $M(n) \leq M_t(n)$ . More interestingly, it turns out that  $M_t(n) = \Omega(M(n))$ ; that is the problem of multiplying two triangular matrices is asymptotically no easier than multiplying two full matrices of the same dimensions.

Lemma 9.5  $M_t(n) = \Omega(M(n))$

Proof: We show that  $P_1$  reduces to  $P_2$  in  $O(n^2)$  time. Note that  $M(n) = \Omega(n^2)$  since there are  $2n^2$  elements in the input and  $n^2$  in the output. Let the two matrices to be multiplied be  $A$  and  $B$  and of size  $n \times n$  each. The instance of  $P_2$  to be created is the following:

$$A' = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & A & 0 \end{pmatrix} \quad B' = \begin{pmatrix} 0 & 0 & 0 \\ B & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Here  $O$  stands for the zero matrix, that is, an  $n \times n$  matrix all of whose entries are zeros. Both  $A'$  and  $B'$  are of size  $3n \times 3n$  each. Multiplying the two matrices, we get

$$A'B' = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ AB & 0 & 0 \end{pmatrix}$$

Thus the product  $AB$  is easily obtainable from the product  $A'B'$ . Problem  $P_1$  reduces to  $P_2$  in  $O(n^2)$  time. This reduction implies that  $M(n) \leq M(3n) + O(n^2)$ ; this in turn means  $M_1(n) \leq M_1(n/3) + O(n^2)$ . Since  $M(n) = \Omega(n^2)$ ,  $M_1(n/3) = \Omega(M(n))$ . Hence,  $M_1(n) = \Omega(M(n))$ .

Note that the above lemma also implies that  $M_1(n) = \Theta(M(n))$ .

### 9.4.5 Inverting a Lower Triangular Matrix

Let  $A$  be an  $n \times n$  matrix. Also, let  $I$  be the  $n \times n$  identity matrix, that is the matrix for which  $i_{kk} = 1$ , for  $1 \leq k \leq n$ , and

whose every other element is zero. The elements  $a_{kk}$  of any matrix  $A$  are called the diagonal elements of  $A$ . Every element of  $I$  is zero except for the diagonal elements which are all ones. If there exists an  $n \times n$  matrix  $B$  such that  $AB = I$ , when we say  $B$  is the inverse of  $A$  and  $A$  is said to be invertible. The inverse of  $A$  is also denoted as  $A^{-1}$ . Not every matrix is invertible. For example, the zero matrix has no inverse.

In this section we concentrate on obtaining a lower bound on the inversion time of a lower triangular matrix. It can be shown that a triangular matrix is invertible if and only if all its diagonal elements are nonzero. Let  $P_1$  be the problem of multiplying two full matrices and  $P_2$  be the problem of inverting a lower triangular matrix. Let  $M(n)$  and  $I_1(n)$  be the corresponding time complexities. We show that  $M(n) = O(I_1(n))$ .

Lemma 9.6  $M(n) = O(I_1(n))$ .

Proof: The claim is that  $P_1$  reduces to  $P_2$  in  $O(n^2)$  time, from which the lemma follows. Let  $A$  and  $B$  be the two  $n \times n$  full matrices to be multiplied. We construct the following lower triangular matrix in  $O(n^2)$  time:

$$C = \begin{pmatrix} I & O & O \\ B & I & O \\ O & A & I \end{pmatrix}$$

where the  $O$ 's and  $I$ 's are  $n \times n$  zero matrices and identity matrices, respectively.  $C$  is a  $3n \times 3n$  matrix. The inverse of  $C$  is

$$C^{-1} = \begin{pmatrix} I & O & O \\ -B & I & O \\ AB & -A & I \end{pmatrix}$$

where  $-A$  refers to  $A$  with all the elements negated. Here also we see that the product  $AB$  is obtainable easily from the inverse of  $C$ . Thus we get  $M(n) \leq I_1(3n) + O(n^2)$ , and hence  $M(n) = O(I_1(n))$ .

Lemma 9.7  $It(n) = O(M(n))$ .

Proof: Let  $A$  be the  $n \times n$  lower triangular matrix to be inverted. Partition  $A$  into four submatrices of size  $\frac{n}{2} \times \frac{n}{2}$  each as follows:

$$A = \begin{pmatrix} A_{11} & O \\ A_{21} & A_{22} \end{pmatrix}$$

Both  $A_{11}$  and  $A_{22}$  are lower triangular matrices and  $A_{21}$  could possibly be a full matrix. The inverse of  $A$  can be verified to be

$$A^{-1} = \begin{pmatrix} A^{-1}_{11} & O \\ -A^{-1}_{22}A_{21}A^{-1}_{11} & A^{-1}_{22} \end{pmatrix}$$

The above equation suggests a divide-and-conquer algorithm for inverting  $A$ . To invert  $A$  which is of size  $n \times n$ , it suffices to invert two lower triangular matrices ( $A_{11}$  and  $A_{22}$ ) of size  $\frac{n}{2} \times \frac{n}{2}$  each and perform two matrix multiplications (i.e.,

compute  $D = A^{-1}_{22}(A^{-1}_{21} A^{-1}_{11})$  and negate a matrix (D). D can be negated in  $n^2_4$  time. The run time of such a divide-and-conquer algorithm satisfies the following recurrence relation:

$$I_t(n) \leq 2I_t\left(\frac{n}{2}\right) + 2M\left(\frac{n}{2}\right) + \frac{n^2}{4}$$

using repeated substitution, we get

$$I_t(n) \leq 2M\left(\frac{n}{2}\right) + 2^2M\left(\frac{n}{2}\right) + \dots + O(n^2)$$

Since  $M(n) = \Omega(n^2)$ , the above simplifies to

$$I_t(n) = O(M(n) + n^2) = O(M(n)).$$

Lemma 9.6 and 9.7 together imply that  $I_t(n) = \theta(M(n))$ .

#### 9.4.6 Computing the Transitive Closure

Let  $G$  be a directed graph whose adjacency matrix is  $A$ . Recall that the reflexive transitive closure (or simply the transitive closure) of  $G$ , denoted  $A^*$ , is a matrix such that  $A^*(i,j) = 1$  if and only if there is a directed path of length zero or more from node  $i$  to node  $j$  in  $G$ . In this section we wish to compute lower bounds on the computing time of  $A^*$  given  $A$ .

In the following discussion we assume that all the diagonal elements of  $A$  are zeros. There is an interesting relationship between the different powers of  $A$  and  $A^*$  captured in the following lemma.

**Lemma 9.8** Let  $A$  be the adjacency matrix of a given directed graph  $G$ . Then,  $A_k(i,j)=1$  if and only if there is a path from node  $i$  to node  $j$  of length exactly equal to  $k$ , for any  $0 \leq k \leq n$ . Here the matrix products are interpreted as follows: Scalar addition corresponds to boolean or and scalar multiplication corresponds to boolean and.

**Proof:** We prove the lemma by induction on  $k$ . When  $k=0$ ,  $A^0$  is the identity matrix and the lemma holds, since there is a path of length zero from every node to itself. When  $k=1$ , the lemma is also true, since  $A(i,j)=1$  if and only if there is an edge from  $i$  to  $j$ . Assume that the lemma is true for all path lengths up to  $k-1$ ,  $k>1$ . We prove it for a path length of  $k$ .

If there is a path of length  $k$  from node  $i$  to node  $j$ , this path consists of an edge from node  $i$  to some other node  $q$  and then a path from  $q$  to  $j$  of length  $k-1$ . In other words, using the induction hypothesis, there exists a  $q$  such that  $A(i,q)=1$  and  $A^{k-1}(q,j)=1$ . If there is such a  $q$ , then  $A_k(i,j) = (A * A^{k-1})(i,j)$  is surely 1.

Conversely, if  $A^k(i,j)=1$ , then since  $A^k = A * A^{k-1}$ , there exists a  $q$  such that  $A(i,q)=1$  and  $A^{k-1}(q,j)=1$ . This means that there is a path from node  $i$  to node  $j$  of length  $k$ .

If there is a path at all from node  $i$  to node  $j$  in  $G$ , the shortest such path will be length  $\leq (n-1)$ ,  $n$  being the number of nodes in  $G$ .

**Lemma 9.9**  $A^* = I + A + A^2 + \dots + A^{n-1}$ .

Lemma 9.9 gives another algorithm for computing  $A^*$ . (We saw search-based algorithms in Section 6.3) Let  $T(n)$  be the time



needed to compute the transitive closure of an  $n$ -node graph.

Lemma 9.10  $M(n) = T(3n) + O(n^2)$ , and hence  $M(n) = O(T(n))$ .

Proof: If  $A$  and  $B$  are the given  $n \times n$  matrices to be multiplies, form the following  $3n \times 3n$  matrix  $C$  in  $O(n^2)$  time:

$$C = \begin{pmatrix} O & A & O \\ O & O & B \\ O & O & O \end{pmatrix}$$

$C^2$  is given by

$$C^2 = \begin{pmatrix} O & O & AB \\ O & O & O \\ O & O & O \end{pmatrix}$$

also  $C^k = O$   $k \geq 3$ . Therefore using lemma 9.9

$$C^* = I + C + C^2 + \dots + C^{n-1} = I + C + C^2 = \begin{pmatrix} O & O & AB \\ O & I & B \\ O & O & I \end{pmatrix}$$

Also,  $C^k = O$  for  $k \geq 3$ . Therefore, using Lemma 10.9

Given  $C^*$ , it is easy to obtain the product  $AB$ .

Lemma 9.11  $T(n)=O(M(n))$ .

Proof: The proof is analogous to that of Lemma 10.7. Let  $G(V,E)$  be the graph under consideration and  $A$  its adjacency matrix. The matrix  $A$  is partitioned into four submatrices of size  $n_2 \times n_2$  each:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$$

Recall that row  $i$  of  $A$  corresponds to edges going out of node  $i$ . Let  $V_1$  be the set of nodes corresponding to rows  $1, 2, \dots, n_2$  of  $A$  and  $V_2$  be the set of nodes corresponding to the rest of the rows.

The entry  $A^*_{11}(i,j)=1$  if and only if there is a path from node  $i \in V_1$  to node  $j \in V_1$  all of whose intermediate nodes are also from  $V_1$ . A similar property holds for  $A^*_{22}$ .

Let  $D = A_{12}A_{21}$  and let  $u$  and  $v \in V_1$ . Then,  $D(u,v)=1$  if and only if there exists a  $w \in V_2$  such that  $\langle u,w \rangle$  and  $\langle w,v \rangle$  are in  $E$ . A similar statement holds for  $A_{21}A_{12}$ .

Let the transitive closure of  $G$  be given by

$$A^* = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

Our goal is to derive a divide-and-conquer algorithm for computing  $A^*$ . Therefore, we should find a way of computing  $C_{11}, C_{12}, C_{21}$ , and  $C_{22}$  from  $A^*_{11}$  and  $A^*_{22}$ .

First we consider the computation of  $C_{11}$ . Note that  $C_{11}$  corresponds to paths from  $i$  to  $j$ , where  $i$  and  $j$  are in  $V_1$ . Of course the intermediate nodes in such paths could as well be from  $V_2$ . Any such path from  $i$  to  $j$  can have several segments, where each segment starts from a node, say  $u_q$  from  $V_1$ , goes to  $w_q \in V_2$  through an edge, goes to  $x_q$  of  $V_2$  through a path of arbitrary length, and goes to  $y_q \in V_1$  through an edge (see Figure 10.6). Any such segment corresponds to  $A_{11} + A_{12} A_{22}^* A_{21}$ . Since there could be an arbitrary number of such segments, we get

$$C_{11} = (A_{11} + A_{12} A_{22}^* A_{21})^*$$

Using similar reasoning, the rest of  $A^*$  can also be determined:  $C_{12} = C_{11} A_{12} A_{22}^*$ ,  $C_{21} = A_{22}^* A_{21} C_{11}$ , and  $C_{22} = A_{22}^* + A_{22}^* A_{21} C_{11} A_{12} A_{22}^*$ .

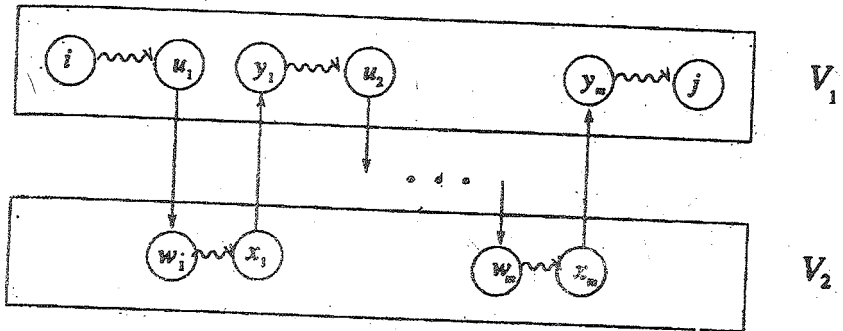


Figure 9.6 A possible path in  $C_{11}$

Thus the above divide-and-conquer algorithm for computing  $A^*$  performs two transitive closures on matrices of size  $\frac{n}{2} \times \frac{n}{2}$  each ( $A_{22}^*$  and  $(A_{11} + A_{12}A_{22}^*A_{21})^*$ ), six matrix multiplication, and two matrix addition on matrices of size  $\frac{n}{2} \times \frac{n}{2}$  each. Therefore we get

$$T(n) \leq 2T\left(\frac{n}{2}\right) + 6M\left(\frac{n}{2}\right) + O(n^2)$$

Repeated substitution yields

$$T(n) \leq \left( 6M\left(\frac{n}{2}\right) + 12M\left(\frac{n}{2}\right) + 24M\left(\frac{n}{8}\right) \dots \right) + O(n^2)$$

But  $M(n) \leq n^2$ , and hence  $M(n/2) \leq 4M(n)$ . Using this fact, we see that  $T(n) = O(M(n) + n^2) = O(M(n))$ .

Lemmas 9.10 and 9.11 show that  $T(n) = \theta(M(n))$ .

## Lesson - 10

**NP-HARD AND NP-COMPLETE PROBLEMS****OBJECTIVE**

After studying this lesson you can able to understand the following

- Basic concepts of NP-Hard and NP-complete problems
- Non deterministic algorithm

**10.1 BASIC CONCEPT**

In this chapter we are concerned with the distinction between problems that can be solved by a polynomial time algorithm and problems for which no polynomial time algorithm is known. It is an unexplained phenomenon that for many of the problems we know and study, the best algorithms for their solutions have computing times that cluster into two groups. The first group consists of problems whose solution times are bounded by polynomials of small degree. Examples we have seen in this book include ordered searching, which is  $O(\log n)$ , polynomial evaluation which is  $O(n)$ , sorting which is  $O(n \log n)$ , and string editing which is  $O(mn)$ .

The second group is made up of problems whose best-known algorithms are nonpolynomial. Examples we have seen include the traveling salesperson and the knapsack problems for

which the best algorithms given in this text have complexities  $O(n^{2^n})$  and  $O(2n/2)$  respectively. In the quest to develop efficient algorithms, no one has been able to develop a polynomial time algorithm for any problem in the second group. This is very important because algorithms whose computing times are greater than polynomial (typically the time is exponential) very quickly require such vast amounts of time to execute that even moderate-size problems cannot be solved.

The theory of NP-completeness which we present here does not provide a method of obtaining polynomial time algorithms for problems in the second group. Nor does it say that algorithms of this complexity do not exist. Instead, what we do is show that many of the problems for which there are no known polynomial time algorithms are computationally related. In fact, we establish two classes of problems. These are given the names NP-hard and NP-complete. A problem that is NP-complete has the property that it can be solved in polynomial time if and only if all other NP-complete problems can also be solved in polynomial time. If an NP-hard problem can be solved in polynomial time, then all NP-complete problems can be solved in polynomial time. All NP-complete problems are NP-hard, but some NP-hard problems are not known to be NP-complete.

Although one can define many distinct problem classes having the properties stated above for the NP-hard and NP-complete classes, the classes we study are related to nondeterministic computations (to be defined later). The relationship of these classes to nondeterministic computations together with the apparent power of nondeterminism leads to the intuitive (though as yet unproved) conclusion that no NP-complete or NP-hard problem is polynomially solvable.

We see that the class of NP-hard problems (and the subclass of NP-complete problems) is very rich as it contains many interesting problems from a wide variety of disciplines. First, we formalize the preceding discussion of the classes.

### 10.1.1 Nondeterministic Algorithms

Up to now the notion of algorithm that we have been using has the property that the result of every operation is uniquely defined. Algorithms with this property are termed deterministic algorithms. Such algorithms agree with the way programs are executed on a computer. In a theoretical framework we can remove this restriction on the outcome of every operation. We can allow algorithms to contain operations whose outcomes are not uniquely defined but are limited to specified sets of possibilities. The machine executing such operations is allowed to choose any one of these outcomes subject to a termination condition to be defined later. This leads to the concept of a nondeterministic algorithm. To specify such algorithms, we introduce three new functions:

1. `Choice(S)` arbitrarily chooses one of the elements of set `S`.
2. `Failure()` signals an unsuccessful completion.
3. `Success()` signals a successful completion.

The assignment statement `x:=Choice(1,n)` could result in `x` being assigned any one of the integers in the range `[1,n]`. There is no rule specifying how this choice is to be made. The `Failure()` and `Success()` signals are used to define a computation of the algorithm. These statements cannot be used to effect a return.

Whenever there is a set of choices that leads to a successful completion, then one such set of choices is always made and the algorithm terminates successfully. A nondeterministic algorithm terminates unsuccessfully if and only if there exists no set of choices leading to a success signal. The computing times for Choice, Success, and Failure are taken to be  $O(1)$ . A machine capable of executing a nondeterministic algorithm in this way is called a nondeterministic machine. Although nondeterministic machines (as defined here) do not exist in practice, we see that they provide strong intuitive reasons to conclude that certain problems cannot be solved by fast deterministic algorithms.

**Example 10.1** Consider the problem of searching for an element  $x$  in a given set of elements  $A[1:n], n \geq 1$ . We are required to determine an index  $j$  such that  $A[j]=x$  or  $j=0$  if  $x$  is not in  $A$ . A nondeterministic algorithm for this is Algorithm 10.1

```

1 j:=Choice(1,n);
2 if A[j]=x then {write(j); Success();}
3 write(0); Failure();
```

#### Algorithm 10.1 Nondeterministic search

From the way a nondeterministic computation is defined, it follows that the number 0 can be output if and only if there is no  $j$  such that  $A[j]=x$ . Algorithm 10.1 is of nondeterministic complexity  $O(1)$ . Note that since  $A$  is not ordered, every deterministic search algorithm is of complexity  $\Omega(n)$ .

**Example 10.2[Sorting]** Let  $A[i], 1 \leq i \leq n$ , be an unsorted array of positive integers. The nondeterministic algorithm



NSort(A,n) (Algorithm 10.2) sorts the numbers into nondecreasing order and then outputs them in this order. An auxiliary array B[1:n] is used for convenience. Line 4 initializes B to zero though any value different from all the A[i] will do. In the for loop of lines 5 to 10, each A[i] is assigned to a position in B. Line 7 nondeterministically determines this position. Line 8 ascertains that B[j] has not already been used. Thus, the order of the numbers in B is some permutation of the initial order in A. The for loop of lines 11 and 12 verifies that B is sorted in nondecreasing order. A successful completion is achieved if and only if the numbers are output in nondecreasing order. Since there is always a set of choices at line 7 for such an output order, algorithm NSort is a sorting algorithm. Its complexity is  $O(n)$ . Recall that all deterministic sorting algorithms must have a complexity  $\Omega(n \log n)$ .

```

1 Algorithm NSort(A,n)
2 //Sort n positive integers.
3 {
4 for i:=1 to n do B[i]:=0; // Initialize B[].
5 for i:=1 to n do
6 {
7 j:=Choice(1,n);
8 if B[j]≠0 then Failure();

```

```

9 B[j]:=A[i];

10 }

11 for i:=1 to n-1 do //Verify order.

12 if B[i]>B[i+1] then Failure();

13 write (B[1:n]);

14 Success();

15 }
```

#### Algorithm 10.2 Nondeterministic sorting

A deterministic interpretation of a nondeterministic algorithm can be made by allowing unbounded parallelism in computation. In theory, each time a choice is to be made, the algorithm makes several copies of itself. One copy is made for each of the possible choices. Thus, many copies are executing at the same time. The first copy to reach a successful completion terminates all other computations. If a copy reaches a failure completion, then only that copy of the algorithm terminates. Although this interpretation may enable one to better understand nondeterministic algorithms, it is important to remember that a nondeterministic machine does not make any copies of an algorithm every time a choice is to be made. Instead, it has the ability to select a “correct” element from the set of allowable choices (if such an element exists) every time a choice is to be made. A correct element is defined relative to a shortest sequence of choices that leads to a successful termination. In case there is no sequence of choices that leads to a successful termination, we assume that the algorithm terminates in one unit

of time with output “ unsuccessful computation”. Whenever successful termination is possible, a nondeterministic machine makes a sequence of choice that is a shortest sequence leading to a successful termination. Since, the machine we are defining is fictitious, it is not necessary for us to concern ourselves with how the machine can make a correct choice at each step.

**DEFINITION 10.1** Any problem for which the answer is either zero or one is called a decision problem. An algorithm for a decision problem is termed a decision algorithm. Any problem that involves the identification of an optimal (either minimum or maximum) value of a given cost function is known as an optimization problem. An optimization algorithm is used to solve an optimization problem.

It is possible to construct nondeterministic algorithms for which many different choice sequences lead to successful completions. Algorithm NSort of Example 10.2 is one such algorithm. If the number  $A[i]$  are not distinct, then many different permutations will result in a sorted sequence. If NSort were written to output the permutation used rather than the  $A[i]$ 's in sorted order, then its output would not be uniquely defined. We concern ourselves only with those nondeterministic algorithms that generate unique outputs. In particular we consider only nondeterministic decision algorithms. A successful completion is made if and only if the output is 1. A 0 is output if and only if there is no sequence of choice leading to a successful completion. The output statement is implicit in the signals Success and Failure. No explicit output statements are permitted in a decision algorithm. Clearly, our earlier definition of a nondeterministic computation implies that the output from a decision algorithm is uniquely defined by the input parameters and algorithm specification.

Although the idea of a decision algorithm may appear very restrictive at this time, many optimization problems can be recast into decision problems with the property that the decision problem can be solved in polynomial time if and only if the corresponding optimization problem can. In other cases, we can at least make the statement that if the decision problem cannot be solved in polynomial time, then the optimization problem cannot either.

**Example 10.3 [Maximum clique]** A maximal complete subgraph of a graph  $G=(V,E)$  is a clique. The size of the clique is the number of vertices in it. The max clique problem is an optimization problem that has to determine the size of a largest clique in  $G$ . The corresponding decision problem is to determine whether  $G$  has a clique of size at least  $k$  for some given  $k$ . Let  $DClique(G,k)$  be a deterministic decision algorithm for the clique decision problem. If the number of vertices in  $G$  is  $n$ , the size of a max clique in  $G$  can be found by making several applications of  $DClique$ .  $DClique$  is used once for each  $k, k=n, n-1, n-2, \dots$ , until the output from  $DClique$  is 1. If the time complexity of  $DClique$  is  $f(n)$ , then the size of a max clique can be found in time  $\leq n f(n)$ . Also, if the size of a max clique can be determined in time  $g(n)$ , then the decision problem can be solved in time  $g(n)$ . Hence, the max clique problem can be solved in polynomial time if and only if the clique decision problem can be solved in polynomial time.

**Example 10.4 [0/1 knapsack]** The knapsack decision problem is to determine whether there is a 0/1 assignment of values to  $x_i$ ,  $1 \leq i \leq n$ , such that  $\sum p_i x_i = r$  and  $\sum w_i x_i \leq m$ . The  $r$  is a given number. The  $p_i$ 's and  $w_i$ 's are nonnegative numbers. If the knapsack decision problem cannot be solved in deterministic polynomial time, then the optimization problem cannot either.

Before proceeding further, it is necessary to arrive at a uniform parameter  $n$  to measure complexity. We assume that  $n$  is the length of the input to the algorithm (that is,  $n$  is the input size). We also assume that all inputs are integer. Rational inputs can be provided by specifying pairs of integers. Generally, the length of an input is measured assuming a binary representation; that is, if the number 10 is to be input, then in binary it is represented as 1010. Its length is 4. In general, a positive integer  $k$  has a length of  $\lceil \log_2 k \rceil + 1$  bits when represented in binary. The length of the binary representation of 0 is 1. The size, or length,  $n$  of the input to an algorithm is the sum of the lengths of the individual numbers being input. In case the input is given using a different representation (say radix  $r$ ), then the length of a positive number  $k$  is  $\lceil \log_r k \rceil + 1$ . thus, in decimal notation,  $r=10$  and the number 100 has a length  $\log_{10} 100 + 1 = 3$ . Since  $\log_r k = \log_2 k / \log_2 r$ , the length of any input using radix  $r$  ( $r > 1$ ) representation is  $c(r)n$ , where  $n$  is the length using a binary representation and  $c(r)$  is a number that is fixed for a given  $r$ .

When inputs are given using the radix  $r=1$ , we say the input is in unary form. In unary form, the number 5 is input as 11111. Thus, the length of a positive integer  $k$  is  $k$ . It is important to observe that the length of a unary input is exponentially related to the length of the corresponding  $r$ -ary input for radix  $r$ ,  $r > 1$ .

**Example 10.5 [Max clique]** The input to the max clique decision problem can be provided as a sequence of edges and an integer  $k$ . Each edge in  $E(G)$  is a pair of numbers  $(i, j)$ . the size of the input for each edge  $(i, j)$  is  $\lceil \log_2 i \rceil + \lceil \log_2 j \rceil + 2$  if a binary representation is assumed. The input size of any instance is

$$n = \sum_{(i,j) \in E(G)} ([\log_2 i] + [\log_2 j] + 2) + [\log_2 k] + 1$$

Note that if  $G$  has only one connected component, then  $n \geq |V|$ . Thus, if this decision problem cannot be solved by an algorithm of complexity  $p(n)$  for some polynomial  $p()$ , then it cannot be solved by an algorithm of complexity  $p(|V|)$ .

Example 10.6[0/1 knapsack] Assuming  $p, w, m$  and  $r$  are all integers, the input size for the knapsack decision problem is

$$q = \sum_{(i=1, \dots, n)} ([\log_2 p_i] + [\log_2 w_i]) + 2n + [\log_2 m] + [\log_2 r] + 2$$

Note that  $q > n$ . If the input is given in unary notation, then the input size  $s$  is  $\sum p_i + \sum w_i + m + r$ . Note that the knapsack decision and optimization problems can be solved in time  $p(s)$  for some polynomial  $p()$  (see the dynamic programming algorithm). However, there is no known algorithm with complexity  $O(p(n))$  for some polynomial  $p()$ .

We are now ready to formally define the complexity of a nondeterministic algorithm.

**DEFINITION 10.2** The time required by a nondeterministic algorithm performing on any given input is the minimum number of steps needed to reach a successful completion if there exists a sequence of choices leading to such a completion. In case successful completion is not possible, then the time required is  $O(1)$ . A nondeterministic algorithm is of complexity  $O(f(n))$  if for all inputs of size  $n, n \geq n_0$ , that result in a successful

completion, the time required is at most  $cf(n)$  for some constants  $c$  and  $n_0$ .

In Definition 10.2 we assume that each computation step is of a fixed cost. In word-oriented computers this is guaranteed by the finiteness of each word. When each step is not of a fixed cost, it is necessary to consider the cost of individual instructions. Thus, the addition of two  $m$ -bit numbers takes  $O(m)$  time, their multiplication takes  $O(m^2)$  time (using classical multiplication), and so on. To see the necessity of this, consider the algorithm Sum (Algorithm 10.3). This is a deterministic algorithm for the sum of subsets decision problem. It uses an  $(m+1)$ -bit word  $s$ . The  $i$ th bit in  $s$  is zero if and only if no subset of the integers  $A[j]$ ,  $1 \leq j \leq n$ , sums to  $i$ . Bit 0 of  $s$  is always 1 and the bits are numbered  $0, 1, 2, \dots, m$  right to left. The function Shift shifts the bits in  $s$  to the left by  $A[i]$  bits. The total number of steps for this algorithm is only  $O(n)$ . However, each step moves  $m+1$  bits of data and would take  $O(m)$  time on a conventional computer. Assuming one unit of time is needed for each basic operation for a fixed word size, the complexity is  $O(nm)$  and not  $O(n)$ .

The virtue of conceiving of nondeterministic algorithms is that often what would be very complex to write deterministically is very easy to write nondeterministically. In fact, it is very easy to obtain polynomial time nondeterministic algorithms for many problems that can be deterministically solved by a systematic search of a solution space of exponential size.

Example 10.7 [knapsack decision problem] DKP (Algorithm 10.4) is a nondeterministic polynomial time algorithm for the knapsack decision problem. The for loop of lines 4 to 8

assigns 0/1 values to  $x[i]$ ,  $1 \leq i \leq n$ . It also computes the total weight and profit corresponding to this choice of  $x[]$ . Line 9 checks to see whether this assignment is feasible and whether the resulting profit is at least  $r$ . A successful termination is possible iff the answer to the decision problem is yes. The time complexity is  $O(n)$ . If  $q$  is the input length using a binary representation, the time is  $O(q)$ .

```

1 Algorithm Sum (A,n,m)
2 {
3 s:=1;
4 // s is an (m+1) bit word. Bit zero is 1.
5 for i:=1 to n do
6 s:=s or Shift(s,A[i]);
7 if the mth bit in s is 1 then
8 write ("A subset sums to m");
9 else write ("No subset sums to m");
10 }
```

### Algorithm 10.3 Deterministic sum of subsets

Example 10.8 [Max clique] Algorithm DCK (Algorithm 10.5) is a nondeterministic algorithm for the clique decision problem. The algorithm begins by trying to form a set of  $k$  distinct vertices. Then it tests to see whether these vertices form a complete subgraph. If  $G$  is given by its adjacency matrix and  $|V|=n$ , the



input length  $m$  is  $n^2 + \lceil \log_2 k \rceil + \lceil \log_2 n \rceil + 2$ . The for loop of lines 4 to 9 can easily be implemented to run in nondeterministic time  $O(n)$ . The time for the for loop of lines 11 and 12 is  $O(k^2)$ . Hence the overall nondeterministic time is  $O(n + k^2) = O(n^2) = O(m)$ . There is no known polynomial time deterministic algorithm for this problem.

**Example 10.9 [Satisfiability]** Let  $x_1, x_2, \dots$  denote boolean variables (their value is either true or false). Let  $\bar{x}_i$  denote the negation of  $x_i$ . A literal is either a variable or its negation. A formula in the propositional calculus is an expression that can be constructed using literal and the operation **and** and **or**. Examples of such formulas are  $(x_1 \wedge x_2) \vee (x_3 \wedge x_4)$  and  $(x_3 \vee x_4) \vee (\bar{x}_1 \vee x_2)$ . The symbol  $\vee$  denotes **or** and  $\wedge$  denotes **and**. A formula is in conjunctive normal form (CNF) if and only if it is represented as  $\bigwedge_{i=1}^k c_i$ , where the  $c_i$  are clauses each represented as  $\bigvee_{j \in I_i} l_{ij}$ . The  $l_{ij}$  are literals. It is in disjunctive normal form (DNF) if and only if it is represented as  $\bigvee_{i=1}^k c_i$  and each clause  $c_i$  is represented as  $\bigwedge_{j \in I_i} l_{ij}$ . Thus  $(x_1 \wedge x_2) \vee (x_3 \wedge x_4)$  is in DNF whereas  $(x_3 \vee x_4) \wedge (\bar{x}_1 \vee x_2)$  is in CNF. The satisfiability problem is to determine whether a formula is true for some assignment of truth values to the variables. CNF-satisfiability is the satisfiability problem for CNF formulas.

It is easy to obtain a polynomial time nondeterministic algorithm that terminates successfully if and only if a given propositional formula  $E(x_1, \dots, x_n)$  is satisfiable. Such an algorithm could proceed by simply choosing (nondeterministically) one of the  $2^n$  possible assignments of truth values to  $(x_1, \dots, x_n)$  and verifying that  $E(x_1, \dots, x_n)$  is true for that assignment.

```

1 Algorithm DKP (p,w,n,m,r,x)
2 {
3 W:=0; P:=0;
4 for i:=1 to n do
5 {
6 x[i]:=Choice(0,1);
7 W:=W + x[i] * w[i]; P:=P + x[i] * p[i];
8 }
9 if ((W > m) or (P < r)) then Failure();
10 else Success();
11 }

```

Algorithm 10.4 Nondeterministic knapsack algorithm

```

1 Algorithm DCK(G,n,k)
2 {
3 S:=∅; //S is an initially empty set.
4 for i:=1 to k do
5 {
6 t:=Choice(1,n);
7 if t ∈ S then Failure();
8 S:= S ∪ {t} // Add t to set S.
9 }
10 // At this point S contains k distinct vertex indices.

```

```

11 for all pairs (i,j) such that $i \in S, j \in S$, and $i \neq j$ do
12 if (i,j) is not an edge of G then Failure();
13 Success();
14 }
```

#### Algorithm 10.5 Nondeterministic clique pseudocode

Eval (Algorithm 10.6) does this. The nondeterministic time required by the algorithm is  $O(n)$  to choose the value of  $(x_1, \dots, x_n)$  plus time needed to deterministically evaluate  $E$  for that assignment. This time is proportional to the length of  $E$ .

```

1 Algorithm Eval(E,n)
2 // Determine whether the propositional formula E is
3 // satisfiable. The variables are $x_1, x_2, \dots, x_n$
4 {
5 for i:= 1 to n do // Choose a truth value assignment.
6 $x_i := \text{Choice}(\text{false}, \text{true});$
7 if $E(x_1, \dots, x_n)$ then Success();
8 else Failure();
9 }
```

#### Algorithm 10.6 Nondeterministic satisfiability

### 10.1.2 The Classes NP-hard and NP-complete

In measuring the complexity of an algorithm, we use the input length as the parameter. An algorithm  $A$  is of polynomial complexity if there exists a polynomial  $p()$  such that the computing

time of  $A$  is  $O(p(n))$  for every input of size  $n$ .

**DEFINITION 10.3:**  $P$  is the set of all decision problems solvable by deterministic algorithms in polynomial time.  $NP$  is the set of all decision problem solvable by nondeterministic algorithm in polynomial time.

Since deterministic algorithms are just a special case of nondeterministic ones, we conclude that  $P \subseteq NP$ . What we do not know, and what has become perhaps the most famous unsolved problem in computer science is whether  $P = NP$  or  $P \neq NP$ .

Is it possible that for all the problems in  $NP$ , there exist polynomial time deterministic algorithm that have remained undiscovered? This seems unlikely, atleast because of the tremendous effort that has already been expended by so many people on these problems. Nevertheless, a proof of that  $P \neq NP$

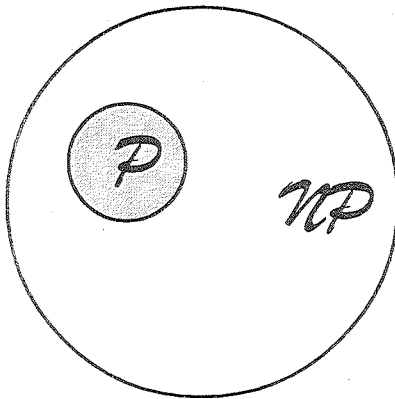


Figure 10.1 Commonly belived relationship between  $P$  and  $NP$

is just as elusive and seems to require as yet undiscovered techniques. But as with many famous unsolved problems, they serve to generate other useful results, and the question of whether  $NP \subseteq P$  is no exception. Figure 10.1 displays the relationship between  $P$  and  $NP$  assuming that  $P \neq NP$ .

S. Cook formulated the following question: Is there any single problem in  $NP$  such that if we showed it to be in  $P$ , then that would imply that  $P = NP$ ? Cook answered his own question in the affirmative with the following theorem.

**Theorem 10.1[cook]** Satisfiability is in  $P$  if and only if  $P = NP$ .

We are now ready to define the  $NP$ -hard and  $NP$ -complete classes of problems. First we define the notion of reducibility.

**DEFINITION 11.4.** Let  $L_1$  and  $L_2$  be problems. Problem  $L_1$  reduces to  $L_2$  (also written  $L_1 \alpha L_2$ ) if and only if there is a way to solve  $L_1$  by a deterministic polynomial time algorithm using a deterministic algorithm that solves  $L_2$  in polynomial time.

This definition implies that if we have a polynomial time algorithm for  $L_2$ , then we can solve  $L_1$  in polynomial time. One can readily verify that  $\alpha$  is a transitive relation (that is if  $L_1 \alpha L_2$  and  $L_2 \alpha L_3$ , then  $L_1 \alpha L_3$ ).

**DEFINITION 10.5** A problem  $L$  is  $NP$ -hard if and only if satisfiability reduces to  $L$  (satisfiability  $\alpha L$ ). A problem  $L$  is  $NP$ -complete if and only if  $L$  is  $NP$ -hard and  $L \in NP$ .

It is easy to see that there are NP-hard problems that are not NP-complete. Only a decision problem can be NP-complete. However, an optimization problem may be NP-hard. Furthermore if  $L1$  is a decision problem and  $L2$  an optimization problem, it is quite possible that  $L1 \leq L2$ . One can trivially show that the knapsack decision problem reduces to the knapsack optimization problem. For the clique problem one can easily show that the clique decision problem reduces to the clique optimization problem. In fact, one can also show that these optimization problems reduce to their corresponding decision problems (see the exercises). Yet, optimization problem cannot be NP-complete whereas decision problems can. There also exist NP-hard decision problems that are not NP-complete. Figure 10.2 shows the relationship among these classes.

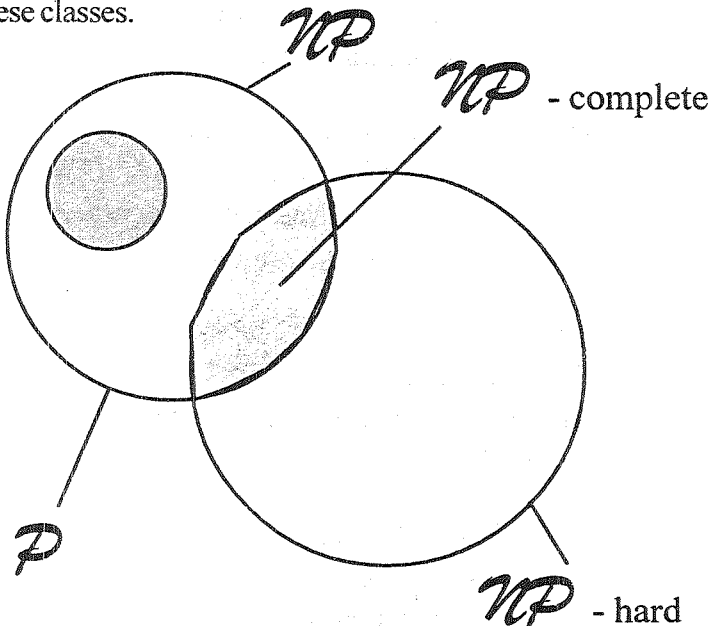


Figure 10.2 Commonly believed relationship among  $P$ ,  $NP$ ,  $NP$ -complete, and  $NP$ -hard problems.

**Example 10.10** As an extreme example of an NP-hard decision problem that is not NP-complete consider the halting problem for deterministic algorithms. The halting problem is to determine for an arbitrary deterministic algorithm  $A$  and an input  $I$  whether algorithm  $A$  with input  $I$  ever terminates (or enters an infinite loop). It is well known that this problem is undecidable. Hence, there exists no algorithm (of any complexity) to solve this problem. So, it clearly cannot be in NP. To show satisfiability of the halting problem, simply construct an algorithm  $A$  whose input is a propositional formula  $X$ . If  $X$  has  $n$  variables, then  $A$  tries out all  $2^n$  possible truth assignments and verifies whether  $X$  is satisfiable. If it is, then  $A$  stops. If it is not, then  $A$  enters an infinite loop. Hence,  $A$  halts on input  $X$  if and only if  $X$  is satisfiable. If we had a polynomial time algorithm for the halting problem, then we could solve the satisfiability problem in polynomial time using  $A$  and  $X$  as input to the algorithm for the halting problem. Hence, the halting problem is an NP-hard problem that is not in NP.

**DEFINITION 10.6** Two problems  $L_1$  and  $L_2$  are said to be polynomially equivalent if and only if  $L_1 \leq L_2$  and  $L_2 \leq L_1$ .

To show that a problem  $L_2$  is NP-hard, it is adequate to show  $L_1 \leq L_2$ , where  $L_1$  is some problem already known to be NP-hard. Since  $\leq$  is a transitive relation, it follows that if satisfiability  $\leq L_1$  and  $L_1 \leq L_2$ , then satisfiability  $\leq L_2$ . To show that an NP-hard decision problem is NP-complete, we have just to exhibit a polynomial time nondeterministic algorithm for it.

Later sections show many problems to be NP-hard. Although we restrict ourselves to decision problems, it should be clear that the corresponding optimization problems are also NP-hard. The NP-completeness proofs are left as exercises (for those problems that are NP-complete).

---

